WEBSITE BOOK EDITION

Pixels → Perception

From Pixels to Understanding

Foundations • Vision • Language • Learning • Theory

Volume 4 of 4

Optimization, Information Theory and Essays

English Edition • Volume 4

2026-10-31

About this edition

This is volume 4 of 4 of the Pixels → Perception course, *Optimization, Information Theory and Essays*. It contains 16 published tutorials and 2 blog posts, in the order of the website's learning tracks. This is a typeset edition of the website, not a separately peer-reviewed publication.

The prose, code, equations, and references retain the website's content. Exercise hints are expanded for print. Static figures remain in the book, image comparisons show their input and output images, and interactive demonstrations link to the website. Reference numbers are local to each tutorial.

The website is still under review. AI tools assisted its production, and errors or omissions may remain. Read it alongside the original sources and consult the website for subsequent corrections.

English and Traditional Chinese, and each volume, are separate PDFs. Links to tutorials in this volume navigate within the PDF. Links to tutorials in other volumes, to specific web sections, interactive demonstrations, notebooks, and external sources remain online. Content snapshot: 2026-10-31

Source revision: 8532535

https://blog.shyandram.dev/

Contents

I Optimization 1

OPT 01 · Modeling Decisions and Convex Sets 2

OPT 02 · Convex Functions, Composition and Conjugates 17

OPT 03 · Linear, Quadratic, Conic and Geometric Programs 33

OPT 04 · Duality, KKT Conditions and Certificates 48

OPT 05 · Approximation, Statistical Estimation and Experiment Design 63

OPT 06 · Gradient Descent, Newton Steps and Numerical Reliability 79

OPT 07 · Equality Constraints and Interior-Point Methods 95

OPT 08 · Geometric Applications, Splitting and Modern Methods 111

II Information Theory 127

IT 01 · Entropy, Relative Entropy and Mutual Information 128

IT 02 · Typical Sequences, AEP and Entropy Rates 144

IT 03 · Lossless Coding and Universal Compression 159

IT 04 · Channel Capacity and Reliable Communication 174

IT 05 · Differential Entropy and Gaussian Channels 188

IT 06 · Rate–Distortion and Lossy Representations 204

IT 07 · Statistical Evidence, Maximum Entropy and Growth 220

IT 08 · Networks, Information Inequalities and Learning 236

III Blog Essays 251

Introducing Pixels → Perception 252

Welcome to Pixels → Perception 255

Part I

Optimization

OPT 01 • Modeling Decisions and Convex Sets

Turn an engineering decision into variables, objectives and constraints; understand convex geometry, projections, separation and the limits of numerical solutions.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-01-modeling-convex-sets/

What you'll learn

- Translate a decision into variables, an objective, constraints and fixed data.
- Distinguish feasible, optimal, infeasible, unbounded and unattained problems.
- Understand affine sets, convex combinations, convex hulls and cones.
- Recognize halfspaces, polyhedra, norm balls, ellipsoids and positive-semidefinite cones.
- Use convexity-preserving operations, projections and separating hyperplanes.
- Check what a numerical solution establishes about the model and the underlying application.

The big picture

An image restoration system must choose pixel values. A scheduler must choose resource allocations. A fitting procedure must choose coefficients. In each case there are many possible choices, a way to compare them, and restrictions on what choices are allowed. Optimization makes those three ingredients explicit.

This course follows Stephen Boyd and Lieven Vandenberghe's *Convex Optimization* [1]. Its emphasis is learning to recognize, formulate and solve problems whose geometry supports global guarantees. The first chapters develop sets, functions, standard problem forms and duality. Applications connect that theory to fitting, statistical estimation and geometric design. The final chapters examine numerical algorithms and modern extensions.

> **Explain like I'm five**
>
> Imagine choosing ingredients for a smoothie. You want a tasty drink, have a fixed budget, and cannot use ingredients that have run out. The ingredient amounts are your choices; taste is what you improve; budget and availability are rules. If mixing any two allowed recipes still gives an allowed recipe, the allowed recipes form a convex set.

The analogy is deliberately limited: taste need not be a convex function, and real ingredients can come in indivisible packages. The first skill is to identify which parts of a real decision fit a convex model and which parts do not. A convincing model is more valuable than a fast solver applied to the wrong equations.

Convex optimization is also useful around nonconvex learning. It can solve a linear classifier on fixed features, compute a constrained prediction, fit a calibration curve or construct a lower

bound for a harder problem. It does not turn an arbitrary neural network's training objective into a convex one. The guarantees belong to the particular mathematical problem being solved.

From a sentence to an optimization model

Plain version. Decide what you can change, what is already known, what makes a choice good, and which rules it must obey. Write these down before choosing an algorithm.

A general scalar optimization problem can be expressed as

$$\begin{aligned} \text{minimize}_{x \in \mathbb{R}^n} \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p. \end{aligned}$$

Here x is the vector of n decision variables; f_0 is the scalar objective to minimize; f_i are m inequality constraint functions; and h_j are p equality constraint functions. Fixed measurements and design parameters enter these functions but are not decision variables unless the model explicitly makes them so.

For example, suppose two operations use amounts x_1 and x_2 of a shared resource. A capacity constraint might be $2x_1 + x_2 \le 5$, while $x_1, x_2 \ge 0$ rules out negative allocations. If the target allocation is b, an objective $\|x - b\|_2^2$ prefers choices close to that target. The numbers in this example specify a synthetic model; they are not measurements of a production system.

Units are part of the formulation. Adding a cost in dollars to a reconstruction error in intensity units requires a weighting convention. A coefficient that converts between their scales should be interpreted and tested. Changing image normalization from zero-to-255 to zero-to-one can change the effective regularization if other terms are left unchanged.

It is helpful to keep a small modeling ledger: each variable's meaning and unit, each fixed input's provenance, the justification for each constraint, and the interpretation of the objective. This avoids treating a convenient formula as if it were the only possible representation of the task.

Constraints and penalties express different commitments

A hard constraint says a choice is disallowed. A penalty says a violation or deviation is undesirable but can be exchanged against other benefits. The distinction matters even if both produce similar answers on one example.

Consider nonnegative image intensities. Enforcing $x \ge 0$ makes all feasible outputs nonnegative. Adding a quadratic penalty on negative entries may still permit some negativity if it improves the data term enough. Increasing a penalty is not automatically the same as enforcing an exact constraint, especially with finite numerical precision.

Likewise, a budget constraint sets a fixed resource limit. A cost penalty expresses a preference that depends on the tradeoff coefficient. Choosing that coefficient is part of the application, not

something convexity decides. Later chapters derive how dual multipliers quantify the sensitivity to changing hard constraints.

Worked model: a target allocation under a capacity limit

A target need not obey the available capacity. Optimization finds the nearest allowed allocation, rather than reducing every coordinate by the same amount.

Take the synthetic target $b = (3, 3)$ and minimize $\frac{1}{2}\|x - b\|_2^2$ with $x \geq 0$ and $2x_1 + x_2 \leq 5$. The target uses nine units of capacity, so it is infeasible. At the solution the capacity boundary is active. Its normal is $a = (2, 1)$, so the shortest correction points along a:

$$x^* = b - \alpha a, \qquad \alpha = \frac{a^\top b - 5}{a^\top a} = \frac{4}{5}, \qquad x^* = \left(\frac{7}{5}, \frac{11}{5}\right).$$

Here x^* is the candidate allocation, α is the correction scale, and a is the capacity normal. Substituting gives $2x_1^* + x_2^* = 5$ and both coordinates are positive, so the nonnegativity constraints do not alter this projection.

Why is this the closest point? For any feasible x, $a^\top(x - x^*) \leq 0$. Expand the objective difference:

$$\frac{1}{2}\|x - b\|_2^2 - \frac{1}{2}\|x^* - b\|_2^2 = \frac{1}{2}\|x - x^*\|_2^2 - \alpha a^\top(x - x^*) \geq 0.$$

Here x is any feasible competitor; all other symbols are defined above. Both terms are nonnegative. The candidate therefore minimizes the objective globally, whose value is $8/5$. This calculation connects the variables, the active rule, the correction direction and the optimum before introducing a solver [1].

Feasibility and optimal value

Plain version. An allowed answer and the best allowed answer are different objects. Sometimes there are no allowed answers; sometimes the model prefers choices that run away without bound.

The feasible set is

$$\mathcal{F} = \{x : f_i(x) \leq 0 \text{ for every } i,\ h_j(x) = 0 \text{ for every } j,\ x \text{ in all required domains}\}.$$

Here $\mathcal{F}$ is the set of points satisfying all constraints and function-domain restrictions. A point in $\mathcal{F}$ is feasible. The optimal value is

$$p^* = \inf_{x \in \mathcal{F}} f_0(x).$$

p^* is the greatest lower bound on achievable objective values. An optimizer x^* is a feasible point attaining that value: $f_0(x^*) = p^*$. The infimum can exist without an optimizer.

Four cases are worth distinguishing. A feasible problem can have an attained finite optimum. An infeasible problem has an empty feasible set. An unbounded-below problem has objective

values tending to negative infinity. An unattained problem can have a finite limiting value that no feasible point reaches.

For a concrete unattained example, minimize x subject to $x > 0$. The optimal value is zero, but zero is excluded. For an unbounded example, minimize $-x$ on $x \geq 0$. For an infeasible example, require both $x \geq 1$ and $x \leq 0$. These have very different implications even if a solver returns no finite optimizer.

Closedness and boundedness help establish existence. A continuous objective on a nonempty compact feasible set attains a minimum. A closed but unbounded set can also admit a minimizer if the objective grows sufficiently at infinity. The existence question concerns the model before any algorithm starts.

Numerical status is evidence about a finite-precision computation

A solver typically assesses constraint residuals, optimality conditions or a primal–dual gap against tolerances. A successful status does not imply exact real-number feasibility. It also does not establish that the measurements, objective or application constraints were correct.

If the application requires a genuinely safe limit, decide how numerical tolerances relate to that limit. Rescaling a constraint can change its raw residual without changing the feasible set. For example, $x \leq 1$ and $10^6 x \leq 10^6$ are equivalent mathematically but have very different unnormalized residual magnitudes.

Record the scale, tolerance and units of residuals. Do not compare objective values between runs whose objective definitions differ. If an algorithm stops because iterations are exhausted, report that condition rather than interpreting a nearly unchanged iterate as a proof of optimality.

Affine and convex sets

Plain version. An affine set contains complete lines through its points. A convex set only needs to contain the line segments between its points.

For points $u, v \in \mathbb{R}^n$ and scalar θ, a two-point affine combination is

$$x = \theta u + (1 - \theta)v.$$

Here u and v are vectors, x is their combination, and θ controls position along their line. Allowing every real θ traces the entire line. Restricting $0 \leq \theta \leq 1$ traces the segment between the endpoints.

A set C is affine if it contains all affine combinations of its points. It is convex if it contains all convex combinations, where coefficients are nonnegative and sum to one. Thus every affine set is convex, while a disk is convex without being affine.

For finitely many points the convex-combination rule is

$$x = \sum_{i=1}^{r} \theta_i v_i, \qquad \theta_i \geq 0, \qquad \sum_{i=1}^{r} \theta_i = 1.$$

v_i are the r endpoint vectors and θ_i are their weights. The combination is a weighted average. Nonnegative weights are essential: extrapolation can leave a convex set.

Convexity says nothing by itself about smooth boundaries, boundedness or closedness. An open ball is convex; a line is unbounded and convex; a single point is convex. The empty set is conventionally convex because there are no endpoint pairs violating the definition.

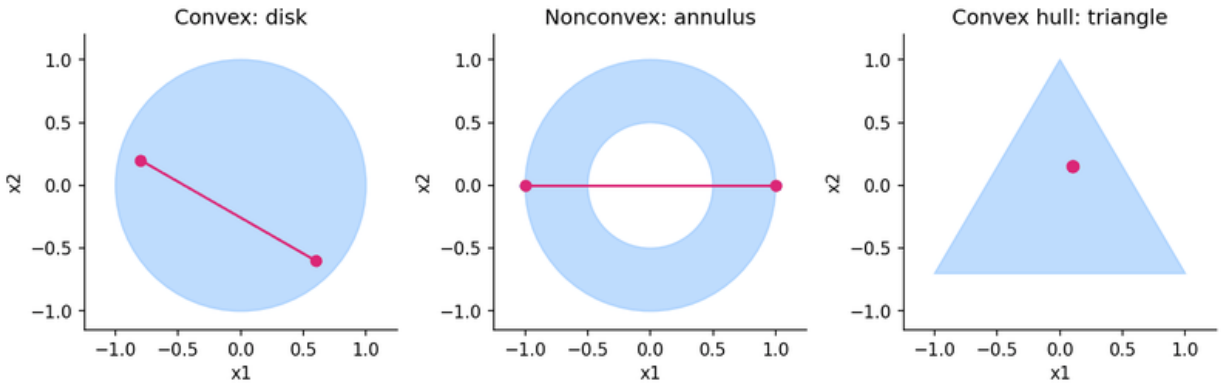


Figure 1 — Original synthetic geometry. A disk and a filled triangle are convex. The annulus is not: two allowed endpoints have a segment that passes through the excluded hole.

A counterexample disproves; sampled tests do not prove

To disprove convexity, find two points in the set whose midpoint or another segment point lies outside. This is often simpler than analyzing a complicated boundary. A ring fails because opposite points average to its excluded center.

To prove convexity, use the definition or a preservation rule. Testing many random pairs can reveal a bug but cannot establish the property for every pair. A tiny nonconvex notch may escape all sampled checks.

The distinction is similar to numerical gradient checks. They are valuable implementation diagnostics, while the mathematical argument addresses a universal statement. Use both for different purposes, and label their roles clearly.

Code: combinations and a nonconvex counterexample

This self-contained NumPy example verifies particular points. The mathematical proof for the disk follows from the triangle inequality, not from the assertions alone.

```
import numpy as np

u = np.array([0.8, 0.1])
v = np.array([-0.2, 0.7])
theta = np.linspace(0, 1, 101)
```

```
segment = theta[:, None] * u + (1 - theta[:, None]) * v
assert np.linalg.norm(segment, axis=1).max() <= 1 + 1e-12
ring_u, ring_v = np.array([1.0, 0.0]), np.array([-1.0, 0.0])
midpoint = (ring_u + ring_v) / 2
assert not (0.5 <= np.linalg.norm(midpoint) <= 1.2)
print("disk sampled-segment maximum norm:", np.linalg.norm(segment, axis=1).max())
print("ring midpoint:", midpoint, "norm:", np.linalg.norm(midpoint))
```

Convex hulls and relaxations

Plain version. A convex hull fills in every weighted average of a collection of choices. It is useful when discrete choices are hard, but its added points may be unavailable in the original application.

The convex hull of a set S is

$$\operatorname{conv}(S) = \left\{ \sum_{i=1}^{r} \theta_i v_i : r \text{ finite}, \ v_i \in S, \ \theta_i \geq 0, \ \sum_i \theta_i = 1 \right\}.$$

Here S is the original collection; r is the number of selected points in a finite combination; and $\operatorname{conv}(S)$ is the smallest convex set containing all of S.

Suppose an allocation must choose one of three configurations. The convex hull also permits mixtures. If mixtures correspond to time sharing, these points may be operationally meaningful. If the decision must select a single indivisible machine, a fractional hull point is a relaxation, not a feasible original answer.

For a minimization problem, enlarging the feasible set cannot increase the optimal value. Therefore a convex relaxation can provide a lower bound on a discrete problem. Turning its optimizer into an actual discrete choice requires another step, and the resulting rounded choice may violate constraints or lose objective quality.

Do not describe an exact optimum of the relaxed problem as an exact optimum of the original problem. Keep the original feasible set, relaxed set, relaxed lower bound and achieved original objective separate. The applications chapter returns to this distinction.

A working vocabulary of convex sets

Plain version. Most useful convex models are assembled from a few reusable geometric shapes. Learning those shapes makes an unfamiliar equation easier to classify.

Hyperplanes, halfspaces and polyhedra

A hyperplane and a closed halfspace have forms

$$H = \{x : a^\top x = b\}, \qquad P = \{x : a^\top x \leq b\}.$$

$x \in \mathbb{R}^n$ is a point, $a \in \mathbb{R}^n$ is a nonzero normal vector and $b \in \mathbb{R}$ is an offset. A hyperplane is affine. A halfspace contains points on one side and its boundary.

A polyhedron is an intersection of finitely many halfspaces and hyperplanes, often written $Ax \leq b$, $Ex = d$. Matrices A, E hold constraint coefficients; vectors b, d hold bounds. The inequality is componentwise. Polyhedra can be bounded, unbounded, lower-dimensional or empty.

A box is a polyhedron with coordinate bounds. A probability simplex adds nonnegativity and a sum-to-one equality. These forms show why probability vectors and bounded pixel values frequently fit convex constraints.

Norm balls and ellipsoids

A norm ball is

$$B = \{x : \|x - c\| \leq r\}.$$

c is its center, $r \geq 0$ is its radius and $\|\cdot\|$ is a norm. Norm triangle inequalities imply convexity. Different norms encode different geometry: a Euclidean ball is round, an infinity-norm ball is a box, and a one-norm ball has corners.

An ellipsoid can be written

$$\mathcal{E} = \{c + Au : \|u\|_2 \leq 1\}.$$

A is a matrix mapping the unit Euclidean ball, u is the auxiliary ball coordinate and c is the center. If A is invertible, an equivalent quadratic description uses a positive-definite shape matrix. Singular A produces a lower-dimensional image, which remains convex.

Cones and matrix inequalities

A convex cone is closed under nonnegative combinations:

$$\alpha u + \beta v \in K \quad \text{for } u, v \in K,\ \alpha, \beta \geq 0.$$

K is the cone, u, v are its points and α, β are nonnegative scalars. The weights do not have to sum to one. The nonnegative orthant is a simple example.

The second-order cone consists of pairs (t, z) satisfying $\|z\|_2 \leq t$, where t is scalar and z is a vector. It represents many norm constraints.

The positive-semidefinite cone contains symmetric matrices X satisfying

$$v^\top X v \geq 0 \quad \text{for every } v \in \mathbb{R}^n.$$

$X \in \mathbb{R}^{n \times n}$ is symmetric, and v is an arbitrary test vector. This condition is written $X \succeq 0$. It is not the same as requiring every matrix entry to be nonnegative. Covariance matrices naturally lie in this cone.

Generalized inequalities use a cone to define order: $x \preceq_K y$ means $y - x \in K$. For the orthant this becomes coordinatewise order; for symmetric matrices it becomes positive-semidefinite order. Stating the cone avoids confusing these different comparisons.

Operations that preserve convexity

Plain version. Once a set is known to be convex, certain operations create another convex set automatically. These rules are stronger and easier to reuse than repeated plotting.

Intersections of convex sets are convex. A point on the segment between two members remains in every set being intersected. This is why combining convex constraints is straightforward.

Affine images and inverse images preserve convexity. If C is convex, both the transformed set $\{Ax + b : x \in C\}$ and the set $\{x : Ax + b \in C\}$ are convex. Here A is a fixed matrix and b a fixed vector. Nonlinear transformations do not have this general guarantee.

Cartesian products and sums of convex sets are convex. Projection onto selected coordinates is an affine image, so eliminating auxiliary variables preserves convexity of the feasible set. This fact underlies many extended formulations: a model may be easier to express with extra variables.

Unions are usually not convex. Two disjoint balls illustrate the failure. Replacing "and" with "or" in a constraint specification can fundamentally change the problem, even when each branch is individually convex.

For matrix variables, the same reasoning works in a vector space of symmetric matrices. If X and Y are positive semidefinite, then $\theta X + (1 - \theta)Y$ is positive semidefinite for $0 \le \theta \le 1$: test the quadratic form against every vector.

Two short proofs you can reuse

Convexity proofs usually track a segment through the construction. Suppose $u, v \in C_1 \cap C_2$, where C_1, C_2 are convex. For $0 \le \theta \le 1$, the mixture $\theta u + (1 - \theta)v$ belongs to each set, hence to their intersection. The same argument works for any collection of convex sets.

For an affine image, let $y_1 = Au + b$ and $y_2 = Av + b$ with $u, v \in C$. Their mixture is

$$\theta y_1 + (1 - \theta)y_2 = A[\theta u + (1 - \theta)v] + b.$$

Here C is convex, A, b are fixed transformation data, and y_1, y_2 are image points. The bracketed point belongs to C, so the mixture belongs to the image. This is why eliminating auxiliary coordinates preserves a convex feasible set [1, Section 2.3].

The corresponding union proof fails: two endpoints can come from different sets. For instance, $-1 \in [-2, -1]$ and $1 \in [1, 2]$, but their midpoint zero lies in neither interval. Each interval is convex; the union is not. In modeling language, simultaneous convex rules preserve convexity, while an either-or choice can break it.

Projections and separating hyperplanes

Plain version. Projection finds the closest allowed point to an arbitrary point. The direction from that point toward its projection also reveals which side of a boundary contains the feasible set.

For a nonempty closed convex set C, Euclidean projection is

$$\Pi_C(y) = \arg\min_{x \in C} \frac{1}{2}\|x - y\|_2^2.$$

$y \in \mathbb{R}^n$ is the query point, x is the candidate feasible point and $\Pi_C(y)$ is the unique minimizing point. Strict convexity of squared Euclidean distance ensures uniqueness. Other distances or nonconvex sets need not have the same property.

Let $z = \Pi_C(y)$. Its optimality condition is

$$(y - z)^\top (x - z) \leq 0 \quad \text{for every } x \in C.$$

Here z is the projection and x ranges over feasible points. The condition follows by considering a small step from z toward any $x \in C$ and requiring that squared distance cannot decrease.

If $y \notin C$, the vector $y - z$ is nonzero. It defines a hyperplane through z with the entire feasible set on the opposite side from y. This is a concrete separating-hyperplane construction.

More general separation theorems compare disjoint convex sets [1, Section 2.5]. Separation need not have a positive margin without additional assumptions. Closedness, compactness and distance between sets matter when asserting strict separation.

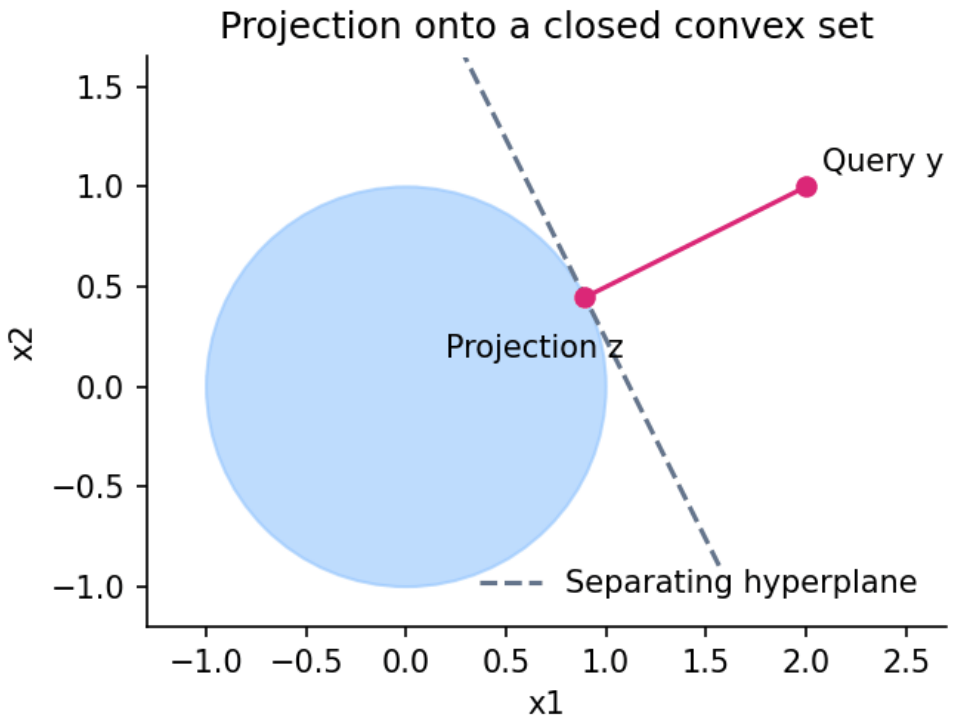


Figure 2 — Original analytical illustration of Euclidean projection onto the unit disk. The projection direction supplies a separating normal for this external point.

Code: box and ball projections

Projection onto a box clips each coordinate. Projection onto a Euclidean ball rescales a point only if it lies outside. These formulas follow from the geometry and are covered by the geometric applications in [1, Section 8.1].

```
import numpy as np

y = np.array([2.0, -1.5])
lower, upper = np.array([0.0, -1.0]), np.array([1.0, 1.0])
box_projection = np.clip(y, lower, upper)
radius = 1.0
ball_projection = y * min(1.0, radius / np.linalg.norm(y))
rng = np.random.default_rng(2)
points = rng.normal(size=(1000, 2))
points /= np.maximum(1.0, np.linalg.norm(points, axis=1))[:, None]
inequality = (points - ball_projection) @ (y - ball_projection)
assert inequality.max() <= 1e-12
assert np.all(box_projection >= lower) and np.all(box_projection <= upper)
print("box projection:", box_projection)
print("ball projection:", ball_projection)
```

The sampled check illustrates the projection inequality. It does not replace a proof for all feasible points. More complicated projections can themselves require solving an optimization problem; "projected gradient" does not imply that projection is inexpensive.

Deriving the projection test

The projection inequality comes from trying every feasible direction, not from drawing a tangent that looks right. Let $z = \Pi_C(y)$ and choose any $x \in C$. Convexity makes $z + \theta(x - z)$ feasible for $0 \le \theta \le 1$. Because z minimizes squared distance, its one-sided directional derivative must be nonnegative:

$$\frac{d}{d\theta}\frac{1}{2}\|z + \theta(x - z) - y\|_2^2\Big|_{\theta=0} = (z - y)^\top (x - z) \ge 0.$$

Here y is the query, z its projection, x a feasible comparison point and θ the segment parameter. This gives the stated separation inequality. It is also sufficient: expanding the distance at x yields the distance at z, plus a nonnegative inner product and $\frac{1}{2}\|x - z\|_2^2$.

Uniqueness follows just as directly. If distinct z_1, z_2 both minimized squared distance, their midpoint would be feasible and have a strictly smaller value. Closedness supplies existence: a minimizing sequence stays in a bounded distance sublevel set and has a convergent subsequence whose limit remains in C. Thus nonempty, closed and convex each plays a specific role [1].

Interior, relative interior and hidden equalities

Plain version. A feasible set can have room to move within its own plane even if it has no room to move in the full surrounding space.

Consider probability vectors whose coordinates sum to one. They lie in a hyperplane, so their ordinary interior in the full vector space is empty. They can nevertheless have strictly positive coordinates and an interior relative to that hyperplane.

The affine hull is the smallest affine set containing C. The relative interior is the interior measured within that affine hull. This distinction matters in feasibility conditions and duality. Requiring a full-dimensional interior would incorrectly exclude many useful equality-constrained models.

Equality constraints can also be hidden inside inequalities. If both $a^\top x \le b$ and $a^\top x \ge b$ are present, they force equality. Recognizing such structure can improve interpretation and numerical treatment.

Strict feasibility does not mean the same thing as having a small computed violation. It is a mathematical property requiring relevant inequalities to hold strictly while equalities remain satisfied. The duality and interior-point chapters specify exactly when this condition is needed.

Convexity of a problem requires both sets and functions

Plain version. Convex allowed choices are only half of the story. The objective must also have the right shape, and equality constraints must preserve that geometry.

A convex minimization problem uses a convex objective, convex inequality functions and affine equalities on their appropriate domains [1]. The next chapter defines convex functions and proves the rules used to recognize them.

A convex feasible set alone does not guarantee a convex optimization problem. Minimizing a function with many valleys over a box remains nonconvex. Conversely, a convex objective with disconnected discrete constraints remains nonconvex.

When the whole problem is convex, every local minimizer is global. This does not guarantee a unique minimizer: a linear objective can be constant along an entire face. Stronger objective properties, such as strict convexity on the feasible set, can establish uniqueness when a minimum exists.

Neither globality nor uniqueness guarantees numerical ease at every scale. A huge dense problem may require more memory than available. An extremely ill-conditioned model may be difficult at tight tolerance. Convexity supplies structural advantages, while complexity and implementation still matter.

Why a local minimum must be global

Convexity excludes a better distant answer by connecting it to the current point. Suppose x is a local minimizer of convex f on convex feasible set C, but some feasible y has $f(y) < f(x)$. For any $0 < \theta \leq 1$, define $z_\theta = (1-\theta)x + \theta y$. Then

$$f(z_\theta) \leq (1-\theta)f(x) + \theta f(y) < f(x).$$

Here z_θ is a feasible segment point and θ its mixing weight. For arbitrarily small positive θ, the point lies arbitrarily close to x and improves the objective, contradicting local minimality. No differentiability is needed [1].

This proof gives globality, not uniqueness. For example, minimizing x_1 on $0 \leq x_1, x_2 \leq 1$ gives every point $(0, x_2)$ as a global minimizer. Adding a strictly convex objective makes distinct tied minimizers impossible, because their feasible midpoint would have lower cost. Existence remains separate: minimizing x on $x > 0$ is convex but has no minimizer.

These three questions guide solver expectations. Convexity makes local minima global; strict curvature can select one answer; closedness and objective growth can ensure an answer exists. A numerical procedure needs an appropriate convergence argument in addition to all three.

Modern view

Plain version. Modern convex optimization still begins with modeling and geometry. Recent methods change how a feasible set is accessed and how much computation is spent obtaining an accurate certificate.

Grant, Boyd and Ye's disciplined convex programming framework [2] organizes models through verified composition rules. It is a modeling methodology, not an independent guarantee that any convenient expression is convex. An algebraically convex expression may need reformulation so its convexity is visible to the rule system.

Pokutta's 2024 survey of Frank–Wolfe methods [3] distinguishes access through projection, barrier functions or a linear minimization oracle. Its useful lesson here is geometric: a feasible set that is easy to describe may still be expensive to project onto. Linear minimization can sometimes be cheaper, and convex combinations preserve feasibility. Later algorithm chapters explain those updates and their assumptions.

The PDLP report by Applegate and colleagues [4], first posted in 2025 and revised in 2026, develops a practical first-order linear-programming solver. It connects a primal–dual formulation with scaling, restarts and feasibility polishing. This is an original solver study, not a survey. We use it as evidence that implementation details and the desired accuracy remain important even for a familiar convex problem class.

Current challenges include representing complex constraints without excessive auxiliary structure, exploiting sparse matrices, making tolerances meaningful under scaling, and maintaining certificates when models are embedded in larger pipelines. A learned feature extractor can be nonconvex while a subsequent allocation problem is convex; describe each stage's guarantees separately.

These sources are a curated reading path, not a claim that every optimization survey has been reviewed. The core theory in this course follows [1], while the modern sections identify precisely which extensions each source supports.

Key takeaways

- Variables, fixed data, objectives, constraints and units must be explicit.
- Feasibility, boundedness, attainment and numerical termination are different questions.
- Convex sets contain all weighted averages with nonnegative weights summing to one.
- Affine sets, polyhedra, norm balls and important cones provide reusable building blocks.
- Intersections and affine transformations preserve convexity; unions generally do not.
- Projection onto a closed convex set is unique and supplies a useful geometric certificate.
- Convex optimization guarantees concern the formulated problem and its stated assumptions.

Exercises

1. Formulate allocating three nonnegative resources under a total budget, with squared deviation from a chosen target. State all units and distinguish variables from data.

   Hint

   Use a vector x, nonnegativity, one sum constraint and a fixed target vector.

2. Prove directly that the intersection of two convex sets is convex. Give a counterexample for their union.

   Hint

   A segment stays in both sets; two separated intervals disprove the union rule.

3. A system can select one of three configurations. Explain when a convex mixture is an actual operational choice and when it is only a relaxation.

   Hint

   Consider time sharing versus an indivisible hardware choice.

4. Derive projection onto a centered Euclidean ball. Identify what happens when the query point is already inside.

   Hint

   Outside the ball, the closest point lies on the query's radial direction.

5. Give a finite-valued convex problem whose optimum is not attained, and explain which existence condition fails.

   Hint

   An open interval can exclude its limiting optimum.

6. A solver reports constraint residual 10^{-6}. Explain why its adequacy cannot be judged without constraint scale and application units.

   Hint

   Rescaling the same inequality changes the raw residual.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapters 1–2; Sections 4.1–4.2 and 8.1 for problem terminology and projections.** Official book page, free official PDF.
2. Michael Grant, Stephen Boyd and Yinyu Ye, "Disciplined Convex Programming," in *Global Optimization: From Theory to Implementation*, Springer, 2006. Author's publication page.
3. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.
4. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Verified paper record.

OPT 02 • Convex Functions, Composition and Conjugates

Recognize convex objectives through chords, supporting planes and curvature; build valid compositions, use perspectives and conjugates, and distinguish quasiconvexity.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-02-convex-functions/

What you'll learn

- Define convexity with domains, chords and epigraphs.
- Use first-order and second-order tests without assuming differentiability everywhere.
- Recognize important scalar, norm and matrix functions.
- Build convex models using sums, affine substitutions, maxima, composition and perspectives.
- Derive conjugates and understand their connection to linear lower bounds.
- Distinguish convex, strongly convex, quasiconvex and log-concave structure.

The big picture

The previous chapter described allowed choices. This chapter describes how their scores change. A convex objective has a useful consistency: mixing two choices cannot produce a score worse than the corresponding mixture of their scores. Its local supporting planes provide global lower bounds.

This does not mean every familiar loss is convex in every variable. Squared error is convex in the predicted values and in linear model coefficients. It is generally not convex in all parameters of a multilayer neural network. Always specify the variable being optimized and the domain being considered.

> **Explain like I'm five**
>
> Imagine a bowl and a straight stick joining two points on its rim. The bowl stays below the stick. A little flat board touching the bowl from underneath stays underneath everywhere. Those two pictures explain convex chords and supporting planes.

Boyd and Vandenberghe's Chapter 3 [1] develops several ways to recognize this structure. The goal is not to memorize a huge catalog. It is to combine a small catalog with valid transformation rules, so a complicated objective comes with a reason for being convex.

Convexity is a statement about the complete function, including its domain. A formula may have positive curvature wherever defined, while its domain is disconnected. That does not define a convex function on the whole set. Domain restrictions deserve the same attention as derivatives.

Chords, domains and epigraphs

Plain version. Draw the straight line between two function values. For a convex function, the actual value at every intermediate input is no higher than that line.

For a function $f : C \to \mathbb{R}$ on convex domain C,

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta)f(y), \qquad 0 \le \theta \le 1.$$

$x, y \in C$ are arbitrary inputs, θ is a mixing weight, and f is convex if this inequality holds for every such choice. Reversing the inequality defines concavity. An affine function satisfies equality and is both convex and concave.

The condition compares inputs and outputs simultaneously. Convexity of the domain ensures the mixed input is available. A function does not qualify by passing the inequality only for a convenient subset of endpoint pairs.

The epigraph is

$$\text{epi} f = \{(x,t) : x \in C, f(x) \le t\}.$$

t is a scalar vertical coordinate and (x,t) is a point above or on the function graph. A function is convex exactly when its epigraph is a convex set. This provides a bridge from Chapter 1's geometry to objectives and inequality constraints.

An epigraph formulation introduces a variable t and minimizes it subject to $f(x) \le t$. It can make a nonsmooth objective easier to express using constraints. It does not change the mathematical optimum when the formulation is exact.

Jensen's inequality and expectations

The two-point inequality extends to finite weighted averages:

$$f\left(\sum_i \theta_i x_i\right) \le \sum_i \theta_i f(x_i), \qquad \theta_i \ge 0, \quad \sum_i \theta_i = 1.$$

x_i are inputs and θ_i are weights. For a random vector X with suitable integrability and domain conditions, the analogous statement is $f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$.

This explains why a convex cost applied after averaging can be smaller than the average individual cost. It does not say that averaging arbitrary model parameters improves prediction: the prediction function and its loss must be convex in those parameters for that conclusion.

Jensen's inequality also helps reason about risk and uncertainty. A nonlinear transformation of an expected measurement need not equal the expected transformed measurement. The direction depends on convexity or concavity, not on a vague claim that averages are always representative.

Supporting planes: the first-order test

Plain version. For a differentiable convex function, the tangent approximation is a global lower bound, not merely a local sketch.

On an open convex domain, differentiable convexity is equivalent to

$$f(y) \geq f(x) + \nabla f(x)^{\top}(y - x).$$

x is the point where the approximation is made, y is any domain point, and $\nabla f(x)$ is the gradient with respect to the optimization variable. The affine expression on the right supports the function from below.

One way to derive the inequality is to apply the convex chord rule at $x + \theta(y - x)$, rearrange it, divide by positive θ and let θ tend to zero. This turns a global mixing condition into a directional derivative bound.

If $\nabla f(x) = 0$, the supporting-plane inequality gives $f(y) \geq f(x)$ everywhere in the domain. Thus a stationary point is globally minimizing in an unconstrained differentiable convex problem. It need not be unique.

With constraints, a nonzero gradient can still be optimal because improving directions are disallowed. The duality and constrained-algorithm chapters express that fact precisely. Simply requiring a small gradient is not an appropriate stopping rule for every constrained problem.

Nonsmooth supporting vectors

A corner does not destroy convexity. For convex f, a subgradient g at x satisfies

$$f(y) \geq f(x) + g^{\top}(y - x) \quad \text{for every } y.$$

g is a vector supplying a supporting affine lower bound; the set of all such vectors is the subdifferential $\partial f(x)$. At a differentiable point it contains only the gradient. At the origin, the absolute-value function has all scalar subgradients in $[-1, 1]$.

The notation is useful for absolute residuals, maxima and sparsity penalties. A subgradient is not an ordinary derivative chosen arbitrarily at a corner: it must satisfy a global support inequality. Later chapters use this structure when discussing proximal methods.

For a convex unconstrained function, $0 \in \partial f(x)$ implies global optimality. A zero gradient and a zero included among many subgradients therefore express the same underlying support idea in different settings.

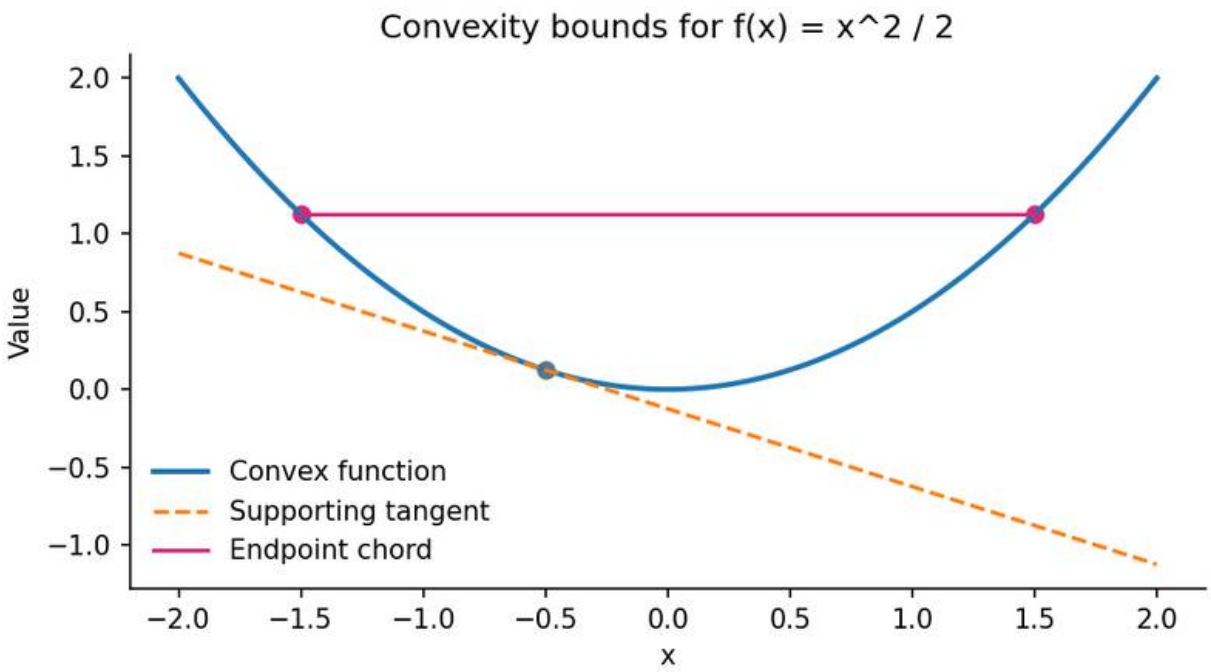


Figure 1 — Original analytical illustration for a scalar quadratic. Convex chords lie above the curve; its tangent at the selected point gives a global affine lower bound.

Curvature: the second-order test

Plain version. A smooth function is convex when it bends upward in every direction. Looking at only one coordinate at a time can miss a downward diagonal direction.

For twice continuously differentiable f on an open convex domain,

$$\nabla^2 f(x) \succeq 0 \quad \text{for every } x$$

is equivalent to convexity. $\nabla^2 f(x)$ is the symmetric Hessian matrix, and $\succeq 0$ means every directional quadratic form $v^\top \nabla^2 f(x) v$ is nonnegative.

Nonnegative diagonal Hessian entries are insufficient. The function $f(x_1, x_2) = x_1 x_2$ has zero coordinate second derivatives but an indefinite Hessian. Along one diagonal it curves upward, along the other downward.

The test is a global condition on the entire domain. Evaluating eigenvalues at ten points is a numerical diagnostic, not a proof. A symbolic Hessian with an analytical positive-semidefinite factorization can establish the property for all points.

Strict and strong convexity

Strict convexity requires a strict chord inequality for distinct endpoints and weights strictly between zero and one. When a minimum exists on a convex set, strict convexity ensures uniqueness.

Strong convexity in the Euclidean norm requires, for some $\mu > 0$,

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2.$$

μ is a uniform curvature lower bound. For twice differentiable functions, it corresponds to $\nabla^2 f(x) \succeq \mu I$, where I is the identity matrix. Strong convexity is stronger than strict convexity; a function can be strictly convex without a positive uniform curvature bound.

Adding a positive quadratic regularizer often supplies strong convexity in the penalized coordinates. An unpenalized intercept or a restricted domain requires separate analysis. Do not infer strong convexity of every parameter merely from one regularization term.

What the Hessian test measures

A multivariable curvature test becomes a scalar test along every line. Fix a point x and direction v, and set $q(t) = f(x + tv)$ on the interval that stays in the domain. Differentiating twice gives

$$q''(t) = v^\top \nabla^2 f(x + tv) v.$$

Here t is the line coordinate, q the restricted scalar function, and v any direction. A positive-semidefinite Hessian makes every such restriction convex. Every segment is part of one of these lines, so the chord definition follows. Conversely, convexity on every line requires its second derivative to be nonnegative. This explains why the test examines quadratic forms in all directions rather than individual matrix entries [1].

Strict and strong convexity differ in a concrete example. The scalar function $f(x) = x^4$ is strictly convex on $\mathbb{R}$: its derivative $4x^3$ is strictly increasing. Yet $f''(0) = 0$, so no positive global μ satisfies the strong-convexity Hessian bound. Its minimum is unique, but the strong-convexity stopping bound in OPT 06 is unavailable. Adding $\frac{\mu}{2}x^2$, with chosen $\mu > 0$, supplies the missing curvature floor and changes the objective.

A catalog of useful functions

Plain version. A few familiar building blocks cover a large share of practical convex objectives. Their domains and signs determine how they can be used.

Affine functions $a^\top x + b$ are both convex and concave. Norms are convex because of positive homogeneity and the triangle inequality. Squared Euclidean norm is convex and smooth; the norm itself is nonsmooth at the origin.

A quadratic

$$f(x) = \frac{1}{2}x^\top Q x + c^\top x + d$$

is convex exactly when symmetric $Q \succeq 0$. Here Q sets curvature, c is a linear coefficient vector and d is a scalar constant. Positive-definite Q gives strong convexity with a bound set by its smallest eigenvalue.

Scalar exponential is convex everywhere. Negative logarithm is convex on positive inputs. Powers need their exponent and domain specified; statements such as “all power functions are convex” are false. For example, square root is concave on nonnegative inputs, while its negative is convex there.

The maximum coordinate and maximum of affine scores are convex and generally nonsmooth. A minimum of affine functions is concave, not generally convex. The sign of a modeling expression is therefore important.

Matrix-valued domains

The function $-\log\det X$ is convex for positive-definite symmetric matrices X [1, Section 3.1]. The variable is a matrix, and the domain excludes singular matrices.

For a symmetric perturbation H, its second directional derivative is

$$\operatorname{tr}(X^{-1}HX^{-1}H) \geq 0.$$

H is a direction in the space of symmetric matrices and tr is trace. After a congruence transformation this becomes a squared Frobenius norm, explaining the nonnegative curvature.

The largest eigenvalue of a symmetric matrix is convex because

$$\lambda_{\max}(X) = \sup_{\|v\|_2=1} v^\top X v.$$

$\lambda_{\max}$ denotes the largest eigenvalue and v ranges over unit vectors. Each quadratic form is affine in the matrix entries of X, so their pointwise supremum is convex. This is a useful example of analyzing which quantity is the variable.

Code: Hessian checks for a quadratic and a counterexample

This code checks constant Hessians, so the eigenvalue test applies at every input. It also verifies one supporting-plane inequality numerically.

```
import numpy as np

Q = np.array([[3.0, 1.0], [1.0, 2.0]])
bad = np.array([[0.0, 1.0], [1.0, 0.0]])
assert np.linalg.eigvalsh(Q).min() > 0
assert np.linalg.eigvalsh(bad).min() < 0
x, y = np.array([0.4, -0.6]), np.array([-1.0, 0.8])
f = lambda z: 0.5 * z @ Q @ z
```

```
support = f(x) + (Q @ x) @ (y - x)
gap = f(y) - support
assert np.isclose(gap, 0.5 * (y - x) @ Q @ (y - x))
print("quadratic Hessian eigenvalues:", np.linalg.eigvalsh(Q))
print("product Hessian eigenvalues:", np.linalg.eigvalsh(bad))
print("supporting-plane gap:", gap)
```

Sums, substitutions and pointwise maxima

Plain version. Build a complicated objective from known functions through operations that preserve convexity. The preservation rule is the proof.

A nonnegative weighted sum of convex functions is convex. Negative coefficients generally reverse or destroy the property. An integral or expectation of convex functions is convex when the result is well-defined and the distribution is fixed independently of the decision variable.

Affine substitution preserves convexity:

$$g(x) = f(Ax + b).$$

f is convex, A is a fixed matrix and b a fixed vector. The domain of g contains points whose transformed values lie in the domain of f. This rule establishes convexity of squared residuals, norm residuals and many regularized linear estimators.

If f_i are convex, then $g(x) = \max_i f_i(x)$ is convex on their common domain. An epigraph variable t expresses this as $f_i(x) \leq t$ for every i. This models the worst residual, worst scenario cost or maximum coordinate without needing a smooth objective.

An infimum over a jointly convex auxiliary variable also preserves convexity under the appropriate formulation:

$$g(x) = \inf_y F(x, y).$$

F is jointly convex in (x, y) and y is the eliminated variable. Joint convexity matters; convexity in each block separately is insufficient. This rule connects partial minimization with the projection of an epigraph.

What does not automatically preserve convexity

Products of convex functions are not generally convex. Dividing one convex function by another is not generally convex. Taking a minimum of convex functions can create multiple valleys.

For example, x^2y^2 is not jointly convex merely because it is a product of two nonnegative convex expressions. A reciprocal or exponential outside an expression needs a composition argument with the right monotonicity. Mathematical plausibility is not a substitute for checking a rule.

Multiplying a fixed parameter by a variable is affine. Multiplying two decision variables is bilinear and generally nonconvex. Many modeling errors arise from accidentally treating a learned or unknown quantity as if it were fixed.

Composition with monotonicity

Plain version. An outer convex function can preserve convexity when it reacts to each inner argument in the right direction.

Suppose

$$F(x) = h(g_1(x), \dots, g_k(x)).$$

h is an outer function and g_i are scalar inner functions. A standard sufficient rule is: h is convex, and for each argument it is nondecreasing when g_i is convex, nonincreasing when g_i is concave, or the inner function is affine. Domain conditions and the appropriate extended-value interpretation are included in the rule [1, Section 3.2].

For instance, exponential is convex and increasing, so exponentiating a convex inner expression preserves convexity. Negative logarithm is convex and decreasing, so applying it to a positive concave inner expression gives a convex function.

The monotonicity qualifier cannot be omitted. Squaring a convex expression is not always convex because square is decreasing on negative inputs and increasing on positive ones. Squaring a nonnegative convex expression is a valid composition, while arbitrary signs require another argument.

These are sufficient recognition rules, not a complete decision procedure for every algebraic expression. An expression may be convex even though its current syntax fails the rules. Rewriting $\sqrt{1 + x^2}$ as a Euclidean norm exposes convexity directly.

A composition rule seen through derivatives

For scalar smooth functions, the chain rule shows where the monotonicity requirement enters. Set $F(x) = h(g(x))$. Then

$$F''(x) = h''(g(x))[g'(x)]^2 + h'(g(x))g''(x).$$

Here primes denote scalar derivatives, g is the inner function and h the outer function. If h is convex and nondecreasing and g is convex, both terms are nonnegative. If h is convex and nonincreasing while g is concave, the second term is again nonnegative. Composition rules also handle nonsmooth functions, but this calculation exposes the signs [1].

Take the convex inner expression $g(x) = x^2 - 1$ and the convex outer function $h(t) = t^2$. Their composition is $(x^2 - 1)^2$, whose second derivative is $12x^2 - 4$. It is negative at zero, so the composition is nonconvex. The inner output is negative near zero, where the square function is decreasing; the required sign agreement fails.

When the domain is convex and a convex inner expression is nonnegative throughout it, squaring is valid because the outer function is nondecreasing there. This is why squared Euclidean residual norms are convex but "square any convex loss expression" is not a valid general rule.

Log-sum-exp and stable evaluation

Plain version. Log-sum-exp is a smooth upper approximation to the largest score. It also provides the normalization behind probability-like weights.

For $x \in \mathbb{R}^n$,

$$f(x) = \log\left(\sum_{i=1}^{n} e^{x_i}\right), \qquad p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

x_i are scalar coordinates; p_i are normalized positive weights summing to one. Its gradient is p and its Hessian is

$$\nabla^2 f(x) = \operatorname{diag}(p) - pp^\top.$$

$\operatorname{diag}(p)$ places the weights on a diagonal. For any direction v, the quadratic form is the weighted variance of v_i, which is nonnegative. This proves convexity.

Adding the same constant to every coordinate shifts the function by that constant. The all-ones direction therefore has zero Hessian curvature; the function is not strongly convex on the entire space.

For temperature $\tau > 0$, define $f_\tau(x) = \tau \log \sum_i e^{x_i/\tau}$. Then

$$\max_i x_i \le f_\tau(x) \le \max_i x_i + \tau \log n.$$

τ controls smoothness and the approximation error. A small temperature approaches the maximum but can make numerical optimization less well-conditioned. Stable evaluation subtracts the largest scaled coordinate before exponentiating.

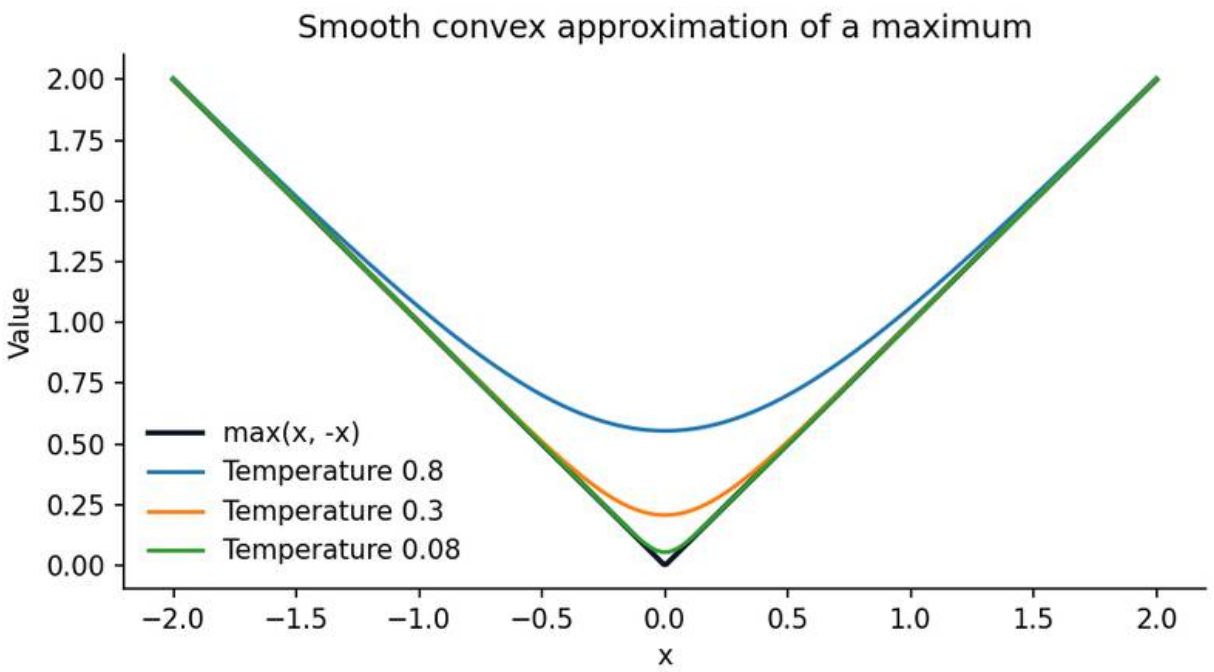


Figure 2 — Original analytical illustration. Temperature-scaled log-sum-exp lies above the maximum of the scores; smaller temperature gives a closer but sharper approximation.

Code: stable log-sum-exp and a Jensen check

This example avoids overflow at large inputs, verifies the Hessian and illustrates one finite Jensen comparison.

```
import numpy as np
from scipy.special import logsumexp

x = np.array([1000.0, 999.0, 997.0])
value = logsumexp(x)
p = np.exp(x - value)
H = np.diag(p) - np.outer(p, p)
assert np.linalg.eigvalsh(H).min() >= -1e-12
assert np.isclose(p.sum(), 1)
u, v = np.array([1., -1., 0.]), np.array([-2., 2., 1.])
theta = 0.3
left = logsumexp(theta * u + (1 - theta) * v)
right = theta * logsumexp(u) + (1 - theta) * logsumexp(v)
assert left <= right + 1e-12
print("stable value:", value, "weights:", p)
print("Jensen gap:", right - left)
```

Perspectives: normalize a variable without losing convexity

Plain version. A perspective combines a cost at a normalized input with the amount of scale used. This creates useful convex expressions that look like ratios.

For convex f, its perspective is

$$g(x,t) = tf(x/t), \qquad t > 0, \quad x/t \in \operatorname{dom} f.$$

x is a vector, t a positive scalar scale and g is jointly convex in both variables. The positive-scale domain is essential.

Taking $f(u) = \|u\|_2^2$ gives $g(x,t) = \|x\|_2^2/t$. This quadratic-over-linear function is jointly convex, even though arbitrary ratios are not.

The rule can be proved by mixing two pairs (x_1,t_1) and (x_2,t_2), then recognizing that their normalized input is a convex combination with weights proportional to θt_1 and $(1-\theta)t_2$. Apply convexity of f and multiply by the combined positive scale.

Perspective formulations arise when cost depends on a normalized rate, allocated duration or positive mass. They can also reveal exact conic representations. Keep physical meaning and domain constraints visible: permitting zero or negative scale without a careful closed extension changes the function.

Perspective proof: the weights change after normalization

Ordinary averaging does not commute with division by a variable scale. The perspective remains convex because normalization produces a different, still valid pair of weights.

Take two domain points $(x_1,t_1),(x_2,t_2)$ with $t_1,t_2 > 0$ and mixing weight $\theta \in [0,1]$. Define

$$\bar{x} = \theta x_1 + (1-\theta)x_2, \qquad \bar{t} = \theta t_1 + (1-\theta)t_2,$$
$$\beta = \frac{\theta t_1}{\bar{t}}, \qquad \frac{\bar{x}}{\bar{t}} = \beta\frac{x_1}{t_1} + (1-\beta)\frac{x_2}{t_2}.$$

Here $\bar{x},\bar{t}$ are the mixed numerator and scale; $\beta \in [0,1]$ is the normalized weight. Apply convexity of f to the last line and multiply by $\bar{t} > 0$. The coefficients become $\bar{t}\beta = \theta t_1$ and $\bar{t}(1-\beta) = (1-\theta)t_2$, giving exactly the convex chord bound for $g(x,t) = tf(x/t)$ [1].

For the quadratic-over-linear example, take $(x_1,t_1) = (2,1)$ and $(x_2,t_2) = (2,3)$ with $\theta = 1/2$. Their costs are 4 and $4/3$. The mixed point $(2,2)$ costs 2, below their average $8/3$. Its normalized input is 1, obtained with weight $\beta = 1/4$ on the first normalized input 2 and $3/4$ on the second $2/3$. The scale-dependent weights are the key step; simply averaging the normalized inputs would give a different number.

Conjugates and linear lower bounds

Plain version. A conjugate asks how much an arbitrary linear score can exceed a function. The answer describes the tightest intercept for a supporting line with that slope.

The convex conjugate is

$$f^*(y) = \sup_{x \in \operatorname{dom} f} \{y^\top x - f(x)\}.$$

y is a candidate slope vector, x is the original variable and $f^*(y)$ is the supremum gap. A finite conjugate value yields the global lower bound

$$f(x) \geq y^\top x - f^*(y).$$

This follows immediately from the supremum definition and is the Fenchel inequality [1, Section 3.3]. Conjugates are useful for deriving dual problems and splitting algorithms.

For $f(x) = \frac{1}{2} x^\top Q x$ with positive-definite Q, maximizing the gap gives $Qx = y$. Therefore

$$f^*(y) = \frac{1}{2} y^\top Q^{-1} y.$$

Q is the quadratic curvature matrix. In numerical code solve $Qx = y$ rather than explicitly inverting Q.

The conjugate of an indicator of a set is its support function. An extended-value indicator is zero on the allowed set and positive infinity elsewhere. This makes a constraint part of an objective without using a finite penalty.

For proper closed convex functions, conjugating twice recovers the original function. These regularity conditions matter: without them, the double conjugate describes the appropriate closed convex envelope rather than blindly reproducing every original formula.

Code: a conjugate certificate

Equality holds in the quadratic Fenchel bound at the matching slope $y = Qx$. The code checks both equality and a nonnegative gap elsewhere.

```
import numpy as np

Q = np.array([[4., 1.], [1., 2.]])
x = np.array([0.5, -0.8])
y = np.array([1.2, 0.3])
f = lambda z: 0.5 * z @ Q @ z
conjugate = lambda slope: 0.5 * slope @ np.linalg.solve(Q, slope)
gap = f(x) + conjugate(y) - x @ y
matching = Q @ x
assert gap >= -1e-12
assert np.isclose(f(x) + conjugate(matching), x @ matching)
```

```
print("Fenchel gap:", gap)
print("matching slope:", matching)
```

A nonsmooth conjugate with a visible domain

The conjugate can turn a nonsmooth penalty into a simple restriction on slopes. Let $f(x) = \lambda\|x\|_1$ with fixed $\lambda > 0$. Separating the coordinates in its definition gives

$$f^*(y) = \sum_i \sup_{x_i \in \mathbb{R}} (y_i x_i - \lambda|x_i|) = \begin{cases} 0, & \|y\|_\infty \le \lambda, \\ +\infty, & \|y\|_\infty > \lambda. \end{cases}$$

Here x_i, y_i are coordinates of the original variable and slope; $\|y\|_\infty = \max_i |y_i|$. When $|y_i| \le \lambda$, each coordinate expression is at most zero, attained at $x_i = 0$. When one slope exceeds that threshold in magnitude, choosing x_i with its sign and increasing magnitude makes the expression unbounded.

Thus the domain of the conjugate is an infinity-norm ball. The corresponding lower bound $y^\top x \le \lambda\|x\|_1$ is tight when nonzero coordinates have matching slopes $y_i = \lambda \operatorname{sign}(x_i)$. This links norm geometry, subgradients and dual feasibility in one calculation [1, Section 3.3]. OPT 08 uses the same threshold structure to derive a proximal update.

Quasiconvexity and logarithmic structure

Plain version. Some functions are not convex, but every set of inputs below a chosen score is convex. Other functions become convex or concave after taking a logarithm. These are useful structures with different guarantees.

A function is quasiconvex if all sublevel sets are convex:

$$C_\alpha = \{x : f(x) \le \alpha\}.$$

α is any real threshold and C_α is its sublevel set. Equivalently, values at mixed inputs do not exceed the larger endpoint value. This is weaker than the convex chord inequality.

Every convex function is quasiconvex. A monotone scalar function on an interval is quasiconvex even when its curvature is not consistently upward. Under appropriate formulations, quasiconvex minimization can use bisection on a threshold and solve a convex feasibility problem at each step [1, Sections 3.4 and 4.2].

For positive f, log-convexity means $\log f$ is convex; log-concavity means $\log f$ is concave. Positivity and the domain are part of these definitions. A density with concave log density has log-concave structure, but that does not make every parameter-estimation problem involving it convex in every parameterization.

Some geometric programs become convex after substituting logarithms of positive variables. This is an exact variable transformation in a specified problem class, not permission to take logarithms of arbitrary objectives and constraints. The next chapter derives the relevant form.

Convexity of a formula versus convexity of an implementation

A mathematically convex expression can be evaluated poorly. Overflow in an exponential, rounding near a matrix boundary or an accidentally broadcast array can invalidate the computed quantity while leaving the intended function convex.

Keep a symbolic curvature argument and a numerical evaluation check as separate evidence. A Hessian sampled at a few points may reveal a counterexample, but cannot prove nonnegative curvature everywhere. A stable implementation likewise does not prove the intended expression is convex.

When parameters are fixed during a solve, the convexity claim concerns decision variables at those fixed parameter values. A coefficient changing sign can invalidate a composition rule. Record which parameter ranges preserve the claim, rather than calling an entire parameterized family convex without qualification.

The same distinction applies to a learned function. Convexity in an input used later as a decision variable does not establish convexity in the weights being trained. Curvature rules describe one optimization problem at a time.

Modern view

Plain version. Convexity-preserving design is still useful when functions are learned from data. A learned function can be constrained to remain convex in its decision input without being convex in its training parameters.

Disciplined convex programming [2] formalizes composition rules so a model carries a recognizable convexity argument. The taxonomy separates affine, convex and concave expressions, tracks monotonicity and respects domains. Failing a syntactic rule can mean a reformulation is needed; it does not automatically disprove mathematical convexity.

Schaller, Bemporad and Boyd's 2025 manuscript on learning parametric convex functions [3] studies fitting functions that remain convex in the variable for valid parameter values and are compatible with disciplined programming. It supports a specific modern connection: learning a component of an optimization model while preserving decision-variable convexity. It is an original research manuscript, not a review or a guarantee about unrestricted neural losses.

Pokutta's 2024 survey [4] organizes constrained smooth methods around geometric access and uses convexity, smoothness and strong convexity to analyze updates. Its practical lesson is to keep curvature assumptions separate: global convexity, uniform lower curvature and an upper smoothness bound play different roles.

Open problems include choosing useful structured function classes without losing expressiveness, certifying domains under noisy learned parameters, handling nondifferentiable solution maps, and evaluating generalization of learned models. An optimization certificate proves a property of the fitted objective; it does not prove that the objective matches future observations.

Key takeaways

- Convexity includes a convex domain and a chord inequality.
- Tangents and subgradients give global lower bounds.
- Positive-semidefinite Hessians establish smooth convexity in every direction.
- Strict convexity, strong convexity and smoothness are different assumptions.
- Composition rules need monotonicity, signs and domains.
- Perspectives and conjugates expose useful structure hidden in ratios and constraints.
- Quasiconvex and log-convex formulations require their own reasoning.

Exercises

1. Prove the first-order supporting-plane inequality from the chord definition for differentiable functions.

   Hint

   Rearrange the chord inequality, divide by the mixing weight and take its limit to zero.

2. Show why nonnegative Hessian diagonal entries do not prove convexity, using a two-variable example.

   Hint

   Use $x_1 x_2$ and test diagonal directions.

3. Reformulate $\sqrt{1 + x^2}$ to make its convexity visible through a norm.

   Hint

   Write the norm of $(1, x)$.

4. Derive the perspective of squared Euclidean norm and identify its domain.

   Hint

   Substitute x/t and multiply by positive t.

5. Derive the conjugate of $\frac{1}{2}\|x\|_2^2$ and find when Fenchel's inequality is equality.

   Hint

   Maximize the quadratic gap by differentiation.

6. Give a quasiconvex function on an interval that is not convex, and verify its sublevel sets.

Hint

A monotone concave function such as square root on positive inputs has interval sublevel sets.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 3; Section 4.2 for quasiconvex feasibility.** Official book page, free official PDF.
2. Michael Grant, Stephen Boyd and Yinyu Ye, "Disciplined Convex Programming," in *Global Optimization: From Theory to Implementation*, Springer, 2006. Author's publication page.
3. Michael Schaller, Alberto Bemporad and Stephen Boyd, "Learning Parametric Convex Functions," manuscript, June 2025. Verified manuscript page.
4. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.

OPT 03 • Linear, Quadratic, Conic and Geometric Programs

Recognize standard convex problem families, derive exact epigraph and cone reformulations, transform geometric programs, and inspect scaling and multiobjective tradeoffs.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-03-convex-programs/

What you'll learn

- Identify linear, quadratic and convex quadratically constrained programs.
- Turn absolute values, maxima and norm constraints into exact formulations.
- Express second-order and semidefinite cone constraints with the right ordering.
- Transform a geometric program into a convex problem using positive variables and logarithms.
- Separate exact reformulation, convex relaxation and approximation.
- Explain scaling, feasibility checks and multiobjective tradeoffs before selecting a solver.

The big picture

A model can be convex while its written form hides that fact. A sum of absolute residuals looks nonsmooth but becomes a linear program with auxiliary variables. A norm bound becomes a cone constraint. A positive-variable design can become convex after a logarithmic change of variables.

Boyd and Vandenberghe's Chapter 4 [1] organizes these problem families. Learning them serves two purposes: it supplies reusable modeling patterns and helps choose numerical methods that exploit structure. The class is determined by equations and domains, not by the label attached to an application.

> **Explain like I'm five**
>
> Different puzzles use different boxes of pieces. Sometimes a puzzle looks unfamiliar because its pieces are turned sideways. Rewriting the rules can reveal that it belongs to a familiar box. But adding new pieces changes the puzzle, so distinguish a rewrite from a relaxation.

The examples in this chapter use synthetic small problems. They show exact mathematical relationships and numerical checks, not benchmark results. The code uses the installed numerical libraries without introducing a separate modeling-system dependency.

The convex standard form

Plain version. Convexity requires the objective, inequalities and equalities to fit together. It is not enough for the objective alone to be convex.

The standard form is

$$\begin{aligned} \text{minimize}_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & Ax = b. \end{aligned}$$

$x \in \mathbb{R}^n$ is the decision vector; $f_0, \dots, f_m$ are convex functions on their relevant common domain; A is a fixed matrix and b a fixed equality target. The feasible set is convex because convex sublevel sets and affine equalities are intersected.

A concave function can appear on the other side: $g(x) \ge a$ is convex-feasible when g is concave, because $a - g(x) \le 0$ is a convex inequality. Equality to a nonlinear convex function is generally not convex. For example, $\|x\|_2^2 = 1$ describes a sphere surface, while $\|x\|_2^2 \le 1$ describes a convex ball.

Writing an equality as two inequalities does not fix this issue. One of the two directions reverses the curvature. If the physical problem truly requires the surface, replacing it by a ball is a relaxation and may change the answer.

Exact reformulation versus changing the problem

An exact reformulation preserves the achievable original decisions and objective after auxiliary variables are eliminated or a one-to-one transformation is reversed. A relaxation enlarges the allowed set. An approximation changes functions or constraints to nearby ones.

For an epigraph variable t, minimizing t with $f(x) \le t$ is exact because the optimizer can choose the smallest permitted t. Merely imposing the inequality inside another model may not force equality if the remaining objective does not prefer smaller t.

When a reformulation adds variables, prove both directions: every original feasible point can be extended to a reformulated point, and every reformulated point maps back to an original feasible decision with the correct objective relation. This small proof prevents many modeling errors.

Linear programming

Plain version. A linear program uses straight-line costs and rules. Its geometry is a polyhedron, which can have corners, faces and directions extending to infinity.

An inequality-and-equality linear program is

$$\begin{aligned} \text{minimize}_x \quad & c^\top x \\ \text{subject to} \quad & Gx \le h, \qquad Ax = b. \end{aligned}$$

c is the fixed objective coefficient vector; G, h define inequality bounds; A, b define equalities. Bounds on individual variables can be included separately or as rows of G.

A linear objective is both convex and concave. The restrictions need not force a unique solution. When an objective is parallel to a feasible face, every point on that face can be optimal.

A standard nonnegative form writes $Ax = b$, $x \ge 0$. Introduce nonnegative slack variables to turn inequalities into equalities, and represent a free scalar as the difference of two nonnegative scalars if this form is needed.

These transformations are exact but can increase size and change numerical conditioning. A solver interface may accept bounds and inequalities directly, so there is no need to expand everything manually for its own sake.

Absolute residuals become linear constraints

To fit measurements b using design matrix A, consider

$$\min_x \|Ax - b\|_1.$$

A is a fixed design matrix, x its coefficient vector and b the measurement vector. Introduce one variable t_i for each residual:

$$-t \le Ax - b \le t, \qquad \min_{x,t} \mathbf{1}^\top t.$$

t bounds residual magnitudes and $\mathbf{1}$ is the all-ones vector. Each feasible t_i is at least the absolute residual, and minimizing their sum tightens the bounds. Nonnegativity of t follows from the paired inequalities but can be specified explicitly.

Minimax fitting instead introduces one scalar t and minimizes it subject to $-t\mathbf{1} \le Ax - b \le t\mathbf{1}$. This controls the largest absolute residual rather than their sum. The modeling change expresses a different application preference.

Code: solve a small allocation LP

The SciPy interface is documented in [2]. This example checks success and feasibility independently, rather than printing the optimizer without checking its status.

```
import numpy as np
from scipy.optimize import linprog

c = np.array([-3.0, -2.0])
G = np.array([[1.0, 1.0], [2.0, 1.0]])
h = np.array([4.0, 5.0])
result = linprog(c, A_ub=G, b_ub=h, bounds=[(0, None), (0, None)])
```

```
assert result.success, result.message
x = result.x
assert np.all(G @ x <= h + 1e-8)
assert np.all(x >= -1e-8)
assert np.allclose(x, [1, 3], atol=1e-7)
print("allocation:", x, "objective:", c @ x)
print("slacks:", h - G @ x)
```

The negative objective coefficients encode maximizing a positive benefit through minimization. The problem is bounded because the capacity constraints restrict nonnegative choices. Removing a relevant bound can make the model unbounded; that is a modeling outcome, not a solver defect.

Quadratic programming and convex QCQPs

Plain version. A quadratic program replaces the linear cost with a curved bowl while keeping linear rules. Quadratic inequality constraints are also possible if their bowls face upward.

A convex quadratic program, QP, has objective

$$f_0(x) = \frac{1}{2}x^\top Qx + c^\top x + d, \qquad Q \succeq 0,$$

with affine constraints. Q is a symmetric positive-semidefinite curvature matrix, c is a linear coefficient vector and d a constant that does not change the optimizer.

Least squares is a QP: expanding $\frac{1}{2}\|Ax - b\|_2^2$ gives $Q = A^\top A$ and $c = -A^\top b$. A quadratic regularizer adds positive diagonal curvature. If the design matrix is rank deficient, the unregularized minimizer may not be unique.

A convex quadratically constrained quadratic program, QCQP, also has inequalities

$$\frac{1}{2}x^\top Q_i x + a_i^\top x + r_i \leq 0, \qquad Q_i \succeq 0.$$

Q_i is each constraint's curvature matrix, a_i a linear coefficient vector and r_i its offset. Arbitrary quadratic constraints do not meet this condition. An indefinite quadratic constraint can be nonconvex even if the objective is strongly convex.

Numerical structure matters

An unconstrained positive-definite QP solves $Qx = -c$. A constrained QP may have an optimizer at a boundary, so solving the unconstrained system and clipping afterward is generally wrong.

Clipping is correct for projection onto a box in ordinary Euclidean distance. A general quadratic couples coordinates; adjusting one coordinate changes the best value of others. The correct constrained solution must respect that coupling.

The size and sparsity of Q matter. Forming $A^\top A$ explicitly can destroy useful sparsity or worsen conditioning. A model can be written as a quadratic while its implementation retains residual operators. The algorithms chapters explain factorization and matrix-vector-product choices.

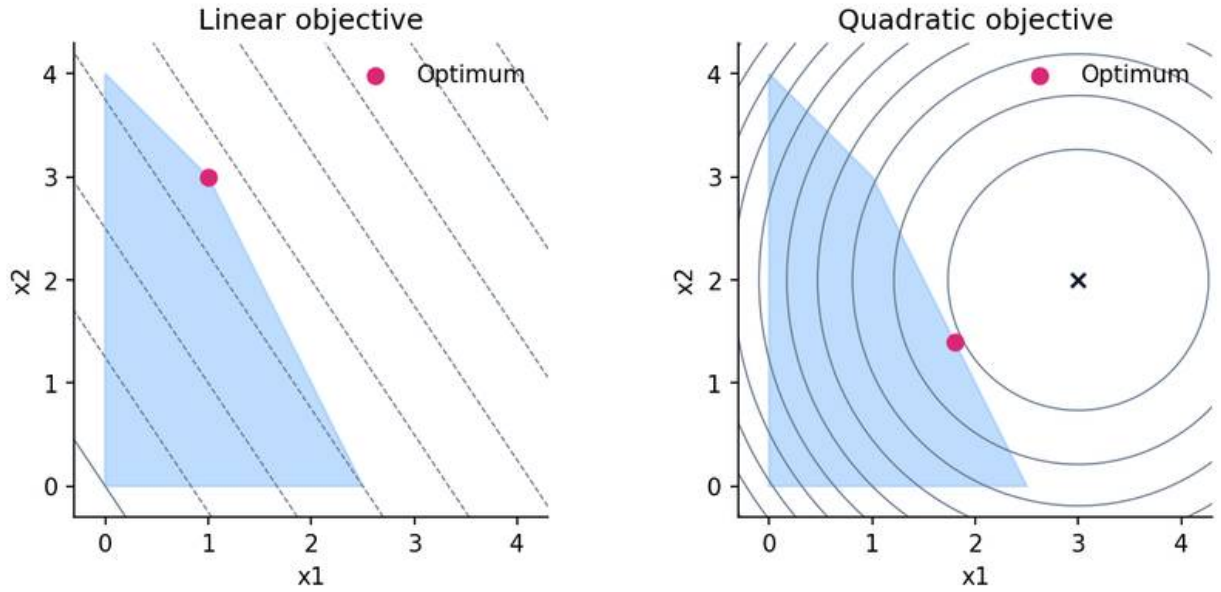


Figure 1 — Original analytical comparison. A linear cost favors a polygon corner in this example; a quadratic cost selects the closest point to its target. Both use the same convex feasible set.

Worked counterexample: why clipping a QP is wrong

Coupled curvature can make changing one coordinate shift the best value of another. Consider the convex quadratic

$$f(x) = \frac{1}{2}(x-a)^\top Q(x-a), \qquad Q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad a = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad x_1 \le 1.$$

Here $x \in \mathbb{R}^2$ is the variable, a the unconstrained target and Q the positive-definite curvature matrix. Simply clipping a produces $(1, 0)$ with objective one. With $x_1 = 1$, however, stationarity in the unrestricted second coordinate gives $(x_1 - 2) + 2x_2 = 0$, hence $x_2 = 1/2$.

The candidate $(1, 1/2)$ has objective $3/4$. Its gradient is $(-3/2, 0)$, balanced by multiplier $3/2$ on $x_1 - 1 \le 0$. Feasibility, multiplier sign, stationarity and complementary slackness therefore verify the optimum using OPT 04's conditions. Coordinate clipping ignores the off-diagonal entries of Q; Euclidean box projection works because its quadratic separates by coordinate.

Second-order cone programming

Plain version. A second-order cone expresses a norm bound with a scalar on the right. It turns many curved constraints into a standard geometric form.

A common SOCP inequality is

$$\|A_i x + b_i\|_2 \leq c_i^\top x + d_i.$$

A_i, b_i define the vector inside the norm; c_i, d_i define the affine scalar bound; and x is the decision vector. The inequality automatically requires its right-hand side to be nonnegative.

Do not square both sides without retaining that sign condition. The squared inequality can allow a negative right-hand side and introduce extra points. The cone formulation carries the sign correctly.

A rotated second-order cone can encode quadratic-over-linear expressions. In a common convention, it requires $u, v \geq 0$ and $2uv \geq \|z\|_2^2$. The factor of two is a convention and must match the implementation.

Using $u = t$, $v = 1/2$, $z = Ax - b$ expresses $\|Ax - b\|_2^2 \leq t$. Different scaling choices give the same mathematical set but can change numerical behavior.

Robust linear constraints

Suppose a constraint coefficient is uncertain: $a = \bar{a} + Eu$ with $\|u\|_2 \leq 1$. Requiring $a^\top x \leq b$ for every allowed u gives

$$\bar{a}^\top x + \|E^\top x\|_2 \leq b.$$

$\bar{a}$ is the nominal coefficient vector, E maps unit-ball uncertainty and b is a fixed bound. The norm is the maximum of $u^\top E^\top x$ over the unit ball.

This is an exact robust counterpart for the specified uncertainty set. It does not guarantee protection against all real-world uncertainty; the set itself must be justified. A larger set can be more conservative, while an incorrect set can miss important disturbances.

The geometry of uncertainty determines the reformulation. Box uncertainty gives a one-norm term; ellipsoidal uncertainty gives a Euclidean norm after a transformation. These connections follow from support functions developed in the previous chapter [1].

Derive the robust counterpart rather than guessing a penalty

The robust norm term is the exact worst allowed disturbance. Expand the uncertain linear constraint:

$$\sup_{\|u\|_2 \leq 1} (\bar{a} + Eu)^\top x = \bar{a}^\top x + \sup_{\|u\|_2 \leq 1} u^\top E^\top x = \bar{a}^\top x + \|E^\top x\|_2.$$

Here x is the decision, $\bar{a}$ the nominal coefficient, E the uncertainty map and u an allowed disturbance. The last equality follows from the inner-product norm bound, with equality at $u = E^\top x / \|E^\top x\|_2$ when the denominator is nonzero. When it is zero, every disturbance contributes zero.

For the synthetic constraint $(1+0.2u_1)x_1 + (2+0.1u_2)x_2 \le 4$ with $\|u\|_2 \le 1$, the exact robust rule is $x_1 + 2x_2 + \sqrt{0.04x_1^2 + 0.01x_2^2} \le 4$. At $x = (1,1)$ its worst left side is $3 + \sqrt{0.05}$, so it is robust feasible. The maximizing disturbance is proportional to $(0.2, 0.1)$. No tuning weight was invented: the added term follows from the stated uncertainty geometry [1].

Semidefinite programming

Plain version. A semidefinite program constrains a whole matrix to have nonnegative curvature in every vector direction. Checking entries individually is not enough.

A linear matrix inequality is

$$F(x) = F_0 + \sum_{i=1}^{n} x_i F_i \succeq 0.$$

$F_0, \dots, F_n$ are fixed symmetric matrices; x_i are scalar decision variables; and $F(x) \succeq 0$ requires positive semidefiniteness. A linear objective with affine equalities and such constraints defines an SDP.

If all matrices are diagonal, positive semidefiniteness reduces to nonnegative diagonal entries, giving linear inequalities. General matrices include relationships between coordinates that cannot be captured by entrywise nonnegativity.

For example, the symmetric matrix

$$X = \begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix}$$

is positive semidefinite exactly when $-1 \le r \le 1$. Here r is a scalar off-diagonal correlation. Negative r is allowed despite entrywise negativity, while an excessively large positive r makes the matrix indefinite.

Schur complements expose hidden forms

For positive-definite P, a block matrix condition satisfies

$$\begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \succeq 0 \quad \iff \quad t - q^\top P^{-1} q \ge 0.$$

P is a positive-definite symmetric block, q a vector and t a scalar. The right side is the Schur complement. The positive-definiteness assumption is important; singular blocks need an additional range condition or a generalized argument.

This identity converts some matrix inverse expressions into affine matrix constraints when the block entries are affine in decision variables. Whether the result is convex depends on that affine structure, not just on displaying a block matrix.

Prove the Schur condition by completing a square

A block matrix is positive semidefinite when its quadratic form is nonnegative for every vector. For $P \succ 0$, a vector v and scalar α give

$$\begin{aligned}\begin{bmatrix} v \\ \alpha \end{bmatrix}^\top \begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \begin{bmatrix} v \\ \alpha \end{bmatrix} &= v^\top P v + 2\alpha q^\top v + \alpha^2 t \\ &= (v + \alpha P^{-1} q)^\top P (v + \alpha P^{-1} q) + \alpha^2 (t - q^\top P^{-1} q).\end{aligned}$$

Here P is the positive-definite block, q a compatible vector, t a scalar block, and v, α arbitrary test coordinates. If $t \geq q^\top P^{-1} q$, both terms are nonnegative. Conversely, choose $\alpha = 1$ and $v = -P^{-1} q$; the first term vanishes, forcing that scalar inequality. This proves both directions of the equivalence [1].

It is also a modeling recipe. With fixed P and affine vector $q(x)$, the upper bound $q(x)^\top P^{-1} q(x) \leq t$ becomes a matrix inequality affine in (x, t). If P is a matrix decision variable, the same block representation can still be affine in its entries on the positive-definite domain. A nonlinear term can therefore have a convex epigraph even when its original inverse expression hides the structure.

The assumption $P \succ 0$ enables the square completion and inverse. A singular P cannot be handled by silently using the same formula; the vector's range condition becomes part of the equivalence.

Code: distinguish matrix and entrywise order

This code checks a family with known eigenvalues and demonstrates a valid Schur-complement boundary.

```
import numpy as np

for r in [-0.8, 0.8, 1.2]:
    X = np.array([[1., r], [r, 1.]])
    eigenvalues = np.linalg.eigvalsh(X)
    print("r:", r, "eigenvalues:", eigenvalues)
    assert (eigenvalues.min() >= -1e-12) == (abs(r) <= 1)
P = np.array([[2., .3], [.3, 1.]])
q = np.array([.4, -.2])
t = q @ np.linalg.solve(P, q)
block = np.block([[P, q[:, None]], [q[None, :], np.array([[t]])]])
assert np.linalg.eigvalsh(block).min() >= -1e-12
print("Schur boundary t:", t)
```

This is a spectral check, not a full SDP solver. The course uses small direct calculations to explain the model without implying that a general nonlinear optimizer supplies all the certificates of a dedicated conic method.

Geometric programming

Plain version. Some positive-variable models look nonconvex in their original coordinates but become convex when multiplication turns into addition under logarithms.

In geometric programming, a monomial has the specialized form

$$m(x) = c \prod_{i=1}^{n} x_i^{a_i}, \qquad c > 0, \quad x_i > 0.$$

c is a positive coefficient and a_i are arbitrary real exponents. This differs from the elementary algebra convention requiring nonnegative integer powers.

A posynomial is a sum of such monomials:

$$f(x) = \sum_{k=1}^{K} c_k \prod_{i} x_i^{a_{ik}}, \qquad c_k > 0.$$

K is the number of terms, c_k their positive coefficients and a_{ik} their exponents. Negative coefficients are excluded.

A geometric program minimizes a posynomial, constrains posynomials to be at most one, and constrains monomials to equal one, with positive decision variables [1, Section 4.5]. Arbitrary sums, signs or equalities do not fit this class.

Derive the logarithmic transformation

Set $y_i = \log x_i$, so $x_i = e^{y_i}$. A monomial becomes $\exp(a^\top y + \log c)$; its logarithm is affine. A posynomial's logarithm becomes

$$\log f(e^y) = \log \sum_{k=1}^{K} \exp(a_k^\top y + \log c_k).$$

a_k holds the exponent vector of term k. This is log-sum-exp of affine expressions, which is convex.

The logarithm is strictly increasing, so minimizing the original positive objective and minimizing its log produce the same optimizer after reversing the variable transformation. Posynomial inequalities become convex log-sum-exp inequalities, and monomial equalities become affine equalities.

The objective values are transformed, however: the log objective is not numerically the same as the original cost. Convert back when reporting the application quantity.

This is an exact change of variables on the positive domain. It is different from dropping negative terms or changing a non-monomial equality to an inequality. Those operations can create a different problem.

Code: a positive allocation in log coordinates

Minimize $1/x + 1/y$ subject to $x + y \le 4$ and $x, y > 0$. Symmetry and the convex original objective establish the solution $x = y = 2$; the transformed problem verifies the same answer numerically.

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import logsumexp

objective = lambda z: logsumexp(-z)
constraint = {"type": "ineq",
              "fun": lambda z: np.log(4.) - logsumexp(z)}
result = minimize(objective, np.log([1., 1.]),
                  constraints=[constraint], method="SLSQP",
                  options={"ftol": 1e-12, "maxiter": 100})
assert result.success, result.message
x = np.exp(result.x)
assert x.sum() <= 4 + 1e-7
assert np.allclose(x, [2., 2.], atol=1e-5)
print("positive variables:", x)
print("original objective:", np.sum(1 / x))
print("transformed objective:", result.fun)
```

The numerical interface is documented in [3]. This small demonstration checks against an analytical solution. It does not make this general-purpose method a replacement for specialized geometric-programming solvers on all scales.

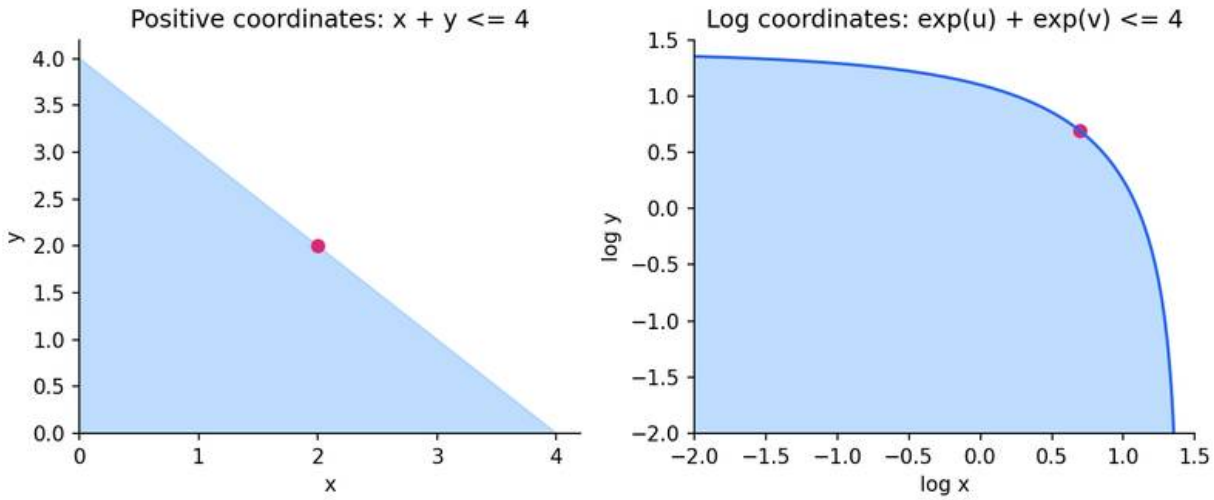


Figure 2 — Original analytical geometry for $x + y \le 4$ with positive variables. In log coordinates its constraint is log-sum-exp $\le \log 4$; axes in the two panels represent different variables.

Worked design: an asymmetric positive allocation

A logarithmic transformation preserves the optimum, even when symmetry cannot supply the answer. Minimize $4/x + 1/y$ under $x + y \le 6$ with $x, y > 0$. Since both terms decrease as their variables grow, the optimum uses the full budget.

Set $y = 6 - x$. Differentiating the remaining scalar objective gives

$$-\frac{4}{x^2} + \frac{1}{(6-x)^2} = 0 \quad \Longrightarrow \quad x = 2(6-x) \quad \Longrightarrow \quad (x^*, y^*) = (4, 2).$$

Here x, y are positive allocations and the star denotes an optimum. The second derivative is positive on $0 < x < 6$, so the stationary point is the unique minimum. Its original cost is $3/2$.

In logarithmic variables $s = \log x$, $t = \log y$, the objective becomes $\log(4e^{-s} + e^{-t})$ and the restriction becomes $\log(e^s + e^t) \le \log 6$. Both are log-sum-exp expressions. The recovered solution is $(s^*, t^*) = (\log 4, \log 2)$, with transformed cost $\log(3/2)$. Reporting $\log(3/2)$ as the original cost would mix two coordinate systems. This example gives an analytical check for the entire transformation [1, Section 4.5].

Quasiconvex problems through feasibility

Plain version. Some useful objectives are not convex, but asking whether a proposed score is achievable gives a convex feasibility problem. A sequence of threshold questions can then find the best score.

Suppose the sets $\{x : f_0(x) \le \alpha\}$ are convex and can be represented by convex constraints for each fixed threshold α. Maintain a lower and upper bound on the optimal score. Test the midpoint threshold: if its feasible set is nonempty, lower the upper bound; if not, raise the lower bound.

Here the threshold is fixed while solving each feasibility problem. Treating it as another free variable can introduce nonconvex products or ratios. The convexity of every threshold problem does not imply joint convexity in the threshold and original decision.

For example, a positive-denominator affine ratio $(a^\top x + b)/(c^\top x + d)$ has convex sublevel sets on a convex domain where $c^\top x + d > 0$. At a fixed threshold α, the score inequality becomes $a^\top x + b \le \alpha(c^\top x + d)$, an affine constraint in x. Positivity of the denominator is essential to multiplying without reversing the inequality.

Numerical feasibility must be handled carefully. An approximate solver may be unable to distinguish an empty set from an extremely narrow feasible region near the optimal threshold. A failed iteration or uncertain status is not automatically an infeasibility certificate. Use an appropriate feasibility procedure and report the threshold interval and tolerances.

This approach is covered by the quasiconvex discussion in [1]. It broadens the course beyond convex objectives while keeping the role of convexity precise: the subproblems are convex, and bisection supplies the outer scalar search.

A modeling audit before solving

Take one original application requirement at a time and translate it to the chosen variables. Check whether its domain is enforced, whether a sign was changed correctly, and whether the formulation preserves its units. Then trace how auxiliary variables are eliminated.

Consider a worst-case norm constraint expressed through a scalar upper bound. If the auxiliary scalar is absent from the objective, it may remain unnecessarily loose. That can still be correct if only an upper limit is required, but it cannot be interpreted as a measured norm without evaluating the norm itself.

Consider a lifted matrix representing outer products. Requiring positive semidefiniteness is convex, while requiring it to equal a vector outer product introduces a rank restriction. Dropping that restriction is a relaxation. The matrix solution may supply a useful bound while lacking a corresponding original vector.

Finally, inspect what happens at domain boundaries. A geometric-program variable cannot simply be set to zero because its logarithm would be undefined. A log-determinant matrix cannot become singular without leaving the finite objective domain. A reformulation that accepts those boundary points needs an explicitly justified extension.

These checks are part of mathematical validation. A numerical solver can correctly optimize an accidentally relaxed or incorrectly signed problem. Comparing against a small analytically solved case is useful because it tests the entire chain from problem statement to arrays and back to interpretation.

Multiobjective decisions and Pareto tradeoffs

Plain version. When two goals conflict, optimization cannot decide your preferences for you. It can describe the tradeoff and compute answers for specified preferences.

For objective vector $F(x) = (f_1(x), \dots, f_k(x))$, a feasible point is Pareto optimal if no other feasible point improves one component without worsening another. Here k is the number of goals.

A positive weighted sum

$$\min_x \sum_{i=1}^{k} w_i f_i(x), \qquad w_i > 0,$$

uses weights w_i encoding preferences and units. Any attained optimizer is Pareto optimal. A zero weight can ignore a component and yield only a weaker efficiency statement.

Weighted sums are not the only choice. One goal can be minimized while others are constrained below prescribed limits. This often matches applications more naturally: minimize reconstruction error subject to a memory budget, rather than inventing a universal conversion from bytes to error.

The scalarization gives a convex problem when the scalar objective and constraints follow convex rules. A tradeoff curve obtained by varying weights describes this model; it does not establish every achievable system behavior if some upstream components or discrete choices were excluded.

Modeling and numerical checks

Plain version. The written problem and the executed arrays must describe the same decision. Small implementation mistakes can silently change constraints.

Check variable bounds explicitly. Some numerical interfaces default to nonnegative variables, while others default to free variables. Leaving defaults unexamined can change an intended regression coefficient into a nonnegative one.

Check inequality signs. A mathematical convention $g(x) \leq 0$ and an interface convention $g(x) \geq 0$ can coexist in the same library ecosystem. Translate the condition deliberately.

Check dimensions and symmetry. A matrix constraint on an asymmetric array does not mean what the symmetric semidefinite notation means. Accidental broadcasting can replace a scalar constraint with a vector of different constraints.

Scale coordinates and objective terms to meaningful ranges, preserving the relationship to original variables. Rescaling can improve numerical conditioning but must be reversed for interpretation and residual reporting. A computed answer should be checked in the original formulation as well as the transformed one.

Finally, separate status from certificates. Success of a generic constrained optimizer can support a small example, especially when an analytical optimum is available. For a production convex model, inspect primal feasibility, dual feasibility and gap using methods appropriate to the problem class.

The written model should remain reviewable after solver conversion. Keep variable units, domain restrictions, transformations and any relaxation in the same report. If a transformed solution is returned, map it back to original coordinates before checking physical constraints and reporting the application objective.

This is especially useful for logarithmic coordinates, auxiliary epigraph variables and lifted matrix models. A feasible transformed point can be misunderstood when reported without its map, and an auxiliary objective can differ from the original expression if its defining restrictions were not enforced accurately.

Modern view

Plain version. Standard problem classes remain useful in current research. New implementations often gain speed by exploiting structure rather than changing the mathematical class.

Disciplined convex programming [4] organizes exact transformations through a graph of verified expressions. Its taxonomy tracks curvature and monotonicity and can turn high-level descriptions into standard forms. The key lesson is to make the convexity argument explicit before selecting a solver.

Applegate and colleagues' PDLP report [5], posted in 2025 and revised in 2026, focuses on large linear programs using a practical first-order primal–dual method. It adds preconditioning, restarting and feasibility polishing around the underlying optimization formulation. Its results concern the tested problems and requested accuracy; they do not imply one universal winner for all linear programs.

Schaller, Bemporad and Boyd [6] study fitting parametric convex functions compatible with disciplined programming. This brings learned objective or constraint components into a convex-modeling workflow, provided the decision-variable convexity and valid parameter domains are preserved.

Open issues include choosing representations that avoid unnecessary expansion, exploiting sparsity in cone constraints, certifying infeasibility reliably under scaling, and differentiating through models near active-set changes. A convex class gives a foundation for analysis; a complete system still needs suitable data, tolerances and engineering validation.

Key takeaways

- Convex standard form uses convex objectives and inequalities with affine equalities.
- Auxiliary variables can expose an exact model, but must be justified in both directions.
- Linear, quadratic, second-order-cone and semidefinite programs encode different geometry.
- Matrix order and coordinatewise order are distinct.
- Geometric programming requires positive variables and restricted monomial/posynomial structure.
- Multiobjective optimization needs explicit preferences or limits.
- Status, original-variable feasibility and optimality certificates must be checked separately.

Exercises

1. Convert minimax absolute-residual fitting into an LP and prove equivalence.

Hint

Use one scalar bound for every signed residual.

2. Explain why squaring a norm bound can add points if its affine right side may be negative.

Hint

The original norm is nonnegative.

3. Derive the allowed range of r in a two-by-two unit-diagonal positive-semidefinite matrix.

Hint

Eigenvalues are $1 + r$ and $1 - r$.

4. Transform minimizing $x^{-1} + y^{-1}$ under $x + y \leq 4$ into logarithmic coordinates and recover the original objective.

Hint

Substitute exponentials and take logarithms of positive expressions.

5. Construct a convex objective whose equality-level set is not convex.

Hint

Use a squared norm at a positive level.

6. Compare minimizing a weighted error–cost sum with minimizing error under a fixed cost limit. State how their application meanings differ.

Hint

One encodes exchange rates, the other a hard commitment.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 4, especially 4.3–4.7; Section 6.1 for norm fitting.** Official book, free PDF.
2. SciPy developers, "linprog," *SciPy API documentation*, accessed 2026. Verified interface.
3. SciPy developers, "minimize," *SciPy API documentation*, accessed 2026. Optimization interface.
4. Michael Grant, Stephen Boyd and Yinyu Ye, "Disciplined Convex Programming," in *Global Optimization: From Theory to Implementation*, Springer, 2006. Author's publication page.
5. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Verified paper record.
6. Michael Schaller, Alberto Bemporad and Stephen Boyd, "Learning Parametric Convex Functions," manuscript, June 2025. Verified manuscript.

OPT 04 • Duality, KKT Conditions and Certificates

Derive Lagrange lower bounds, distinguish weak and strong duality, verify optimality through KKT conditions, and interpret multipliers, sensitivity and infeasibility certificates.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-04-duality-kkt/

What you'll learn

- Derive a lower bound on a constrained minimization problem from a Lagrangian.
- Distinguish dual feasibility, weak duality, strong duality and attainment.
- State the KKT conditions and identify when they certify global optimality.
- Interpret complementary slackness and sensitivity with the correct multiplier signs.
- Construct a simple infeasibility certificate and extend dual order to cones.
- Report residuals and gaps without claiming more than numerical evidence supports.

The big picture

Finding a good feasible answer gives an upper bound on the best possible cost. How do we know that a much better answer is not hidden elsewhere? A dual problem produces lower bounds. When an upper and lower bound meet, they certify the optimum.

Boyd and Vandenberghe's Chapter 5 [1] connects this idea to geometry, supporting planes, multipliers and numerical algorithms. Duality is useful even when we already know how to solve the primal problem: it verifies an answer, describes the value of limited resources and reveals alternative computational structure.

> **Explain like I'm five**
>
> You found a recipe costing ten coins. A clever argument says every allowed recipe must cost at least ten coins. Together they prove your recipe is cheapest. Finding the recipe and proving the lower bound are different jobs.

The guarantee must respect signs and assumptions. A value computed from arbitrary multipliers may not be a lower bound. A small stationarity residual alone is not a complete certificate. Strong duality is not automatic for every convex formulation.

This chapter develops the derivations with explicit domains and uses small examples whose exact answers can be checked. The resulting reasoning extends to large problems, but the implementation details and tolerances remain part of the numerical assessment.

From constraints to the Lagrangian

Plain version. Attach a price to every inequality and equality. Then minimize the priced objective without explicitly enforcing those constraints.

Consider

$$\min_{x \in D} f_0(x) \quad \text{subject to} \quad f_i(x) \le 0, \quad h_j(x) = 0.$$

x is the decision vector, D is the common domain retained during minimization, f_0 is the objective, f_i are inequalities and h_j are equalities.

The Lagrangian is

$$L(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^{m} \lambda_i f_i(x) + \sum_{j=1}^{p} \nu_j h_j(x).$$

λ_i are inequality multipliers and ν_j equality multipliers. For the convention $f_i \le 0$, require $\lambda_i \ge 0$. Equality multipliers are unrestricted because equality residuals vanish at feasible points regardless of sign.

At a feasible point, the inequality contribution is nonpositive and the equality contribution zero. Thus $L(x, \lambda, \nu) \le f_0(x)$ for nonnegative λ.

Calling these multipliers "penalties" can be misleading. This Lagrangian is not simply a large positive cost added to violations during an unconstrained minimization. Its role is to create a lower-bound family, and the sign condition is selected for that purpose.

Define the dual function

$$g(\lambda, \nu) = \inf_{x \in D} L(x, \lambda, \nu).$$

g is the Lagrange dual function; its input is the multiplier pair. It takes the infimum over every point in D, not just feasible points.

For any feasible x and nonnegative λ,

$$g(\lambda, \nu) \le L(x, \lambda, \nu) \le f_0(x).$$

The first inequality follows from taking an infimum. The second follows from primal feasibility and multiplier signs. Therefore g is a lower bound on the primal optimal value.

If the infimum is negative infinity, the bound is valid but uninformative. Dual feasibility includes multipliers with an appropriate finite dual value when we want a useful certificate.

Why the dual problem is concave

Plain version. The lower bound changes with prices. We choose the prices giving the strongest bound, and that maximization has a concave objective.

For each fixed x, L is affine in the multipliers. The pointwise infimum of affine functions is concave. Therefore g is concave even if the original problem is not convex.

The dual problem is

$$\begin{aligned} \text{maximize}_{\lambda,\nu} \quad & g(\lambda, \nu) \\ \text{subject to} \quad & \lambda \geq 0. \end{aligned}$$

Let its optimal value be d^*. Weak duality states $d^* \leq p^*$, where p^* is the primal optimal value. This result does not require convexity, only the lower-bound construction with the correct signs.

A concave maximization is a convex optimization problem in the usual sense. However, evaluating the dual function can itself be difficult if it requires a hard nonconvex minimization. "The dual is concave" does not automatically provide a practical oracle.

The dual also need not have the same dimension as the primal. There is one variable for each explicitly dualized constraint. Retaining some restrictions in D rather than attaching multipliers changes the written dual representation, while the lower-bound logic remains consistent.

The concavity proof in three lines

The dual is concave because the bound at mixed prices is at least the same mixture of the individual bounds. Let $z = (\lambda, \nu)$ collect the multipliers, and take two allowed choices z_1, z_2 and $\theta \in [0, 1]$. Affinity of the Lagrangian in prices gives

$$\begin{aligned} g(\theta z_1 + (1-\theta)z_2) &= \inf_{x \in D} [\theta L(x, z_1) + (1-\theta) L(x, z_2)] \\ &\geq \theta \inf_{x \in D} L(x, z_1) + (1-\theta) \inf_{x \in D} L(x, z_2) \\ &= \theta g(z_1) + (1-\theta) g(z_2). \end{aligned}$$

Here D is the retained primal domain and g the dual function; consider finite dual values for this comparison. Each summand at a shared x is at least its own infimum, which explains the middle inequality. Nothing in that step assumes the primal objective is convex [1].

This distinction matters computationally. For a nonconvex primal, merely finding a local minimum of L does not evaluate its infimum. That local value can exceed $g(z)$ and cease to be a valid lower bound. Dual concavity concerns the exact dual function, not any approximate routine called a dual evaluator.

A complete one-dimensional derivation

Plain version. A small example shows how prices, the unconstrained minimizer and the bound fit together.

Minimize $\frac{1}{2}(x-2)^2$ subject to $x \le 1$. Here x is scalar. The constrained optimizer is $x^* = 1$, with cost $p^* = 1/2$.

Using multiplier $\lambda \ge 0$,

$$L(x,\lambda) = \frac{1}{2}(x-2)^2 + \lambda(x-1).$$

The unconstrained Lagrangian minimizer satisfies $x-2+\lambda = 0$, so $x_\lambda = 2-\lambda$. Substitution yields

$$g(\lambda) = \lambda - \frac{1}{2}\lambda^2.$$

x_λ depends on the selected multiplier and need not be primal feasible. The dual is a concave parabola, maximizing at $\lambda^* = 1$ with $g(\lambda^*) = 1/2$. Upper and lower bounds meet.

The optimum multiplier balances the objective's desire to increase x against the active upper-bound constraint. If λ is too small, the Lagrangian minimizer lies outside the allowed region. If it is too large, the bound becomes weaker despite the larger multiplier.

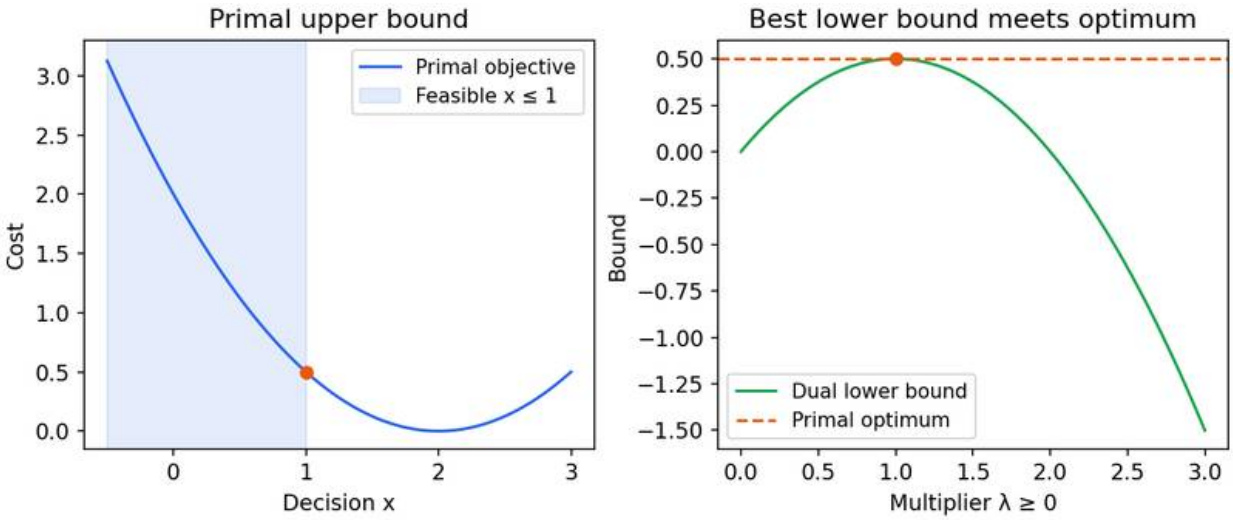


Figure 1 — Original analytical derivation for minimizing $(x-2)^2 / 2$ under $x \le 1$. The feasible primal optimizer and the best dual bound both have value 1/2.

Code: verify the gap and all KKT components

This standalone calculation checks primal feasibility, multiplier sign, stationarity, complementary slackness and equality of the two values.

```
import numpy as np

x, multiplier = 1.0, 1.0
objective = 0.5 * (x - 2)**2
dual_value = multiplier - 0.5 * multiplier**2
constraint = x - 1
stationarity = x - 2 + multiplier
complementarity = multiplier * constraint
assert constraint <= 0 and multiplier >= 0
assert abs(stationarity) < 1e-12
assert abs(complementarity) < 1e-12
assert np.isclose(objective, dual_value)
print("primal value:", objective, "dual value:", dual_value)
print("stationarity:", stationarity, "complementarity:", complementarity)
```

Strong duality and constraint qualifications

Plain version. Weak duality always gives the ordering. Strong duality says the best lower bound is exact, which needs additional conditions.

Strong duality means $d^* = p^*$. It is a statement about optimal values, not automatically about whether optimizers exist. A problem can have a value approached by a sequence without attaining it.

For a convex primal with affine equalities, a standard sufficient condition is Slater's condition [1, Section 5.2]:

$$\exists x \in \operatorname{relint} D : f_i(x) < 0 \text{ for all } i, \qquad Ax = b.$$

D is the common function domain, $\operatorname{relint} D$ its relative interior, and A, b define the affine equality constraints. This says there is a point strictly satisfying inequalities while remaining in the appropriate interior and satisfying equalities.

Under these assumptions, strong duality holds. With a finite optimal value, the dual optimum is also attained. The primal attainment question can still require separate existence reasoning.

There are refined conditions for affine inequalities, so requiring all of them strictly can be stronger than necessary. A failure to find a Slater point does not by itself disprove strong duality. It means this sufficient argument has not been established.

Convexity alone is not enough in every formulation. Domains can be degenerate and constraint qualifications can fail. Rather than saying "convex problems always have zero gap," state the condition being used or verify a valid matching certificate directly.

Domain restrictions are part of duality

If an objective contains a logarithm or inverse, its domain remains in the Lagrangian minimization. Treating it as a function on all real inputs silently changes the problem.

Alternatively, an extended-value function can encode a domain by assigning positive infinity outside it. This is mathematically convenient, but numerical code should not rely on accidental invalid floating-point values as a modeling convention.

In a conic formulation, domain and cone restrictions can be dualized or retained according to a consistent representation. The choice affects the dual expression and sign conventions. Derive the bound from the stated primal rather than copying a formula with incompatible signs.

Zero gap can coexist with no optimal multiplier

A failed constraint qualification does not force a positive duality gap, but it can remove multiplier attainment. Consider minimizing x subject to $x^2 \le 0$, with scalar $x \in \mathbb{R}$. The only feasible point is zero, so $p^* = 0$. There is no strictly feasible point.

The Lagrangian is $L(x, \lambda) = x + \lambda x^2$. Minimizing over x gives

$$g(\lambda) = \begin{cases} -1/(4\lambda), & \lambda > 0, \\ -\infty, & \lambda = 0. \end{cases}$$

Here $\lambda \ge 0$ prices the quadratic restriction and g is its dual bound. For positive λ, the minimizing point is $-1/(2\lambda)$. As λ increases without bound, $g(\lambda)$ approaches zero. Thus $d^* = p^* = 0$, yet no finite multiplier attains the dual optimum.

At the primal solution, stationarity would require $1 + 2\lambda x = 0$, which becomes $1 = 0$. No KKT multiplier exists for this representation. Replacing the equivalent restriction by the equality $x = 0$ gives a multiplier $\nu = -1$. The feasible decision is unchanged, but the regularity and multiplier description improve. This is why OPT 01's hidden equalities and OPT 07's initialization deserve attention [1].

KKT optimality conditions

Plain version. KKT combines allowed primal choices, allowed prices, balanced forces and the rule that slack constraints carry no positive price.

For differentiable convex f_0, f_i and affine equality $Ax = b$, the conditions are

$$\begin{aligned} & f_i(x) \le 0, \qquad Ax = b, \\ & \lambda_i \ge 0, \\ & \lambda_i f_i(x) = 0, \\ & \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu = 0. \end{aligned}$$

x is a primal candidate, λ are inequality multipliers, ν are equality multipliers, and the final row is stationarity of the Lagrangian. These are the Karush–Kuhn–Tucker conditions.

For this convex differentiable problem, satisfying all conditions is sufficient for primal and dual optimality. Nonnegative multipliers make the Lagrangian convex in x; stationarity makes x its global minimizer; feasibility and complementary slackness make the Lagrangian value equal the objective.

Under a suitable constraint qualification such as Slater's condition, optimal points also admit multipliers satisfying KKT. This supplies necessity as well as sufficiency. Keep the two logical directions distinct.

For nonconvex problems, a stationary Lagrangian can be a local maximum or saddle in x. KKT may be necessary under regularity but does not generally certify a global optimum. A concave quadratic at its interior stationary point is a simple counterexample.

Complementary slackness is one-way information

If $f_i(x) < 0$, complementary slackness forces $\lambda_i = 0$. If $\lambda_i > 0$, the constraint is active: $f_i(x) = 0$.

An active constraint can nevertheless have zero multiplier. Degeneracy or redundant constraints can make prices nonunique. Therefore "active" and "positive price" are not interchangeable labels.

At a smooth constrained optimum, the negative objective gradient lies in the combination of active constraint normals and equality normals with appropriate signs. This gives a geometric explanation of stationarity: feasible descent directions cannot reduce the objective.

Numerical near-activity is tolerance-dependent. A constraint with a tiny slack and tiny multiplier may be difficult to classify discretely. Report actual values where the distinction matters instead of forcing a confident active/inactive label.

Equality-constrained quadratics

Plain version. For a quadratic objective and linear equalities, the KKT equations form one linear system. This reveals the algebra used by later constrained algorithms.

For objective $\frac{1}{2}x^\top Qx + c^\top x$ under $Ax = b$,

$$\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \nu \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}.$$

Q is symmetric positive semidefinite, c is the objective coefficient vector, A the equality matrix, b its target, and ν the equality multiplier.

The block matrix is generally indefinite even though the optimization problem is convex. A solver requiring a positive-definite matrix cannot be used blindly on this full KKT system.

If Q is positive definite and A has independent rows, the system is nonsingular. More general cases require considering curvature on the equality nullspace and redundant constraints. A pseudoinverse may produce one solution but should not hide the underlying rank issue.

The equality multiplier balances the gradient in directions normal to the feasible affine set. Its sign is unconstrained. Removing a constraint changes both the feasible set and the meaning of its multiplier.

Deriving an LP dual carefully

Plain version. A dual formula is not universal notation. Its signs follow from the exact primal convention.

Take the LP $\min c^\top x$ subject to $Ax = b$, $x \geq 0$. Retain nonnegativity in the domain. The Lagrangian is $c^\top x + \nu^\top (Ax - b)$.

Taking the infimum over nonnegative x is finite exactly when $c + A^\top \nu \geq 0$. If any coefficient is negative, that coordinate can grow without bound and send the infimum to negative infinity.

Set $y = -\nu$. The dual becomes

$$\max_y b^\top y \quad \text{subject to} \quad A^\top y \leq c.$$

y is an unrestricted equality-price vector. For any feasible pair,

$$c^\top x - b^\top y = x^\top (c - A^\top y) \geq 0.$$

The gap is the inner product of nonnegative primal coordinates and nonnegative dual slack. This is weak duality in a particularly transparent form.

Using a primal maximization or reversed inequalities changes the dual signs. Rather than memorizing separate tables, deriving the Lagrangian and checking the bound can prevent mistakes.

A fitting dual built from the norm conjugate

Duality can turn a parameter penalty into a residual restriction. Consider sparse linear fitting with objective $\frac{1}{2}\|Ax - b\|_2^2 + \lambda \|x\|_1$, where A, b are fixed and $\lambda > 0$. Introduce residual variable $r = Ax - b$ and an unrestricted equality multiplier y:

$$L(x, r, y) = \frac{1}{2}\|r\|_2^2 + \lambda \|x\|_1 + y^\top (Ax - b - r).$$

Here x is the parameter vector and y has one coordinate per observation. Minimizing in r gives $r = y$ and value $-\|y\|_2^2/2$ for the residual terms. Minimizing in x is finite exactly when $\|A^\top y\|_\infty \leq \lambda$, by the norm-conjugate calculation in OPT 02. The dual is therefore

$$\text{maximize}_y \quad -\frac{1}{2}\|y\|_2^2 - b^\top y \qquad \text{subject to} \quad \|A^\top y\|_\infty \leq \lambda.$$

Here y is the dual variable; the norm restriction is dual feasibility. At an optimum it coincides with the fitted residual, while its correlations with the design columns must stay within the penalty threshold. This links the lower-bound construction to the geometry of sparse fitting [1, Chapters 5–6].

For synthetic $A = I$, $b = (2, -0.3, 0.1)$ and $\lambda = 0.4$, the primal solution is $x = (1.6, 0, 0)$ and dual candidate $y = x - b = (-0.4, 0.3, -0.1)$. Both are feasible for their respective problems and both objective values equal 0.77. Matching values certify the fit; observing zero coordinates alone would not.

Sensitivity and the value of a constraint

Plain version. A multiplier measures how the best cost responds to changing a constraint, under specified regularity. Its units depend on the constraint's units.

Perturb inequalities to $f_i(x) \le u_i$ and equalities to $Ax - b = v$. Let the best cost be $p^*(u, v)$. If strong duality holds at zero perturbation and optimal multipliers exist,

$$p^*(u, v) \ge p^*(0, 0) - \lambda^{*\top} u - \nu^{*\top} v.$$

u and v are right-side perturbations; λ^*, ν^* are optimal unperturbed multipliers. Positive u_i relaxes an inequality under the chosen convention.

The inequality gives a global affine lower bound on the perturbed value. If the value function is differentiable at the original problem, its derivatives are $-\lambda^*$ and $-\nu^*$.

Without differentiability, multipliers can describe supporting slopes rather than one unique derivative. A large multiplier signals sensitivity in that local modeling direction, not a universal physical importance score.

A scalar example with a changing bound

Minimize $\frac{1}{2}(x - 2)^2$ under $x \le b$. The best point is $\min(2, b)$ and the value is

$$p^*(b) = \frac{1}{2} \max(2 - b, 0)^2, \qquad \lambda^*(b) = \max(2 - b, 0).$$

b is the allowed upper bound. Where differentiable, $dp^*/db = -\lambda^*(b)$. Once the bound exceeds the unconstrained target, relaxing it further does not help and the multiplier is zero.

Scaling the constraint changes the multiplier reciprocally. Multiplying $x - b \le 0$ by a positive factor preserves feasible points but rescales its associated price. Compare sensitivity only after accounting for units and normalization.

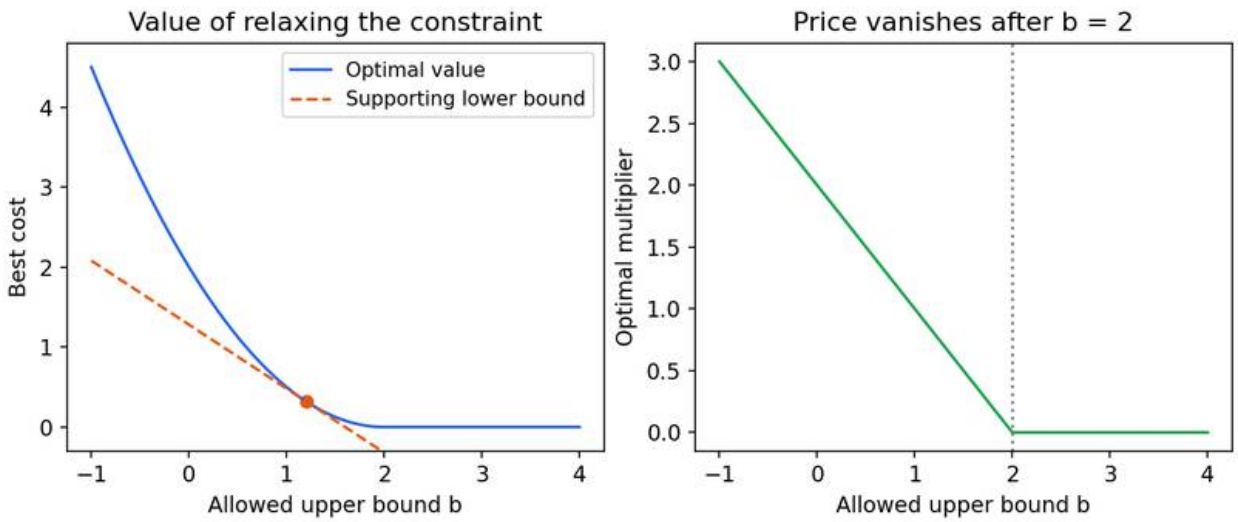


Figure 2 — Original analytical sensitivity example. Relaxing the bound reduces cost while it is binding; after the unconstrained target is allowed, the value and multiplier are zero.

Code: a finite-difference sensitivity check

```
import numpy as np

value = lambda b: 0.5 * max(2 - b, 0)**2
b, eps = 1.2, 1e-5
multiplier = max(2 - b, 0)
derivative = (value(b + eps) - value(b - eps)) / (2 * eps)
assert np.isclose(derivative, -multiplier, atol=1e-8)
test_bounds = np.linspace(-1, 4, 50)
lower_bound = value(b) - multiplier * (test_bounds - b)
assert np.all([value(t) for t in test_bounds] >= lower_bound - 1e-12)
print("finite-difference derivative:", derivative)
print("negative multiplier:", -multiplier)
```

Deriving sensitivity from the same lower bound

The sensitivity inequality needs no new principle. Use the original optimal multipliers on the perturbed Lagrangian. For any point feasible under $f_i(x) \le u_i$ and $h_j(x) = v_j$,

$$f_0(x) \ge g(\lambda^*, \nu^*) - {\lambda^*}^\top u - {\nu^*}^\top v.$$

Here u, v are the perturbation vectors and the starred multipliers are optimal for the unperturbed problem. Taking the infimum over perturbed feasible points and using strong duality at the original problem produces the previously stated value-function bound. The minus sign follows from subtracting the perturbed right-hand sides inside the Lagrangian.

For the scalar bound example at $b = 1$, the value is $1/2$ and $\lambda^* = 1$. Increasing the bound by δ with $0 < \delta < 1$ gives actual value $(1 - \delta)^2/2$. Its decrease is $\delta - \delta^2/2$, while the multiplier predicts first-order decrease δ. The lower-bound prediction $1/2 - \delta$ stays below the actual value, with difference $\delta^2/2$.

If the restriction is instead written $10(x - b) \leq 0$, its multiplier is $1/10$. The derivative of this restriction with respect to b is -10, so its product with the multiplier still predicts slope -1. Prices depend on representation; the predicted change in the original cost is consistent [1, Section 5.6].

Infeasibility certificates

Plain version. A valid weighted combination of constraints can prove that no choice satisfies them. That is stronger evidence than an algorithm failing to find a point.

For inequalities $Gx \leq h$, suppose a vector y satisfies

$$y \geq 0, \qquad G^\top y = 0, \qquad h^\top y < 0.$$

G, h define the inequality system and y is a nonnegative certificate vector. If a feasible x existed, multiplying and summing would give $0 = y^\top Gx \leq y^\top h < 0$, a contradiction.

This is the geometry behind a theorem of alternatives such as the forms discussed in [1, Section 5.8]. It distinguishes a mathematical certificate from an unsuccessful search.

In floating point, the equalities and strict sign must be assessed relative to scale. A nearly zero transpose residual and a tiny negative certificate value may be inconclusive under accumulated errors. Numerical certification requires a suitable tolerance analysis.

Code: incompatible scalar inequalities

The constraints are $x \leq 0$ and $x \geq 1$. Their nonnegative sum proves impossibility.

```
import numpy as np

G = np.array([[1.], [-1.]])
h = np.array([0., -1.])
certificate = np.array([1., 1.])
assert np.all(certificate >= 0)
assert np.allclose(G.T @ certificate, 0)
assert h @ certificate < 0
print("transpose residual:", G.T @ certificate)
print("certificate contradiction:", h @ certificate)
```

Cones, saddle points and numerical gaps

Plain version. The lower-bound argument extends to cone inequalities by choosing prices from the matching dual cone. A primal–dual solution can also be viewed as a saddle point.

For a cone K, its dual cone is

$$K^* = \{y : y^\top z \geq 0 \text{ for every } z \in K\}.$$

z ranges over the original cone and y is a price vector. When a constraint is expressed as belonging to K, choose signs so the dual-cone inner product supplies the intended lower bound. Orthants, second-order cones and positive-semidefinite cones have familiar self-dual structures under their usual inner products.

For matrix variables, the inner product is trace: $\langle X, Y \rangle = \operatorname{tr}(XY)$ for symmetric matrices. Complementary slackness is then a trace condition, not entrywise multiplication.

Under the appropriate primal–dual assumptions, a saddle point satisfies

$$L(x^*, \lambda, \nu) \leq L(x^*, \lambda^*, \nu^*) \leq L(x, \lambda^*, \nu^*).$$

x^* and (λ^*, ν^*) are optimal primal and dual points; x ranges over the domain and $\lambda \geq 0$. The first side maximizes prices at the feasible point; the second minimizes decisions at the optimal prices.

In practice, an objective difference is a certified gap only when both points are appropriately feasible and the dual value is correctly evaluated. Approximate feasibility can require corrected bounds. A relative gap should use a documented denominator that remains meaningful near zero or with negative objective values.

A complete LP certificate and an invalid near-solution

Consider minimizing $x_1 + 3x_2$ subject to $x_1 + x_2 = 1$ and $x \geq 0$. The primal point $(1, 0)$ has cost one. In the LP dual convention derived above, the scalar dual variable $y = 1$ is feasible because $(y, y) \leq (1, 3)$, and its value is one. These matching bounds prove optimality without enumerating every feasible point.

The dual slack is $(0, 2)$. Its inner product with the primal point is zero, making complementary slackness explicit. The second coordinate is expensive relative to the equality price and receives zero allocation. This interpretation follows from the coefficients and the stated equality; it is not a general feature-importance conclusion.

Now take the approximate point $(1.01, -0.01)$. It satisfies the equality exactly and has an apparently better objective, 0.98. It violates nonnegativity. Comparing that objective with the dual value produces a negative difference, but does not refute weak duality: the candidate is outside the feasible set.

A routine that displays only equality residual and objective could therefore report a misleading result. Check every bound and cone restriction, including those encoded through a solver's variable-bound argument rather than an explicit matrix row.

Clipping the negative coordinate gives $(1.01, 0)$, which then violates the equality. A subsequent normalization gives $(1, 0)$ in this particular example. Neither clipping nor normalization is a universally valid repair procedure for coupled constraints.

Approximate residuals need corrected reasoning

For an approximately feasible LP pair, the algebraic objective difference is

$$c^\top x - b^\top y = x^\top s + y^\top r, \qquad s = c - A^\top y, \quad r = Ax - b.$$

x, y are candidate primal and dual vectors, s the dual slack and r the primal equality residual. The identity is exact algebra; a nonnegative certified gap additionally needs the required feasibility properties.

If $x \geq 0$ and $s \geq 0$ but $r \neq 0$, the residual term can have either sign. A small residual multiplied by a large price can matter. This is why an absolute equality tolerance by itself is not a universal objective-accuracy guarantee.

If dual slack is only bounded below by $-\varepsilon$ componentwise and $x \geq 0$, then $x^\top s \geq -\varepsilon \|x\|_1$. Here $\varepsilon \geq 0$ measures the worst negative slack. Large primal magnitude can amplify a tiny dual violation.

This inequality does not turn the approximate dual objective into a lower bound for every feasible primal point unless a valid bound on those points is also available. An unbounded feasible set makes that correction especially delicate. Avoid describing an arbitrary approximate multiplier as an exact global certificate.

Report primal residuals, negative-slack magnitudes, multiplier norms and objective differences together. If a rigorous certificate is required, use a valid repair, verified error bound or higher-precision check appropriate to the problem structure.

Modern view

Plain version. Modern large-scale methods use primal–dual structure directly, while applications increasingly differentiate through constrained solutions. Both rely on the same certificate and regularity ideas.

Applegate and colleagues' PDLP report [2], first posted in 2025 and revised in 2026, develops large-scale LP computation with practical primal–dual steps and feasibility polishing. Its relevance here is that a useful system monitors feasibility as well as cost improvement; a low objective error with unusable constraint violations is a different outcome.

Pokutta's 2024 survey [3] explains primal and dual convergence for Frank–Wolfe methods. A linear minimization over the feasible set gives a first-order gap from the supporting-plane inequality. This is related to a lower-bound certificate, while its assumptions and computation differ from a generic Lagrange multiplier dual.

Agrawal and colleagues' differentiable convex optimization layers [4] connect structured formulations with differentiation through solutions. The original paper is from 2019; it is included as a foundation for current optimization-layer work, not mislabeled as a new 2026 result. Differentiability and conditioning can fail near degeneracy or active-set changes, so a valid convex solution does not automatically imply a well-behaved gradient.

Current questions include obtaining reliable certificates at very large scale, interpreting uncertainty when constraints are learned, differentiating through nonunique solutions and balancing precision against application latency. Keep optimization accuracy, model validity and learning generalization as separate evaluations.

Key takeaways

- Feasible primal points provide upper bounds; feasible dual points provide lower bounds.
- Multiplier signs follow the exact inequality convention.
- The dual function is concave even when the primal is nonconvex.
- Strong duality needs suitable conditions or a directly verified matching certificate.
- KKT is sufficient for convex differentiable problems and needs regularity for necessity.
- Complementary slackness and sensitivity have precise one-way and sign rules.
- Numerical residuals and gaps must be interpreted with scale and feasibility.

Exercises

1. Derive the dual of minimizing $\frac{1}{2}(x-a)^2$ under $x \le b$, treating a, b as fixed data.

   Hint

   Minimize the Lagrangian over the scalar variable before maximizing its multiplier.

2. Give an active inequality with zero multiplier at an optimum.

   Hint

   A redundant bound at an unconstrained minimizer works.

3. Derive the LP dual using nonnegative variables retained in the domain.

   Hint

   The linear coefficient must be nonnegative for a finite infimum.

4. Explain why KKT stationarity at the origin does not certify minimizing $-x^2$ on $[-1, 1]$.

Hint

The origin is a maximum of the objective.

5. Construct a nonnegative weighted contradiction proving a pair of scalar inequalities infeasible.

Hint

Add an upper bound below a required lower bound.

6. Rescale an inequality by ten. Derive the corresponding multiplier rescaling and explain why primal sensitivity stays consistent.

Hint

Preserve the Lagrangian product.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 5, including duality, KKT, perturbation, alternatives and generalized inequalities.** Official book, free PDF.
2. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Verified paper.
3. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.
4. Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond and J. Zico Kolter, "Differentiable Convex Optimization Layers," *NeurIPS*, 2019. Official publication page.

OPT 05 • Approximation, Statistical Estimation and Experiment Design

Choose convex residual losses, regularizers and shape constraints, formulate likelihood and detector problems, and allocate measurements through optimal experiment design.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-05-fitting-estimation/

What you'll learn

- Select a residual norm by the errors that matter, rather than habit.
- Distinguish parameter regularization, robust residual losses and uncertain measurement models.
- Impose monotonicity and convexity on fitted functions using linear inequalities.
- Recognize convex likelihood and probability-estimation formulations.
- Design randomized detectors and understand moment-based risk bounds.
- Allocate measurement effort using information matrices and explicit design criteria.

The big picture

An optimization model can fit a curve to observations, estimate a distribution or decide which observations to collect next. These tasks share a common pattern: choose a representation, state how disagreement is measured, and specify which choices are allowed.

Boyd and Vandenberghe's Chapters 6–7 [1] connect approximation and statistical estimation. Read them together: the same squared residual can mean a geometric approximation error, a likelihood under a noise model, or one component of a constrained design objective. Those interpretations impose different assumptions even when the numerical formula is identical.

> **Explain like I'm five**
>
> You want a ruler that passes near several dots. One rule worries about the average mismatch; another worries about the worst dot; a third refuses to bend too much. Before finding the best ruler, decide which meaning of "best" matches the job.

A successful solver answers the stated optimization problem. It does not prove that the measurement assumptions are true, that a fitted curve predicts new data, or that a sensor allocation is physically achievable. This chapter keeps those questions visible alongside the mathematics.

The examples use synthetic observations and tiny measurement models. They show how the pieces work without hiding downloads, pretrained systems or application-specific assumptions inside the code.

Residual norms express priorities

Plain version. A residual is the difference between a prediction and an observation. Choosing a norm determines which patterns of error are expensive.

For a linear model,

$$r = Ax - b, \qquad \min_x \|Ax - b\|.$$

$A \in \mathbb{R}^{m \times n}$ is a fixed design matrix, $b \in \mathbb{R}^m$ the observations, $x \in \mathbb{R}^n$ the parameters and $r \in \mathbb{R}^m$ the residual vector. The chosen norm acts on residuals, not directly on parameters.

The Euclidean norm or its square emphasizes large residuals through their squared contribution. The absolute-value norm adds residual magnitudes. The maximum norm minimizes the largest absolute residual and can be appropriate when every measured error must remain below a uniform tolerance [1, Section 6.1].

$$\|r\|_2^2 = \sum_i r_i^2, \qquad \|r\|_1 = \sum_i |r_i|, \qquad \|r\|_\infty = \max_i |r_i|.$$

r_i is residual coordinate i. All three objectives are convex when r is affine in x. Squaring a nonnegative Euclidean norm preserves its minimizers, but changes objective values, derivatives and the tradeoff with any additional penalty.

For absolute residuals, introduce $t_i \geq r_i$ and $t_i \geq -r_i$, then minimize $\sum_i t_i$. For a uniform tolerance introduce one scalar t and impose $-t \leq r_i \leq t$. These are LP representations when there are only affine additional restrictions.

Three meanings of the best constant

A single fitted number exposes the difference between residual objectives. Take the synthetic observations $b = (0, 1, 8)$ and predict the same scalar c for all three. The resulting design matrix is a column of ones.

For squared loss, differentiating $c^2 + (c-1)^2 + (c-8)^2$ gives $6c - 18 = 0$, hence $c = 3$. For absolute loss, the slope is the number of observations below c minus the number above it, away from the data points. The slope changes from negative to positive at the median, so $c = 1$ minimizes the sum. For maximum absolute loss, balance the two extreme errors: $c = 8 - c$, hence $c = 4$.

Here c is the fitted constant and b the fixed observations. At their respective optima, squared residual sum is 38, absolute residual sum is 8, and maximum residual is 4. These are different quantities with different units; comparing their raw numerical sizes would be meaningless. The important comparison is the returned prediction: average fit chooses three, median fit chooses one, and uniform-error fit chooses four. The observation eight pulls the mean strongly but does not move the median beyond the middle observation [1].

For a real task, ask which error pattern is costly. Uniform instrument tolerances suggest maximum error. A specified Gaussian measurement model suggests squared error. Contaminated responses

can motivate bounded-influence residual losses. None of these choices follows solely from the convenience of a solver.

Units and weighted errors

Measurements with different units cannot be added sensibly without a stated scaling convention. A residual of one pixel and a residual of one second are not automatically comparable.

A weighted least-squares objective can be written

$$(Ax - b)^\top W(Ax - b), \qquad W \succeq 0.$$

W is a fixed positive-semidefinite weighting matrix. If W is the inverse of a known positive-definite noise covariance, the objective has a statistical interpretation. If weights encode priorities, explain that interpretation instead.

Off-diagonal entries can describe correlated measurement errors. Replacing a full covariance by diagonal weights is a modeling approximation, not just an implementation detail. Whitening with a matrix factor transforms the residual before fitting and should preserve the intended quadratic form.

A large residual is not automatically an outlier. It could reveal a wrong model, a poorly scaled feature, genuine rare behavior or a corrupted observation. Inspect the generating process before deciding which errors should lose influence.

Least-norm solutions and regularization

Plain version. Matching the observations may leave many parameter choices. A norm on parameters chooses among them or trades fit against complexity.

A least-norm problem has the form

$$\min_x \|x\| \quad \text{subject to} \quad Ax = b.$$

x is the parameter vector, and A, b are fixed. This differs from a residual-norm problem: here exact consistency is required, and the objective selects a small parameter vector [1, Section 6.2].

If the equality is inconsistent, the problem is infeasible. Replacing it with a soft residual term changes the question. In an underdetermined consistent system, the Euclidean least-norm solution can be expressed through a suitable pseudoinverse, but numerical rank and singular values still matter.

Regularized approximation instead balances two goals:

$$\min_x \frac{1}{2}\|Ax - b\|_2^2 + \lambda R(x), \qquad \lambda \geq 0.$$

R is a convex penalty and λ its nonnegative weight. The factor one-half simplifies derivatives. Changing that factor while keeping λ fixed changes the tradeoff.

For $R(x) = \frac{1}{2}\|x\|_2^2$, the objective is a ridge or quadratic regularization model [1, Section 6.3]. Positive λ makes the Hessian $A^\top A + \lambda I$ positive definite. For $R(x) = \|x\|_1$, the penalty encourages exact zero coordinates through its nonsmooth geometry.

An intercept is often treated separately because penalizing it has a different meaning from penalizing feature effects. Feature units also change a parameter norm. Standardizing columns or using an explicitly scaled penalty makes this choice visible.

Derive quadratic shrinkage direction by direction

Quadratic regularization reduces the instability caused by weakly observed parameter directions. For the ridge objective already defined, setting its gradient to zero gives

$$(A^\top A + \lambda I)x^* = A^\top b.$$

Here x^* is the optimizer, A, b are the fixed design and observations, I is the identity, and $\lambda > 0$ is the penalty weight. Let $A = U\Sigma V^\top$ be a singular-value decomposition, with orthonormal singular vectors in U, V and singular values σ_i in Σ. In direction v_i, the coefficient is

$$v_i^\top x^* = \frac{\sigma_i}{\sigma_i^2 + \lambda} u_i^\top b.$$

Here u_i, v_i are paired singular vectors. Without regularization, a nonzero singular value divides the observation coefficient by σ_i; a tiny value can amplify measurement error. With regularization, the denominator never falls below λ. A nullspace direction receives zero in this fully penalized model.

For $A = \mathrm{diag}(1, 0.1)$ and $b = (1, 0.1)$, the unregularized solution is $(1, 1)$. With $\lambda = 0.1$, the solution becomes $(10/11, 1/11)$. The weakly observed second direction is shrunk much more strongly. This explains both the numerical stability and the introduced bias: regularization does not learn that the second true coefficient is small. It imposes a preference when the observations constrain that direction weakly [1, Section 6.3].

Constraints and penalties are related but different

The constrained model $\min \|Ax - b\|_2^2$ subject to $R(x) \le \rho$ directly specifies an allowable complexity budget. Its Lagrange multiplier can connect it to a penalty formulation under appropriate conditions.

That connection does not mean a particular numerical λ corresponds to an obvious ρ in advance. Nonunique solutions and inactive restrictions can complicate the correspondence. Choose the form that expresses the actual requirement.

Regularization can improve conditioning, but that numerical advantage is distinct from predictive improvement. A well-conditioned biased model may predict poorly. Select tuning parameters with a valid evaluation split when prediction is the goal, and keep the fitting objective separate from the final evaluation metric.

Robust losses and robust measurement models

Plain version. "Robust" can mean reducing the influence of unusually large residuals, or protecting against uncertainty in the measurement operator. These are different models.

One convex residual loss is the Huber function:

$$\phi_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta|r| - \frac{1}{2}\delta^2, & |r| > \delta. \end{cases}$$

r is a scalar residual and $\delta > 0$ the transition scale. The derivative equals r near zero and saturates at magnitude δ outside. Summing this loss over affine residuals gives a convex problem [1, Section 6.1].

The transition has the units of the residual. Multiplying all observations by a scale while leaving δ unchanged alters which points receive reduced influence. The loss is not immune to every corruption pattern; high-leverage errors in the design matrix require separate attention.

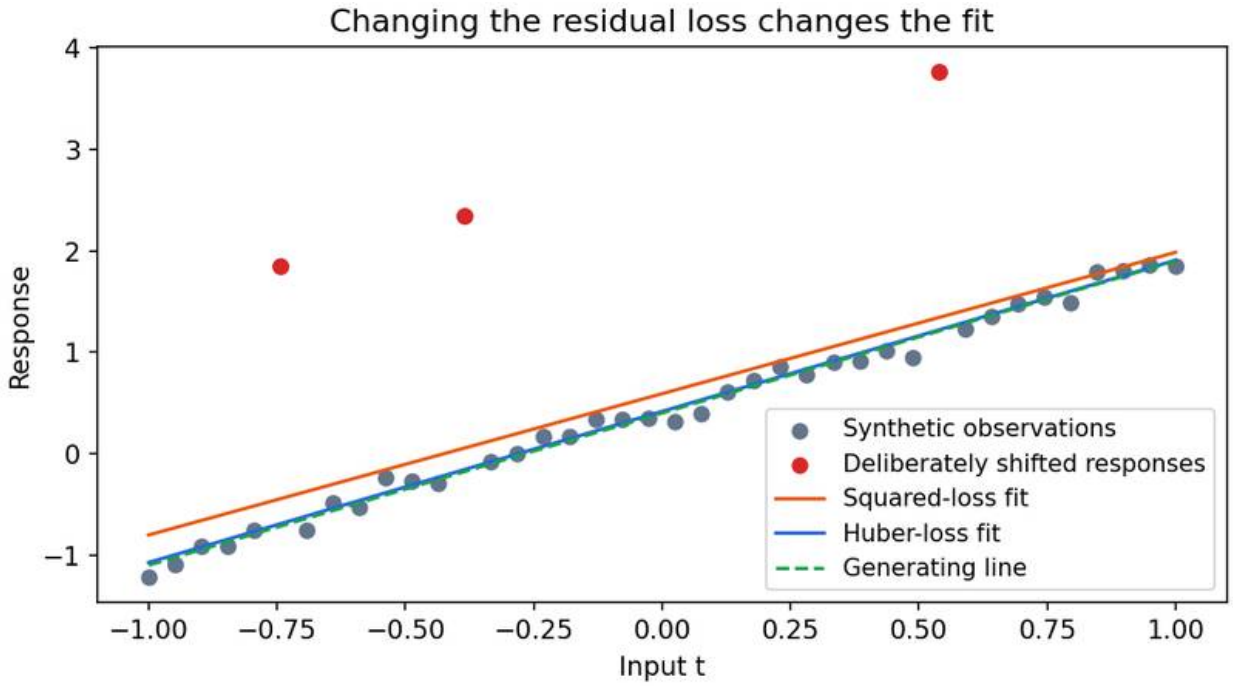


Figure 1 — Original synthetic observations and fitted solutions. Some responses were deliberately shifted upward. This illustrative example compares residual losses; it is not a benchmark or a general robustness guarantee.

Code: fit a line with two losses

The data and shift are generated locally. The smooth optimization call follows the official SciPy interface [2]. We compare solutions and verify the robust objective's gradient instead of asserting that one estimator always wins.

```
import numpy as np
from scipy.optimize import minimize

rng = np.random.default_rng(15)
t = np.linspace(-1, 1, 40)
A = np.column_stack([np.ones_like(t), t])
b = 0.4 + 1.5*t + rng.normal(0, 0.08, len(t))
b[[5, 12, 30]] += 2.5
least_squares = np.linalg.lstsq(A, b, rcond=None)[0]
delta = 0.2

def huber_objective(x):
    r = A @ x - b
    loss = np.where(abs(r) <= delta, 0.5*r*r,
                    delta*abs(r) - 0.5*delta**2)
    return loss.sum(), A.T @ np.clip(r, -delta, delta)

fit = minimize(huber_objective, least_squares, jac=True,
               method="BFGS", options={"gtol": 1e-9})
assert np.linalg.norm(huber_objective(fit.x)[1]) < 1e-6
print("squared-loss coefficients:", np.round(least_squares, 4))
print("Huber-loss coefficients:", np.round(fit.x, 4))
print("robust objective:", round(fit.fun, 6))
```

Uncertain operators

Suppose the matrix itself may vary in a specified set $\mathcal{A}$. A worst-case approximation model is

$$\min_{x} \sup_{A \in \mathcal{A}} \|Ax - b\|.$$

x is the decision, b the fixed target, and $\mathcal{A}$ the allowed measurement matrices. Each fixed-matrix norm objective is convex in x, so their pointwise supremum is convex [1, Section 6.4].

Convexity does not guarantee that evaluating this supremum is easy. A finite uncertainty set can yield finitely many epigraph restrictions. Structured ellipsoidal uncertainty can permit conic reformulations. A vaguely described "small perturbation" is not enough to select the correct counterpart.

An expected squared-error model is different. If $A = \bar{A} + U$, with $\mathbb{E}[U] = 0$, then

$$\mathbb{E}\|Ax - b\|_2^2 = \|\bar{A}x - b\|_2^2 + x^\top P x, \qquad P = \mathbb{E}[U^\top U].$$

$\bar{A}$ is the mean operator, U its random variation and P a positive-semidefinite second-moment

matrix. The cross term vanishes under the zero-mean assumption. Here regularization follows from a specified uncertainty model, rather than an arbitrary complexity preference.

Worst-case and expected protection answer different questions. An uncertainty set is not a probability distribution. A covariance model does not promise control of every possible realization.

Function fitting with shape constraints

Plain version. We can ask a fitted function to be increasing or convex while keeping the optimization convex in its coefficients.

For fixed basis functions $\psi_j(t)$,

$$f(t) = \sum_{j=1}^{n} x_j \psi_j(t).$$

t is the input, ψ_j are known basis functions, and x_j are fitted coefficients. Evaluating at fixed sample inputs makes the predictions affine in x, even when the basis functions are nonlinear in t.

This distinction matters: convexity in the optimization variables is not the same as convexity in the function's input. A polynomial regression may be a convex coefficient-fitting problem while the fitted polynomial is neither increasing nor convex.

At ordered knots $t_1 < \cdots < t_m$, let $z_i = f(t_i)$ be fitted values. Monotonicity can be imposed by $z_{i+1} \geq z_i$. Convexity of the piecewise-linear interpolant requires

$$\frac{z_{i+1} - z_i}{t_{i+1} - t_i} \leq \frac{z_{i+2} - z_{i+1}}{t_{i+2} - t_{i+1}}.$$

z_i are variables and the knot spacings are fixed positive numbers. These slope-order inequalities are linear in fitted values. Unlike sampling a derivative of an arbitrary flexible function, they guarantee the shape of this particular interpolant throughout its knot interval.

Code: monotone absolute-error fitting as an LP

We fit five ordered measurements. The auxiliary variables represent absolute residual bounds. SciPy's LP bounds are explicitly specified [3], including unrestricted fitted values.

```
import numpy as np
from scipy.optimize import linprog

observed = np.array([0., 2., 1., 3., 4.])
n = len(observed)
I = np.eye(n)
order = np.zeros((n-1, n))
for i in range(n-1):
    order[i, i], order[i, i+1] = 1, -1
```

```
G = np.vstack([np.hstack([I, -I]),
               np.hstack([-I, -I]),
               np.hstack([order, np.zeros((n-1, n))])])
h = np.r_[observed, -observed, np.zeros(n-1)]
fit = linprog(np.r_[np.zeros(n), np.ones(n)], A_ub=G, b_ub=h,
              bounds=[(None, None)]*n + [(0, None)]*n,
              method="highs")
assert fit.success
z = fit.x[:n]
assert np.all(np.diff(z) >= -1e-9)
assert np.isclose(abs(z-observed).sum(), fit.fun)
print("monotone fitted values:", z)
print("total absolute residual:", fit.fun)
```

Absolute-error solutions may be nonunique. The code verifies the property and objective rather than requiring one particular plateau value. Adding a strictly convex secondary objective would select one solution, but would be a different stated problem.

Constraints outside the observed interval require care. Piecewise-linear interpolation supplies a clear in-range function; extrapolation is an additional rule. Shape restrictions can improve plausibility while introducing bias if the underlying process violates them.

Likelihood and convex parameterizations

Plain version. A likelihood model can generate a convex objective, but the chosen parameterization and domain determine whether that is true.

For independent observations y_i under a distribution with natural parameter θ,

$$p(y \mid \theta) = \exp(\theta^\top T(y) - A(\theta) + c(y)).$$

$T(y)$ is a sufficient-statistic vector, $A(\theta)$ the log-partition function and $c(y)$ a parameter-independent base term. For an exponential-family model, A is convex on its natural domain.

Ignoring terms independent of θ, the negative log-likelihood is

$$\ell(\theta) = NA(\theta) - \theta^\top \sum_{i=1}^{N} T(y_i).$$

N is the number of observations. This is convex in the natural parameter. An arbitrary nonlinear change of parameter does not necessarily preserve convexity [1, Section 7.1].

For Bernoulli observations with success probability $p \in (0, 1)$ and k successes out of N, the negative log-likelihood is $-k \log p - (N - k) \log(1 - p)$. Its interior optimum is $p = k/N$ when both outcomes are observed.

If every observation is a success or every observation a failure, the optimum over the open domain may only be approached at a boundary. Allowing boundary distributions or adding a prior

changes the existence question. A program that clips probabilities should record that clipping as a domain choice.

Maximum a posteriori estimation adds the negative log prior. A convex negative log prior preserves convexity in the selected coordinates. "Bayesian" alone does not guarantee a convex optimization problem, and a point estimate does not represent the entire posterior uncertainty.

Nonparametric probabilities

For discrete outcomes, estimate a probability vector q on the simplex:

$$q_j \geq 0, \qquad \sum_j q_j = 1.$$

q_j is the probability of outcome j. Given counts n_j, the negative log-likelihood is $-\sum_j n_j \log q_j$, with the appropriate extended-value convention for positive counts at zero probability.

Known moments become affine restrictions, such as $\sum_j q_j a_j = \mu$ for fixed outcome features a_j and known mean μ. Maximizing entropy, equivalently minimizing $\sum_j q_j \log q_j$, selects a distribution subject to such constraints [1, Section 7.2].

Moment restrictions should come from the same population and units as the data. Inconsistent empirical constraints can make the simplex model infeasible. Softening them adds a modeling decision about acceptable uncertainty, rather than repairing an algebraic nuisance.

Detector design as a linear program

Plain version. Choosing decisions after observing discrete evidence can be optimized directly. Allowing randomized decisions makes the feasible set convex.

Suppose p_{kj} is the known probability of observation k under hypothesis j. Let T_{ik} be the probability of deciding hypothesis i after seeing k.

$$T_{ik} \geq 0, \qquad \sum_i T_{ik} = 1, \qquad D_{ij} = \sum_k T_{ik} p_{kj}.$$

D_{ij} is the probability of deciding i when j is true. T is the decision variable; the conditional probabilities p_{kj} are fixed. Each entry of D is linear in T.

Weighted error costs, lower bounds on correct detection and upper bounds on false decisions can therefore be expressed as linear objectives and restrictions [1, Section 7.3]. A prior distribution on hypotheses can weight overall expected error, but should not be silently substituted for a conditional-error guarantee.

For binary hypotheses, likelihood-ratio threshold rules emerge from suitable optimal detector formulations. A boundary observation may need randomization to meet an exact error budget. Deterministic rules are a restricted subset; forcing them introduces discrete choices.

An optimized detector is only as valid as its conditional distributions. Distribution shift, estimated likelihoods and calibration uncertainty belong in the assessment. Numerical optimality under a specified probability table does not validate that table.

Worked detector: why randomization can be useful

A false-positive budget can fall between two deterministic rules. Suppose the observation is either a or b. Under hypothesis H_0, its probabilities are $(0.8, 0.2)$; under H_1, they are $(0.2, 0.8)$. These are synthetic conditional probability tables, not measured frequencies.

Let $t_a, t_b \in [0, 1]$ be the probabilities of deciding H_1 after each observation. Maximizing true-positive probability with false-positive budget 0.1 is the LP

$$\begin{aligned} \text{maximize}_{t_a,t_b} \quad & 0.2t_a + 0.8t_b \\ \text{subject to} \quad & 0.8t_a + 0.2t_b \le 0.1, \\ & 0 \le t_a, t_b \le 1. \end{aligned}$$

Here the objective is detection probability under H_1, and the first restriction is error probability under H_0. Spending one unit of error budget on b yields four units of detection probability, whereas spending it on a yields only one quarter. Allocate the budget to b: $t_a = 0$, $t_b = 1/2$, giving true-positive probability 0.4.

A deterministic decision on b would cost 0.2, beyond the budget. With this discrete observation alphabet, every nontrivial deterministic rule violates the stated budget, so the only allowed deterministic rule always decides H_0 and detects nothing. Randomization fills the convex space between deterministic rules and achieves the desired operating point [1, Section 7.3]. This is randomizing a decision after a known observation, not treating unknown probabilities as decision variables.

Moment and exponential risk bounds

Plain version. Sometimes we know moments but not the whole distribution. Bounds can control a tail event conservatively, with assumptions stated explicitly.

For a scalar random variable Y with mean μ and finite variance σ^2, Chebyshev's inequality gives

$$\Pr(|Y - \mu| \ge a) \le \frac{\sigma^2}{a^2}, \qquad a > 0.$$

a is a deviation threshold. The bound uses only the first two moments and can be loose. An upper bound above one can be replaced by one; it is not an estimated event probability [1, Section 7.4].

For a threshold b, an exponential or Chernoff bound uses

$$\Pr(Y \ge b) \le \inf_{s>0} \exp(-sb)\, \mathbb{E}[\exp(sY)].$$

s is a positive auxiliary parameter, and the expectation must be finite where used. Optimizing its logarithm gives $\inf_{s>0}\{\log \mathbb{E}[\exp(sY)] - sb\}$.

The log moment-generating function is convex in s on its domain. This does not automatically make every joint design-and-risk problem convex in all its variables. Check how the random variable depends on decisions and which distribution assumptions are retained.

Tail bounds can support conservative design constraints. They do not replace checking whether moments were reliably estimated. Heavy tails, dependence and finite-sample uncertainty can make an assumed moment-generating function unsuitable.

Choosing the next measurements

Plain version. Estimation quality depends on what we measure, not only on how we fit. Convex experiment design allocates continuous measurement effort before any discrete rounding.

For candidate measurement vectors $a_j \in \mathbb{R}^n$, independent unit-variance noise and normalized weights w_j, define

$$M(w) = \sum_j w_j a_j a_j^\top, \qquad w_j \geq 0, \quad \sum_j w_j = 1.$$

M is the information matrix per normalized measurement effort. The covariance of a corresponding linear estimator scales with its inverse when the matrix is positive definite, the measurement model is correct and total effort is accounted for.

D-optimal design minimizes $-\log\det M$, which targets an uncertainty-volume criterion. A-optimal design minimizes $\operatorname{tr}(M^{-1})$, targeting total coordinate variance. E-optimal design maximizes the smallest eigenvalue of M, targeting the weakest measured direction [1, Section 7.5].

These are different scalar summaries. Units and coordinate changes can affect their interpretation differently. A small total variance may hide one poorly measured direction; a small uncertainty volume does not promise a bound on every coordinate.

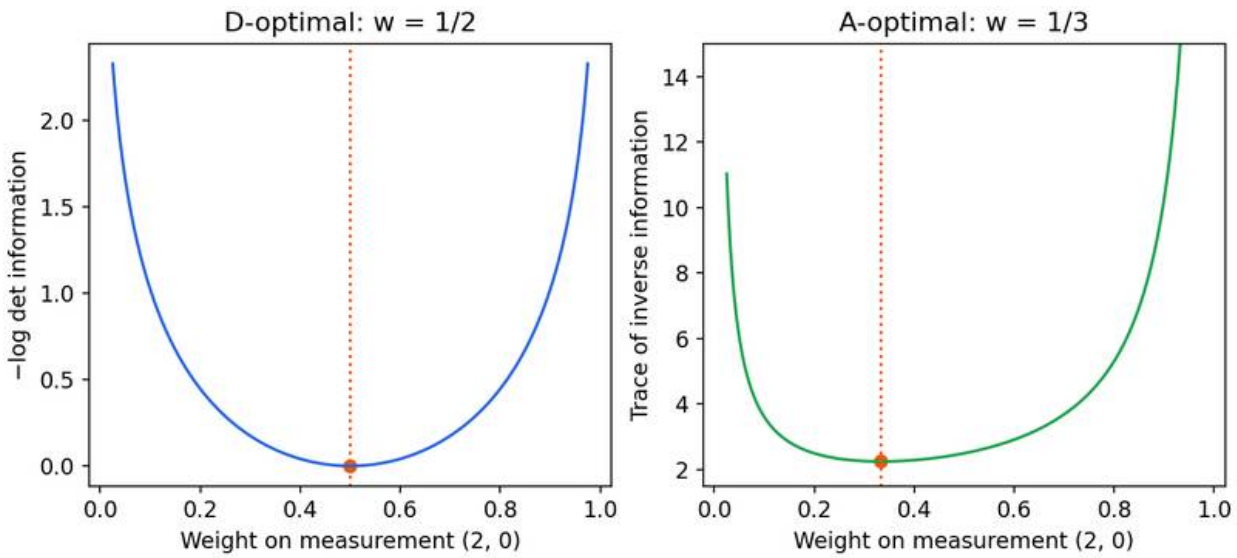


Figure 2 — Original analytical two-direction design. The D-optimal and A-optimal allocations differ because volume and total coordinate variance are different goals. The curves assume independent unit-variance noise and positive effort in both directions.

Code: two design criteria with exact checks

Use candidate vectors $(2, 0)$ and $(0, 1)$, with first weight w and second weight $1 - w$. The information matrix is diagonal: $M = \mathrm{diag}(4w, 1 - w)$. Both criteria diverge at an unobserved direction.

```
import numpy as np
from scipy.optimize import minimize_scalar

def d_loss(w):
    return -np.log(4*w) - np.log(1-w)

def a_loss(w):
    return 1/(4*w) + 1/(1-w)

d = minimize_scalar(d_loss, bounds=(1e-6, 1-1e-6), method="bounded")
a = minimize_scalar(a_loss, bounds=(1e-6, 1-1e-6), method="bounded")
assert np.isclose(d.x, 0.5, atol=1e-5)
assert np.isclose(a.x, 1/3, atol=1e-5)
print("D-optimal first weight:", round(d.x, 6))
print("A-optimal first weight:", round(a.x, 6))
```

With a fixed number of experiments, actual counts are integers. Continuous weights are a relaxation or a randomized sampling policy. Rounding can destroy a budget or leave a direction unmeasured, so reassess the resulting matrix after converting weights into a schedule.

If candidate vectors do not span the parameter space, no weighting makes the information positive definite. Adding a prior matrix may regularize the design, but changes its interpretation to prior-plus-measurement information.

Derive why the two design criteria allocate differently

The two-direction experiment can be solved before running numerical code. With candidate vectors $(2,0)$ and $(0,1)$ and weights $w, 1-w$, its information matrix is $M = \operatorname{diag}(4w, 1-w)$. Let $0 < w < 1$ so both directions are measured.

The D-optimal objective and its derivative are

$$f_D(w) = -\log(4w) - \log(1-w), \qquad f_D'(w) = -\frac{1}{w} + \frac{1}{1-w}.$$

Here f_D is the negative log-determinant criterion. Stationarity gives $w = 1/2$. The strength factor four changes the objective by a constant and does not change this D-optimal allocation.

For A-optimal design,

$$f_A(w) = \frac{1}{4w} + \frac{1}{1-w}, \qquad f_A'(w) = -\frac{1}{4w^2} + \frac{1}{(1-w)^2}.$$

Here f_A is the inverse-information trace criterion. Positive denominators turn its stationary equation into $1 - w = 2w$, giving $w = 1/3$. Each scalar objective has positive second derivative and diverges at the endpoints, so these stationary points are the unique global minima [1, Section 7.5].

The interpretation is useful: the first measurement already observes its direction more strongly, so minimizing total variance sends more effort to the weaker second direction. Minimizing uncertainty volume balances multiplicative directional information instead and gives equal effort. If measurement cost or noise variance differs, incorporate those changes in the information matrix and budget before repeating the derivation. Neither allocation is universally best; each answers its chosen uncertainty question.

A fitting and design audit

Start with the application quantity that must be accurate. Decide whether it is a prediction, parameter component, worst residual or a distributional probability. Then state the representation and units before selecting a numerical loss.

For each constraint, ask whether it is physically required, statistically estimated or introduced for numerical convenience. Those roles imply different responses to infeasibility. A physical limit may need a revised design; an empirical moment may need uncertainty; redundant algebra may need removal.

After fitting, verify domain membership, shape restrictions and solver residuals. Then assess generalization or uncertainty using an appropriate independent procedure. An optimum of a training objective and a reliable future prediction are separate claims.

For design, inspect the information spectrum, available budget, noise assumptions and schedule conversion. Report the final implementable schedule alongside the relaxed objective. This keeps the most consequential approximation visible.

What an uncertainty ellipsoid actually says

An inverse information matrix describes a covariance only under the corresponding measurement and estimator assumptions. Turning it into a region with a stated coverage probability needs a distributional argument and a correctly chosen threshold. Drawing an ellipse from that matrix alone does not establish a confidence level.

For a positive-definite covariance C, an ellipsoid can be written as $(x - \hat{x})^\top C^{-1}(x - \hat{x}) \leq c$. Here $\hat{x}$ is a center and $c > 0$ is a threshold. The matrix specifies shape; the threshold specifies size. Coverage additionally depends on the actual distribution of estimation error.

A prior covariance, a sampling covariance and a numerical Hessian inverse need not have the same interpretation. They can coincide under particular models, but should not be exchanged without checking those models. Likewise, an estimator's optimization tolerance and its sampling uncertainty have different units and sources.

When measurements are repeated, independent equal-variance assumptions justify a simple additive information model. Correlated repeats can provide less new information than their count suggests. A design that ignores that correlation may confidently allocate effort to redundant measurements.

This distinction affects stopping precision. Solving far below the uncertainty in the input coefficients may offer little practical value, while violating a hard physical bound by a seemingly small amount may still be unacceptable. Set optimization tolerances against the relevant application scale, rather than choosing precision solely from the number of available floating-point digits.

Finally, uncertainty visualization should state whether it displays an analytical model, an estimated covariance or repeated synthetic experiments. The figures in this chapter show illustrative fits and objective curves. They do not display empirically calibrated coverage regions or claim a particular confidence probability.

Modern view

Plain version. Current work combines structured convex estimation with large-scale first-order computation and learned convex components. The convexity guarantee belongs to the stated variables and domains.

Pokutta's 2024 survey [4] organizes Frank–Wolfe methods around linear minimization over a convex feasible set, convergence guarantees and useful variants. Simplex-constrained experiment design is a natural setting for this viewpoint: an extreme-point search can avoid a full projection. The objective still needs the required smoothness and positive-definite domain control.

Schaller, Bemporad and Boyd's 2025 manuscript [5] studies learning parametric convex functions for subsequent optimization. It connects fitting with disciplined curvature rules: a learned component can preserve convexity in decision variables for valid fixed parameters. Training the component is a separate problem and need not itself be convex.

These sources cover complementary questions: how to move through a structured feasible set, and how to construct functions usable inside such a set. Neither replaces uncertainty modeling, held-out evaluation or the physical feasibility of a design.

Current challenges include reliable uncertainty under model mismatch, informative measurements under resource limits, shape restrictions that remain appropriate outside the sample interval, and scalable updates when observations arrive sequentially. Distinguish a provably convex estimator from a complete validated statistical pipeline.

Key takeaways

- Residual norms, parameter norms and evaluation metrics answer different questions.
- Regularization weights depend on objective normalization and feature units.
- Robust residual losses and uncertain-operator models protect against different errors.
- Fixed nonlinear bases can still yield convex coefficient fitting.
- Likelihood convexity depends on parameterization and domain.
- Randomized discrete-evidence detectors can be formulated as LPs.
- Experiment-design criteria differ, and continuous weights require an implementation plan.

Exercises

1. Formulate maximum absolute residual fitting as an LP, including unrestricted parameters.

   Hint

   Use one nonnegative residual bound.

2. Scale observations and predictions by ten. Determine how a Huber transition and a quadratic regularization weight should change to preserve a chosen equivalent objective.

   Hint

   Write the scaled objective explicitly; the answer depends on whether parameters and penalty units are also scaled.

3. Add convexity constraints to the five-knot monotone-fitting example, using unequal positive knot spacings.

   Hint

   Order adjacent slopes.

4. Derive the Bernoulli maximum-likelihood estimate, and explain the open-domain boundary cases.

Hint

Set the likelihood derivative to zero, then inspect endpoints.

5. Construct a two-hypothesis randomized detector LP with a stated false-positive budget.

Hint

Use decision probabilities per observation, and conditional error as a linear expression.

6. Derive the two-direction D- and A-optimal weights in the code, then explain why rounding a small measurement budget needs another check.

Hint

Differentiate the scalar criteria; inspect whether each rounded direction receives a measurement.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 6, Sections 6.1–6.5; Chapter 7, Sections 7.1–7.5.** Official book, free PDF.
2. SciPy developers, "scipy.optimize.minimize" and "scipy.optimize.minimize_scalar," *SciPy API documentation*, accessed 2026. Multivariate interface, scalar interface.
3. SciPy developers, "scipy.optimize.linprog," *SciPy API documentation*, accessed 2026. Official documentation.
4. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.
5. Michael Schaller, Alberto Bemporad and Stephen Boyd, "Learning Parametric Convex Functions," manuscript, 2025. Official manuscript page.

OPT 06 • Gradient Descent, Newton Steps and Numerical Reliability

Derive descent directions and line searches, compare gradient and Newton methods, interpret convergence assumptions, and implement a stable convex logistic solver.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-06-gradient-newton/

What you'll learn

- Separate a search direction from the step length chosen along it.
- Derive a safe gradient step under a Lipschitz-gradient assumption.
- Implement Armijo backtracking with domain and iteration safeguards.
- Explain conditioning, norm-dependent steepest descent and Newton curvature.
- Interpret a Newton decrement and the assumptions behind rapid local convergence.
- Verify derivatives, linear solves and stopping evidence in a CPU example.

The big picture

Convexity tells us which answers are globally optimal. An algorithm tells us how to move toward one using information that can be computed. Those are complementary questions: a convex problem can still be badly scaled, expensive or numerically delicate.

Boyd and Vandenberghe's Chapter 9 [1] develops unconstrained descent methods, gradient descent, steepest descent, Newton's method, self-concordance and implementation. This chapter follows that progression, keeping the cost of each step and the meaning of a stopping test visible.

> **Explain like I'm five**
>
> You are walking down a hill. A slope arrow tells you which way goes down. A map of how the hill bends can suggest a smarter route. Either way, you still check that the next step lands safely and actually lowers your height.

The examples solve deterministic convex objectives. Their guarantees should not be silently transferred to a neural network's nonconvex training objective, a noisy derivative or a nonsmooth penalty. Useful ideas can transfer, but the proof assumptions must be checked again.

We first use a quadratic with an exact solution, then implement a smooth regularized logistic objective from scratch. Both run on a CPU with locally generated data and no solver package beyond NumPy.

A direction and a distance

Plain version. A descent iteration chooses where to point, then how far to travel. Combining those choices too early hides why a step is safe.

Write an iteration as

$$x_{k+1} = x_k + t_k d_k.$$

x_k is the current decision vector, d_k a search direction and $t_k > 0$ the step length. For a differentiable objective f, a strict descent direction satisfies $\nabla f(x_k)^\top d_k < 0$.

The inner product is the directional derivative at zero step. By differentiability, a sufficiently small positive step lowers the objective and remains in an open domain around the current point. A large step can still increase it or leave the domain.

The direction may depend on a gradient, a Hessian, a metric or other structure. The line search evaluates candidate distances. Counting only accepted iterations can hide many rejected objective evaluations.

If the gradient is zero for a differentiable convex objective on an open convex domain, the supporting-plane inequality certifies a global minimizer. With constraints, zero unconstrained gradient is generally not the correct optimality condition; the next chapter uses constrained systems.

Exact and inexact line searches

An exact line search minimizes $f(x + td)$ over allowed positive distances. That auxiliary one-dimensional problem can be costly or unnecessary.

An inexact search accepts sufficient decrease rather than solving the line problem precisely. It should also preserve the objective's domain. For a logarithmic term, testing a candidate with an invalid argument is a failed domain test, not evidence about its objective.

Floating-point evaluation adds another distinction: a mathematically decreasing step can become indistinguishable from the current value at machine precision. Repeatedly shrinking until a comparison happens to pass is not a robust termination rule.

Gradient descent and its assumptions

Plain version. Gradient descent points opposite the gradient. A curvature bound tells us how small a fixed step must be.

The update is

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

$\eta > 0$ is the step size. Assume the gradient is Lipschitz with constant L on the relevant region:

$$\|\nabla f(y) - \nabla f(x)\|_2 \le L\|y - x\|_2.$$

x, y are points whose connecting segment stays in that region. L measures how rapidly the gradient changes; it is not a regularization coefficient or a loss value.

The smoothness inequality is

$$f(x + d) \le f(x) + \nabla f(x)^\top d + \frac{L}{2}\|d\|_2^2.$$

d is a trial displacement. Substituting $d = -\nabla f(x)/L$ gives

$$f\left(x - \frac{1}{L}\nabla f(x)\right) \le f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2.$$

The step $1/L$ therefore supplies descent whenever the assumptions and domain requirement hold. For a twice-differentiable objective, an upper Hessian eigenvalue bound can establish such an L [1, Section 9.3].

When only convexity and smoothness are assumed, objective convergence is typically sublinear under the standard fixed-step analysis. Strong convexity adds a positive lower curvature bound and permits linear geometric convergence. Those words describe mathematical rates, not a promise that every implementation will be fast.

From one-step decrease to a convergence rate

Descent proves that the objective improves; a convergence rate quantifies how much work reaches a target error. For a convex differentiable function on $\mathbb{R}^n$ with L-Lipschitz gradient and an attained minimizer x^*, use $x^+ = x - \nabla f(x)/L$. The gradient supporting bound gives

$$f(x) - f(x^*) \le \nabla f(x)^\top (x - x^*).$$

Here x is the current point, x^+ its next iterate and x^* any minimizer. Expanding the squared distance after the update gives

$$\|x^+ - x^*\|_2^2 = \|x - x^*\|_2^2 - \frac{2}{L}\nabla f(x)^\top (x - x^*) + \frac{1}{L^2}\|\nabla f(x)\|_2^2.$$

All symbols retain their definitions above. Combine the first inequality with the earlier smoothness decrease bound to obtain

$$f(x^+) - f(x^*) \le \frac{L}{2}\left(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2\right).$$

Summing this inequality over the first k updates cancels the intermediate distances. Since objective values decrease, the last error is no larger than the average of those k errors:

$$f(x_k) - f(x^*) \le \frac{L\|x_0 - x^*\|_2^2}{2k}, \qquad k \ge 1.$$

Here x_0 is the start and x_k the point after k updates. This is the concrete meaning of an $O(1/k)$ objective rate: halving the guaranteed error generally requires twice as many iterations under this bound. It is a worst-case guarantee with these assumptions, not a claim about every observed trace [1].

Strong convexity and a stopping bound

If f is μ-strongly convex, its supporting model includes $\frac{\mu}{2}\|y - x\|_2^2$. Minimizing that lower model yields

$$f(x) - f^* \leq \frac{\|\nabla f(x)\|_2^2}{2\mu}.$$

f^* is the global minimum and $\mu > 0$ a valid strong-convexity constant. A small gradient then gives an objective-gap bound with known units.

Without a verified positive μ, the same formula is not a certificate. A small gradient can coexist with a large distance to the solution on a very flat objective. A small parameter update can instead reflect an excessively small step.

Avoid stopping solely because several printed objective values look identical. Check a suitable residual, the model's scale, the progress trend and an iteration or time limit.

Conditioning changes the path

Plain version. An elongated bowl has steep and flat directions. A step safe for the steep direction can make progress painfully slow along the flat one.

For a positive-definite quadratic,

$$f(x) = \frac{1}{2}x^\top Qx + c^\top x, \qquad \nabla f(x) = Qx + c.$$

Q is symmetric positive definite and c fixed. The Hessian is Q. Its smallest and largest eigenvalues supply μ and L.

The ratio $\kappa = L/\mu$ describes this Euclidean curvature conditioning. A large ratio means a single scalar step size must accommodate very different directional scales. Coordinate units can therefore change the behavior of gradient descent substantially.

Standardizing variables or choosing a suitable metric can improve a path, but must transform constraints, coefficients and stopping norms consistently. A rescaled algorithm is not judged fairly by an untransformed parameter-distance threshold.

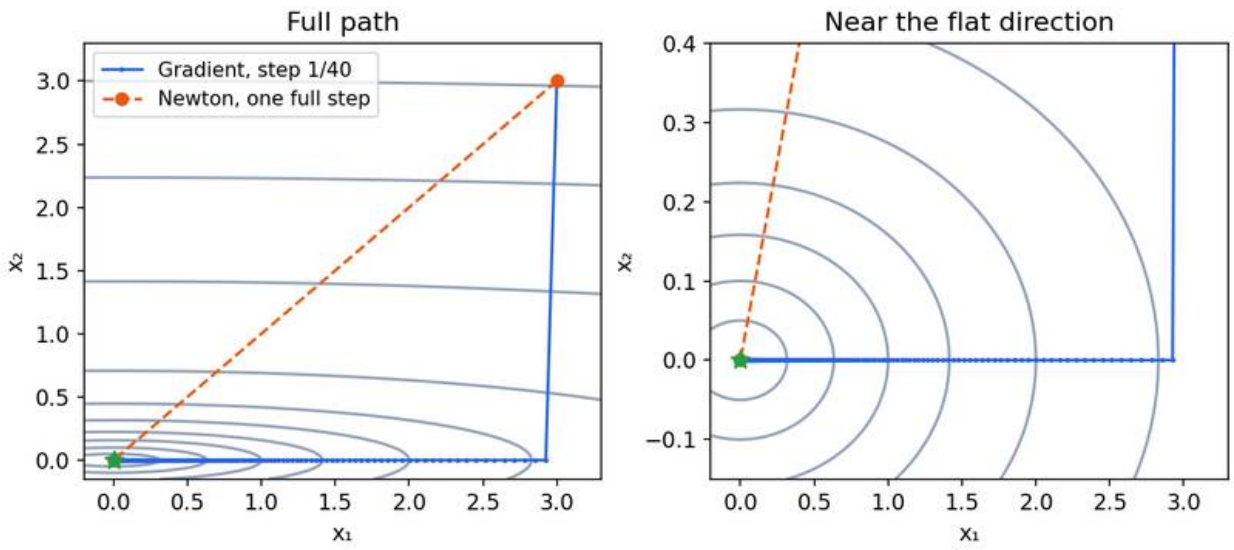


Figure 1 — Original analytical quadratic $f(x) = (x_1^2 + 40x_2^2)/2$. The gradient path uses step 1/40; Newton reaches the exact quadratic minimizer in one full step. This example isolates conditioning and does not compare total runtime on general problems.

Code: compare a fixed gradient step and a Newton step

The objective is minimized at zero. We compare objective decrease and verify the Newton linear system rather than using an explicit matrix inverse.

```
import numpy as np

Q = np.diag([1., 40.])
x = np.array([3., 3.])
start = x.copy()
values = [0.5*x @ Q @ x]
for _ in range(200):
    x = x - (Q @ x)/40
    values.append(0.5*x @ Q @ x)
direction = np.linalg.solve(Q, -(Q @ start))
newton = start + direction
assert np.all(np.diff(values) <= 1e-12)
assert np.allclose(Q @ direction + Q @ start, 0)
assert np.linalg.norm(newton) < 1e-12
print("gradient objective after 200 steps:", round(values[-1], 8))
print("Newton objective after one step:", 0.5*newton @ Q @ newton)
```

A one-step Newton solution is special to an exact positive-definite quadratic with an accurate linear solve. On a general smooth function, curvature changes as we move and a full step may fail.

Read the quadratic path one eigenvector at a time

A quadratic makes the effect of the step size exact. Let $e_k = x_k - x^*$ be the error for a positive-definite quadratic with curvature Q. Gradient descent gives $e_{k+1} = (I - \eta Q)e_k$. In an eigenvector of Q with eigenvalue q_i, the error coefficient is multiplied by $1 - \eta q_i$.

Here η is the fixed step, I the identity, and $q_i > 0$ the directional curvature. All coefficients contract precisely when $0 < \eta < 2/L$, where L is the largest eigenvalue. If $\eta > 2/L$, the steepest-direction error grows in magnitude even though the objective is convex.

For the chapter's $Q = \mathrm{diag}(1, 40)$ and $\eta = 1/40$, the steep coordinate disappears after one update, while the flat coordinate is multiplied by $39/40$ each time. Starting from $(3, 3)$ therefore leaves first-coordinate error $3(39/40)^k$ after k updates. The gradual path is a consequence of directional curvature, not random numerical failure.

For a quadratic with smallest eigenvalue μ, choosing $\eta = 2/(L+\mu)$ balances the magnitudes of the extreme contraction factors. Their worst value is $(L-\mu)/(L+\mu)$. This choice improves the bound relative to a conservative step, but relies on known spectral endpoints and does not replace a line search for arbitrary nonquadratic functions [1].

Armijo backtracking

Plain version. Start with a candidate distance and shrink it until the decrease is large enough relative to the local slope.

The sufficient-decrease condition is

$$f(x + td) \le f(x) + \alpha t \nabla f(x)^\top d.$$

d is a descent direction, $t > 0$ the tested distance, and $\alpha \in (0, 1)$ a fixed decrease fraction. Backtracking repeatedly multiplies t by $\beta \in (0, 1)$ until the domain and decrease tests pass [1, Section 9.2].

For Newton methods, a usual choice keeps $\alpha < 1/2$ so full steps are accepted near a regular optimum under the standard local analysis. This is an assumption-aware choice, not a universal optimal parameter.

An implementation needs a maximum number of trials, a minimum meaningful step, finite-value checks and a clear failure report. Otherwise an invalid direction or an incorrect gradient can produce an endless shrinking loop.

What a rejected step tells us

A rejection can mean the initial distance was too ambitious, the curvature model was inaccurate far from the solution, the domain was crossed or the derivatives were wrong. It does not by itself identify which reason applies.

Check the directional derivative first. If it is nonnegative, ordinary descent backtracking lacks its intended justification. A modified direction may be needed, or the candidate may already meet an optimality test.

Repeated rejection near an optimum can come from floating-point cancellation. Compare objective and derivative scales before demanding an improvement below representable precision. Termination should explain which numerical condition was met.

Steepest descent depends on a norm

Plain version. "Steepest" requires a definition of one unit of movement. Different norms choose different directions.

The normalized steepest direction solves

$$\min_{\|d\|\le 1} \nabla f(x)^\top d.$$

d is the direction, $\|\cdot\|$ the chosen movement norm, and x the current point. Under the Euclidean norm, a nonzero gradient gives direction $-\nabla f(x)/\|\nabla f(x)\|_2$.

For a positive-definite metric matrix P, define $\|d\|_P = \sqrt{d^\top P d}$. A corresponding unnormalized descent direction is

$$d = -P^{-1}\nabla f(x).$$

P determines directional cost. The directional derivative is $-\nabla f(x)^\top P^{-1}\nabla f(x)$, strictly negative for a nonzero gradient.

Choosing P resembles preconditioning: we account for known scale or curvature without necessarily forming the true Hessian. The usefulness depends on how cheaply the metric system can be solved and how well it represents the problem.

With a coordinate-based movement norm, steepest directions may select a coordinate or distribute movement differently. The book's treatment [1, Section 9.4] explains why steepest descent is a family rather than one coordinate-independent procedure.

Newton's local quadratic model

Plain version. Newton uses a quadratic approximation of the objective, then solves for the minimizer of that approximation.

At the current point, the model is

$$m(d) = f(x) + g^\top d + \frac{1}{2} d^\top H d, \qquad g = \nabla f(x),\ H = \nabla^2 f(x).$$

d is the displacement, g the gradient and H the Hessian. When H is positive definite, the model's unique minimizing displacement solves

$$Hd = -g.$$

This is the Newton direction. Compute it through a linear solve or factorization, not by forming an inverse. Positive definiteness ensures a descent direction whenever $g \neq 0$.

Convexity gives a positive-semidefinite Hessian, which may be singular. A full Newton step is not automatically defined. A regularized Hessian, restricted subspace or another direction can be appropriate, but each changes the stated numerical procedure.

The Newton decrement

Define

$$\lambda_N(x)^2 = g^\top H^{-1} g = -g^\top d.$$

λ_N is the Newton decrement and H is assumed positive definite. The quadratic model predicts a decrease of $\lambda_N^2/2$ at its minimizing step.

For an exact quadratic, that prediction equals the objective gap. For a general function, it is a local model quantity; a rigorous gap interpretation needs additional assumptions. Do not label the predicted decrease an unconditional global error certificate.

The direction and decrement transform coherently under an invertible affine change of coordinates when derivatives and solves are exact. This is one reason Newton behaves differently from Euclidean gradient descent. Floating-point conditioning and implementation still matter.

Local convergence is conditional

With suitable smoothness, a nonsingular positive-definite Hessian near the minimizer and an appropriate line search, Newton eventually accepts full steps and can converge quadratically [1, Section 9.5].

Quadratic convergence describes error shrinking approximately as the square of the previous error in a sufficiently small neighborhood. It does not imply that the first few steps from any initial point exhibit that behavior.

Errors in gradients, Hessians and linear solves can limit the rate. Far from the solution, safeguards may be essential. Near the solution, requesting a residual below derivative accuracy can waste work or trigger misleading failures.

Why Newton respects a change of coordinates

Newton adjusts direction using curvature, so changing variable units does not change its ideal physical step. Let $x = Tz + b$ with invertible matrix T and fixed offset b, and define $\tilde{f}(z) = f(Tz+b)$. At corresponding points, the chain rule gives

$$\tilde{g} = T^\top g, \qquad \widetilde{H} = T^\top H T.$$

Here g, H are the gradient and positive-definite Hessian in x coordinates, while tildes denote the same derivatives in z coordinates. The transformed Newton equation is

$$T^\top H T\,\Delta z = -T^\top g \quad \Longrightarrow \quad H(T\Delta z) = -g.$$

Here Δz is the displacement in new coordinates. Uniqueness of the solve gives $T\Delta z = \Delta x$, the original Newton displacement. Its predicted decrease is invariant too: $\tilde{g}^\top \widetilde{H}^{-1} \tilde{g} = g^\top H^{-1} g$.

Ordinary gradient descent instead maps its transformed step to $-\eta T T^\top g$, generally a different direction from $-\eta g$. This connects coordinate scaling to the metric in steepest descent. Newton's invariance assumes exact derivatives and solves; an ill-conditioned coordinate transformation can still harm a floating-point implementation [1, Section 9.5].

Self-concordance and controlled curvature

Plain version. Some convex functions control how quickly curvature changes through their own local curvature. This gives a special route to analyzing Newton steps.

For a standard self-concordant one-dimensional function,

$$|f'''(t)| \le 2\big(f''(t)\big)^{3/2}.$$

t is the scalar input and primes are derivatives, with nonnegative second derivative. In multiple dimensions, this condition is applied along lines in the domain [1, Section 9.6].

The logarithmic barrier $-\log t$ on $t > 0$ satisfies the scalar relation. This links unconstrained Newton analysis to the interior-point methods in the next chapter.

Self-concordance is more specific than ordinary smooth convexity. Its usefulness is that local norms can control changes without a single global Euclidean Hessian-Lipschitz constant. The theory includes precise decrement-based regions and complexity statements; those require the proper normalization and barrier assumptions.

A function containing logarithms is not automatically a valid barrier for every constraint representation. Its domain, boundary behavior and scaling must match the optimization problem. Distinguish a smooth likelihood term from a constraint barrier even if both contain a logarithm.

A stable convex logistic solver

Plain version. A complete implementation should evaluate the objective stably, verify its derivatives and stop using an optimality quantity.

For fixed rows $a_i^\top$ and binary outcomes $y_i \in \{0, 1\}$, use

$$f(w) = \frac{1}{N}\sum_{i=1}^{N}[\log(1+\exp(a_i^\top w)) - y_i a_i^\top w] + \frac{\rho}{2}\|w\|_2^2.$$

w is the parameter vector, N the observation count and $\rho > 0$ a regularization coefficient. Every coordinate is regularized in this example. The fixed-design logistic likelihood follows the convex-estimation reasoning in [1, Chapter 7].

Let $p_i = 1/(1+\exp(-a_i^\top w))$. The derivatives are

$$g = \frac{1}{N}A^\top(p-y) + \rho w, \qquad H = \frac{1}{N}A^\top \operatorname{diag}(p_i(1-p_i))A + \rho I.$$

A stacks the rows, p, y are observation vectors, and I is the identity. The positive ρI gives a valid global strong-convexity constant $\mu = \rho$.

Use a stable softplus evaluation instead of taking the logarithm of a potentially overflowed exponential. The following bounded synthetic scores permit a direct sigmoid; for unrestricted extreme scores, use a numerically stable sigmoid implementation as well.

Derive the logistic curvature bounds

A stable objective is also easier to optimize when its curvature constants are known. For one score $s = a_i^\top w$, the derivative of $\log(1+e^s)$ is p_i, and its second derivative is $p_i(1-p_i)$. Thus subtracting the linear term $y_i s$ gives gradient $(p_i - y_i)a_i$ and Hessian $p_i(1-p_i)a_i a_i^\top$.

Here a_i is a fixed feature vector, w the parameter vector, $y_i \in \{0,1\}$ the observed label and p_i the sigmoid probability. Average over observations and add the quadratic penalty to obtain the derivatives displayed above. Since $0 \le p_i(1-p_i) \le 1/4$,

$$\rho I \preceq \nabla^2 f(w) \preceq \left(\rho + \frac{\|A\|_2^2}{4N}\right) I.$$

Here f is the regularized average logistic objective, A the feature matrix, N the observation count, $\rho > 0$ the penalty, and $\|A\|_2$ its largest singular value. The lower bound follows from full-coordinate regularization; the upper bound follows by testing $v^\top A^\top \operatorname{diag}(p_i(1-p_i))Av$ for arbitrary direction v.

Consequently $\mu = \rho$ and $L = \rho + \|A\|_2^2/(4N)$ are valid global constants. They provide a safe gradient step and an objective-gap stopping bound for this specific model. If the intercept is left unpenalized, the same positive lower bound for every coordinate no longer follows. The mathematical formula, the penalty implementation and the stopping test must agree [1].

Code: Newton with derivative and stopping checks

This is an optimization demonstration, not a classification benchmark. It checks a directional derivative before solving and verifies the final gradient, decrease sequence and strong-convexity gap bound.

```
import numpy as np

rng = np.random.default_rng(16)
A = rng.normal(size=(160, 5))
truth = np.array([0.8, -0.5, 0.3, 0., 0.4])
prob = 1/(1+np.exp(-(A @ truth)))
y = (rng.random(len(A)) < prob).astype(float)
rho = 0.1

def evaluate(w):
    z = A @ w
    p = 1/(1+np.exp(-z))
    value = np.mean(np.logaddexp(0, z)-y*z) + rho*(w @ w)/2
    gradient = A.T @ (p-y)/len(y) + rho*w
    Hessian = (A.T * (p*(1-p))) @ A/len(y) + rho*np.eye(A.shape[1])
    return value, gradient, Hessian

w = np.zeros(A.shape[1])
direction = rng.normal(size=len(w))
eps = 1e-5
finite_difference = (evaluate(w+eps*direction)[0] -
                     evaluate(w-eps*direction)[0])/(2*eps)
assert np.isclose(finite_difference, evaluate(w)[1] @ direction,
                  atol=1e-8)
history = []
for iteration in range(30):
    value, g, H = evaluate(w)
    history.append(value)
    if np.linalg.norm(g) < 1e-9:
        break
    step_direction = np.linalg.solve(H, -g)
    slope = g @ step_direction
    assert slope < 0
    step = 1.0
    for trial in range(50):
        candidate = w + step*step_direction
        candidate_value = evaluate(candidate)[0]
        if np.isfinite(candidate_value) and candidate_value <= value+0.1*step*slope:
            w = candidate
            break
        step *= 0.5
    else:
        raise RuntimeError("backtracking did not accept a step")
else:
    raise RuntimeError("iteration limit reached")

value, g, H = evaluate(w)
assert np.linalg.norm(g) < 1e-9
assert np.all(np.diff(history) <= 1e-12)
assert np.linalg.eigvalsh(H).min() >= rho-1e-12
print("accepted Newton steps:", len(history)-1)
print("objective:", round(value, 8))
print("gradient norm:", np.linalg.norm(g))
```

```
print("objective-gap upper bound:", (g @ g)/(2*rho))
```

The derivative check tests one direction and one point; it cannot prove every derivative correct. Add checks at representative nonzero points when changing the model. Hessian symmetry, eigenvalue expectations and solve residuals provide additional independent evidence.

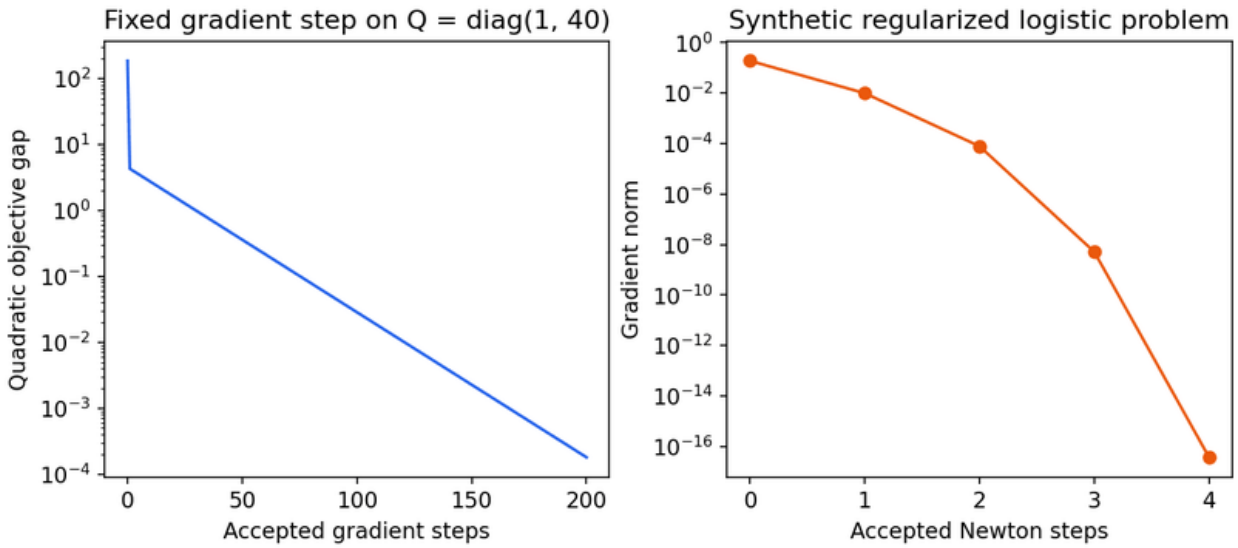


Figure 2 — Original computed convergence traces. The left panel has a known zero quadratic optimum; the right panel shows a gradient norm for a synthetic logistic objective. Accepted steps are not a runtime or cross-problem benchmark.

Implementation and stopping reports

Plain version. A solver result should describe why it stopped, what it verified and what computational work it performed.

Dense Newton stores a matrix whose size grows quadratically with the number of variables. Factorization can dominate the cost. Gradient methods often need only matrix-vector products, making many cheap iterations attractive for large problems.

Sparse matrices, low-rank terms and block structure can alter that comparison substantially. Exploit structure only after checking that the transformed system remains equivalent and that factorization fill-in is manageable [1, Section 9.7 and Appendix C].

A Hessian-vector product can support an iterative linear solve without explicitly storing the Hessian. Then the inner solve has its own residual and tolerance. A loosely solved Newton system need not reproduce the exact Newton direction or convergence rate.

Report objective values, a suitable gradient or stationarity residual, domain checks, iteration count

and the stated stopping condition. Where timing matters, include derivative evaluations and line-search trials rather than only accepted steps.

Scaling the objective changes gradient magnitudes. A fixed absolute gradient tolerance is therefore not comparable across arbitrary loss normalization. Relative criteria need a documented scale and should remain meaningful near zero.

An iteration limit is a resource stop, not an optimality certificate. A line-search failure is an algorithmic diagnosis, not proof of nonexistence. Retain the final candidate and residual evidence so that the result can be assessed.

Exact quadratic line searches reveal a tradeoff

For the positive-definite quadratic above and a nonzero descent direction d, the objective along the line is a scalar quadratic. Differentiating with respect to distance gives

$$t^* = -\frac{g^\top d}{d^\top Q d}.$$

$g = Qx + c$ is the current gradient, Q the positive-definite curvature and $t^* > 0$ the minimizing line distance. This formula is an exact line search for this particular quadratic, not a general smooth objective.

For the negative gradient direction, the numerator becomes $g^\top g$ and the denominator is $g^\top Q g$. The chosen distance accounts for curvature along that direction. It can outperform a conservative fixed step without correcting the underlying elongated geometry.

Successive directions can still zigzag. Exact minimization along one line does not imply that the line points directly toward the global minimizer. This makes the roles of direction and distance particularly clear: better distance selection cannot fully replace better directional geometry.

A Newton direction uses the whole quadratic curvature system, reaching the minimizer with distance one in exact arithmetic. A metric approximation may reduce the system cost while giving a less accurate route. The fair comparison includes how the metric is built and reused, not just the length of the visible path.

If a matrix factorization is reusable across many right-hand sides, its initial cost can be amortized. If the matrix changes at each iteration, that advantage may shrink. Likewise, an objective evaluation can be cheap in a tiny tutorial but expensive in an application with a simulation inside it. State the actual computational primitives before making a speed claim.

Derivative checks need a scale

A central finite difference divides the difference of two nearby objective values by twice the perturbation. Making the perturbation smaller reduces the ideal truncation error at first, but eventually amplifies floating-point cancellation.

Use several perturbation sizes and compare relative as well as absolute discrepancies. Scale the direction sensibly, choose test points away from a domain boundary, and inspect whether the

result stabilizes over a useful interval. One failed tiny-step comparison may indicate numerical resolution rather than an incorrect derivative.

For a Hessian, compare a gradient difference with a Hessian-vector product. This tests a different layer of the implementation from checking objective differences alone. Also verify symmetry when the objective is twice continuously differentiable and the intended Hessian should be symmetric.

An automatic derivative system can remove much manual algebra but still differentiate the implemented expression rather than the intended model. Incorrect broadcasting, a misplaced reduction or an accidentally regularized intercept can yield a perfectly consistent derivative of the wrong objective.

Record these checks beside the formulation. The goal is to connect symbols to arrays, not merely to produce a green test. Once the implementation matches the model, numerical tolerances and algorithm assumptions can be assessed meaningfully.

A reproducible comparison also needs a starting point, objective normalization, stopping rule and numerical precision. Changing any of these can alter apparent convergence. Store the parameters and random seed with a synthetic experiment so that a later figure can be regenerated from the same stated problem.

For deterministic algorithms, small platform differences may still affect the final few digits or a marginal line-search comparison. Verify properties with justified tolerances instead of asserting one exact decimal trajectory. The examples here check descent, linear-system consistency and gradient evidence, which are the quantities tied to the formulation.

Modern view

Plain version. Modern convex algorithms trade curvature information, projections, proximal operations and communication costs. The right choice depends on structure and required accuracy.

Dvurechensky, Shtern and Staudigl's first-order survey [2] provides a broad foundation, including non-Euclidean proximal methods, acceleration and projection-free approaches. It was published in 2021; it is a background map rather than a new 2026 result.

Condat and colleagues' 2023 SIAM Review survey [3] unifies proximal splitting for high-dimensional nonsmooth convex problems through primal–dual product spaces and chosen metrics. It explains why adding an absolute-value penalty changes the appropriate primitive: an ordinary smooth Newton formula cannot simply differentiate through the kink.

Pokutta's 2024 Frank–Wolfe survey [4] focuses on linear minimization over constrained sets and its convergence and certificate structure. It complements the gradient/Newton comparison by asking whether a projection or expensive system solve can be avoided.

Together these reviews give a practical taxonomy: smooth versus nonsmooth objectives, inexpensive versus expensive feasible-set operations, Euclidean versus structured geometry, and local

curvature versus cheap first-order steps. Recent variants do not remove those modeling distinctions.

Current issues include adaptive methods with reliable certificates, precision limits in large linear systems, communication in distributed problems and differentiating through solver outputs. Match the algorithm to the model, the hardware and the requested error measure; compare total work at a comparable accuracy.

Key takeaways

- A descent direction and a step length are separate choices.
- Fixed gradient steps require a valid curvature and domain argument.
- Poor conditioning slows a scalar-step gradient method.
- Backtracking needs safeguards and correct derivative signs.
- Newton uses a linear solve and has conditional local convergence.
- A decrement is a model quantity unless its gap assumptions are established.
- Reliable reports include stopping evidence and the cost of rejected work.

Exercises

1. Derive the smoothness decrease inequality for a step of size $1/L$.

   Hint

   Substitute the negative scaled gradient into the upper quadratic model.

2. Replace the quadratic eigenvalue 40 by 400. Predict and test how fixed-step gradient progress changes.

   Hint

   The flat-direction contraction depends on the eigenvalue ratio.

3. Implement backtracking for $-\log x + x^2$ on $x > 0$, checking the domain before every objective evaluation.

   Hint

   Reject nonpositive candidates immediately.

4. Verify the logistic gradient in three random directions at a nonzero parameter vector.

   Hint

   Compare central differences and inner products using several finite-difference scales.

5. Show why a singular positive-semidefinite Hessian does not always define a unique Newton direction.

Hint

Inspect the linear system's nullspace and consistency.

6. Rescale the logistic objective by one hundred. Determine which stopping quantity and tolerance changes preserve the original requested objective accuracy.

Hint

Transform the gradient, strong-convexity constant and desired gap together.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 9, Sections 9.1–9.7; Appendix C for linear systems; Chapter 7 for the logistic likelihood connection.** Official book, free PDF.
2. Pavel Dvurechensky, Shimrit Shtern and Mathias Staudigl, "First-Order Methods for Convex Optimization," *EURO Journal on Computational Optimization*, 9, 100015, 2021. Verified open manuscript, published article.
3. Laurent Condat, Daichi Kitahara, Andrés Contreras and Akira Hirabayashi, "Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists," *SIAM Review*, 65(2), 375–435, 2023. Published survey, open manuscript.
4. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.

OPT 07 • Equality Constraints and Interior-Point Methods

Solve constrained Newton systems, follow the logarithmic central path, distinguish feasibility from strict feasibility, and check primal–dual certificates.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-07-constrained-interior-point/

What you'll learn

- Extend Newton's method to a convex objective with affine equalities.
- Derive a logarithmic barrier, its central path and the associated duality gap.
- Implement a small feasible barrier solver with an explicit line search.
- Distinguish feasible, strictly feasible and infeasible constraint systems.
- Interpret primal–dual residuals, complementarity and infeasibility certificates.
- Compare modern interior-point and first-order solver designs using their numerical demands.

The big picture

Constraints describe what a solution is allowed to do. A solver cannot ignore them while improving the objective. Equality constraints restrict every permitted step to a lower-dimensional space; inequality constraints create boundaries the solver must handle. Interior-point methods approach useful boundary solutions through a sequence of problems whose iterates remain inside those boundaries.

> **Explain like I'm five**
>
> Imagine finding the lowest place in a fenced garden. You must stay in the garden, and a painted line might require you to walk along one particular path. Start away from the fence. An imaginary cushion pushes you back whenever you get too close. Gradually make the cushion weaker, so you can approach the low point beside the fence without stepping outside.

The reading is Boyd and Vandenberghe, Chapters 10–11 [1]. Chapter 10 treats equality-constrained minimization; Chapter 11 develops barriers, the central path, initialization and primal–dual methods. Read them after gradient and Newton methods and return to duality and optimality when a certificate seems mysterious. Bubeck's algorithmic monograph [2] supplies a complementary view of complexity and solver families.

This chapter uses synthetic low-dimensional problems so you can check every claim independently. The code is a teaching implementation, with assertions and transparent assumptions. A production conic solver adds scaling, sparse factorizations, numerical regularization, termination policies and infeasibility handling. Understanding the small problem helps explain those engineering choices.

1. Equality constraints change the permitted directions

An affine equality must remain satisfied while the objective decreases. If the current point satisfies $Ax = b$, a step Δx preserves feasibility exactly when $A\Delta x = 0$. This gives a geometric explanation before any matrix derivation: permitted steps lie in the nullspace of the equality matrix.

Consider the convex problem

$$\begin{aligned} \text{minimize}_x \quad & f(x) \\ \text{subject to} \quad & Ax = b. \end{aligned}$$

Here $x \in \mathbb{R}^n$ is the variable; f is a twice-differentiable convex objective; $A \in \mathbb{R}^{p\times n}$ is fixed; and $b \in \mathbb{R}^p$ is fixed. We initially assume A has independent rows and the equality system has a solution. The Lagrange multiplier $\nu \in \mathbb{R}^p$ accounts for the equality restriction.

At a feasible iterate, minimize the quadratic approximation along feasible directions. The resulting Newton system is

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = - \begin{bmatrix} \nabla f(x) \\ 0 \end{bmatrix}.$$

Here $H = \nabla^2 f(x)$ is the Hessian; Δx is the proposed primal step; w is the equality multiplier of the local quadratic subproblem; and the zero block has size $p \times p$. The second block equation enforces $A\Delta x = 0$. The multiplier w solves the local model; it is not automatically the increment of a separately maintained multiplier estimate.

The Hessian need only have positive curvature along nonzero feasible directions to make the constrained quadratic model well behaved. A direction excluded by the equalities need not have positive curvature. This distinction explains why a full Hessian can be singular while the constrained minimizer is unique. Check the actual feasible subspace before diagnosing singularity as nonuniqueness.

The block matrix is a Karush–Kuhn–Tucker, or KKT, matrix [1]. It is generally indefinite even when H is positive definite. Do not apply a positive-definite factorization to the entire matrix merely because the original objective is convex. Its structure reflects simultaneous primal and dual equations. For a small dense example, a general linear solve is adequate.

An alternative eliminates equalities. Choose one feasible point x_0 and a matrix F whose columns span the nullspace of A. Write $x = x_0 + Fz$, where $z \in \mathbb{R}^{n-p}$ is the reduced variable. Then solve the unconstrained problem in z. Elimination can simplify a tiny problem but destroy sparsity in a large one. Both formulations express the same geometry; the computational representation determines the cost.

Reduced curvature and block elimination

A singular Hessian need not make the constrained step ambiguous. Minimize $f(x) = x_1^2/2$ under $x_2 = 3$. The Hessian is $\operatorname{diag}(1, 0)$, but the only feasible directions change x_1. The unique minimizer is $(0, 3)$: the equality removes precisely the flat direction.

More generally, let F contain independent columns spanning the nullspace of A. The reduced Hessian is $F^\top HF$. Positive definiteness of this smaller matrix gives a unique local quadratic minimizer along feasible directions, even when H is singular. Here H is the current Hessian and A the full-row-rank equality matrix [1, Chapter 10].

If H itself is positive definite, the KKT step can also be derived by eliminating the primal direction. Write $g = \nabla f(x)$ and use the first block equation:

$$\begin{aligned} \Delta x &= -H^{-1}(g + A^\top w), \\ (AH^{-1}A^\top)w &= -AH^{-1}g. \end{aligned}$$

Here Δx is the step and w the local equality multiplier. The second line follows by substituting the first into $A\Delta x = 0$. The smaller matrix is positive definite when A has independent rows: for nonzero v, its quadratic form is $(A^\top v)^\top H^{-1}(A^\top v) > 0$.

These equations explain the block solve, but the inverse notation is algebraic. Compute products through solves with H and compare the cost of this elimination with a direct sparse KKT factorization. In the singular example above, the nullspace formulation remains valid while this full-Hessian elimination does not.

2. A constrained quadratic with an inspectable certificate

A quadratic objective makes the KKT equations exact rather than approximate. Solving the block system produces a candidate; checking equality feasibility and stationarity determines whether the candidate is an optimality certificate for this convex problem.

For

$$f(x) = \tfrac{1}{2}x^\top Qx + c^\top x,$$

$Q \in \mathbb{R}^{n\times n}$ is a symmetric positive-semidefinite matrix, $c \in \mathbb{R}^n$ is fixed, and x is the variable. An optimal pair (x^*, ν^*) satisfies $Qx^* + c + A^\top \nu^* = 0$ and $Ax^* = b$. The superscript star denotes an optimal value. These equations are sufficient for a feasible convex quadratic; uniqueness requires additional curvature on feasible directions.

```
import numpy as np

Q = np.diag([2.0, 4.0])
c = np.array([-2.0, -1.0])
A = np.array([[1.0, 1.0]])
b = np.array([1.0])
K = np.block([[Q, A.T], [A, np.zeros((1, 1))]])
solution = np.linalg.solve(K, np.concatenate([-c, b]))
x, nu = solution[:2], solution[2:]
primal = A @ x - b
dual = Q @ x + c + A.T @ nu
print("Primal solution:", x)
print("Equality multiplier:", nu)
print("Residual norms:", np.linalg.norm(primal), np.linalg.norm(dual))
```

```
assert np.allclose(x, [5/6, 1/6])
assert np.linalg.norm(primal) < 1e-12
assert np.linalg.norm(dual) < 1e-12
```

The coordinates sum to one. The unconstrained optimum would have coordinates one and one quarter, so it would violate the total constraint. The multiplier adjusts the stationarity equation to account for that restriction. A small objective value alone would not establish this adjustment; the residual checks make it explicit.

Residuals depend on units. Multiplying the equality by a large constant leaves the feasible set unchanged while changing the raw primal residual and multiplier scale. Report the units and scaling of the original constraints, and compare residuals against meaningful tolerances. A solver's internal normalized stopping test should be translated back into the application's measurements.

Do not form a matrix inverse to compute this solution. A linear solve is more direct and usually more stable. In a repeated problem, factorization reuse can reduce work, but only when the matrix structure permits it. Updating a linear term is different from updating the Hessian or the constraint matrix; distinguish these changes when planning warm starts and caching.

For nonlinear objectives, recompute the local curvature and use a feasible line search. A full Newton step can leave the objective's domain even if it preserves the equalities. The domain is an additional restriction: a logarithm or a positive-definite matrix variable is not defined everywhere. The equality equations alone do not protect it.

3. Inequalities become a smooth interior problem

A boundary can be represented by a barrier that grows without bound as the iterate approaches it. The barrier is finite only at strictly feasible points. This lets a smooth constrained Newton method handle the inequality through the objective, while retaining explicit affine equalities.

Consider convex inequalities $f_i(x) \le 0$ for $i = 1, \dots, m$, with affine equalities $Ax = b$. Define

$$\phi(x) = -\sum_{i=1}^{m} \log(-f_i(x)), \qquad F_t(x) = t f_0(x) + \phi(x).$$

Here f_0 is the original convex objective; each f_i is a convex inequality function; m is the number of scalar inequalities; ϕ is the logarithmic barrier; and $t > 0$ controls the relative importance of the objective. The domain requires every $f_i(x) < 0$. The barrier problem minimizes F_t subject to $Ax = b$.

Constraint functions should be expressed consistently in chosen units, or normalized by a positive reference scale before taking a logarithm. Positive rescaling changes the barrier by an additive constant and does not change its minimizer, although finite-precision calculations can still be affected. The mathematical invariance is not a reason to neglect numerical scaling.

For a twice-differentiable inequality, its barrier contribution has gradient $\nabla f_i(x)/(-f_i(x))$ and Hessian

$$\frac{\nabla f_i(x)\nabla f_i(x)^\top}{f_i(x)^2} + \frac{\nabla^2 f_i(x)}{-f_i(x)}.$$

Here ∇f_i is the gradient column vector and $\nabla^2 f_i$ is its Hessian. Both terms are positive semidefinite in the strictly feasible domain because f_i is convex. The expression also shows rapidly increasing curvature near an active boundary, where $-f_i(x)$ becomes small.

An inequality representation affects the path. Repeating an identical constraint repeats its barrier contribution, changing the approximate problem for finite t. The exact feasible set remains unchanged. This illustrates an important separation: equivalent constraint sets need not produce identical intermediate iterates. Canonicalization and scaling are algorithmic decisions with observable effects.

The barrier cannot begin on the boundary. If a nonnegative variable starts at zero, $-\log x$ is undefined. Use a strictly positive initialization or reformulate a variable that is forced to zero by equalities. Adding an arbitrary positive offset to a constraint changes the optimization problem and must not be mistaken for an innocuous implementation fix.

4. The central path links interior points to dual bounds

For each barrier weight, solve the interior problem accurately. The resulting minimizers form the central path. Increasing the weight weakens the barrier relative to the original objective, allowing the solution to approach a constrained optimum on the boundary.

Let $x(t)$ minimize F_t subject to $Ax = b$. Define

$$\lambda_i(t) = \frac{1}{t[-f_i(x(t))]}, \qquad \nu(t) = \frac{w(t)}{t}.$$

Here $\lambda_i(t) > 0$ is the inequality multiplier; $w(t)$ is the equality multiplier for the scaled barrier problem; and $\nu(t)$ is the multiplier in the original objective's scale. The variables t, f_i and $x(t)$ retain their definitions above.

Barrier stationarity becomes original Lagrangian stationarity, while complementarity becomes $\lambda_i(t)[-f_i(x(t))] = 1/t$. Each scalar inequality contributes the same positive product. At a true optimum, complementary slackness requires those products to vanish; the barrier path approaches that condition as t increases.

For an exactly centered point satisfying the stated assumptions, the associated primal–dual gap is

$$f_0(x(t)) - g(\lambda(t), \nu(t)) = \frac{m}{t}.$$

Here g is the Lagrange dual function, a lower bound on the optimal objective for dual-feasible multipliers. The gap bounds objective suboptimality because the primal value is an upper bound and the dual value is a lower bound. This is a statement about the formulated convex problem, not a guarantee that the formulation matches the application.

The equality requires exact centering and a valid dual construction. A numerical iterate with a large stationarity residual cannot claim an exact gap merely by printing m/t. Use the theoretical quantity to plan the barrier schedule, then check the actual residuals and, where available, evaluate a genuine dual lower bound. This prevents an outer-loop counter from masquerading as a certificate.

An outer barrier method therefore alternates two activities: solve a fixed-t interior problem, then increase t. Reuse the previous solution as the next starting point. The next problem usually has a nearby minimizer, but very large changes in t can make centering expensive. Choose the increase based on measured centering behavior rather than assuming that fewer outer iterations always mean less computation.

Where the gap m/t actually comes from

The central-path gap is a sum of explicit prices times explicit slacks. At an exactly centered point, divide barrier stationarity by t:

$$\nabla f_0(x(t)) + \sum_i \frac{\nabla f_i(x(t))}{t[-f_i(x(t))]} + A^\top \frac{w(t)}{t} = 0.$$

Here $x(t)$ minimizes the barrier problem, $w(t)$ is its equality multiplier, f_0,f_i are the original objective and inequalities, and $t > 0$ is the barrier weight. Substituting the defined $\lambda_i(t) = 1/[t(-f_i(x(t)))]$ and $\nu(t) = w(t)/t$ makes this exactly the original Lagrangian's stationarity equation.

The Lagrangian is convex in x because the original functions are convex and the inequality multipliers are nonnegative. Stationarity therefore makes $x(t)$ its global minimizer, so evaluating L at this point really computes the dual function. Equality feasibility removes the equality term, giving

$$g(\lambda(t), \nu(t)) = f_0(x(t)) + \sum_i \lambda_i(t) f_i(x(t)) = f_0(x(t)) - \frac{m}{t}.$$

Here g is the dual function and m the scalar inequality count. The gap follows by subtracting this lower bound from the feasible primal objective. The central point is usually not the original optimizer: it trades objective improvement against positive slack. Its multipliers already supply a certificate whose error decreases as the barrier weakens [1, Section 11.2].

For a desired theoretical gap at most $\varepsilon > 0$, choose $t \geq m/\varepsilon$ and then solve the centering problem accurately enough. The choice of t supplies a target; the stationarity solve is what justifies using the formula. Increasing t without centering omits the key equality in the proof.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

5. A complete one-dimensional barrier experiment

Use a constrained quadratic whose true optimum is easy to check. We minimize $\frac{1}{2}(x-2)^2$ subject to $0 \le x \le 1$. The unconstrained minimizer is two; the closest permitted point is one, with objective one half. A barrier solution stays strictly between zero and one and approaches the upper endpoint.

For this example,

$$F_t(x) = \frac{t}{2}(x-2)^2 - \log x - \log(1-x).$$

Here $x \in (0,1)$ is the scalar variable and $t > 0$ is the objective weight. The first and second derivatives are $F_t'(x) = t(x-2) - 1/x + 1/(1-x)$ and $F_t''(x) = t + 1/x^2 + 1/(1-x)^2$. Every denominator is positive in the barrier domain.

```
import numpy as np

def center(t, x):
    def value(z):
        return 0.5*t*(z-2)**2 - np.log(z) - np.log1p(-z)
    for _ in range(100):
        grad = t*(x-2) - 1/x + 1/(1-x)
        hess = t + 1/x**2 + 1/(1-x)**2
        step = -grad/hess
        if grad*grad/hess < 1e-16:
            break
        alpha = 1.0
        for _ in range(100):
            trial = x + alpha*step
            if 0 < trial < 1 and value(trial) <= value(x) + 0.01*alpha*grad*step:
                break
            alpha *= 0.5
        else:
            raise RuntimeError("Line search failed")
        x = trial
    else:
        raise RuntimeError("Centering iteration limit reached")
    return x
x = 0.5
for t in [1., 4., 16., 64., 256., 1024.]:
    x = center(t, x)
    lower, upper = 1/(t*x), 1/(t*(1-x))
    stationarity = x-2-lower+upper
    primal = 0.5*(x-2)**2
    dual = 2-upper-0.5*(-2-lower+upper)**2
    gap = primal-dual
    print(int(t), round(x, 6), round(gap, 7), round(2/t, 7))
    assert 0 < x < 1
    assert abs(stationarity) < 1e-5
    assert abs(gap-2/t) < 1e-8
    assert primal-0.5 <= gap + 1e-8
```

The feasible backtracking check runs before the objective comparison. This prevents logarithms from being evaluated outside their domain. The sufficient-decrease test uses the directional derivative, and the Newton decrement supplies the centering test. Maximum iteration counts turn an unexpected failure into an explicit error instead of an infinite loop.

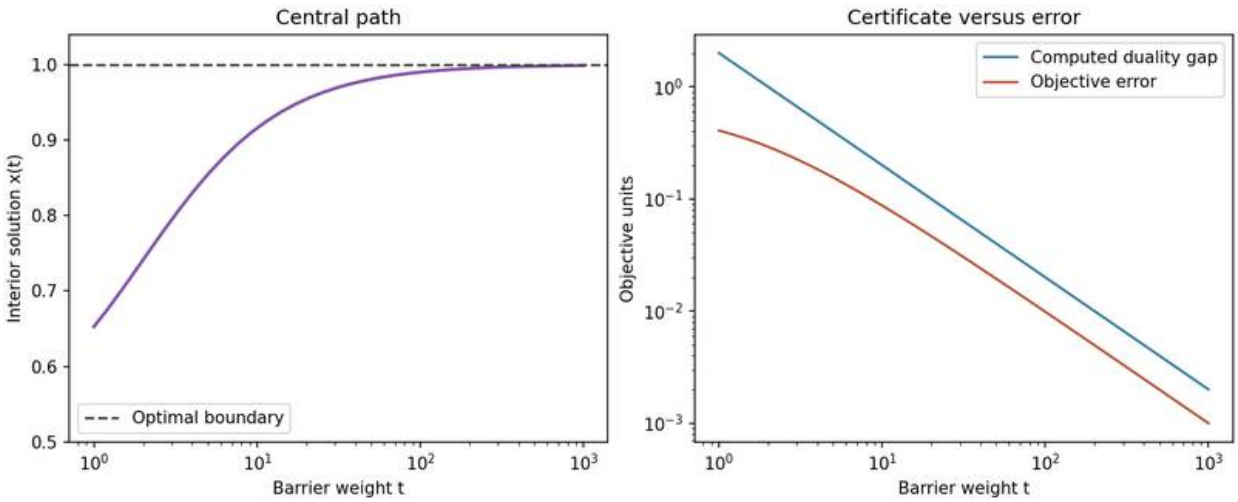


Figure 1 — A synthetic constrained quadratic makes the central-path certificate visible: the solution approaches one from the interior and the computed dual gap bounds its objective error.

The experiment computes a genuine dual value using the unconstrained minimum of the scalar Lagrangian. It therefore checks the gap separately from the prescribed $2/t$. In a larger nonlinear problem, evaluating the dual function may itself require another optimization. A primal–dual residual test is then useful, but describe what it certifies and under which regularity assumptions.

Near the endpoint, `1 - x` becomes small and the barrier Hessian becomes large. A smaller error target can make the numerical system harder even though the mathematical problem is unchanged. Stop when the solution is accurate enough in meaningful units. Demanding every last representable digit can waste work or expose conditioning that the application never needed to resolve.

6. Feasibility, strict feasibility and Phase I

A feasible solution may exist without any strictly feasible solution. A barrier for the original inequality representation then has no domain. This is different from infeasibility, and a solver must distinguish the two cases before trying to follow a central path.

A simple Phase I construction introduces one scalar s and solves

$$\begin{aligned} \text{minimize}_{x,s} \quad & s \\ \text{subject to} \quad & f_i(x) \le s \quad (i = 1, \dots, m), \\ & Ax = b. \end{aligned}$$

Here s bounds the largest inequality violation and is an additional variable. A point with $s < 0$ is strictly feasible for the original inequalities. A certified positive lower bound on the optimal s

proves infeasibility. An optimal value zero can mean boundary-only feasibility; nonattainment and numerical uncertainty require separate attention. Finite iteration failure does not itself prove any of these outcomes.

For the contradictory system $x \leq 0$ and $x \geq 1$, the violation measure is $\max(x, 1-x)$. Its minimum is one half at $x = 1/2$. Equal nonnegative weights on the two inequalities also produce an explicit infeasibility certificate.

```
import numpy as np

A = np.array([[1.0], [-1.0]])
b = np.array([0.0, -1.0])
y = np.array([0.5, 0.5])
print("A transpose y:", A.T @ y)
print("b transpose y:", b @ y)
assert np.all(y >= 0)
assert np.allclose(A.T @ y, 0)
assert b @ y < 0
# If A x <= b existed, multiplying by y would imply 0 <= b@y < 0.
grid = np.linspace(-1, 2, 301)
violation = np.maximum(grid, 1-grid)
assert np.isclose(violation.min(), 0.5)
# A different system is feasible only on its boundary.
A_boundary = np.array([[1.0], [-1.0]])
b_boundary = np.zeros(2)
assert np.all(A_boundary @ np.array([0.0]) <= b_boundary)
```

The grid is an illustration of the already derived minimum, rather than a proof that an arbitrary optimization is globally solved. The certificate is the decisive argument: if $Ax \leq b$, multiplying by $y \geq 0$ would imply $0 = y^\top Ax \leq y^\top b < 0$, a contradiction. The certificate checks data and multipliers directly.

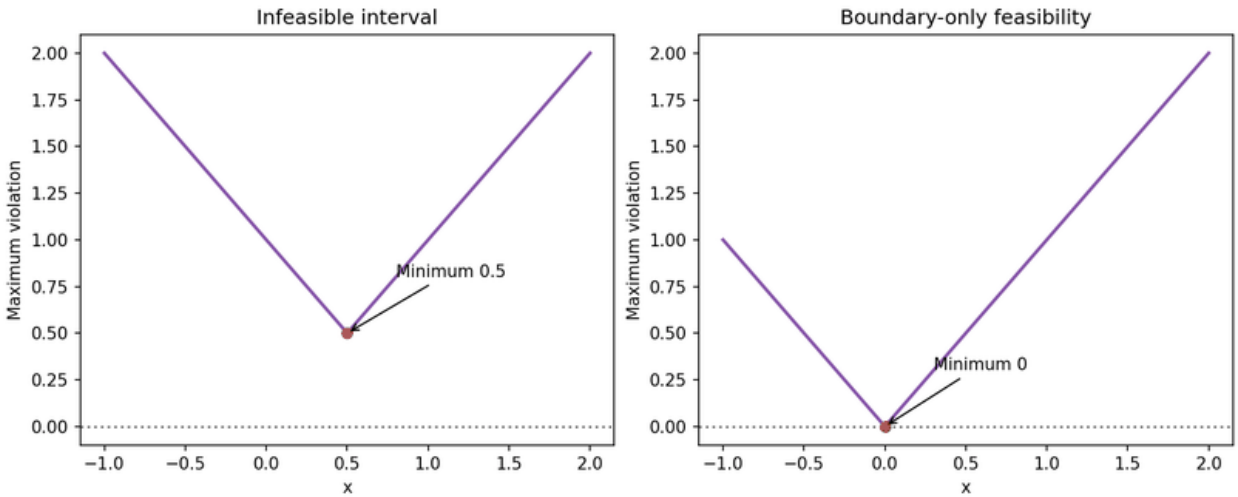


Figure 2 — Positive minimum violation indicates the contradictory interval is infeasible; zero minimum in

the second example identifies a feasible point with no strict interior.

For the boundary-only system $x \leq 0$ and $-x \leq 0$, the only feasible point is zero. Eliminate the forced variable or represent the condition as an equality. Merely nudging the starting point fails because one inequality becomes violated whichever direction you move. The relevant interior is the relative interior after the forced equality is accounted for.

Phase I also depends on the domains of the f_i. Relaxing their inequality values does not automatically make logarithms, matrix inverses or other expressions defined. Initialize inside the function domains or choose an appropriate reformulation. Inspect domain feasibility separately from the affine and inequality residuals.

Phase I has normalized dual prices

The maximum-violation formulation connects initialization to dual certificates. Attach multipliers $\lambda_i \geq 0$ to $f_i(x) - s \leq 0$ and multiplier ν to $Ax - b = 0$. Its Lagrangian is

$$L_I(x, s, \lambda, \nu) = \left(1 - \sum_i \lambda_i\right) s + \sum_i \lambda_i f_i(x) + \nu^\top (Ax - b).$$

Here x, s are Phase I variables, λ, ν its multipliers, and f_i, A, b the original restriction data. Since s is unrestricted, its infimum is finite only when $\sum_i \lambda_i = 1$. The inequality prices therefore form normalized nonnegative weights. A finite lower bound on the remaining weighted function bounds the largest original violation [1, Section 11.4].

For $f_1(x) = x$ and $f_2(x) = 1 - x$, choose $\lambda_1 = \lambda_2 = 1/2$. Their weighted sum is exactly $1/2$ for every x. Thus the Phase I optimum is at least $1/2$; choosing $x = 1/2, s = 1/2$ attains it. This supplies matching primal and dual values, proving that the original inequalities are infeasible.

For the boundary-only pair $f_1(x) = x, f_2(x) = -x$, the same normalized sum is zero. The feasible Phase I point $x = 0, s = 0$ matches it, but no negative s is possible. Zero violation can therefore prove ordinary feasibility without providing the strict interior needed by a logarithmic barrier. Eliminating the forced equality is the appropriate next modeling step.

7. Primal–dual steps and reliable stopping

A primal–dual method works directly with approximate optimality equations rather than completely solving each separate barrier problem. It maintains positive inequality multipliers and positive slacks, then solves a linearized system to reduce stationarity, feasibility and complementarity errors together.

Write the slack of inequality i as $s_i = -f_i(x) > 0$ and its multiplier as $\lambda_i > 0$. For the original problem, the residuals include

$$
\begin{aligned}
r_{\text{dual}} &= \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu, \\
r_{\text{pri}} &= Ax - b, \\
\eta &= \sum_i \lambda_i s_i.
\end{aligned}
$$

Here r_{dual} is the stationarity residual, r_{pri} is the equality residual, and η is the sum of complementarity products. The functions, matrix, multipliers and slacks have been defined above. For affine inequalities, separate slack equations can be included explicitly; nonlinear formulations may represent them through $f_i(x)$.

A small η with a large feasibility residual is not a valid approximate solution. Nor does a small gradient in unconstrained coordinates establish constrained optimality. Require all relevant residuals, multiplier signs and domain conditions to pass. When the multipliers and primal point are feasible and stationarity holds, the complementarity sum corresponds to the duality gap; otherwise that interpretation needs qualification.

A fraction-to-boundary line search limits the step so the slacks and multipliers remain positive. It is a numerical safeguard for the algorithm's domain, not a separate solution constraint. An infeasible-start method can initially violate affine equalities while still respecting the interior conditions needed by its formulas. Its merit function must reduce residuals, rather than simply evaluate the original objective at an infeasible point.

Sparse linear algebra often dominates the solve. The pattern of nonzero entries determines factorization fill, memory use and whether reuse is possible. A small iteration count can coexist with expensive iterations. Record factorization time and peak memory alongside iteration count when comparing methods. A dense educational implementation hides these costs and should not be used to predict large-problem performance.

Check the solution in the original formulation after canonicalization. A solver may introduce auxiliary variables and cones, normalize data, or remove redundant rows. These operations help computation, but the final application needs original-unit constraints and an interpretable objective. Record solver status, residuals, tolerances, formulation version and any postprocessing that modifies the returned point.

Linearize the primal–dual equations

A primal–dual Newton step targets the optimality equations jointly. For a quadratic objective with $Q \succeq 0$, affine inequalities $Gx + s = h$, and equalities $Ax = b$, use positive slack vector s and positive multiplier vector λ. Let c be the objective's linear coefficient and ν the equality multiplier.

Define $r_d = Qx + c + G^\top \lambda + A^\top \nu$, $r_g = Gx + s - h$, and $r_a = Ax - b$. With a chosen complementarity target $\tau > 0$, linearization gives

$$
\begin{aligned}
Q\Delta x + G^{\top}\Delta\lambda + A^{\top}\Delta\nu &= -r_d,\\
G\Delta x + \Delta s &= -r_g,\\
A\Delta x &= -r_a,\\
\Lambda\Delta s + S\Delta\lambda &= \tau\mathbf{1} - S\lambda.
\end{aligned}
$$

Here G, h specify the affine inequality coefficients and bounds; A, b specify the equalities; r_d, r_g, r_a are stationarity, inequality-feasibility and equality-feasibility residuals; and τ is the complementarity target. The quantities $\Delta x, \Delta s, \Delta\lambda, \Delta\nu$ are increments; $S = \operatorname{diag}(s)$, $\Lambda = \operatorname{diag}(\lambda)$, and $\mathbf{1}$ is an all-ones vector. The last row linearizes the products $s_i\lambda_i = \tau$, dropping only the second-order increment products. Unlike a full barrier centering solve, one step simultaneously reduces feasibility, stationarity and complementarity errors [1, Section 11.7].

If an increment has a negative slack coordinate, positivity requires step length $\alpha < -s_i/\Delta s_i$ in that coordinate. Apply the same rule to negative multiplier increments, take the smallest permitted bound, and stay slightly inside it. This derives the fraction-to-boundary safeguard rather than treating it as an arbitrary clipping rule. A further residual-decrease test is still needed: preserving positivity alone does not guarantee convergence.

8. Interpret a certificate at the application boundary

An optimality certificate answers a mathematical question about a specified model. An application decision also depends on whether that model includes the right constraints, quantities and uncertainty. Keep these two checks separate in the final report: numerical correctness cannot repair an omitted requirement.

Suppose a resource-allocation model limits total power but overlooks a per-device limit. The solver can return a globally optimal, accurately certified allocation that one device cannot implement. Add the missing constraint and solve again; do not describe the earlier result as a solver failure. Conversely, if every required constraint is present but the returned point violates one beyond tolerance, the issue belongs to numerical acceptance or the solve itself.

Choose tolerances before seeing the answer. A temperature bound, a budget equality and a dimensionless probability constraint need different interpretations of error. Write the acceptable deviations in original units, then connect them to solver tolerances and scaling. If a downstream rounding operation changes the variables, recheck feasibility after rounding. A certificate for the unrounded vector does not automatically certify the displayed or implemented decision.

Dual multipliers can explain sensitivity when the assumptions from the duality chapter hold. A large multiplier on a tight constraint suggests that a small relaxation may strongly affect the objective locally. It is not a prediction for an arbitrary large change, especially when the active constraints change. Test a small perturbation, resolve the problem and compare the observed objective change with the local prediction.

Near-degenerate problems deserve careful interpretation. Several almost redundant constraints can produce poorly determined multipliers while the primal decision is quite stable. A solver's multiplier variation may then overstate uncertainty in the decision itself. Examine both primal

sensitivity and dual sensitivity, and avoid reporting a shadow price with more precision than the data and conditioning justify.

Warm starts also belong to this boundary. Yesterday's solution may be near today's optimum but violate today's updated constraints. A feasible-start barrier method must first find a new interior point. A primal–dual or homogeneous-embedding solver may handle a different initialization strategy. Pass a warm start only through a method that supports its feasibility state; do not clip variables independently when clipping can break coupled equalities.

A useful result bundle includes the formulation and data versions, original objective, primal variables, relevant multipliers, maximum original-unit violations, a gap or lower bound when valid, and the termination status. Add the measured time and memory if performance matters. This bundle lets another reader distinguish a trustworthy approximate optimum, an infeasibility certificate, an unresolved numerical outcome and a model that needs revision.

Modern view

Interior-point theory remains useful while current solvers make different choices about precision, memory and hardware. The following readings provide an algorithmic taxonomy and verified modern evidence through October 4, 2026.

Bubeck's monograph [2] organizes convex optimization around oracle information, geometry and structure, covering first-order, structural and interior-point approaches. Its useful comparison is the cost of an iteration together with the guarantees attached to the oracle and assumptions. Our synthesis is to choose a method by the operations the problem makes cheap: matrix–vector products, projections, proximal maps or factorizations.

Parikh and Boyd's proximal survey [3] maps nonsmooth, constrained and distributed algorithms through small convex subproblems. It complements the barrier viewpoint: a projection or proximal operation can enforce a constraint without representing its interior through logarithms. These operations can be inexpensive or costly depending on the set. A problem's mathematical convexity does not determine which operation is affordable.

Goulart and Chen's 2026 Clarabel paper [4] describes an interior-point solver for conic problems with quadratic objectives, using a homogeneous embedding and supporting several cone types. It connects textbook ideas to software that also addresses infeasibility and computational structure. Its comparisons apply to the tested problem sets and solver configurations; they do not establish one universal winner for every convex model.

Applegate et al.'s PDLP manuscript [5], first posted in 2025 and revised in 2026, develops a practical first-order approach to large-scale linear programming. It adds techniques such as preprocessing, scaling and adaptive behavior around matrix–vector operations. It is a preprint, and its performance claims are empirical for the reported workloads. The comparison with interior-point methods should include requested accuracy, memory budget and original-unit residuals.

Solver family	Expensive operation	Typical structural question	What to verify
Equality-constrained Newton	KKT linear solve	can sparsity and factorization be reused?	feasibility and stationarity
Barrier/interior-point	changing curvature and constrained solve	is there an appropriate interior and cone representation?	domains, residuals, gap or certificates
Projection/proximal	projection or proximal subproblem	is the constraint or nonsmooth term simple to process?	iterate residual and feasible output
Large-scale first-order LP	matrix–vector products	does memory or factorization dominate the alternative?	scaled and original residuals at target accuracy

The current methodological lesson is broader than a preference for first- or second-order methods. Formulation, preprocessing, scaling, linear algebra and stopping policy are part of the algorithm's practical behavior. Comparing only the named update equation leaves out much of the system that determines reliability.

Open questions include accurate certificates for poorly scaled models, robustness when the feasible region is nearly lower-dimensional, memory limits in large factorizations and effective warm starts across changing constraints. A learned prediction can offer an initial point, but it still requires a feasibility and optimality check. Approximate solutions are useful when their errors are measured in the quantities the application can tolerate.

For a solver experiment, construct a family with one varying property: condition number, number of active constraints, sparsity or requested tolerance. Keep the mathematical optimum or an independently checked bound where possible. Report failures and inaccurate statuses alongside timings. Such a controlled family teaches more than a single successful solve with default settings.

Key takeaways

- Equality-constrained Newton steps remain in the nullspace of the equality matrix.
- The KKT matrix is generally indefinite even for a convex objective.
- A logarithmic barrier requires strict feasibility in the relevant domain.
- Exactly centered points yield a dual construction with gap m/t under the stated assumptions.
- Numerical iterates need actual feasibility, stationarity and complementarity checks.
- Boundary-only feasibility differs from infeasibility; Phase I needs a certificate to distinguish them reliably.
- Compare solver cost, precision and memory in the original problem's units.

Exercises

1. **Constrained curvature.** Construct a positive-semidefinite quadratic whose full Hessian is singular but whose objective has a unique minimizer on an affine equality set.

Hint

Let a zero-curvature direction be excluded by the equality. Check positive curvature only along nonzero vectors satisfying the homogeneous equality.

2. **Equality scaling.** Multiply the quadratic example's equality by 1000. Compare the solution, multiplier and residual scales. Explain which quantities should remain invariant.

Hint

The feasible set is unchanged. The equality multiplier scales inversely, and raw residuals scale with the row. Verify stationarity in both representations.

3. **Duplicate barriers.** Duplicate the upper-bound inequality in the scalar example. Compare the finite-t central path and the count used in the theoretical gap.

Hint

The feasible set is identical, but the upper-bound logarithm appears twice. There are now three scalar barrier contributions.

4. **Initialization.** Start the scalar barrier solver at zero and explain the failure. Give a principled repair for this problem and for a variable forced to zero by equalities.

Hint

A free nonnegative variable can begin strictly positive. A forced zero variable needs elimination or a suitable equality representation, rather than a numerical nudge.

5. **Infeasibility certificate.** Generalize the contradictory interval to $x \leq a$ and $x \geq b$ with $a < b$. Derive its minimum maximum violation and nonnegative certificate.

Hint

Balance the two violations at the midpoint. Equal weights cancel the variable and leave a negative right-hand-side sum.

6. **Stopping policy.** A solver reports tiny complementarity but a nonzero equality residual. Write the additional checks needed before accepting its output, and explain why an objective-only report is insufficient.

Hint

Include original-unit equality and inequality residuals, domain checks, multiplier signs, stationarity and a valid lower bound when available.

References

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapters 10–11. Book and official PDF
2. S. Bubeck, "Convex Optimization: Algorithms and Complexity," Foundations and Trends in Machine Learning, 2015. Monograph
3. N. Parikh and S. Boyd, "Proximal Algorithms," Foundations and Trends in Optimization, 2014. Author page and article
4. P. J. Goulart and Y. Chen, "Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives," Mathematical Programming Computation, 2026. Published paper · Author manuscript
5. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue and W. Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Paper

OPT 08 • Geometric Applications, Splitting and Modern Methods

Turn geometric and signal models into convex programs, choose proximal, ADMM or conditional-gradient operations, and evaluate an optimization pipeline with explicit certificates.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-08-applications-modern-methods/

What you'll learn

- Model distances, centers, separation and placement using convex geometry.
- Distinguish a convex application model from a relaxation of a discrete or nonconvex problem.
- Derive proximal updates for nonsmooth terms and constraints.
- Implement splitting for synthetic signal denoising and conditional gradients on a simplex.
- Choose methods by their expensive operations and check appropriate convergence evidence.
- Build a reproducible optimization capstone and connect it to modern surveys and methods.

The big picture

Convex optimization becomes useful when a problem's structure tells you both how to model it and how to solve it. A geometric model may make projections simple. A signal model may separate a smooth data term from a nonsmooth prior. A distributed model may make local solves cheap while communication is expensive. The right algorithm works with that structure rather than hiding it.

> **Explain like I'm five**
>
> Imagine arranging a workshop. Every machine needs some space, the doors must stay clear, and you want short walking routes. First decide which arrangements are allowed. Then choose what "better" means. Some rules let you improve the layout smoothly; other rules say a machine must be entirely left or entirely right of another. Those either-or rules need special care. A smooth-looking drawing does not make every layout problem convex.

The geometric reading follows Boyd and Vandenberghe's Chapter 8, with modeling and approximation connections to Chapters 4 and 6 [1]. The algorithmic extensions use Parikh and Boyd's proximal survey [2], Boyd et al.'s ADMM review [3], Condat et al.'s splitting survey [4] and Pokutta's conditional-gradient introduction [5]. The chapter closes the track by connecting these operations to a complete modeling, solving and validation workflow.

All experiments use synthetic data and run locally on a CPU. Their purpose is to expose the objective, assumptions, residuals and failure modes. A smaller numerical objective is meaningful only after the feasible set, data scale and evaluation protocol have been checked.

1. Distances, projections and separation

A distance problem asks how close a point or set can get to another while obeying its restrictions. A projection finds the nearest allowed point. These geometric operations become building blocks for both applications and optimization algorithms.

For a nonempty closed convex set $C \subseteq \mathbb{R}^n$, define

$$\Pi_C(v) = \arg\min_{x \in C} \tfrac{1}{2}\|x - v\|_2^2.$$

Here $v \in \mathbb{R}^n$ is a fixed query point; x is the optimization variable; $\|\cdot\|_2$ is Euclidean norm; and $\Pi_C(v)$ is the projection. The objective is strictly convex in x, so the minimizer is unique. Closedness ensures attainment; nonemptiness ensures the problem has a feasible point.

For a box, projection clips each coordinate to its interval. For an affine set, it solves a constrained quadratic. For a positive-semidefinite matrix cone, a projection involves the spectrum of a symmetric matrix. These are all convex projections, but their costs differ substantially. "Projected gradient" is a useful name only after the projection cost is understood.

A projection satisfies $(v - p)^\top (x - p) \le 0$ for every $x \in C$, where $p = \Pi_C(v)$. The inequality says that the displacement from the projection to the query points away from every feasible direction. It supplies an optimality check and, when the query lies outside the set, a separating hyperplane.

Distances between two convex sets can be modeled by minimizing $\|x - y\|_2$ with $x \in C$ and $y \in D$, where D is a second set. The norm is convex in the pair. A finite infimum need not be attained for arbitrary unbounded sets; verify existence before interpreting an algorithm's iterates as converging to a closest pair. For compact sets, the geometry is simpler.

Classification by separation also has a geometric interpretation. A linear score $a^\top x + b$ places a hyperplane between classes, where a is its normal and b its offset. Margin constraints can be convex when the data and labels are fixed. Choosing a representation jointly with all classifier parameters can create a different, nonconvex learning problem. The Pattern Recognition track develops the statistical context of these models.

2. Centers, ellipsoids and placement

A center can mean several different things. The center minimizing maximum distance to observations differs from the center of the largest ball fitting inside a feasible region. The analytic center of an inequality representation is another object. State the geometry before calling a point "the center."

For a polyhedron described by $a_i^\top x \le b_i$, the largest contained Euclidean ball has center c and radius r obtained from

$$\begin{aligned} \text{maximize}_{c,r} \quad & r \\ \text{subject to} \quad & a_i^\top c + r\|a_i\|_2 \le b_i \quad \text{for every } i, \\ & r \ge 0. \end{aligned}$$

Here $a_i \in \mathbb{R}^n$ and $b_i \in \mathbb{R}$ are fixed inequality data; $c \in \mathbb{R}^n$ and $r \in \mathbb{R}$ are variables. The worst displacement of length at most r in direction a_i contributes $r\|a_i\|_2$. Every constraint is affine in (c, r) because the normal vectors are fixed.

This center balances distance to boundaries in Euclidean units. Multiplying an inequality by a positive constant scales both its normal norm and right-hand side, preserving the geometric condition. A zero normal needs separate inspection: its inequality is either always satisfied or impossible. Do not divide by its norm blindly.

An ellipsoid allows different radii in different directions. Write it as $E = \{c + Bu : \|u\|_2 \le 1\}$, where B is a positive-definite symmetric shape matrix, c is the center and u is an auxiliary unit-ball coordinate. Containment in a halfspace becomes $a_i^\top c + \|B^\top a_i\|_2 \le b_i$. Maximizing $\log\det B$ models maximum volume up to an additive constant [1]. The domain $B \succ 0$ is essential.

Centering and enclosing are different constraints. An inscribed ellipsoid must stay inside a region, while an enclosing ellipsoid must contain given points. Their convenient convex parameterizations differ. A visually plausible shape matrix can lead to a nonconvex formulation if inverses and variable products are introduced carelessly. Trace each transformation to its domain and convexity rule.

Placement problems often minimize sums of distances between movable points and fixed anchors. If connectivity and constraints are fixed, Euclidean norms yield a convex objective. Choosing which connections exist introduces discrete variables. Floorplanning can be convex after selecting a relative ordering of rectangles, but unknown nonoverlap choices contain disjunctions. The original physical problem and a convex subproblem need separate labels.

Worked center: a rectangle has more than one answer

The largest inscribed ball need not have a unique center. Take the rectangle $0 \le x_1 \le 6, 0 \le x_2 \le 2$. The ball constraints reduce to $r \le c_1 \le 6 - r$ and $r \le c_2 \le 2 - r$, with $r \ge 0$.

Here $c = (c_1, c_2)$ is the center and r its radius. The shorter side forces $r \le 1$. At maximum radius one, every center $(c_1, 1)$ with $1 \le c_1 \le 5$ is optimal. The problem is convex, yet its optimizer is not unique. The symmetric center $(3, 1)$ is one valid answer, rather than a uniquely implied one.

For an inscribed ellipsoid, a symmetric positive-definite B maps the unit ball to the shape. Its volume is the unit-ball volume multiplied by $\det B$, because a linear map scales volume by its determinant. Maximizing $\log\det B$ therefore preserves volume maximizers. The containment rule follows by maximizing $a_i^\top (c + Bu)$ over $\|u\|_2 \le 1$, producing $a_i^\top c + \|B^\top a_i\|_2$. This combines the support-function calculation of OPT 03 with the matrix curvature of OPT 02 [1, Chapter 8].

The convex formulation minimizes $-\log\det B$ over the containment constraints. The halfspace normals are fixed, so the norm term is convex in B. Allowing B to become singular would leave the finite log-determinant domain and collapse the ellipsoid's volume.

3. Relaxations are models with an explicit interpretation

A relaxation enlarges the feasible set to make a hard problem easier. It can produce a useful bound or a candidate that later needs rounding. A fractional solution is implementable only when the application permits mixtures, sharing or time allocation.

Suppose a binary configuration variable z_i must lie in $\{0, 1\}$. Replacing that constraint by $0 \leq z_i \leq 1$ is a convex relaxation of the discrete requirement when the remaining formulation is convex. For minimization, enlarging the feasible set can only decrease the optimum, so the relaxed value is a lower bound on the original optimum. The bound does not mean the relaxed point is a valid discrete decision.

After rounding, evaluate the original objective and constraints again. Independent coordinate rounding can violate a total budget, a connectivity requirement or an equality. A repaired feasible rounded solution gives an upper bound; the relaxed optimum gives a lower bound. Their difference is meaningful when both values refer to the same original objective and the lower value is the exact relaxed optimum or a certified dual or error-corrected bound. An approximate relaxed primal objective alone is generally an upper bound on the relaxed optimum; it need not be a lower bound on the original discrete optimum.

The same care applies to convex surrogates in vision and learning. A norm penalty can encourage sparsity without solving a general count-minimization problem exactly. A convex data term may be a deliberate approximation to a physical measurement process. State whether the model is a statistical estimator, a computational relaxation or an exact reformulation. These purposes support different claims.

Parameter selection also belongs outside the numerical certificate. Solving a regularized problem to high accuracy does not prove that its regularization strength predicts well on new data. Select the strength using a valid validation protocol, then test once on held-out data. Keep numerical optimization error separate from model-selection and generalization error.

4. Proximal operations handle nonsmooth structure

A nonsmooth term need not be approximated by a steep smooth function. A proximal operation balances that term against staying near a proposed point. Many penalties and constraints admit a simple closed-form operation, making them suitable for first-order methods [2].

For a proper closed convex function g and step scale $\eta > 0$, define

$$\operatorname{prox}_{\eta g}(v) = \arg\min_{z} \left[g(z) + \frac{1}{2\eta} \|z - v\|_2^2 \right].$$

Here $v \in \mathbb{R}^n$ is the proposed point; $z \in \mathbb{R}^n$ is the subproblem variable; g is allowed to take positive infinity to encode restrictions; and η scales the quadratic proximity term. Under the stated conditions the subproblem has a unique minimizer.

For the indicator of a closed convex set, the proximal operation is projection. For $g(z) = \lambda \|z\|_1$, with $\lambda \geq 0$, it is coordinatewise soft thresholding: $\operatorname{sign}(v_i) \max(|v_i| - \eta\lambda, 0)$. Here i indexes coordinates.

The operation can create exact zeros; ordinary gradient descent on the absolute-value penalty is undefined at zero.

To minimize $f(x) + g(x)$ with differentiable convex f, proximal gradient takes

$$x^{k+1} = \operatorname{prox}_{\eta g}\left(x^k - \eta \nabla f(x^k)\right).$$

Here k is the iteration index; ∇f is the smooth gradient; and η is a step size selected to meet the method's assumptions. If the gradient is L-Lipschitz, a conservative fixed choice is $0 < \eta \leq 1/L$. The constant L bounds gradient variation, rather than the objective value itself.

For constrained least squares, this update becomes gradient followed by projection. For sparse least squares, it becomes gradient followed by soft thresholding. More complicated compositions, such as a nonsmooth function applied to a linear difference operator, may make the direct proximal map expensive. Splitting introduces auxiliary variables to keep the cheap operations separate.

A useful residual is the scaled update $G_\eta(x) = [x - \operatorname{prox}_{\eta g}(x - \eta \nabla f(x))]/\eta$. Here G_η is a proximal-gradient mapping. A zero mapping satisfies the first-order optimality condition for this convex composite problem. Its magnitude is a convergence diagnostic; interpret it together with scale, feasibility and an objective bound where available.

Derive soft thresholding at the corner

The proximal map solves the nonsmooth corner exactly. For one coordinate, minimize $\lambda|z| + (z - v)^2/(2\eta)$ with $\eta > 0$ and $\lambda \geq 0$. The optimality condition is

$$0 \in \lambda \partial |z| + \frac{z - v}{\eta}.$$

Here v is the proposed coordinate, z the output and $\partial|z|$ the absolute-value subdifferential. For $z > 0$, it equals one, so $z = v - \eta\lambda$, valid only if $v > \eta\lambda$. For $z < 0$, it equals minus one, giving $z = v + \eta\lambda$, valid only if $v < -\eta\lambda$. At zero, the subdifferential is $[-1, 1]$, so zero is optimal precisely when $|v| \leq \eta\lambda$ [2].

These three cases give the threshold formula. For the synthetic vector $v = (2, -0.3, 0.1)$ and threshold $\eta\lambda = 0.4$, the output is $(1.6, 0, 0)$. A nonzero input can become exactly zero because zero has a whole interval of supporting slopes.

The proximal-gradient update follows by minimizing a local upper model of the smooth term: replace $f(z)$ by $f(x) + \nabla f(x)^\top (z - x) + \|z - x\|_2^2/(2\eta)$ and retain $g(z)$ exactly. Completing the square centers this model at $v = x - \eta \nabla f(x)$. Thus the method uses a gradient step for the smooth cost and an exact small solve for the nonsmooth cost. When $1/\eta \geq L$, the smoothness inequality makes this model an upper bound, explaining the conservative step condition.

5. Split a signal model into simple subproblems

Piecewise-constant signals can be estimated by balancing agreement with observations against neighboring differences. A quadratic data term discourages large corrections; an absolute-difference penalty encourages many neighboring values to match. The resulting convex estimator is useful for a controlled denoising experiment, with behavior different from uniform smoothing.

Let $y \in \mathbb{R}^n$ contain noisy observations and $D \in \mathbb{R}^{(n-1)\times n}$ compute adjacent differences. Solve

$$\operatorname{minimize}_x \quad \tfrac{1}{2}\|x - y\|_2^2 + \lambda \|Dx\|_1.$$

Here $x \in \mathbb{R}^n$ is the estimated signal and $\lambda \geq 0$ is the regularization strength. This is one-dimensional total-variation regularization as treated in proximal and splitting references [2–4]. The term describes local variation; it does not assert that every real signal is piecewise constant.

Introduce $z = Dx$ and apply the alternating direction method of multipliers, or ADMM [3]. In scaled form,

$$\begin{aligned} x^{k+1} &= (I + \rho D^\top D)^{-1}\left[y + \rho D^\top (z^k - u^k)\right], \\ z^{k+1} &= \operatorname{shrink}(Dx^{k+1} + u^k, \lambda/\rho), \\ u^{k+1} &= u^k + Dx^{k+1} - z^{k+1}. \end{aligned}$$

Here $\rho > 0$ is the penalty parameter; I is the identity matrix; u is the scaled dual variable; and shrink denotes coordinatewise soft thresholding defined above. The inverse notation describes the solution mathematically; the implementation uses a factorization and solve. The split equality is satisfied at convergence, rather than necessarily at each intermediate iterate.

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

rng = np.random.default_rng(8)
n = 48
clean = np.r_[np.zeros(16), np.ones(16), np.full(16, 0.3)]
y = clean + rng.normal(0, 0.16, n)
D = np.diff(np.eye(n), axis=0)
lam, rho = 0.4, 4.0
factor = cho_factor(np.eye(n) + rho*D.T @ D)
x, z, u = y.copy(), np.zeros(n-1), np.zeros(n-1)
for k in range(2000):
    x = cho_solve(factor, y + rho*D.T @ (z-u))
    z_old = z.copy()
    v = D @ x + u
    z = np.sign(v) * np.maximum(np.abs(v)-lam/rho, 0)
    u += D @ x-z
    primal = np.linalg.norm(D @ x-z)
    dual = rho*np.linalg.norm(D.T @ (z-z_old))
    if max(primal, dual) < 1e-7:
        break
objective = lambda a: 0.5*np.sum((a-y)**2) + lam*np.abs(D @ a).sum()
```

```
print("Iterations and residuals:", k+1, primal, dual)
print("Objectives, raw and estimated:", objective(y), objective(x))
print("Synthetic MSE, raw and estimated:", np.mean((y-clean)**2), np.mean((x-clean)**2))
assert max(primal, dual) < 1e-7
assert objective(x) <= objective(y) + 1e-8
assert np.mean((x-clean)**2) < np.mean((y-clean)**2)
```

The last assertion checks this specific synthetic example; it is not a theorem that regularization always improves reconstruction. Strong regularization can flatten a real edge or remove a narrow peak. The objective chooses a compromise under the model, while clean-signal error requires ground truth that a real deployment may not have.

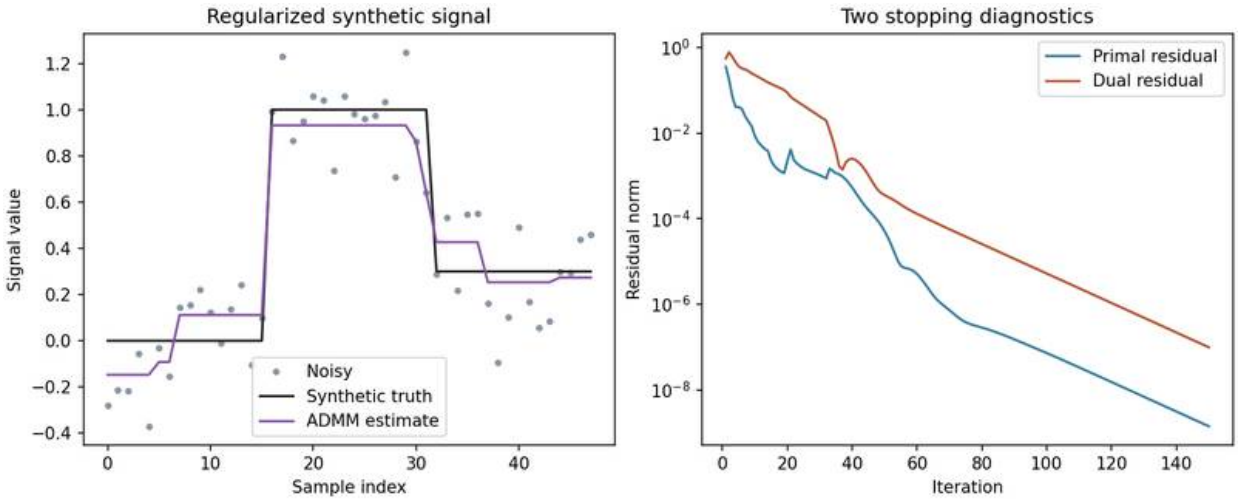


Figure 1 — Splitting separates a quadratic solve from soft thresholding. Convergence requires both residuals to tend to zero, without requiring monotone decrease; the denoising improvement belongs to this constructed signal and regularization choice.

The factorization is reused because D and ρ are fixed. Changing the regularization strength only changes thresholding; changing ρ changes the linear system and requires refactorization. If ρ changes while retaining the same unscaled multiplier, rescale u consistently. Otherwise the update silently changes the dual state.

For images, adjacent differences become horizontal and vertical operators, and the penalty can combine them in different ways. Boundary conditions and pixel scale affect the estimator. This chapter's one-dimensional result cannot be reported as an image-restoration benchmark. Use Image Processing for measurement models and Deep Learning for Computer Vision for learned restoration comparisons.

Derive the split updates and their residuals

The ADMM equations come from retaining an explicit difference variable rather than taking a difficult composite proximal map. With $z = Dx$, form the scaled augmented objective

$$\frac{1}{2}\|x-y\|_2^2+\lambda\|z\|_1+\frac{\rho}{2}\|Dx-z+u\|_2^2-\frac{\rho}{2}\|u\|_2^2.$$

Here x is the signal, y the observations, D the difference matrix, z the split differences, u the scaled multiplier, and $\rho>0$ the penalty. The last term is constant during the primal subproblems. Differentiating in x yields $(I+\rho D^\top D)x=y+\rho D^\top(z-u)$. Minimizing in z is the soft-threshold problem centered at $Dx+u$ with threshold λ/ρ. The multiplier update then accumulates the split-equality error [3].

Why monitor two residuals? The primal residual $r_p=Dx-z$ checks the coupling. The dual residual $r_d=\rho D^\top(z^{k+1}-z^k)$ checks the mismatch left in stationarity by the sequential update. Here k is the iteration index; changing the sign convention does not change the residual norm.

The data term makes this particular objective strongly convex in x, so the denoised signal is unique even though neighboring differences can be zero at nonsmooth corners. The residuals explain whether the split algorithm has reached that solution; they do not measure agreement with an unknown clean signal. The synthetic experiment can inspect both because its generating signal is available.

6. Conditional gradients use a linear oracle

Projection can be expensive even when minimizing a linear function over the set is easy. Conditional-gradient methods replace projection with a linear optimization oracle and a convex-combination update. Their iterates remain feasible when initialized feasibly [5].

For differentiable convex f on a compact convex set C, solve

$$v^k\in\arg\min_{v\in C}\nabla f(x^k)^\top v,\qquad x^{k+1}=(1-\gamma_k)x^k+\gamma_k v^k.$$

Here x^k is the current feasible point; v^k is a linear-oracle solution; and $\gamma_k\in[0,1]$ is the step weight. The new point is feasible by convexity. The Frank–Wolfe algorithm [5] is this basic conditional-gradient construction with suitable step choices and convergence assumptions.

A useful certificate is

$$g_{\mathrm{FW}}(x)=\nabla f(x)^\top(x-v(x)).$$

Here $v(x)$ solves the linear oracle at x. For an exact oracle and convex differentiable objective, the supporting-hyperplane inequality gives $f(x)-f(x^*)\le g_{\mathrm{FW}}(x)$, where x^* is a minimizer in C. The gap is therefore an upper bound on objective error at the current feasible point. An approximate oracle needs an additional error allowance.

On the probability simplex $C=\{x\ge 0:\sum_i x_i=1\}$, the linear oracle selects the coordinate with the smallest gradient. It returns a simplex vertex. This makes the method useful for mixtures where a new component can be selected cheaply. Sparse iterates arise because each update adds a vertex; representation and accuracy trade-offs still need examination.

```
import numpy as np
```

```
target = np.array([-0.1, 0.6, 0.5])
def project_simplex(v):
    sorted_v = np.sort(v)[::-1]
    threshold = (np.cumsum(sorted_v)-1)/np.arange(1, len(v)+1)
    active = np.flatnonzero(sorted_v > threshold)[-1]
    return np.maximum(v-threshold[active], 0)
optimum = project_simplex(target)
x = np.full(3, 1/3)
for k in range(1000):
    grad = x-target
    vertex = np.eye(3)[int(np.argmin(grad))]
    gap = float(grad @ (x-vertex))
    error = 0.5*np.sum((x-target)**2)-0.5*np.sum((optimum-target)**2)
    assert error <= gap + 1e-12
    if gap < 1e-4:
        break
    gamma = 2/(k+2)
    x = (1-gamma)*x + gamma*vertex
# Recompute certificate at the returned iterate.
grad = x-target
vertex = np.eye(3)[int(np.argmin(grad))]
gap = float(grad @ (x-vertex))
print("Simplex iterate:", x)
print("Frank–Wolfe gap:", gap)
assert np.all(x >= 0) and np.isclose(x.sum(), 1)
assert gap < 0.01
```

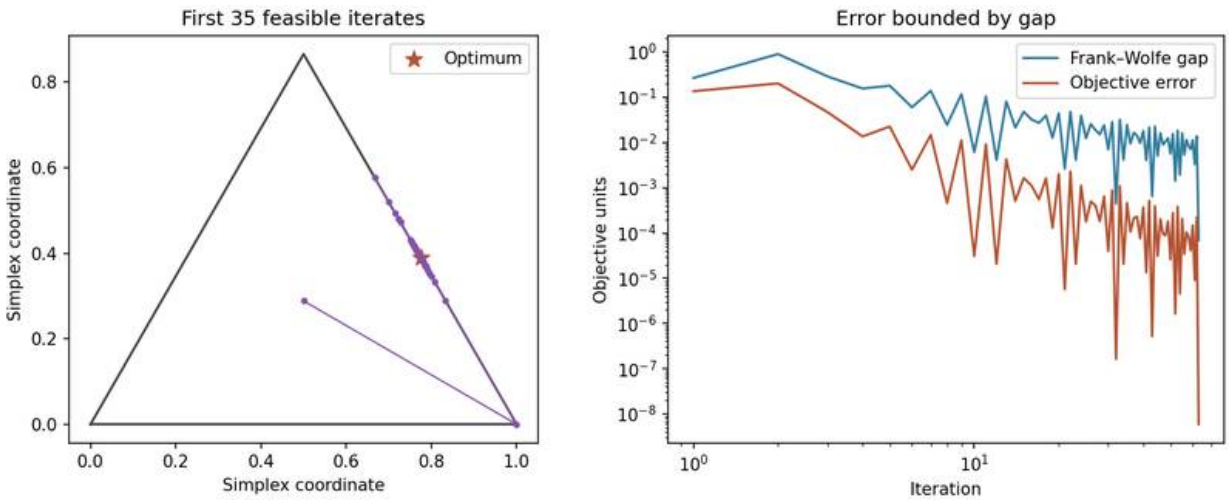


Figure 2 — A linear oracle supplies feasible vertices and a gap certificate. The path can zigzag even while objective error decreases.

The reference projection in the example computes the exact quadratic optimum for comparison. The conditional-gradient method itself never calls that projection. The distinction lets you test its gap bound rather than inferring convergence only from a decreasing trace. Recompute the gap after the last update so it describes the point actually returned.

A compact feasible set and a smooth objective support the usual convergence analysis. An unbounded oracle or a nonsmooth objective changes the problem. Variant methods can address additional structure, but do not transfer the basic guarantee without checking their assumptions. The survey [5] explains alternatives and their geometric motivation.

Prove the oracle gap, including an inexact oracle

The conditional-gradient gap is the improvement allowed by the best linearized feasible direction. Convexity gives $f(x^*) \geq f(x) + \nabla f(x)^\top (x^* - x)$. Since the exact oracle minimizes that linear score over C,

$$f(x) - f(x^*) \leq \nabla f(x)^\top (x - x^*) \leq \nabla f(x)^\top (x - v) = g_{\mathrm{FW}}(x).$$

Here x is the current feasible point, x^* an optimizer, and v the exact linear-oracle solution. The first inequality uses objective convexity; the second uses the oracle's comparison with x^*. These are the two ingredients that turn a computable directional quantity into a global error bound [5].

If a feasible approximate oracle returns $\tilde{v}$ satisfying $\nabla f(x)^\top \tilde{v} \leq \min_{z \in C} \nabla f(x)^\top z + \delta$, with known $\delta \geq 0$, then the valid bound is the computed gap plus δ. Here δ is the oracle's linear-objective error allowance. Feasibility of the update remains intact because $\tilde{v} \in C$, even when the gap needs correction.

For a simplex gradient $(2, -1, 3)$, the exact oracle chooses its second vertex, whose score is -1. Choosing the first vertex gives score 2, requiring allowance $\delta = 3$. At $x = (1/3, 1/3, 1/3)$, the exact gap is $7/3$, but this approximate-oracle gap is $-2/3$. Adding three recovers $7/3$. A negative uncorrected gap here indicates a poor oracle, not an objective error below zero.

7. Choose a method by structure, accuracy and information

There is no universally best update equation. The modeling decisions determine what information is available and which operations are inexpensive. Compare those operations before comparing a method's name or its iteration count.

For a modest smooth problem with exploitable curvature, Newton or an interior-point method may reach a precise solution through relatively few expensive steps. For a huge sparse model, factorization memory can dominate; first-order methods may use cheaper matrix–vector products. For a simple nonsmooth penalty, a proximal operation can be exact and cheap. For a complicated set with an easy linear oracle, conditional gradients may avoid a costly projection.

Distributed optimization introduces communication as another expensive operation. Local objectives may be separable while consensus constraints connect variables. ADMM can expose local subproblems, but convergence time must include communication rounds and local solve cost. A mathematically distributed formulation does not imply the implementation runs in parallel; the denoising example is entirely serial.

Primal–dual hybrid-gradient methods handle saddle-point formulations using primal and dual updates [4]. Their step sizes depend on operator norms and the structure of the split terms. Prac-

tical linear-programming systems such as PDLP [6] add preprocessing and adaptive techniques around that structure. A short theoretical recurrence and a production solver are different levels of description.

Requested precision can reverse a comparison. A method that rapidly produces a useful approximate solution may take much longer to achieve a very small certificate gap. Another method may spend substantial setup time but converge efficiently to high precision. Plot time to several stated tolerances, and report original-unit feasibility alongside the gap. This makes the trade-off reviewable.

Memory and repeated solves matter too. A parameter sweep can reuse data structures, factorization or a previous solution when supported. A different sparsity pattern may prevent that reuse. Include preprocessing and setup in the first-solve measurement, then separately report warm repeated solves. Do not compare one method's warm run with another's cold run silently.

Consensus: local solves and a shared average

Distributed ADMM makes the communication mechanism explicit. Suppose J participants share a vector decision but each has a local convex cost f_j. Introduce local copies x_j and shared variable z, and solve $\min \sum_{j=1}^{J} f_j(x_j)$ with $x_j = z$ for every j.

In scaled form, one consensus iteration is

$$
\begin{aligned}
x_j^{k+1} &= \arg\min_x \left[f_j(x) + \frac{\rho}{2} \|x - z^k + u_j^k\|_2^2 \right], \\
z^{k+1} &= \frac{1}{J} \sum_{j=1}^{J} (x_j^{k+1} + u_j^k), \\
u_j^{k+1} &= u_j^k + x_j^{k+1} - z^{k+1}.
\end{aligned}
$$

Here u_j is participant j's scaled multiplier, $\rho > 0$ the penalty, and k the iteration index. The first row consists of independent local solves. The second row minimizes the sum of quadratic consensus terms in z, whose derivative sets z to their average. The third row accumulates disagreement [3].

For scalar costs $f_j(x) = \frac{1}{2}(x - a_j)^2$, the local solve is $x_j^{k+1} = [a_j + \rho(z^k - u_j^k)]/(1 + \rho)$, where a_j is a local target. Two participants with targets zero and four must agree at the globally optimal value two. Their individually optimal values zero and four violate consensus; averaging and multiplier updates coordinate the compromise.

Communication is therefore part of the algorithm: the shared update needs each local quantity, while each local solve needs the latest shared value. A parallel implementation must measure message cost and synchronization as well as local arithmetic. Additional shared constraints change the z subproblem, often from a simple average to a constrained proximal operation.

8. A complete convex-optimization capstone

A capstone should make the formulation and evidence reproducible. Choose a synthetic signal estimator, geometric placement task or resource model. Write variables and units, a convexity argument, domains and a feasible-set interpretation before running the solver. State any relaxation and how its output becomes an implementable decision.

Build one small instance with an analytic answer or an independently computed certificate. Use it to check signs, shapes, scaling and stopping rules. Then vary one property at a time: noise strength, regularization, condition number, sparsity, problem size or tolerance. Keep the data seed fixed when comparing algorithms on the same instance.

Record objective values and residuals at the point actually returned. For splitting, track the coupling residual and the dual residual. For a conditional-gradient solve, report the current gap and oracle accuracy. For a barrier solve, distinguish the outer target from the computed certificate. For a relaxation, keep the lower bound and feasible rounded upper bound separate.

Assess the application outcome alongside numerical convergence. A denoiser may accurately minimize a penalty that removes a real event. A placement solver may obey every modeled rule while omitting a doorway requirement. A resource optimizer may be sensitive to an uncertain demand estimate. These are model-validation findings, not reasons to blur the meaning of a mathematical certificate.

For learned pipelines, split data before selecting regularization or fitting any preprocessing. Repeat solves on a validation set, choose the rule, then report the held-out outcome. The ML/DL evaluation chapter provides the broader learning protocol. Numerical convergence and predictive evaluation answer different questions and should appear in separate columns of the result table.

Modern view

The modern reading map combines surveys that organize algorithms with recent work that changes practical modeling or solving. The sources below were checked through October 4, 2026; unpublished manuscripts are identified explicitly.

Parikh and Boyd [2] organize proximal methods around the operation of solving a small convex subproblem. **Boyd et al. [3]** organize ADMM around separability, coupling and distributed statistical applications. Our teaching synthesis is to examine which subproblems become easy after a split and which new residuals must then be controlled. Splitting is valuable when it reduces total work, not simply because an objective has several terms.

Condat et al. [4] unify proximal splitting through primal–dual product spaces and monotone inclusions, revisiting parameter ranges and algorithmic variants. The reading connects apparently different recurrences through their operators and metrics. Use it to identify a method's actual

assumptions, especially when a smooth quadratic term permits choices different from a generic smooth function. The unified perspective prevents arbitrary parameter combinations from being treated as guaranteed methods.

Pokutta's 2024 introduction [5] maps conditional gradients through linear oracles and feasible combinations. It helps compare projection-based methods with oracle-based alternatives and explains why feasible-set geometry affects behavior. The practical takeaway is to price the oracle and the representation, then use a valid gap. A cheap oracle can still require many iterations to obtain the precision an application needs.

The 2024 multi-robot survey by Shorinwa et al. [7] distinguishes distributed first-order, sequential convex-programming and ADMM approaches. It places communication, coordination and physical constraints alongside mathematical updates. Some robotics formulations are nonconvex or solved by successive convex subproblems, so convergence of a convex subproblem does not certify a global solution of the original trajectory or coordination task.

Applegate et al.'s PDLP manuscript [6], posted in 2025 and revised in 2026, provides recent evidence for practical large-scale first-order linear programming. **Schaller, Bemporad and Boyd's 2025 manuscript [8]** instead fits parameterized convex functions compatible with disciplined programming. The former concerns a solver pipeline; the latter concerns learned functions that retain convexity in decision variables. Both are manuscripts, and neither makes arbitrary neural-network training convex.

Question	Useful family or reading	Evidence to collect
Is a constraint projection cheap?	projected/proximal methods [2]	projection cost and optimality residual
Can terms be split into easy pieces?	ADMM and primal–dual splitting [3,4]	local solve cost and coupling residuals
Is linear minimization easier than projection?	conditional gradients [5]	oracle cost, feasible iterate and gap
Does communication dominate?	distributed-method taxonomy [7]	rounds, messages, local work and physical feasibility
Is the linear model too large for factorization?	practical first-order LP [6]	memory, accuracy and original-unit residuals
Can learned functions preserve decision-variable convexity?	parametric convex modeling [8]	structural restrictions and held-out model adequacy

Our synthesis is that modeling and computation increasingly meet in reusable structured components: convex subproblems, inexpensive proximal maps, learned convex functions and hardware-aware solver operations. Their value still depends on an explicit formulation, valid assumptions and measured end-to-end behavior. A solver benchmark does not establish generalization; a prediction benchmark does not establish feasibility.

Open problems include automatic structure detection, robust certificates under poor scaling, communication-efficient solves, differentiability when active constraints change and reliable

integration of learned predictions with exact constraints. A useful research question isolates one issue and preserves an independently checkable outcome. For example: "Does this warm start reduce total solve time while preserving the same certificate tolerance?"

The track now supplies a complete path: convex sets and functions, problem formulation, duality, estimation, numerical methods, constrained solvers and structured applications. Continue by choosing one textbook exercise family and building a small original experiment around its assumptions. The central habit is to state what is optimized, why the model is convex, what the algorithm computes and what evidence justifies accepting the result.

Key takeaways

- Geometric modeling includes projection, separation, centers, ellipsoids and structured placement.
- Unknown discrete choices and disjunctions can make the original application nonconvex.
- Relaxed lower bounds and feasible implementable solutions have different roles.
- Proximal operations handle simple nonsmooth penalties and constraints directly.
- ADMM splits coupling into tractable operations and requires primal and dual residual checks.
- Conditional gradients maintain feasibility through a linear oracle and supply a gap under stated assumptions.
- Compare original-unit accuracy, setup, memory, repeated solves and application outcomes.

Exercises

1. **Geometric center.** Derive the largest inscribed ball for an axis-aligned rectangle. Explain why its center may be nonunique when one side is longer than the other.

   Hint

   The shorter side limits the radius. The center can move along the longer dimension while maintaining enough boundary clearance.

2. **Floorplanning model.** Write convex nonoverlap constraints for two rectangles after choosing which one is left of the other. Explain what changes if the ordering is unknown.

   Hint

   A fixed order gives an affine separation inequality. Unknown order requires an either-or choice, which is a disjunction rather than one convex halfspace.

3. **Regularization sweep.** Run the denoising example with five regularization strengths. Compare objective value, synthetic reconstruction error and the number of nearly zero adjacent differences.

Hint

Objectives with different regularization strengths are different functions, so their raw values are not a fair convergence comparison. Keep model quality and solve accuracy separate.

4. **Penalty update.** Change the ADMM penalty halfway through the experiment. Update the factorization and scaled dual variable consistently, then compare residual traces with the fixed-penalty version.

Hint

The unscaled multiplier is the penalty times the scaled dual variable. Preserve that product when changing the penalty.

5. **Inexact oracle.** Replace the simplex linear oracle with a deliberately suboptimal vertex. Identify how its objective error changes the certificate bound and whether the iterate remains feasible.

Hint

Feasibility follows from convex combinations of vertices. The gap based on an inexact oracle needs an additive bound on the oracle's linear-objective error.

6. **Capstone report.** Compare two appropriate algorithms on one synthetic convex application. Include the formulation, convexity proof, stopping policy, original-unit residuals, timing, memory and one model failure.

Hint

Use identical data and acceptance tolerances. Separate setup from repeated-solve costs, and report an unresolved numerical run rather than counting it as a successful solve.

References

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapter 8, with Chapters 4 and 6 for modeling and approximation. Book
2. N. Parikh and S. Boyd, "Proximal Algorithms," Foundations and Trends in Optimization, 2014. Survey
3. S. Boyd, N. Parikh, E. Chu, B. Peleato and J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," Foundations and Trends in Machine Learning, 2011. Review

4. L. Condat, D. Kitahara, A. Contreras and A. Hirabayashi, “Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists,” SIAM Review, 2023. Author manuscript
5. S. Pokutta, “The Frank-Wolfe Algorithm: A Short Introduction,” Jahresbericht der Deutschen Mathematiker-Vereinigung, 2024. Survey
6. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O’Donoghue and W. Schudy, “PDLP: A Practical First-Order Method for Large-Scale Linear Programming,” arXiv preprint, 2025, revised 2026. Paper
7. O. Shorinwa, T. Halsted, J. Yu and M. Schwager, “Distributed Optimization Methods for Multi-Robot Systems: Part 2—A Survey,” IEEE Robotics & Automation Magazine, 2024. Survey
8. M. Schaller, A. Bemporad and S. Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025. Author page

Part II

Information Theory

IT 01 • Entropy, Relative Entropy and Mutual Information

Build information measures from probabilities, derive chain rules and data processing, and distinguish exact identities from the uncertainty of estimating them from data.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-01-entropy-mutual-information/

What you'll learn

- Explain why a logarithm converts probability into an additive measure of surprise.
- Calculate entropy, conditional entropy, relative entropy and mutual information.
- Derive chain rules and identify the equality conditions in fundamental inequalities.
- Explain what processing can discard, preserve or add without confusing randomness with information.
- Check information identities using finite probability tables and complete CPU examples.
- Distinguish a population quantity, an empirical estimate and a learning objective.

The big picture

A sensor sends a sequence of measurements. Some values are predictable; others are surprising. A compressor benefits from predictability, while a receiver benefits from measurements that reduce uncertainty about something it wants to know. These two benefits are related but different. A stream can be highly random and contain almost no useful evidence about a scene.

> **Explain like I'm five**
>
> Imagine guessing what is inside a box. "It is a toy" may not help much if every box contains a toy. "It is the only red train" can settle the question. Information theory measures how much uncertainty a message removes, using a probability model of the possibilities.

This track follows Thomas M. Cover and Joy A. Thomas's *Elements of Information Theory*, second edition [1]. Its quantities have operational roles: entropy will guide compression, mutual information will describe communication limits, and relative entropy will connect probability mismatch to extra description length and statistical discrimination. We begin with finite alphabets so every quantity can be checked by explicit summation.

The Stanford entropy notes [2] give an accessible companion treatment. We use explicit synthetic probability models so the definitions and numerical checks can be inspected directly.

How the track fits together

Tutorial	Main question	Core second-edition chapters
IT 01	what does information measure?	1–2, basic tools from 17
IT 02	why do long sequences become predictable in aggregate?	3–4
IT 03	how can descriptions become short without losing symbols?	5, 13–14
IT 04	how much can a noisy channel communicate reliably?	7
IT 05	what changes for continuous and Gaussian variables?	8–9
IT 06	what rate is required when some distortion is allowed?	10
IT 07	how do information measures support inference and decisions?	6, 11–12, 16
IT 08	what changes with several users, constraints and learning?	15, advanced tools from 17

This grouping covers the book's main themes without pretending that every theorem fits into a short derivation. Work through definitions, then numerical checks, then proofs. If a calculation feels mysterious, make the probability table explicit before adding a neural estimator or a communication system.

Random variables, outcomes and the probability model

A random variable describes what could be observed before the observation is known. Its probabilities belong to a model of the experiment. The same observed value can be surprising under one model and ordinary under another.

Let X take values in a finite alphabet $\mathcal{X}$. Write

$$p_X(x) = \Pr(X = x), \qquad p_X(x) \geq 0, \qquad \sum_{x \in \mathcal{X}} p_X(x) = 1.$$

Here X is the random variable, x is a possible outcome, $\mathcal{X}$ is the set of outcomes, p_X is its probability mass function, and Pr denotes probability. A joint distribution $p_{XY}(x, y)$ assigns probabilities to pairs. Marginals come from summation:

$$p_X(x) = \sum_y p_{XY}(x, y), \qquad p_Y(y) = \sum_x p_{XY}(x, y).$$

Here Y is another finite random variable, y is one of its outcomes, and p_Y is its marginal mass function. Conditional probabilities use $p_{X|Y}(x \mid y) = p_{XY}(x, y)/p_Y(y)$ when $p_Y(y) > 0$. Conditioning on an event with zero probability needs separate conventions and will not contribute to the finite sums below.

We use base-two logarithms unless explicitly stated otherwise. Results are then in **bits**. Natural logarithms give **nats**; dividing a value in nats by $\ln 2$ converts it to bits. A bit is a unit of information, not necessarily a literal stored binary digit in a particular file format.

Keep the experiment fixed while comparing distributions. If one model describes grayscale intensities and another describes whole images, their numerical entropies are not directly comparable. Changing the variable, alphabet, quantization or conditioning information changes the question.

Surprise becomes additive through a logarithm

An unlikely outcome brings more surprise. Two independent observations should bring the sum of their individual surprises. A logarithm makes this addition agree with probability multiplication.

Define the self-information of an outcome:

$$\iota_X(x) = -\log_2 p_X(x).$$

Here $\iota_X(x)$ is the surprise associated with outcome x under model p_X. If independent outcomes have probabilities a and b, their joint probability is ab, and $-\log_2(ab) = -\log_2 a - \log_2 b$. This is why doubling the number of equally likely possibilities adds one bit rather than multiplying the information value.

An outcome with probability one has zero surprise. A probability-1/8 outcome has three bits. These statements describe the probability model before seeing the outcome; observing an unusual symbol does not by itself make that symbol intrinsically valuable.

If a model gives an observed outcome probability zero, its surprise is infinite. In a prediction application, that may indicate a model-support error: the model declared an actually possible event impossible. Adding a small numerical constant changes the model. It can be useful, but it must not be confused with evaluating the original distribution.

The expected surprise is entropy:

$$H(X) = -\sum_x p_X(x) \log_2 p_X(x) = \mathbb{E}[\iota_X(X)].$$

Here $H(X)$ is the entropy of X, and $\mathbb{E}$ averages over its probability distribution. The convention $0 \log 0 = 0$ follows by taking a limit; outcomes that never occur do not contribute to the average.

Entropy is therefore a property of a distribution, not of one realized sample. A rare outcome can have high surprise even when the average entropy is small. An image may contain a visually striking pattern and still be easy for a suitable probability model to predict.

Entropy bounds and the binary example

Uncertainty is zero when the outcome is certain. For a fixed number of possibilities, uncertainty is greatest when all are equally likely.

If an alphabet has K possible values,

$$0 \le H(X) \le \log_2 K.$$

Here $K = |\mathcal{X}|$ is the alphabet size. The lower bound is attained by a point mass; the upper bound is attained by the uniform distribution. If only K_+ outcomes have positive probability, the sharper bound uses $\log_2 K_+$.

For a binary variable with probability p of outcome one,

$$h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

Here h_2 is the binary entropy function and $p \in [0, 1]$. It is symmetric around $p = 1/2$. Its maximum is one bit, and it approaches zero near either endpoint. A highly biased source is easier to predict than a fair source even though both use the same two-symbol alphabet.

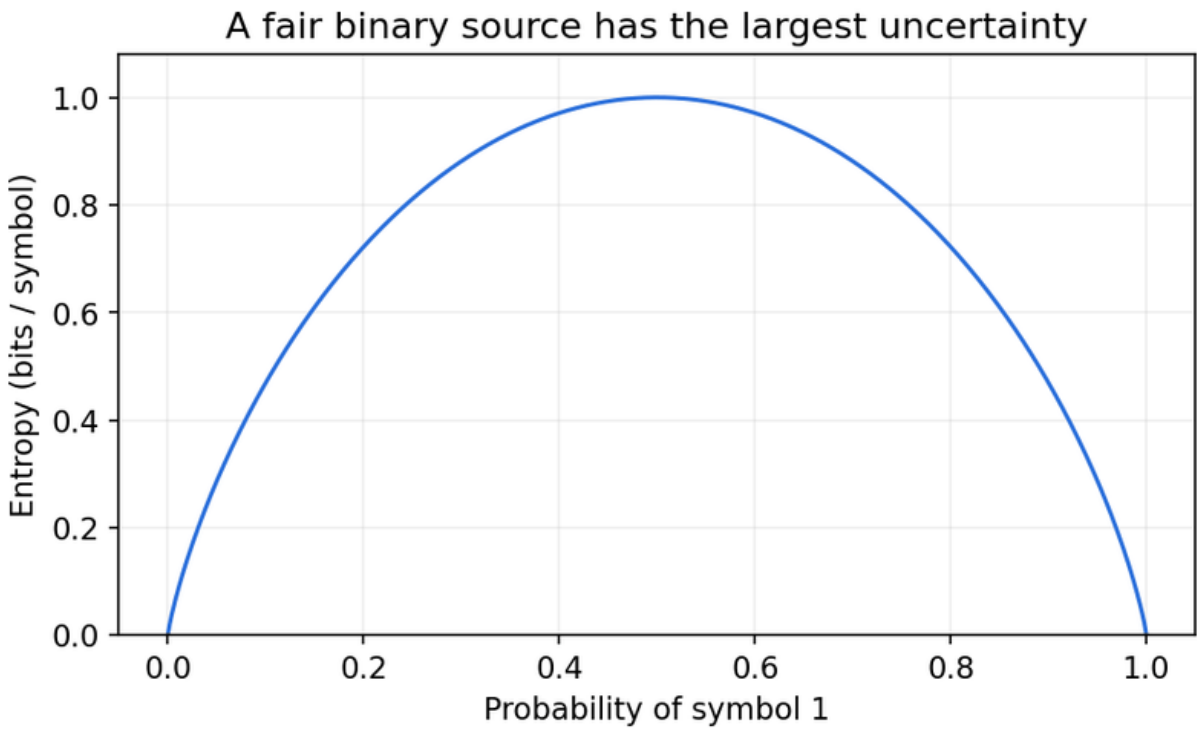


Figure 1 — Analytical binary entropy. The number of possible symbols stays fixed while their probabilities change.

Entropy is concave in the distribution. Mixing two source distributions while hiding which one was selected cannot reduce entropy below their weighted average. Revealing that selection may remove part of the added uncertainty. This explains why mixtures often appear less predictable than their individual components.

The maximum-entropy bound can be proved using relative entropy to a uniform model; we will obtain it after defining divergence. The proof is useful because the same nonnegativity argument will also establish why conditioning reduces average entropy and why mutual information cannot be negative.

Joint entropy and the chain rule

Describing two variables can be cheaper than describing each independently. Once the first is known, shared structure may make the second easier to predict.

Joint entropy is

$$H(X,Y) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{XY}(x,y).$$

Here $H(X,Y)$ measures uncertainty about the pair (X,Y). Conditional entropy is

$$H(Y \mid X) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{Y|X}(y \mid x).$$

Here $H(Y \mid X)$ is average remaining uncertainty about Y when X is available. It weights the entropy of each conditional distribution by the probability of the corresponding x.

Using $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$ gives

$$H(X,Y) = H(X) + H(Y \mid X).$$

Every term refers to the same joint model. Taking the logarithm of the product creates two terms, and summing the first over y leaves the marginal $p_X(x)$. This derivation explains the identity rather than merely providing a formula to memorize.

For a sequence $X_1, \ldots, X_n$:

$$H(X_1, \ldots, X_n) = \sum_{i=1}^{n} H(X_i \mid X_1, \ldots, X_{i-1}).$$

Here n is the number of variables and i is the sequence index; the first term has no conditioning variables. This rule requires no independence assumption. Under independence, every conditional term equals its marginal entropy, so the joint entropy is the sum.

For an image, this distinction matters. A histogram describes marginal pixel frequencies but discards spatial dependencies. Two images with the same histogram can have very different predictability when nearby pixels are known. The sequence model determines whether those dependencies can shorten a description.

Conditioning helps on average, not necessarily for every outcome

Knowing another variable cannot make the optimal average prediction worse. But a particular observation can reveal that you are in an unusually uncertain situation.

$$H(X \mid Y) \le H(X), \qquad H(X,Y) \le H(X) + H(Y).$$

Here the first inequality compares average conditional uncertainty with marginal uncertainty; the second follows from the chain rule. Equality holds when X and Y are independent.

The phrase "on average" is essential. Suppose most days a sensor always outputs zero, but on rare diagnostic days it produces a fair binary outcome. Learning that today is a diagnostic day increases uncertainty relative to the marginal mixture. Learning the day type nevertheless lowers average uncertainty when averaged over both normal and diagnostic days.

Another distinction concerns more conditioning:

$$H(X \mid Y, Z) \leq H(X \mid Y).$$

Here Z is additional information. This inequality concerns the entropies of one fixed joint distribution. It does not say that a finite neural network trained with extra inputs must generalize better. Optimization, limited data and spurious correlations can prevent a learned predictor from reaching the optimal conditional prediction.

Conditional entropy is nonnegative for finite variables. Later, differential entropy of continuous variables will behave differently. Do not transfer every finite-alphabet inequality to raw continuous entropy without checking its assumptions.

A numerical conditioning check

Make the diagnostic-day example explicit: let $Y = 1$ denote a diagnostic day with probability 0.1. On ordinary days $X = 0$; on diagnostic days X is a fair bit. Then the marginal probability of $X = 1$ is $0.1 \times 0.5 = 0.05$, so $H(X) = h_2(0.05) = 0.286397$ bits.

Knowing that this is a diagnostic day raises uncertainty to $H(X \mid Y = 1) = 1$ bit. Averaging over all days instead gives $H(X \mid Y) = 0.9 \times 0 + 0.1 \times 1 = 0.1$ bits. The conditioning inequality holds because it compares 0.1 with 0.286397, rather than comparing every individual conditional entropy with the marginal entropy. The day label removes $I(X; Y) = 0.186397$ bits on average.

Relative entropy measures probability mismatch

A probability model can waste description length by expecting the wrong outcomes. Relative entropy measures that mismatch in the direction of the distribution that actually generates observations.

$$D(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}.$$

Here p and q are probability mass functions on the same alphabet, and $D(p\|q)$ is their relative entropy, also called Kullback–Leibler divergence [1]. Terms with $p(x) = 0$ contribute zero. If $p(x) > 0$ but $q(x) = 0$, the divergence is infinite.

Nonnegativity follows from concavity of the logarithm:

$$\begin{aligned} -D(p\|q) &= \mathbb{E}_p\left[\log_2 \frac{q(X)}{p(X)}\right] \\ &\leq \log_2 \sum_{x:p(x)>0} q(x) \leq 0. \end{aligned}$$

Here $\mathbb{E}_p$ means expectation under p. The first inequality is Jensen's inequality; the final sum is at most one. Equality requires matching distributions. This proof also handles a model that places some probability outside the source support.

The direction matters. $D(p\|q)$ generally differs from $D(q\|p)$. Divergence is not a metric: it does not obey all the requirements of a geometric distance, including symmetry and a general triangle inequality. Calling it a "distance" informally should not justify operations that require a genuine metric.

Compare a fair source with a model that gives probability 0.9 to outcome one. The model is overconfident in one direction and pays for the frequent unexpected zeros. Reversing the distributions describes a different experiment and a different average penalty.

For a uniform distribution $u(x) = 1/K$, expanding the divergence gives $D(p\|u) = \log_2 K - H(p)$. Since it is nonnegative, the entropy upper bound follows. The equality condition also establishes why the uniform source uniquely maximizes entropy on a fixed finite alphabet.

Cross-entropy connects prediction and coding

Cross-entropy scores observations using a proposed probability model. It contains both the source's unavoidable uncertainty and the cost of choosing an inaccurate model.

$$H(p, q) = -\sum_x p(x) \log_2 q(x) = H(p) + D(p\|q).$$

Here $H(p, q)$ is cross-entropy, $H(p)$ is the entropy of the true distribution, and q supplies the predicted probabilities. Under idealized probability coding, $-\log_2 q(x)$ is a description-length target. Actual codes also involve integer lengths, framing, model description and implementation overhead.

If p stays fixed while parameters change q, minimizing population cross-entropy also minimizes divergence. With data, the empirical mean negative log-probability estimates this objective:

$$\widehat{H}_N = -\frac{1}{N} \sum_{j=1}^{N} \log_2 q_\theta(x_j).$$

Here N is the number of observations, x_j is observation j, and θ denotes model parameters. The hat marks an empirical quantity. Its connection to the population average depends on how observations were sampled and whether they were reused to select the model.

A classifier applies the same idea conditionally to labels given features. A generative model applies it to observations or sequences. A low training value is evidence about fit to training observations, not automatically about future data. This distinction connects information theory to the evaluation rules in the ML/DL track.

When training code uses natural logarithms, the loss is in nats. Multiplying by a positive constant changes gradient scale but not the mathematical optimum. Comparing numerical losses across papers requires consistent units, normalization and prediction targets.

Worked example: one table, three information questions

Read the following table as a population model: each cell is the probability of observing that pair. It describes two fair bits that agree more often than they disagree.

Joint probability	$Y = 0$	$Y = 1$	Row total
$X = 0$	0.4	0.1	0.5
$X = 1$	0.1	0.4	0.5
Column total	0.5	0.5	1.0

Rows and columns sum to the marginal probabilities. Both marginals are fair, so $H(X) = H(Y) = 1$ bit. After observing $X = 0$, divide that row by its total 0.5: the conditional probabilities of Y are 0.8 and 0.2. The other row reverses them and has the same entropy.

$$H(Y \mid X) = 0.5h_2(0.2) + 0.5h_2(0.2) = 0.721928,$$
$$H(X, Y) = 1 + 0.721928 = 1.721928,$$
$$I(X; Y) = 1 - 0.721928 = 0.278072.$$

Here h_2 is binary entropy, $H(Y \mid X)$ averages the two conditional rows, $H(X, Y)$ describes the pair, and $I(X; Y)$ is the reduction in uncertainty about either bit after seeing the other. All displayed values are in bits and rounded to six decimals; the code below computes them without rounding.

The same mutual information comes from comparing this joint table with the independent table, which has probability 0.25 in every cell. In agreeing cells the ratio is $0.4/0.25 = 1.6$; in disagreeing cells it is $0.1/0.25 = 0.4$. Their weighted log ratios sum to 0.278072. A single cell's log ratio can be negative even though the average divergence is nonnegative.

Now keep a fair one-bit source but predict probabilities 0.1 and 0.9. Its entropy stays one bit. Its cross-entropy becomes $-0.5 \log_2(0.1) - 0.5 \log_2(0.9) = 1.736966$ bits, leaving 0.736966 bits of model mismatch. The joint-table question measures dependence; this prediction question measures a wrong model. Both use divergence, but they compare different distributions.

This table also gives a simple interpretation for classification. Predicting $Y = X$ is correct with probability 0.8, while the conditional distribution still contains 0.721928 bits of uncertainty. Accuracy counts correct decisions; entropy evaluates an entire predictive distribution. Neither number can be substituted for the other.

Code: exact finite-table checks

The following complete CPU example masks zero-probability terms instead of replacing them with a small constant. The helper functions also check that inputs are normalized probability arrays.

```
import numpy as np

def probability(a):
```

```
    a = np.asarray(a, dtype=float)
    assert np.all(a >= 0) and np.isclose(a.sum(), 1)
    return a

def entropy(a):
    a = probability(a)
    positive = a > 0
    return -np.sum(a[positive] * np.log2(a[positive]))

def divergence(p, q):
    p, q = probability(p), probability(q)
    assert p.shape == q.shape
    positive = p > 0
    if np.any(q[positive] == 0):
        return np.inf
    return np.sum(p[positive] * np.log2(p[positive] / q[positive]))

joint = probability([[0.4, 0.1], [0.1, 0.4]])
px, py = joint.sum(axis=1), joint.sum(axis=0)
independent = np.outer(px, py)
hx, hy, hxy = entropy(px), entropy(py), entropy(joint)
information = divergence(joint, independent)
print("H(X), H(Y), H(X,Y):", hx, hy, hxy)
print("H(Y|X), I(X;Y):", hxy - hx, information)
assert np.isclose(information, hx + hy - hxy)
assert np.isclose(divergence(independent, independent), 0)

p, q = probability([0.5, 0.5]), probability([0.1, 0.9])
cross = -np.sum(p * np.log2(q))
print("Source entropy / cross-entropy / mismatch:",
      entropy(p), cross, divergence(p, q))
assert np.isclose(cross, entropy(p) + divergence(p, q))
assert np.isinf(divergence(probability([0.5, 0.5]), probability([0, 1])))
```

The identities are checked on a specified population table. An empirical table built from finite observations adds estimation uncertainty; the algebra can be exact while the estimated value is statistically inaccurate.

Mutual information measures dependence

Two variables share information when knowing one changes what we expect about the other. Shared information is not limited to linear relationships.

$$\begin{aligned} I(X;Y) &= D(p_{XY} \| p_X p_Y) \\ &= H(X) + H(Y) - H(X,Y) \\ &= H(X) - H(X \mid Y). \end{aligned}$$

Here $I(X;Y)$ is mutual information and $p_X p_Y$ is the product of the marginals. It is nonnegative, symmetric and zero exactly under independence. Its upper bound is the smaller of the two

marginal entropies.

The divergence form asks how different the actual pair distribution is from an independent pair model. The entropy form asks how much average uncertainty one variable removes about the other. These viewpoints agree but emphasize different applications.

Correlation is narrower: zero correlation does not generally establish independence. Nonlinear dependencies can be strong even when a particular linear summary is zero. Conversely, estimating mutual information well in a high-dimensional space is often harder than estimating a correlation.

Conditional mutual information is

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z).$$

Here Z specifies the conditioning information. This quantity averages dependence within each conditional distribution. It is nonnegative, but it can be larger or smaller than unconditional $I(X;Y)$.

For independent fair binary X and Y, let $Z = X \oplus Y$ be their parity. Unconditionally, X tells us nothing about Y. Given Z, knowing X determines Y. Thus the unconditional mutual information is zero and the conditional value is one bit. Conditioning can reveal dependencies rather than merely subtracting a fixed amount of information.

The information chain rule is

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y).$$

Here (Y,Z) is a joint observation. The second term measures the extra information supplied by Z after Y is already available. Adding both unconditional values would double-count shared evidence or miss joint effects.

Mutual information as average predictive improvement

Expand the joint-distribution ratio using $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$. The marginal factor cancels, giving

$$I(X;Y) = \sum_x p_X(x) D(p_{Y|X=x} \| p_Y).$$

Here $p_{Y|X=x}$ is the conditional distribution of Y after observing $X = x$, p_Y the marginal prediction made without that observation, and divergence uses bits. This form expresses information as the average change from an unconditional prediction to a conditional one.

It also proves independence is the equality case. Every conditional divergence is nonnegative. Their weighted sum is zero only when each positive-probability x leaves the prediction unchanged, which is exactly the finite-table independence condition. In the worked table, each conditional row differs from the fair marginal by the same divergence, $1 - h_2(0.2)$.

Under logarithmic prediction loss, the best unconditional predictor pays $H(Y)$, and the best conditional predictor pays $H(Y \mid X)$. Their difference is mutual information. This is a population

statement about optimal distributions: a fitted predictor can fail to realize the gain because it lacks data, capacity or an adequate optimization procedure. The derivation identifies the achievable predictive benefit and the reason its empirical estimate needs separate validation.

Data processing, invertibility and added randomness

Processing cannot create new evidence about an original source if it has access only to the current observation. It can add random variation, but variation is not the same as information about that source.

A Markov relation $X \to Y \to Z$ means

$$p(x, y, z) = p_X(x)p_{Y|X}(y \mid x)p_{Z|Y}(z \mid y).$$

Here Z is generated from Y without extra access to X. This condition implies the data-processing inequality:

$$I(X; Z) \leq I(X; Y).$$

The same X appears on both sides. The inequality does not assert $H(Z) \leq H(Y)$ for arbitrary randomized processing. Adding independent noise can increase output entropy while decreasing information about the input.

A useful proof expands $I(X; Y, Z)$ in two orders. The Markov relation makes $I(X; Z \mid Y) = 0$. The other expansion includes nonnegative $I(X; Y \mid Z)$, leaving $I(X; Y) = I(X; Z) + I(X; Y \mid Z)$. Equality means that Z loses no relevant information about X beyond what Y already supplies.

A deterministic transformation obeys $H(f(X)) \leq H(X)$ because it merges possible outcomes. If it is one-to-one on the support, no uncertainty is lost. An invertible transformation of either variable also preserves mutual information. Compression through a many-to-one quantizer is different: it can discard dependencies.

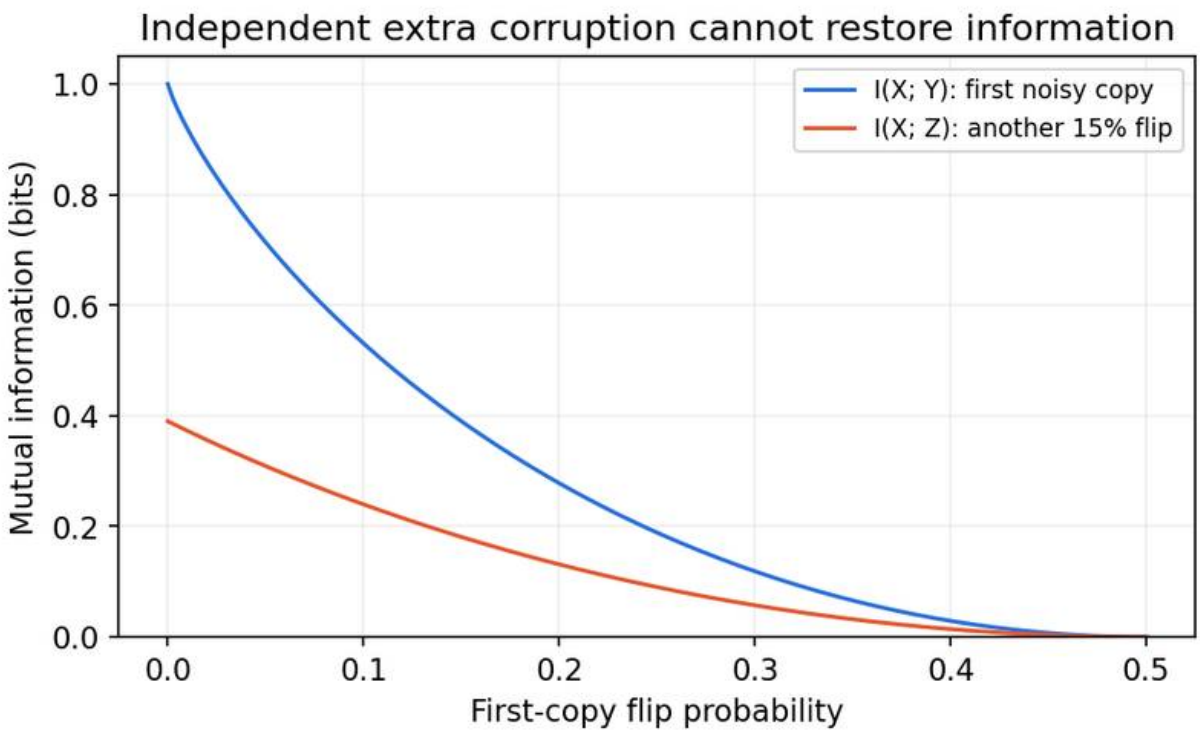


Figure 2 — Analytical data processing. The second channel independently flips its input with probability 0.15; the curves compare information about the same original source.

Code: processing and parity

This second standalone example constructs the full three-variable probability table. It checks the processing inequality and the parity example without a fitted estimator.

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    assert np.all(p >= 0) and np.isclose(p.sum(), 1)
    q = p[p > 0]
    return -np.sum(q * np.log2(q))

def mutual(joint):
    return entropy(joint.sum(axis=1)) + entropy(joint.sum(axis=0)) - entropy(joint)

pxy = np.array([[0.4, 0.1], [0.1, 0.4]])
channel = np.array([[0.85, 0.15], [0.15, 0.85]])
pxyz = pxy[:, :, None] * channel[None, :, :]
pxz = pxyz.sum(axis=1)
print("Before / after processing:", mutual(pxy), mutual(pxz))
assert mutual(pxz) <= mutual(pxy) + 1e-12
```

```
parity = np.zeros((2, 2, 2))
for x in range(2):
    for y in range(2):
        parity[x, y, x ^ y] = 0.25
pxz, pyz = parity.sum(axis=1), parity.sum(axis=0)
pz = parity.sum(axis=(0, 1))
conditional = entropy(pxz) + entropy(pyz) - entropy(pz) - entropy(parity)
print("Parity I(X;Y) / I(X;Y|Z):",
      mutual(parity.sum(axis=2)), conditional)
assert np.isclose(conditional, 1)
```

When a reported estimated mutual information violates data processing, possible causes include sampling error, different estimator biases or a violated Markov assumption. The exact theorem has not ceased to hold.

Estimation requires more than an identity

Counting observed outcomes is easy. Deciding how much unobserved probability remains is not. Large alphabets and strong dependence can make information estimates fragile.

For counts N_x among N observations, the empirical distribution and plug-in entropy are

$$\hat{p}_N(x) = \frac{N_x}{N}, \qquad \widehat{H}_N = -\sum_x \hat{p}_N(x) \log_2 \hat{p}_N(x).$$

Here N_x is the count of outcome x, N is the total sample count and hats denote estimates. Unobserved outcomes have empirical probability zero even if their true probability is positive.

For a fixed finite alphabet and independent sampling, the estimate converges under suitable conditions as samples grow. At small sample sizes, it typically underestimates entropy because the empirical distribution misses possible outcomes and appears too concentrated. Estimating a joint distribution requires covering a larger alphabet than either marginal alone.

Quantization also defines the variable. Binning a continuous feature into 8 levels and into 256 levels asks different finite-alphabet questions. Finer bins can add genuine resolution but also create more sparse cells. A smooth-looking information curve does not remove that trade-off.

The following standalone simulation checks average plug-in estimates for a known uniform source. It uses repeated synthetic samples, not a confidence claim from one draw.

```
import numpy as np

rng = np.random.default_rng(31)
alphabet = 20
truth = np.log2(alphabet)
for n in [40, 400, 4000]:
    estimates = []
    for _ in range(200):
```

```
        sample = rng.integers(alphabet, size=n)
        p = np.bincount(sample, minlength=alphabet) / n
        positive = p > 0
        estimates.append(-np.sum(p[positive] * np.log2(p[positive])))
    print("N:", n, "true entropy:", round(truth, 4),
          "mean estimate:", round(np.mean(estimates), 4))
assert abs(np.mean(estimates) - truth) < 0.02
```

For practical work, specify the sampling unit, feature transformations, estimator family, hyperparameter selection and uncertainty assessment. Repeated frames from one recording are not equivalent to independently sampled scenes. Evaluating an estimator on distributions with known information values is a useful first check.

Modern view

The basic quantities remain the same, but current applications often involve high-dimensional learned representations. The difficult step is frequently estimating or optimizing the quantity under a model, rather than writing its definition.

Butakov and colleagues [3] study mutual-information estimation through normalizing flows in NeurIPS 2024. Their method uses invertible transformations to simplify estimation while preserving the target information under stated assumptions. This illustrates a distinction: changing coordinates can help an estimator without creating information. The paper is an original method, not a survey or evidence that every neural estimator is unbiased.

Aubret, Matignon and Hassas's 2023 survey [4] organizes intrinsic motivation through **surprise, novelty and skill learning**. It shows why an information quantity must be tied to its target: unpredictable observations, newly visited situations and controllable skills are different objectives. Our interpretation is that "maximize entropy" needs a specified variable and conditioning structure before it becomes a useful design rule.

El Gamal's 2026 *Information Theory and Statistical Learning* manuscript [5] connects divergence-based objectives with predictive and generative modeling. It is a **draft chapter proposed for a forthcoming third edition**, not the published third edition or a replacement for the second-edition chapter numbers used here.

Open problems include reliable estimation in sparse high-dimensional spaces, comparison of estimators with different biases, dependence-aware evaluation and the relationship between a convenient information objective and the actual task. A learning method can improve an estimated lower bound without proving that its downstream decisions improved.

Use this chapter as a checklist when reading such claims: Which random variables? Which joint distribution? Which units? Is the number exact, estimated, bounded or optimized? What sup-

ports the conditional-independence assumption? These questions connect mathematical precision with useful empirical evidence.

Key takeaways

- Entropy is expected surprise under a probability distribution, not a property of one observed value.
- Chain rules account for shared structure and require no independence assumption.
- Conditioning reduces average uncertainty; individual conditional outcomes can behave differently.
- Relative entropy is directional, support-sensitive and nonnegative.
- Cross-entropy separates source uncertainty from model mismatch.
- Mutual information measures dependence; conditional dependence can differ sharply from unconditional dependence.
- Data processing limits evidence about a fixed source, while output randomness may increase.
- Empirical information estimates inherit sampling, quantization and model assumptions.

Exercises

1. A three-symbol source has probabilities $(1/2, 1/4, 1/4)$. Compute each outcome's surprise and the entropy. Compare it with a uniform three-symbol source.

   Hint

   Average the logarithmic surprises with their probabilities.

2. Construct a joint binary table with uniform marginals and mutual information zero, then another with mutual information one bit. Explain their conditional entropies.

   Hint

   Compare an outer product with a diagonal table.

3. Expand $D(p\|u)$ for a uniform model on K symbols and use nonnegativity to prove the entropy upper bound. State its equality condition.

   Hint

   The divergence becomes $\log_2 K - H(p)$.

4. Give a concrete model where $H(X \mid Y = y) > H(X)$ for one value y, while the average conditioning inequality still holds.

Hint

Use rare fair diagnostic days and common deterministic days.

5. Extend the processing example with a third noisy channel. Check mutual information after every stage and identify the independence assumptions.

Hint

Multiply transition probabilities before summing out intermediate states.

6. Repeat the entropy-estimation simulation with a larger alphabet and the same sample sizes. Describe which conclusions concern the estimator rather than the true entropy.

Hint

Record unseen-symbol counts as well as estimated entropy.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 1–2 and 17. Publisher book and contents.
2. T. Weissman, "Entropy, Relative Entropy, and Mutual Information," Stanford EE276 Lecture 2 notes, 2024. Primary lecture notes.
3. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, "Mutual Information Estimation via Normalizing Flows," NeurIPS, 2024. Proceedings.
4. A. Aubret, L. Matignon and S. Hassas, "An Information-Theoretic Perspective on Intrinsic Motivation in Reinforcement Learning: A Survey," *Entropy*, 25(2), 327, 2023. Published survey.
5. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

IT 02 • Typical Sequences, AEP and Entropy Rates

Derive why long source blocks concentrate near entropy, connect typical sets to coding, and distinguish stationary dependent sources from independent and nonergodic models.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-02-typicality-entropy-rates/

What you'll learn

- Derive the asymptotic equipartition property for an independent finite source.
- Bound the probability and size of a typical set without confusing the two.
- Explain why the most likely sequence need not be representative.
- Derive an entropy rate for a stationary finite-state Markov source.
- Distinguish stationarity, independence and ergodicity through a concrete counterexample.
- Connect conditional and joint typicality to compression and communication proofs.

The big picture

A single draw is uncertain, yet a long sequence often has stable aggregate behavior. A biased binary source might produce an unpredictable next bit while consistently spending about the same description length per symbol over a large block. Typicality makes that observation precise.

> **Explain like I'm five**
>
> You cannot predict every bead pulled from a bag. But after many pulls, you can predict roughly how many colors will appear and how surprising the whole string will be. Most likely strings then fit into a smaller catalog than all imaginable strings.

This chapter follows Cover and Thomas's Chapters 3–4 [1]. Stanford's primary AEP notes [2] provide a compact independent-source proof. We will derive the result, explain its finite-block limitations, and extend the viewpoint to dependence. Later tutorials turn these existence arguments into actual encoders and decoders.

Typicality describes sequences relative to a specified source distribution. It does not mean visually ordinary, semantically correct or typical of every population. A sample can be probable under a misspecified model and still be unsuitable for the application.

Block probabilities and average surprise

Instead of asking how surprising one symbol is, ask how many bits of surprise the whole block has, then divide by its length. For independent symbols, this becomes an average of individual

surprises.

Let $X_1, \ldots, X_n$ be independent and identically distributed according to a mass function p on finite alphabet $\mathcal{X}$. Define the block $X^n = (X_1, \ldots, X_n)$:

$$p(x^n) = \prod_{i=1}^{n} p(x_i), \qquad J_n = -\frac{1}{n} \log_2 p(X^n) = \frac{1}{n} \sum_{i=1}^{n} -\log_2 p(X_i).$$

Here n is block length, x^n is a particular sequence, i is its position, and J_n is the random average surprise in bits per symbol. The equality uses independence. Its expectation is $H(X)$, the entropy of one source symbol.

The finite-alphabet setting ensures the surprise is integrable on positive-probability outcomes. An alphabet may formally include impossible symbols, but a sampled block does not contain them. The law of large numbers therefore gives

$$J_n \xrightarrow{\Pr} H(X).$$

The arrow denotes convergence in probability: for any fixed positive tolerance, the chance that J_n differs from entropy by more than that tolerance goes to zero. This is the **asymptotic equipartition property**, or AEP [1,2].

"Equipartition" is approximate and asymptotic. It does not say that every block has the same probability, nor that all symbols become equally frequent. It says most probability mass concentrates on blocks with approximately the same exponential probability scale.

For a Bernoulli source with probability p of symbol one, a block containing k ones has

$$-\frac{1}{n} \log_2 p(x^n) = -\frac{k}{n} \log_2 p - \left(1 - \frac{k}{n}\right) \log_2 (1 - p).$$

Here k is the observed count and k/n its empirical fraction. When that fraction approaches p, average surprise approaches binary entropy $h_2(p)$. The number of ones is not generally close to half the block length.

The typical set separates probability from counting

Most likely observations fit into a catalog of plausible blocks. Each block in that catalog may be individually unlikely, but together they carry almost all source probability.

For tolerance $\epsilon > 0$, define

$$A_\epsilon^{(n)} = \left\{ x^n : \left| -\frac{1}{n} \log_2 p(x^n) - H(X) \right| \leq \epsilon \right\}.$$

Here $A_\epsilon^{(n)}$ is the weakly typical set, $|\cdot|$ inside the braces denotes absolute value, and ϵ is a tolerance in bits per symbol. Its elements satisfy

$$2^{-n(H(X)+\epsilon)} \leq p(x^n) \leq 2^{-n(H(X)-\epsilon)}.$$

These are bounds on the probability of one typical block. The AEP separately states that $\Pr(X^n \in A_\epsilon^{(n)})$ approaches one.

For sufficiently large n, suppose that typical-set probability is at least $1-\delta$, where $\delta > 0$ is a chosen probability tolerance. Then

$$(1-\delta)2^{n(H(X)-\epsilon)} \leq |A_\epsilon^{(n)}| \leq 2^{n(H(X)+\epsilon)}.$$

Here $|A_\epsilon^{(n)}|$ denotes the number of typical sequences, not their probability. The upper bound follows because each typical block has probability at least $2^{-n(H+\epsilon)}$ and the total probability cannot exceed one. The lower bound follows because the set carries probability at least $1-\delta$ while each of its blocks has probability at most $2^{-n(H-\epsilon)}$.

Separating ϵ and δ makes the roles clear: one controls description-length tolerance, the other controls excluded probability mass. A textbook may use the same small parameter for both after choosing a sufficiently large block. That shorthand should not hide the distinction in finite calculations.

There are $|\mathcal{X}|^n$ possible sequences. If $H(X) < \log_2 |\mathcal{X}|$, the typical catalog has an exponentially smaller scale. This is the counting fact behind compression: not every conceivable block needs a short description if a small error probability is allowed.

The most probable block is not necessarily typical

A single very likely-looking string can be unrepresentative. Many slightly less likely strings can collectively dominate the probability.

For an independent binary source with $p = 0.2$, the all-zero block is the most likely individual block. Its average surprise is

$$-\frac{1}{n}\log_2(0.8^n) = -\log_2(0.8).$$

Here 0.8 is the zero probability. This value differs from $h_2(0.2)$. For a narrow tolerance, the all-zero string remains outside the typical set even though no individual string is more probable.

Its total probability 0.8^n tends to zero. Meanwhile there are many blocks with approximately one fifth ones; their collective probability approaches one. Probability mass and individual ranking are different objects.

The same distinction appears in generative modeling. A maximum-probability sequence need not resemble a representative sample from the model. Typicality provides a mathematical example of the difference, not a guarantee that a particular sampling algorithm produces useful language or images.

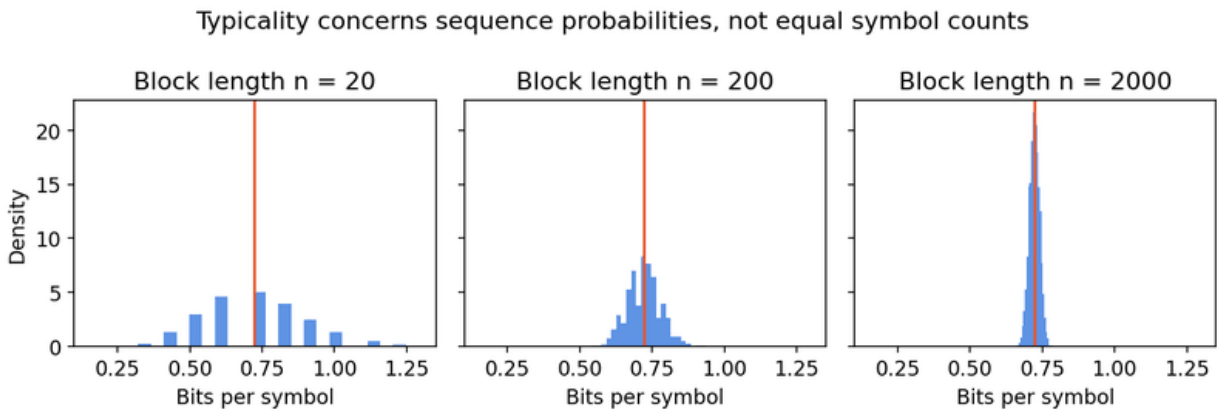


Figure 1 — Synthetic binomial draws, 5,000 blocks per length. The orange line is the exact source entropy; the horizontal axis describes surprise, not the fraction of ones.

Code: exact typical probability and finite counts

The first complete example uses a binomial count distribution to sum typical-set probability exactly up to floating-point arithmetic. A separate small enumeration checks the cardinality bound.

```
import itertools
import numpy as np
from scipy.stats import binom

p, epsilon = 0.2, 0.1
entropy = -p * np.log2(p) - (1 - p) * np.log2(1 - p)
for n in [20, 200, 2000]:
    k = np.arange(n + 1)
    per_symbol = -(k / n) * np.log2(p) - (1 - k / n) * np.log2(1 - p)
    typical = np.abs(per_symbol - entropy) <= epsilon
    probability = binom.pmf(k, n, p)[typical].sum()
    print("Length:", n, "typical probability:", probability)

n, epsilon = 10, 0.15
count, mass = 0, 0.0
for block in itertools.product([0, 1], repeat=n):
    k = sum(block)
    probability = p**k * (1 - p)**(n - k)
    surprise = -np.log2(probability) / n
    if abs(surprise - entropy) <= epsilon:
        count += 1
        mass += probability
assert count <= 2**(n * (entropy + epsilon))
assert count >= mass * 2**(n * (entropy - epsilon))
print("Small-block typical count / mass:", count, mass)
```

Small-block typical mass need not be close to one. “Asymptotic” is a statement about growing

block length with a fixed tolerance, not permission to apply the large-block conclusion at any length.

Worked example: count strings, then add their probabilities

Use the small enumeration above: $n = 10$, $p = 0.2$ and $\epsilon = 0.15$. A block with k ones has average surprise $0.321928 + 0.2k$ bits per symbol, while source entropy is 0.721928. Only $k = 2$ satisfies the tolerance. This determines membership before we count anything.

There are $\binom{10}{2} = 45$ such strings. Each has probability $0.2^2 \times 0.8^8 = 0.0067108864$, so the typical set has probability $45 \times 0.0067108864 = 0.301989888$. In contrast, the single all-zero string has probability $0.8^{10} = 0.1073741824$. It outranks each typical string individually, but carries less probability than the 45 strings together.

These calculations answer three separate questions: **membership** is decided by surprise tolerance,

cardinality counts accepted strings, and **mass** sums their probabilities. A catalog of 45 strings needs six fixed-length index bits because five bits identify only 32 alternatives. Yet that catalog covers only about 30.2% of this short-block source. The length-10 catalog alone is therefore a poor almost-lossless code, despite its rate of six tenths of a bit per symbol.

The entropy bound does not fail. Its high-probability conclusion requires longer blocks at this fixed tolerance. Cardinality rounding and excluded probability are finite-block costs that the limiting argument does not erase from a short experiment.

For comparison, scoring the same blocks with a fair-bit model would give exactly one bit per symbol to every block. That model cannot distinguish the all-zero string from a two-one string by surprise. The underlying source has not changed; the scoring model has. A typical-set computation must always specify which of those distributions supplies the probabilities.

Empirical frequencies and combinatorial intuition

A block's symbol frequencies define a group of many strings with the same source probability. Counting that group explains how individually unlikely strings can dominate collectively.

For a binary block with exactly k ones, the number of arrangements and the total probability of that count are

$$N_{n,k} = \binom{n}{k}, \qquad \Pr(K_n = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Here $N_{n,k}$ counts strings, K_n is the random number of ones, $\binom{n}{k}$ is the binomial coefficient, and p is the source probability of one. Every arrangement with that count has the same probability, but the number of arrangements varies dramatically.

At the extremes, there is just one all-zero string and one all-one string. Near the center, there are many arrangements. For a biased source, the probability assigned to each arrangement and

the number of arrangements pull in different directions; their product peaks near the source's expected count.

This viewpoint also clarifies why empirical-frequency typicality and surprise typicality are different. Under a fair binary source, every length-n string has probability 2^{-n} and average surprise exactly one bit. Therefore every possible string is weakly typical for any positive tolerance, including all zeros.

The probability of observing an all-zero string still tends to zero. A frequency-based typical set can exclude it by requiring the fraction of ones to stay near one half. Its probability also tends to one, but it has a stricter membership rule.

Neither definition is "wrong." Each is useful for a different proof. A probability-scale statement may need only weak typicality; a construction that matches empirical conditional distributions may require stronger frequency control. Always check the definition actually used, especially when a proof moves from marginal source coding to joint input-output relationships.

A source-coding argument and its limitation

Number the typical blocks and send their catalog index. With enough symbols in each block, this uses close to entropy bits per symbol while failing only on a small excluded set.

Since the typical set has at most $2^{n(H+\epsilon)}$ members, an index takes at most approximately $n(H+\epsilon)$ bits, plus rounding and framing. A decoder uses the same catalog to reconstruct the selected block.

This is an **almost-lossless fixed-length** construction. It does not encode every possible block correctly. It can reserve one extra symbol for failure, but that flag does not reconstruct an atypical block. To make a truly lossless scheme, provide an escape representation with enough information to recover it.

A simple converse explains why rates well below entropy cannot cover most probability mass. Suppose a code can correctly represent at most 2^{nR} blocks. Intersect those blocks with the typical set. Each typical block has probability at most $2^{-n(H-\epsilon)}$, so their accepted mass is at most

$$2^{nR}2^{-n(H(X)-\epsilon)} = 2^{-n(H(X)-\epsilon-R)}.$$

Here R is the fixed code rate in bits per source symbol. If $R < H(X)-\epsilon$, this typical accepted mass approaches zero. The omitted atypical mass also approaches zero, so the code cannot reconstruct the source with high probability.

This argument assumes the encoder-decoder pair has a limited number of distinct messages. It is a counting proof, not an efficient implementation. Enumerating a huge catalog is generally impractical. The next chapter studies prefix coding, probability coding and adaptive models that turn theoretical rates into computable procedures.

Add an escape branch to make the catalog lossless

A catalog that sometimes fails can be turned into a code that reconstructs every allowed finite block. Send a one-bit branch flag. If the block is typical, send its fixed-length catalog index. Otherwise, send an uncompressed representation of the block using a fixed number of bits per symbol.

Let $m = \lceil \log_2 |\mathcal{X}| \rceil$ and $\ell_n = \lceil \log_2 |A_\epsilon^{(n)}| \rceil$ for a nonempty typical set. The expected length satisfies

$$\frac{\mathbb{E}[L_n]}{n} = \frac{1}{n} + \frac{P_T \ell_n}{n} + (1 - P_T)m \le \frac{2}{n} + P_T(H(X) + \epsilon) + (1 - P_T)m.$$

Here L_n is total block length in bits, m is the fixed raw symbol width, ℓ_n the typical index width, $P_T = \Pr(X^n \in A_\epsilon^{(n)})$ the typical probability, and $n, \epsilon, H(X)$ retain their earlier meanings. The equality counts the flag and the two branches; the inequality uses typical-set cardinality and upward integer rounding.

The decoder reads the flag first, so it knows which fixed branch length to read. The typical branch returns a shared catalog entry; the escape branch reconstructs the raw symbols. No block is silently dropped. If a small finite block has an empty typical set, use only the raw branch until a usable catalog exists.

For fixed ϵ, AEP gives $P_T \to 1$, making the escape contribution vanish per symbol. The flag and rounding costs also vanish as n grows. Then the expected rate approaches no more than $H(X) + \epsilon$. Choosing smaller tolerance after establishing sufficiently long blocks brings the rate arbitrarily close to entropy.

The construction reveals the difference between fixed-rate almost-lossless and variable-length fully lossless coding. The escape branch can be long on rare blocks, so the latter conclusion concerns **expected length**, not a short bound for every block. It also assumes encoder and decoder share the catalog and source model. This is a proof of attainable rates; explicit enumeration is generally too expensive to be a practical compression algorithm.

Finite-block concentration depends on the source

Long blocks are useful, but how long is enough depends on the source and the tolerance. Rare symbols can produce large surprise fluctuations.

For independent symbols with finite surprise variance,

$$\mathrm{Var}(J_n) = \frac{V}{n}, \qquad V = \mathrm{Var}[-\log_2 p(X)].$$

Here V is the source surprise variance and Var denotes variance. This follows from adding independent variances and dividing by n^2. A general variance bound gives

$$\Pr(|J_n - H(X)| > \epsilon) \le \frac{V}{n\epsilon^2}.$$

Here ϵ is the same surprise tolerance as before. The bound can be loose, but it makes the dependence explicit. Very small probabilities can increase surprise variance, and demanding a narrow tolerance increases the required block length.

An empirical standard deviation across blocks is useful, but it is not a universal finite-block coding guarantee. Dependence changes the variance calculation; rare events may be missed by a small simulation; model mismatch changes the center of concentration.

If a source p is scored by model q, independent block average surprise converges to $H(p, q)$ under integrability conditions, not to $H(p)$. The difference is relative entropy. Typical under the model and typical under the actual source are separate questions.

Entropy rates account for dependence

Neighboring symbols often help predict one another. The information required per symbol over a long sequence can therefore be smaller than the entropy of a single marginal symbol.

For a stochastic process $X_1, X_2, \ldots$, define its entropy rate when the limit exists:

$$\bar{H}(X) = \lim_{n \to \infty} \frac{1}{n} H(X_1, \ldots, X_n).$$

Here $\bar{H}(X)$ is the average new uncertainty per symbol and n is block length. This definition concerns block entropy, not a fitted model's loss on one realized sequence.

The chain rule gives

$$\frac{1}{n} H(X^n) = \frac{1}{n} \sum_{i=1}^{n} H(X_i \mid X^{i-1}).$$

Here $X^{i-1} = (X_1, \ldots, X_{i-1})$ is the preceding history, with empty history at $i = 1$. Dependence makes these conditional terms different from marginal entropy.

A process is **stationary** when shifting all time indices leaves every finite joint distribution unchanged. This is stronger than a constant mean or covariance. Under finite-alphabet stationarity, the conditional entropy given a longer past is nonincreasing and bounded below by zero, so its limit exists. The block-average entropy has the same limit.

The equality of these limits comes from averaging a convergent conditional-entropy sequence. Stationarity permits moving a fixed-length history to a common time index, where adding more past information reduces conditional entropy.

Independent identically distributed sources are a special case: each conditional entropy equals $H(X_1)$. A dependent stationary source may have the same marginal distribution at every time while requiring much less new information per step.

Markov sources provide an explicit calculation

A first-order Markov model needs only the current state to predict the next. Its long-run uncertainty averages the uncertainty in each transition row.

Let a finite Markov source have transition matrix P:

$$P_{ij} = \Pr(X_{t+1} = j \mid X_t = i), \qquad \pi_j = \sum_i \pi_i P_{ij}.$$

Here i and j index source states, t is time, P_{ij} is a transition probability, and π is a stationary state distribution. Each row of P sums to one; π is nonnegative and sums to one.

If the process starts in π, stationarity and the Markov property give

$$\bar{H}(X) = H(X_{t+1} \mid X_t) = -\sum_i \pi_i \sum_j P_{ij} \log_2 P_{ij}.$$

The inner sum is the entropy of transition row i; the outer sum weights rows by stationary occupancy. The finite block entropy is

$$H(X^n) = H(\pi) + (n-1)\bar{H}(X).$$

Here $H(\pi)$ is the entropy of the initial state. The chain rule contributes one initial term and $n-1$ identical transition terms.

For a symmetric binary source that flips with probability a, the stationary distribution is uniform. Its marginal entropy is one bit, while its entropy rate is $h_2(a)$. With $a = 0.1$, most next symbols repeat the current symbol, reducing fresh uncertainty.

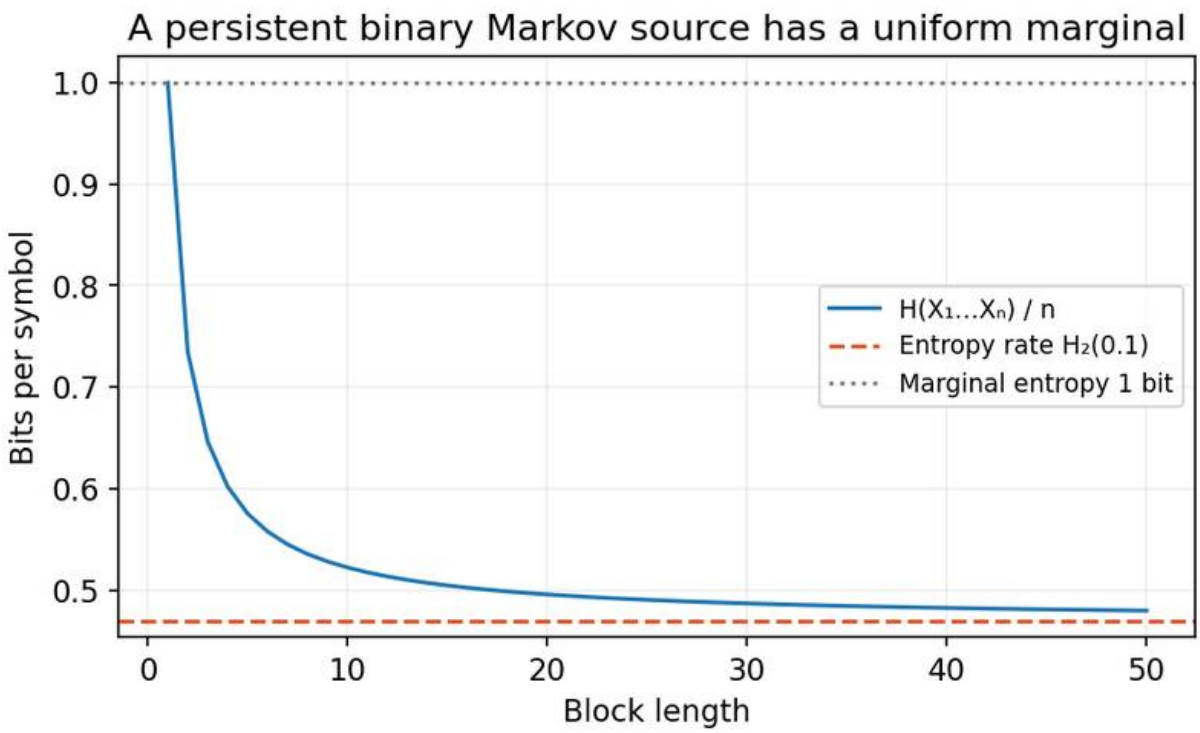


Figure 2 — Analytical stationary binary Markov example. Uniform single-symbol frequencies do not imply an independent source.

Code: predictability without changing the marginal

This standalone experiment generates stationary binary Markov blocks using independent flip events. It compares observed average surprise with the exact block-entropy formula.

```
import numpy as np

rng = np.random.default_rng(32)
a = 0.1
rate = -a * np.log2(a) - (1 - a) * np.log2(1 - a)
for n in [64, 256, 1024]:
    initial = rng.integers(2, size=1000)
    flips = rng.binomial(1, a, size=(1000, n - 1))
    states = np.column_stack([
        initial, initial[:, None] ^ np.bitwise_xor.accumulate(flips, axis=1)])
    surprise = 1 + (-flips * np.log2(a) - (1 - flips) * np.log2(1 - a)).sum(axis=1)
    expectation = (1 + (n - 1) * rate) / n
    print("Length:", n, "one-frequency:", states.mean(),
          "mean surprise rate:", surprise.mean() / n,
          "exact entropy per symbol:", expectation)
    assert abs(states.mean() - 0.5) < 0.08
```

```
    assert abs(surprise.mean() / n - expectation) < 0.02
print("Entropy rate:", rate, "marginal entropy:", 1.0)
```

The initial state is sampled uniformly, which makes this source stationary. Starting every block at zero changes finite-block distributions; the asymptotic rate can still agree under suitable transition assumptions, but the exact initial-entropy term changes.

Why a fair marginal can still be cheap to predict

For the symmetric Markov example with flip probability $a = 0.1$, the first bit costs one bit of uncertainty. Each later bit contributes $h_2(0.1) = 0.468996$ bits. At length ten, the block entropy is $1 + 9h_2(0.1) \approx 5.220960$ bits, or 0.522096 bits per symbol. Ten independent fair bits would instead have ten bits of block entropy.

A useful representation is the initial bit followed by nine flip indicators. This mapping is invertible: starting from the first bit, exclusive-or each flip indicator to recover the next state. It makes the dependence visible without throwing away any source information. The indicators are biased independent bits under this particular model, so their distribution is easier to code than ten marginal fair states.

The long-run rate approaches 0.468996 because the one-bit initialization cost is divided by an increasing block length. Predictability comes from the transition relationship, even though every time position is marginally fair. An image histogram similarly cannot reveal all gains available from predicting pixels using their neighbors.

Hidden states and the choice of observation

A transition model may describe an internal state while the receiver sees only a noisy measurement. The observed sequence then has its own prediction problem.

Suppose S_t is a finite hidden Markov state and Y_t an emitted observation. A model may factor a block as

$$p(s^n, y^n) = p(s_1) \prod_{t=1}^{n-1} p(s_{t+1} \mid s_t) \prod_{t=1}^{n} p(y_t \mid s_t).$$

Here s^n is a state sequence, y^n an observation sequence, and the factors describe transitions and emissions. Summing over hidden sequences gives the observed likelihood $p(y^n)$.

The entropy rate of Y is generally not just the stationary average of emission-row entropies. Past observations help infer the hidden state, and residual state uncertainty affects prediction. The relevant terms are $H(Y_t \mid Y^{t-1})$, not only $H(Y_t \mid S_t)$.

This is a useful warning for vision: a model of object dynamics and a model of camera measurements refer to different random processes. More sensor noise may increase observation entropy

while decreasing useful information about the scene. Rate calculations must state which process is being described.

Ergodicity matters for a common typical value

Stationarity says distributions do not drift with time. It does not guarantee that one long realization explores all parts of a mixed population.

Under appropriate stationary ergodic finite-source assumptions, the long-block normalized surprise converges to a single entropy-rate value. Ergodicity rules out a persistent hidden choice that divides the process into separately behaving stationary components.

Consider choosing a component C once: with probability one half, produce zeros forever; with probability one half, produce independent fair bits. Each component is stationary, and the mixture is stationary.

Conditional block entropy is $H(X^n \mid C) = n/2$. The identity

$$H(X^n) = H(X^n \mid C) + I(C; X^n)$$

uses component label C, block X^n and mutual information I. Since $I(C; X^n) \leq H(C) = 1$, the block-average entropy tends to one half bit.

Nevertheless, a realized sequence from the deterministic component has normalized surprise approaching zero; one from the fair component usually has normalized surprise approaching one. There is no single common value near one half for most realizations. An entropy-rate limit alone does not justify an ergodic AEP conclusion.

Code: a stationary nonergodic counterexample

This complete example computes the mixture block probability analytically from the number of ones. It avoids exponentiating a very small fair-block probability before taking its logarithm.

```
import numpy as np

rng = np.random.default_rng(33)
for n in [128, 1024]:
    component = rng.integers(2, size=2000)
    counts = rng.binomial(n, 0.5, size=2000) * component
    # A nonzero sequence can only come from the fair component:
    # its probability is 0.5 * 2**(-n), so surprise is n + 1.
    all_zero_surprise = -np.log2(0.5 + 0.5 * np.exp2(-float(n)))
    per_symbol = np.where(counts == 0, all_zero_surprise, n + 1) / n
    print("Length:", n, "deterministic mean:", per_symbol[component == 0].mean(),
          "fair mean:", per_symbol[component == 1].mean(),
          "fraction within 0.1 of 0.5:", np.mean(abs(per_symbol - 0.5) <= 0.1))
    assert np.mean(abs(per_symbol - 0.5) <= 0.1) == 0
```

This example is a model of persistent heterogeneity, not time drift. A dataset containing different source types can require conditional models or explicit mixtures rather than assuming one long recording represents the whole population.

Joint and conditional typicality

Two individually ordinary sequences can form an extraordinary pair. Communication proofs need the relationship between input and output, not only separate histograms.

For independent draws of pairs (X_i, Y_i) from joint mass function p_{XY}, consider the joint and conditional average surprises:

$$-\frac{1}{n}\log_2 p(X^n, Y^n) \to H(X, Y), \qquad -\frac{1}{n}\log_2 p(Y^n \mid X^n) \to H(Y \mid X).$$

Here arrows mean convergence in probability under the specified independent-pair model. X^n and Y^n are paired blocks; dependence within each pair is allowed.

A jointly typical pair is checked against the joint source relationship. Marginal typicality alone does not ensure this. If Y is a noisy copy of X, two unrelated fair-bit sequences can each have typical marginal counts but the wrong mismatch frequency.

The per-block information density is

$$\imath(x^n; y^n) = \log_2 \frac{p(x^n, y^n)}{p(x^n)p(y^n)}.$$

Here $\imath$ measures evidence for the joint pair relative to an independent pair model. Under independent paired sampling, its normalized average approaches $I(X; Y)$ when integrable.

This connects to channel decoding: a receiver looks for codewords whose relationship to the received block is consistent with the channel. It also connects to matching and representation learning, where positive pairs must be defined carefully. A negative pair drawn from the wrong population changes the reference distribution.

Strong typicality for finite alphabets additionally constrains empirical frequencies of each symbol or pair, respecting impossible events. Weak typicality constrains average log-probability. Neither notion should be silently substituted for the other in a proof whose assumptions use one specific definition.

Modern view

The distinction between average surprise and meaningful output is particularly relevant to modern probability models. Long contexts and learned predictors can model dependence beyond a fixed Markov order, but their loss is still scored under a particular observation and tokenization model.

El Gamal's 2026 proposed learning chapter [3] gives a current divergence-based bridge to predictive and generative models. It is a draft supplement, not a published third edition. Our inference from the source-coding viewpoint is that sequence likelihood should always be reported with its prediction target, units and conditioning context.

Huang and Wu's 2024 review [4] surveys learned image compression through the distinction between coding for human reconstruction and coding for machine analysis. Its taxonomy highlights that a useful compressed representation depends on the consumer. Entropy rate supplies a source-description limit, but it does not alone specify a task's acceptable distortion.

Three practical questions remain open in many applications: how quickly asymptotic behavior becomes useful at actual block lengths; whether a fitted source model remains valid across populations; and how dependencies created by sampling, tokenization and preprocessing affect estimated rates.

Finite evaluation should compare independent blocks or acquisition groups and retain atypical cases. Reporting only average surprise can hide a rare source component with a much higher rate. Measuring likelihood under one model cannot establish that all source components share the same typical set.

The next chapter makes the bridge concrete: a coder must define its probability model, update rules, termination convention and treatment of unexpected symbols. A rate theorem guides those choices; a round-trip test checks whether the actual implementation reconstructs the intended data.

Key takeaways

- AEP concentrates normalized block surprise under stated source assumptions.
- Typical-set probability and cardinality are different quantities with different bounds.
- Most-probable individual blocks need not carry most collective probability mass.
- Typical catalogs explain almost-lossless coding limits, not efficient implementations.
- Entropy rate measures new uncertainty after dependence is exploited.
- Stationarity does not guarantee ergodicity or a common realized surprise rate.
- Joint typicality checks relationships that marginal typicality can miss.
- Finite-block behavior, source mismatch and heterogeneity must accompany asymptotic claims.

Exercises

1. For a Bernoulli source with $p = 0.1$, calculate the all-zero block surprise rate and source entropy. Choose a tolerance that excludes all-zero blocks for every length.

Hint

Compare $-\log_2(0.9)$ with $h_2(0.1)$.

2. Repeat exact typical-mass computation at tolerances 0.05, 0.1 and 0.2. Explain why smaller tolerance can require much longer blocks.

Hint

The average-surprise window narrows.

3. Derive the typical-set lower and upper cardinality bounds using separate ϵ and δ. Identify which individual-probability bound is used in each step.

Hint

Bound a sum by count times the largest or smallest allowed block probability.

4. A binary Markov chain has transition matrix $P = \begin{pmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{pmatrix}$. Find its stationary distribution, marginal entropy and entropy rate.

Hint

Solve $\pi = \pi P$ and weight transition-row entropies.

5. In the nonergodic mixture example, change the component probabilities to one quarter deterministic and three quarters fair. Find the block entropy-rate limit and realized surprise-rate limits.

Hint

The persistent component still prevents one common realized value.

6. Construct two marginally typical fair-bit sequences that are not jointly typical under a 10% flip channel. Explain why separate symbol counts fail to detect the mismatch.

Hint

Use a pair with too many or too few disagreements.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 3–4. Publisher book and contents.
2. T. Weissman, "Asymptotic Equipartition Property," Stanford EE276 Lecture 3 notes, 2024. Primary lecture notes.
3. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.
4. C.-H. Huang and J.-L. Wu, "Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression," *Entropy*, 26(5), 357, 2024. Published review.

IT 03 • Lossless Coding and Universal Compression

Build decodable Huffman and arithmetic examples, derive the entropy limit, and connect adaptive universal coding with prediction and algorithmic complexity.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-03-lossless-universal-coding/

What you'll learn

- Distinguish injective, uniquely decodable and prefix-free descriptions.
- Derive the entropy lower bound using the Kraft–McMillan inequality.
- Construct a Huffman code and test exact reconstruction.
- Explain arithmetic intervals and implement a short exact-arithmetic round trip.
- Relate adaptive probability prediction, universal coding and model overhead.
- Separate computable compression from ideal algorithmic complexity.

The big picture

Compression replaces a message with a shorter representation that a receiver can interpret. The crucial word is **interpret**. Assigning every symbol a short string is insufficient if concatenated strings become ambiguous. A useful coder specifies how a message begins and ends, how probabilities are obtained, and exactly how the receiver reverses the representation.

> **Explain like I'm five**
>
> You and a friend invent nicknames for toys. Common toys get short nicknames. But if two nicknames run together and look like another nickname, your friend cannot unpack the box correctly. A compression rule needs both short names and an unambiguous way to read them.

The reading map is Cover and Thomas's Chapter 5 for lossless codes, Chapter 13 for universal source coding, and Chapter 14 for Kolmogorov complexity [1]. Stanford's variable-length and Huffman lecture notes [2,3] give accessible primary derivations. This tutorial develops original examples rather than reproducing the book's tables or exercises.

The preceding chapter described an almost-lossless typical-set catalog. Here the requirement is exact reconstruction of every allowed message. The price includes escape representations for unexpected symbols and any information needed to identify the model. Later, rate–distortion theory relaxes reconstruction explicitly; it must not be mixed into a claim of losslessness.

What a lossless code must preserve

A compressed representation must identify the original message, including its boundaries. Two different messages cannot become the same transmitted object.

An encoder f and decoder g satisfy

$$g(f(x)) = x$$

for every allowed source message x. Here $f(x)$ is its representation and g is the reconstruction rule. This identity requires f to be injective on the allowed message set.

For symbol-by-symbol codes, a source string is represented by concatenating its symbols' codewords. A **uniquely decodable** code gives a different concatenation for every different source sequence. A **prefix-free** code has no codeword that begins with another complete codeword. Prefix freedom permits decoding immediately upon reaching a leaf of a code tree.

Prefix-free implies uniquely decodable, but the reverse need not hold. An injective mapping of isolated symbols does not even imply unique decodability of concatenations. For example, giving three symbols representations 0, 1 and 01 makes the third symbol indistinguishable from the first followed by the second.

Supplying a separator can repair ambiguity, but the separator occupies transmitted space. Supplying the message length helps determine termination, but length alone does not repair an ambiguous symbol parse. Any side information used by the decoder belongs to the communication protocol and needs explicit accounting.

An empty codeword is possible for a source with only one known symbol and externally supplied length. It is unsuitable for the ordinary multi-symbol tree used below. Our example assigns at least one bit to each symbol and assumes the alphabet, tree and message boundary are shared.

Kraft–McMillan turns a tree into a probability budget

Very short codewords consume a large share of the available binary tree. Too many short words leave no room for the remaining symbols.

For a binary uniquely decodable code with integer lengths ℓ_i,

$$K = \sum_i 2^{-\ell_i} \leq 1.$$

Here i indexes source symbols, ℓ_i is the number of bits in symbol i's codeword, and K is the Kraft sum. Conversely, any finite nonnegative integer length assignment satisfying the inequality can be realized by a prefix code, subject to the usual singleton convention [1,3].

For a prefix tree, each length-ℓ leaf occupies a fraction $2^{-\ell}$ of the full tree at any sufficiently deep level. Prefix freedom makes the occupied descendants disjoint, so their fractions cannot sum

above one. The uniquely decodable extension needs a counting argument across concatenated codewords; it is not simply the tree picture applied to a nonprefix code.

This inequality is an existence test for lengths. It does not give a unique tree, specify a bitstream format, or guarantee that a code is efficient under the actual source distribution. Many valid trees have unnecessarily large expected length.

The entropy lower bound

A reversible binary description cannot, on average, be shorter than the source's uncertainty. An apparently better rate means some cost or assumption has changed.

Let p_i be source probabilities and define normalized code-induced probabilities $q_i = 2^{-\ell_i}/K$. Then

$$\mathbb{E}[\ell(X)] = H(p) + D_{\mathrm{KL}}(p\|q) - \log_2 K \geq H(p).$$

Here X is a random source symbol, $\mathbb{E}$ is expectation, $H(p)$ its entropy, D_{KL} relative entropy in bits, and K the Kraft sum. Both extra terms are nonnegative because divergence is nonnegative and $K \leq 1$.

The equality decomposes inefficiency into distribution mismatch and unused tree capacity. For a complete tree, $K = 1$; a mismatch between p and the dyadic probabilities induced by integer lengths still causes overhead. Equality is possible when the probabilities match a suitable complete binary tree.

Choosing $\ell_i = \lceil -\log_2 p_i \rceil$ satisfies the Kraft inequality and gives

$$H(X) \leq \mathbb{E}[\ell(X)] < H(X) + 1.$$

The ceiling rounds upward to an integer. The strict upper bound applies to positive-probability finite symbols with the usual nondegenerate convention. The extra bit is per coded unit; coding length-n source blocks as units reduces the corresponding per-symbol rounding cost to less than $1/n$.

Block coding is not free: the block alphabet can grow exponentially, storing the table can be expensive, and delay increases. Arithmetic coding avoids assigning a separate fixed tree leaf to every possible long message.

Huffman coding: merge the least likely alternatives

Repeatedly combine the two least likely symbols into a temporary symbol. Undoing those merges yields a binary tree whose common symbols have shorter paths.

Huffman coding minimizes

$$L = \sum_i p_i \ell_i$$

over binary prefix codes for the specified finite source distribution. Here L is expected length, p_i is symbol probability, and ℓ_i is the resulting leaf depth. Optimality concerns this model and this class of symbol codes, not every possible block or context-dependent coder [1,3].

An optimal tree can place two least likely symbols at sibling leaves of maximum depth. Swapping a smaller probability into a longer path cannot increase expected length. Merging those siblings reduces the optimization to a smaller alphabet; recursively solving the smaller problem yields the greedy algorithm.

Equal probabilities may produce multiple equally optimal trees. A deterministic tie rule matters when encoder and decoder rebuild a tree independently. Our implementation uses an increasing counter to make heap comparisons unambiguous, without comparing heterogeneous tree objects.

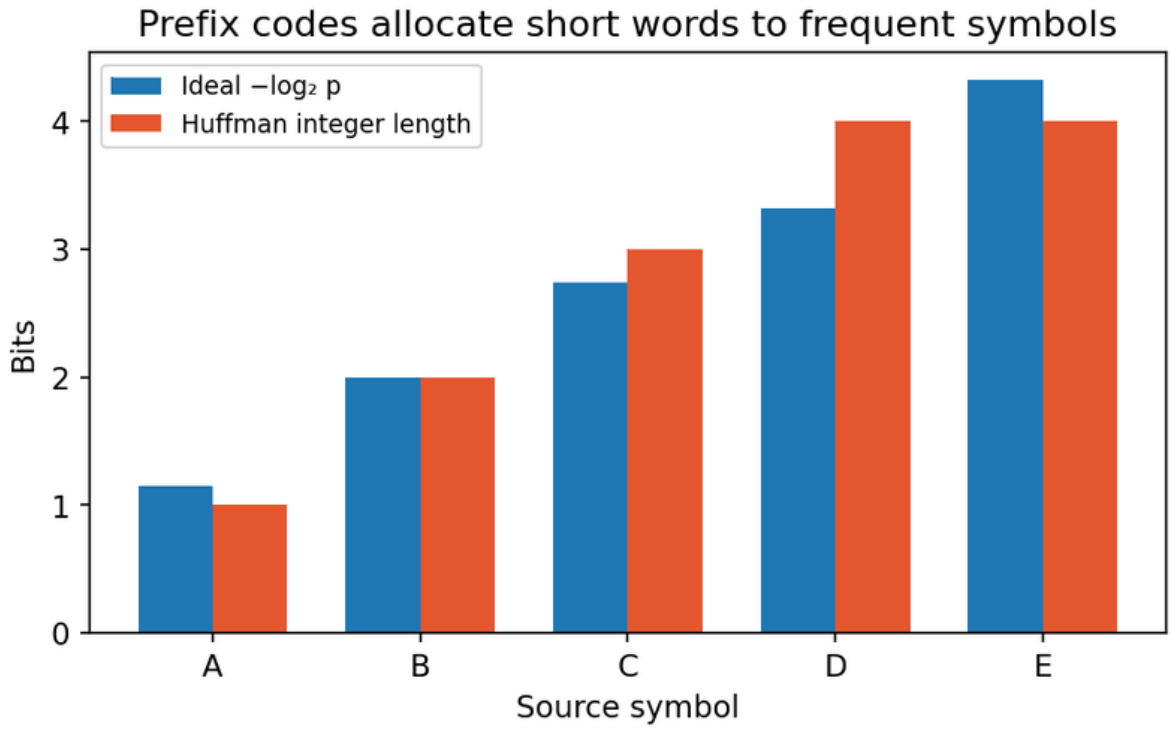


Figure 1 — Probabilities 0.45, 0.25, 0.15, 0.10 and 0.05. Integer symbol-code lengths approximate ideal surprise; they need not equal it for each symbol.

Why the greedy merge preserves optimality

The exchange argument has a concrete cost calculation. If $p_i \geq p_j$ but symbol i is deeper than j, swapping their leaf labels changes expected length by

$$\Delta L = (p_i - p_j)(\ell_j - \ell_i) \leq 0.$$

Here p_i, p_j are the two probabilities, $\ell_i > \ell_j$ their original depths and ΔL the swapped cost minus

the original cost. A higher-probability symbol never benefits from being placed deeper than a lower-probability one.

In an optimal binary tree, deepest leaves can be chosen as siblings; otherwise an unnecessary lone branch can be shortened. The exchange argument permits putting the two smallest probabilities in such sibling leaves without increasing cost. Replace them by one temporary symbol of probability $p_i + p_j$. Its depth is one less than either original leaf, so

$$L_{\text{original}} = L_{\text{merged}} + (p_i + p_j).$$

Here the two L values are expected lengths before and after merging. The added term is fixed, independent of how the rest of the reduced tree is arranged. Therefore an optimal reduced tree gives an optimal original tree, and a supposedly better original tree would produce a better reduced tree. Repeating this argument justifies the greedy algorithm rather than merely observing that its output looks reasonable [1,3].

Work through the five-symbol tree

For probabilities 0.45, 0.25, 0.15, 0.10 and 0.05, the merge sequence is easy to follow by hand. Combine 0.05 with 0.10 to form 0.15. Combine that temporary weight with the original 0.15 to form 0.30. Then combine 0.25 with 0.30 to form 0.55, and finally 0.45 with 0.55 to form one.

A symbol's code length is the number of merges between its leaf and the root. The resulting lengths are 1, 2, 3, 4 and 4. One valid assignment is:

Symbol	Probability	Codeword	Length
A	0.45	`0`	1
B	0.25	`10`	2
C	0.15	`110`	3
D	0.10	`1110`	4
E	0.05	`1111`	4

The code is prefix-free: after reading `0`, the receiver immediately knows A; after reading `111`, it still needs one more bit to decide between D and E. Left–right swaps can change codewords while preserving their lengths and expected cost.

The Kraft sum is $1/2 + 1/4 + 1/8 + 1/16 + 1/16 = 1$. Expected length is $0.45 \times 1 + 0.25 \times 2 + 0.15 \times 3 + 0.10 \times 4 + 0.05 \times 4 = 2$ bits per symbol. Source entropy is approximately 1.977235 bits, so this symbol code pays about 0.022765 bits of rounding mismatch per symbol. A fixed three-bit code would be sufficient for five symbols, but would spend an extra bit on every symbol relative to this expected length.

Notice that A's one-bit description is shorter than its ideal surprise $-\log_2(0.45)$. Other symbols compensate. The theorem constrains the probability-weighted average, rather than requiring every individual codeword to be at least its own surprise. The program below may choose different bit labels for tied nodes, but reproduces these lengths and the two-bit expectation.

Code: build a tree, encode, decode and account for length

This complete example uses the same synthetic probabilities as the figure. It tests every pair of codewords for prefix freedom and reconstructs an independently sampled message. The printed payload excludes a transmitted tree and length header; those exclusions are explicit.

```
import heapq
import itertools
import numpy as np

symbols = list("ABCDE")
p = np.array([0.45, 0.25, 0.15, 0.10, 0.05])
counter = itertools.count()
heap = [(float(w), next(counter), s) for s, w in zip(symbols, p)]
heapq.heapify(heap)
while len(heap) > 1:
    wa, _, a = heapq.heappop(heap)
    wb, _, b = heapq.heappop(heap)
    heapq.heappush(heap, (wa + wb, next(counter), (a, b)))
tree = heap[0][2]
code = {}
def visit(node, prefix=""):
    if isinstance(node, str):
        code[node] = prefix or "0"
    else:
        visit(node[0], prefix + "0")
        visit(node[1], prefix + "1")
visit(tree)
assert all(not b.startswith(a) for i, a in enumerate(code.values())
           for j, b in enumerate(code.values()) if i != j)

rng = np.random.default_rng(34)
message = "".join(rng.choice(symbols, size=2000, p=p))
payload = "".join(code[s] for s in message)
inverse = {v: k for k, v in code.items()}
decoded, prefix = [], ""
for bit in payload:
    prefix += bit
    if prefix in inverse:
        decoded.append(inverse[prefix])
        prefix = ""
assert not prefix and "".join(decoded) == message
entropy = -np.sum(p * np.log2(p))
expected = sum(w * len(code[s]) for s, w in zip(symbols, p))
assert entropy <= expected < entropy + 1
print("Code:", code)
print("Entropy / expected length:", entropy, expected)
print("Payload bits per symbol:", len(payload) / len(message))
```

The entropy bound concerns expected length under the source, not every individual finite message. A particular sample can be shorter or longer than the expected rate. Tree transmission may

dominate a tiny message; sharing a stable alphabet model across many messages amortizes it.

If probabilities are estimated from the message itself, the receiver cannot use that estimate unless it is transmitted or reproducible from already decoded symbols. Fitting a histogram and reporting only the resulting payload hides a real model cost.

Arithmetic coding: identify an interval for the whole message

Start with a unit interval. Each observed symbol selects a probability-sized subinterval. Repeated selection leaves a small interval that uniquely describes the whole message.

With cumulative model probability $F_t(s)$ below symbol s and conditional probability $q_t(s)$, update bounds by

$$L_t = L_{t-1} + (U_{t-1} - L_{t-1})F_t(x_t),$$
$$U_t = L_{t-1} + (U_{t-1} - L_{t-1})[F_t(x_t) + q_t(x_t)].$$

Here $[L_t, U_t)$ is the interval after t symbols, x_t is the current symbol, F_t and q_t use the same already decoded history at both endpoints, and the initial interval is $[0, 1)$. The second update uses the old lower bound, not the newly updated one.

The final width is

$$U_n - L_n = \prod_{t=1}^{n} q_t(x_t) = q(x^n).$$

Here n is message length and $q(x^n)$ the model's joint probability. A sufficiently short binary description selecting a point or contained dyadic interval identifies the message. Ideal interval schemes incur only a bounded termination overhead for the whole block, rather than integer rounding at every symbol.

Practical implementations use finite integer registers, renormalization and explicit termination. A real number is not a transmitted object, and infinitely precise arithmetic is not a production codec. The short example below deliberately uses exact rational values to demonstrate the interval invariant without pretending to implement a full file format.

Read the first three interval updates

Use the example's ordered model: A has probability 1/2, B has 1/3, and C has 1/6. Their cumulative starts are 0, 1/2 and 5/6. For the prefix `ABA`, the updates are:

Symbols read	Lower bound	Upper bound	Width
none	0	1	1
`A`	0	1/2	1/2
`AB`	1/4	5/12	1/6
`ABA`	1/4	1/3	1/12

After A, the available interval has width $1/2$. B selects the portion from one half to five sixths **within that interval**, giving lower bound $0 + (1/2)(1/2) = 1/4$ and upper bound $0 + (1/2)(5/6) = 5/12$. The next A takes the lower half of this width-$1/6$ interval. The final width is $1/2 \times 1/3 \times 1/2 = 1/12$, exactly the model probability of `ABA`.

To decode, normalize the tag's position inside the current interval. That normalized position selects A, B or C using the same cumulative partition. The receiver then updates the interval and repeats. The model and message length make the process reversible; the tiny interval explains the ideal description cost. Exact rational tags teach the mechanism, while finite integer implementations decide which actual bits to emit.

Code: a short exact interval round trip

The encoder outputs a rational tag, while the decoder is supplied the shared model and message length. The numerator and denominator are conceptual side information here; their serialized size is not claimed to be the optimal arithmetic-code length.

```
from fractions import Fraction

model = {"A": Fraction(1, 2), "B": Fraction(1, 3), "C": Fraction(1, 6)}
message = "ABACAB"
cumulative = {}
running = Fraction(0)
for symbol, probability in model.items():
    cumulative[symbol] = running
    running += probability
assert running == 1
low, high = Fraction(0), Fraction(1)
joint = Fraction(1)
for symbol in message:
    width = high - low
    old_low = low
    low = old_low + width * cumulative[symbol]
    high = old_low + width * (cumulative[symbol] + model[symbol])
    joint *= model[symbol]
assert high - low == joint
tag = (low + high) / 2

low_d, high_d = Fraction(0), Fraction(1)
decoded = []
for _ in range(len(message)):
    position = (tag - low_d) / (high_d - low_d)
    symbol = next(s for s in model
                  if cumulative[s] <= position < cumulative[s] + model[s])
    width, old_low = high_d - low_d, low_d
    low_d = old_low + width * cumulative[symbol]
    high_d = old_low + width * (cumulative[symbol] + model[symbol])
    decoded.append(symbol)
assert "".join(decoded) == message
```

```
print("Final interval:", low, high, "width:", high - low)
print("Exact decoded message:", "".join(decoded))
```

A zero model probability for a real symbol makes the interval vanish. The codec needs a positive escape route or an alphabet model covering all valid inputs. Merely clipping probabilities in the encoder is insufficient unless the decoder applies the same normalization and integer quantization.

Modeling cost is cross-entropy, not just entropy

An excellent bit-packing algorithm cannot repair a poor probability model. Unexpected symbols still need long descriptions.

For actual block distribution p and coding model q,

$$\mathbb{E}_p[-\log_2 q(X^n)] = H_p(X^n) + D_{\mathrm{KL}}(p_{X^n} \| q_{X^n}).$$

Here $\mathbb{E}_p$ averages under the actual source, $H_p(X^n)$ is actual block entropy, and the divergence compares full block distributions. Finite coder and framing overhead adds further bits.

A model that uses preceding symbols can approach an entropy rate rather than just marginal entropy. An independence model on a persistent Markov sequence throws away predictable repetition. Conversely, a richer context model can overfit, require excessive memory, or assign fragile probabilities to unseen histories.

For evaluation, reserve independent recordings or acquisition groups. Report actual serialized size, including headers, model selection flags and required side streams. If weights are pretrained and permanently shared, state that convention; if weights are adapted per message, count their updates.

The relevant comparison is a complete encoder–decoder protocol under common assumptions. Comparing an ideal log-probability from one method with actual bytes from another can be useful diagnostically, but it is not a direct codec comparison.

Framing and malformed streams

An honest experiment also specifies what happens at the boundary of a message. A prefix tree tells the decoder where a symbol ends; it does not necessarily tell the decoder when the entire stream ends. A length header, end-of-message symbol or container boundary provides that separate fact.

Padding to a byte boundary can add bits that look like valid codewords. The receiver therefore needs the number of meaningful bits or the original symbol count. In the tree example, the payload is a Python string of bits used for teaching, not a byte-efficient container.

The decoder should reject an incomplete final codeword and impossible transitions instead of silently returning a shorter result. A checksum or other integrity mechanism may detect corrup-

tion, but detecting a damaged file is a separate requirement from compressing an undamaged one. If such a mechanism is part of the chosen protocol, its transmitted size also counts.

These details matter most for small messages and fair comparisons. Asymptotic statements can make a fixed header negligible per symbol, while an actual short file still pays every header bit.

Universal coding learns while decoding

When the source probability is unknown, both endpoints can update the same predictor from symbols already transmitted. No future symbol needs to be revealed for that update.

A sequential model factors as

$$q(x^n) = \prod_{t=1}^{n} q_t(x_t \mid x^{t-1}).$$

Here x^{t-1} is the preceding decoded history, and q_t is a normalized conditional distribution. The encoder observes the current symbol; the decoder learns it from the interval and then performs the same state update.

For an unknown independent Bernoulli source, the Krichevsky–Trofimov predictor uses

$$q_t(1 \mid x^{t-1}) = \frac{N_1(t-1) + 1/2}{t}, \qquad q_t(0 \mid x^{t-1}) = 1 - q_t(1 \mid x^{t-1}).$$

Here $N_1(t-1)$ counts previous ones and $t-1$ is the number of previous observations. The denominator is previous count plus one. Half-count smoothing gives positive probability to either next symbol, including the first observation [1].

This predictor is the posterior predictive distribution of a Bernoulli mixture with a beta half-count prior. It is universal for the stated Bernoulli family in the appropriate asymptotic coding sense; it is not automatically universal for arbitrary dependent sources.

Define sequence regret against an oracle model:

$$\mathcal{R}_n(x^n) = -\log_2 q(x^n) + \log_2 p(x^n).$$

Here p is the chosen oracle source and q the adaptive model. Expected regret under that fixed source is a block divergence and nonnegative. Realized regret on one sample may be negative, because the mixture can assign that particular sample higher probability than the oracle.

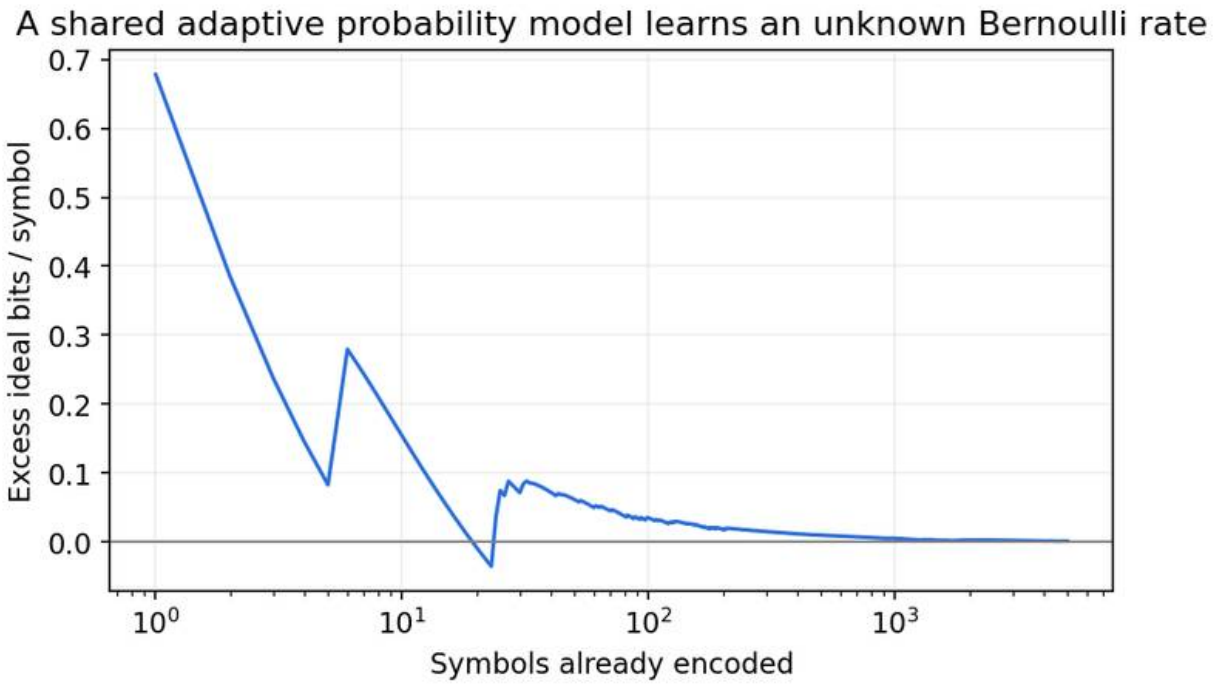


Figure 2 — Seeded synthetic Bernoulli sequence. The curve is realized log-loss difference per symbol, not a serialized byte count or a monotonic guarantee.

Code: adaptation and an abrupt source change

This complete experiment measures ideal sequential lengths. It compares a fixed fair-bit model, a matched oracle and half-count adaptation. A second sequence changes its generating probability halfway through, illustrating the limitation of a stationary count model.

```
import numpy as np

rng = np.random.default_rng(23)
def adaptive_length(bits):
    ones, loss = 0, 0.0
    for t, bit in enumerate(bits):
        q = (ones + 0.5) / (t + 1)
        loss -= np.log2(q if bit else 1 - q)
        ones += int(bit)
    return loss

for n in [100, 1000, 5000]:
    bits = rng.binomial(1, 0.2, size=n)
    oracle = -np.sum(bits * np.log2(0.2) + (1 - bits) * np.log2(0.8))
    print("Length:", n, "oracle:", oracle / n,
          "adaptive:", adaptive_length(bits) / n, "fair model:", 1.0)
bits = np.r_[rng.binomial(1, 0.1, size=2500),
```

```
            rng.binomial(1, 0.9, size=2500)]
one_model = adaptive_length(bits)
two_models = adaptive_length(bits[:2500]) + adaptive_length(bits[2500:])
print("Changed source, whole / reset ideal rate:", one_model / 5000, two_models / 5000)
assert two_models < one_model
```

The reset comparison supplies the change point for free. A real codec must identify it or apply a predetermined reset schedule shared with the receiver. Counting a reset's benefit while omitting its signaling cost overstates the achievable saving.

There are two different benchmarks: regret against a fixed oracle probability, and regret against the best parameter fitted retrospectively to the observed sequence. They are not interchangeable. A finite sample may favor a parameter different from its generating probability, even when the source is correctly specified.

See a causal predictor learn one symbol at a time

Before any observation, the half-count model assigns either bit probability $1/2$. If the decoded prefix is `001`, the next-one probability is $(1+1/2)/(3+1) = 3/8$, while next-zero probability is $5/8$. Both endpoints obtain this state from the same prefix, so neither needs a transmitted histogram.

The model pays uncertainty early because it does not know the source bias. As counts accumulate under a fixed Bernoulli source, that initial learning cost is spread across more symbols. An alternating string has fair overall counts but perfect predictability from its previous symbol. The count-only model misses that relationship and keeps predicting near one half. Universal within a Bernoulli family therefore means learning an unknown bias, not learning every possible sequence structure.

Dictionaries, dependence and universal source classes

A dictionary coder learns recurring phrases rather than only symbol frequencies. Long repeated patterns can then be represented by references to earlier phrases.

Lempel–Ziv families construct descriptions from a growing set of previously observed strings [1]. Their asymptotic universality results specify a source class, such as stationary ergodic finite-alphabet processes, and a precise algorithm. They do not assert that any dictionary implementation beats every probabilistic model on every finite file.

Dictionary entries, phrase boundaries and reference indices must be reproducible by the decoder. As the dictionary grows, indices need more bits. A dictionary parser's phrase count is not itself the compressed length; an integer coding rule and termination format are also needed.

Different source structures favor different mechanisms. A single-symbol predictor captures a bias; a context predictor captures short dependencies; phrase dictionaries exploit repeated substrings; a learned model may exploit structure shared across a training population. All still require reversible symbol transmission for a lossless claim.

Universality is relative to a benchmark and limiting regime. Unknown stationarity parameters are one problem; a changing alphabet or arbitrary adversarial source is another. Memory constraints, latency, short messages and reset policy can dominate the asymptotic benefit.

It is useful to ask four questions before quoting a universal-rate result: what class of sources is covered; what reference length is used; whether convergence is in expectation or along realizations; and whether model, dictionary and framing costs are included.

Kolmogorov complexity is an ideal limit of description

Imagine the shortest computer program that prints a string and stops. Structured strings can have short programs; most long strings cannot.

Relative to a fixed universal description machine U, define

$$K_U(x) = \min_{p:U(p)=x} |p|.$$

Here x is a finite string, p a terminating program for machine U, and $|p|$ its program length in bits. Some formulations use prefix-free programs; the machine and convention must be stated [1].

Changing between suitable universal machines alters complexity by an additive machine-dependent constant. That does not make the value practically machine-independent for short strings. More importantly, the exact shortest-program function is not computable in general. One cannot turn the definition into an algorithm that always finds the best compressor.

A lossless compressor gives an upper bound on description complexity once its decoder and framing are included. Its output length is not the exact Kolmogorov complexity. Nor does a compressor failing to shorten a string prove the string is algorithmically random.

A counting argument shows why no injective compressor can shorten every fixed-length input. There are more length-n strings than binary descriptions shorter than n. Any scheme giving some inputs shorter descriptions must leave other inputs unshortened or longer, under an honest boundary convention.

This is compatible with excellent compression on a structured source distribution. Compression exploits nonuniform probability or repeated structure; it does not remove the counting constraint across all possible inputs.

Modern view

Yang, Mandt and Theis's published 2023 monograph [4] separates lossless probability coding from lossy representation design. Its lossless taxonomy includes autoregressive, latent-variable and invertible models; its discussion emphasizes compatibility between model factorization and the entropy coder. We use that taxonomy to ask how probabilities reach an executable decoder.

A modern autoregressive network can replace the simple count predictor while retaining the chain-rule protocol. Its next-symbol probabilities must be available from the already decoded

context. Latent-variable models need a specific protocol for any transmitted latent information; a low variational training objective is not automatically the final bitstream length.

The 2026 proposed learning chapter by El Gamal [5] provides a current divergence-centered connection between compression and statistical learning. It is a draft chapter. The classical mismatch identity remains useful: improved predictive probability can reduce ideal coding cost, while computation and synchronization determine whether that reduction is practical.

Open problems include adapting efficiently to distribution shifts, reducing sequential decoding cost, comparing complete formats fairly, and making model updates affordable for small messages. A larger predictor can save payload bits and simultaneously consume more shared memory or computation.

For a concrete evaluation, record three quantities separately: held-out negative log-probability, actual file size including required side information, and encoder/decoder runtime under the same hardware assumptions. Only the second establishes an achieved compression ratio; only a round-trip assertion establishes exact reconstruction.

The next chapter switches roles: a source coder removes predictable redundancy, while a channel coder introduces carefully structured redundancy to survive noise. Both use probability, but they solve different reconstruction problems.

Key takeaways

- Losslessness is an exact encoder–decoder identity with explicit boundaries.
- Kraft–McMillan constrains lengths; entropy lower-bounds their expected value.
- Huffman is optimal for the specified binary symbol-prefix problem.
- Arithmetic intervals spread rounding overhead across a whole message.
- Model mismatch adds divergence before finite implementation overhead.
- Adaptive coding requires synchronized causal updates and positive support.
- Universal performance depends on a stated source class and benchmark.
- Computable compression lengths are upper bounds on ideal algorithmic descriptions.

Exercises

1. Construct two different symbol sequences that collide under codewords 0, 1 and 01. Explain why a supplied number of symbols may still leave ambiguity for other choices.

   Hint

   Start with the third symbol versus the first two.

2. Verify the Kraft sum for lengths 1, 2, 3, 4, 4. Find a prefix tree realizing them and compare expected length under the example distribution.

Hint

Sum powers of two and assign leaves without ancestors.

3. Change the Huffman probabilities to a uniform five-symbol source. Explain why ties change the tree but not the optimum expected length.

Hint

Keep a deterministic tie rule.

4. Modify the exact arithmetic example to use a context-dependent binary predictor. Make the decoder update from decoded symbols and verify the final interval width.

Hint

Use exact half-count fractions and identical histories.

5. Compare half-count adaptation on constant, alternating and independent biased binary strings. Identify which source structures a single count model misses.

Hint

Frequencies do not encode ordering.

6. A learned predictor saves 0.1 ideal bits per symbol but requires 8,000 transmitted adaptation bits. Ignoring other changes, calculate the message length needed to offset that overhead.

Hint

Divide total overhead by the per-symbol saving.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 5, 13–14. Publisher book and contents.
2. T. Weissman, "Variable Length Lossless Compression," Stanford EE276 Lecture 4 notes, 2024. Primary lecture notes.
3. T. Weissman, "Kraft-McMillan Inequality and Huffman Coding," Stanford EE276 Lecture 5 notes, 2024. Primary lecture notes.
4. Y. Yang, S. Mandt and L. Theis, "An Introduction to Neural Data Compression," *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. Published monograph, author version.
5. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

IT 04 • Channel Capacity and Reliable Communication

Derive finite-channel capacity, explain coding achievability and Fano's converse, and compute optimal signaling distributions with Blahut–Arimoto.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-04-channel-capacity/

What you'll learn

- Define a discrete memoryless channel and distinguish messages from channel symbols.
- Derive capacities of binary symmetric and binary erasure channels.
- Explain random-code achievability through joint typicality and packing.
- Use Fano's inequality to derive a converse bound on reliable rate.
- Compute capacity and a diagnostic optimality gap with Blahut–Arimoto.
- Separate asymptotic limits from finite-block reliability and task-based communication.

The big picture

A noisy channel does not have to reproduce every transmitted symbol correctly for a receiver to recover the message. A code represents each message by a longer structured sequence. The receiver compares the noisy observation with the possible sequences and decides which message was sent.

> **Explain like I'm five**
>
> If a friend hears some letters incorrectly, sending a carefully chosen longer nickname can help. The extra letters make different nicknames harder to confuse. Sending too many different nicknames through the same noisy space eventually makes them overlap.

Source coding removes predictable redundancy; channel coding adds useful redundancy. These are different objectives. Repeating a bit is an elementary channel code, but not necessarily a good use of the channel. The central question is how many distinct messages can be communicated per channel use while their reconstruction error tends to zero.

The main reading is Cover and Thomas's Chapter 7; the book's computational discussion connects capacity optimization with the Blahut–Arimoto method [1]. Stanford's communication and converse notes [2,3] give primary derivations. We develop the theorem's assumptions and a numerical experiment, then explain why practical short messages need additional evaluation.

Define the channel before its capacity

A channel specifies how an input symbol is randomly transformed into an output symbol. The source distribution and the channel transformation are separate objects.

For finite input alphabet $\mathcal{X}$ and output alphabet $\mathcal{Y}$, a discrete memoryless channel is a row-stochastic matrix

$$W_{xy} = \Pr(Y = y \mid X = x), \qquad p(y^n \mid x^n) = \prod_{t=1}^{n} W_{x_t y_t}.$$

Here x, y index symbols, $W_{xy} \geq 0$ and each row sums to one. The blocks x^n, y^n have length n, and t indexes a use. The factorization defines memorylessness and a time-invariant transition law.

Memorylessness concerns outputs **conditioned on the entire input block**. A code's input symbols may be dependent, because all are determined by one message. Their unconditional outputs can consequently be dependent too. Assuming independent inputs when proving a converse would restrict the permitted codes unnecessarily.

The sender chooses an input distribution $q_x = \Pr(X = x)$ for a single-letter calculation. It induces output probability

$$r_y = \sum_x q_x W_{xy}.$$

Here r_y is the output marginal. The channel W stays fixed as q changes. Altering the propagation law, receiver observations, resource constraints or available feedback defines a different communication problem.

Rows that are identical cannot distinguish their corresponding inputs. Distinct rows may be easier or harder to distinguish depending on overlap. Output alphabet size gives a crude upper bound on information, but the transition probabilities determine the actual usable distinction.

Rate and reliability are operational quantities

A communication rate counts message information divided by channel uses. Reliability refers to recovering the whole message, not simply making most channel symbols correct.

Let a message M be uniform over N alternatives. A length-n code maps it to X^n and estimates $\widehat{M}$ from received Y^n. Define

$$R_n = \frac{\log_2 N}{n}, \qquad P_e^{(n)} = \Pr(\widehat{M} \neq M).$$

Here N is codebook size, R_n is bits per channel use, and $P_e^{(n)}$ is average message-error probability under the specified uniform message and channel. These are different from symbol-error rate.

A rate R is achievable if there is a sequence of codes with asymptotic rates at least R and message error tending to zero. Capacity is the supremum of achievable rates:

$$C = \sup\{R : R \text{ is achievable}\} = \max_q I_q(X;Y).$$

Here q ranges over all input probability distributions on the finite alphabet and I_q is mutual information under $q_x W_{xy}$. The second equality is the channel-coding theorem for the stated finite memoryless channel [1,2].

The theorem describes rates strictly below capacity in its standard achievability statement. It does not promise zero error at finite length or provide a fast decoder. Capacity also differs from **zero-error capacity**, which demands that allowed output sets never confuse messages. A channel can have positive ordinary capacity while every finite block still has a nonzero confusion probability.

The average-error formulation has a uniform message assumption. Maximum error across messages is another operational criterion; standard finite memoryless results connect the capacity formulations by expurgating poor codewords with negligible asymptotic rate loss. At finite length their reliability statistics can differ.

A binary symmetric channel

Each bit is independently flipped with the same probability. A fair input makes full use of the two output alternatives.

For crossover probability e, the binary symmetric channel has

$$W = \begin{pmatrix} 1-e & e \\ e & 1-e \end{pmatrix}, \qquad C_{\mathrm{BSC}} = 1 - h_2(e).$$

Here $e \in [0,1]$ is the flip probability, h_2 is binary entropy, and capacity is bits per use. Conditional output entropy is $h_2(e)$ for either input. Since $H(Y) \le 1$, mutual information is at most $1 - h_2(e)$; a fair input attains the bound.

At $e = 0$, one use conveys one bit without noise. At $e = 1/2$, output is independent of input and capacity is zero. At $e = 1$, every bit flips deterministically, so the receiver can invert it and capacity is again one. Noise severity is therefore not simply the raw flip fraction across the entire interval.

The figure restricts crossover probability to zero through one half, where the ordinary interpretation of increasing uncertainty is natural. The complementary branch follows by relabeling outputs.

A binary erasure channel

The receiver either sees the correct bit or a visible "missing" marker. Knowing which observations are missing is useful information.

A binary erasure channel with erasure probability e has outputs zero, one and an erasure symbol:

$$W = \begin{pmatrix} 1-e & 0 & e \\ 0 & 1-e & e \end{pmatrix}, \qquad C_{\mathrm{BEC}} = 1 - e.$$

Here rows correspond to input zero and one, while columns correspond to the two correct symbols and the erasure. The erasure event is independent of the input.

Conditioned on a nonerased output, the input is known. Conditioned on erasure, its uncertainty remains $H(X)$. Therefore $I(X;Y) = (1-e)H(X)$ and a fair input attains $1-e$ bits per use.

The same numerical event probability produces different capacities for flips and visible erasures. An erasure tells the receiver where information is missing; an unmarked flip can actively suggest the wrong alternative. The distinction follows from their different output laws, not a universal ranking for every physical communication system.

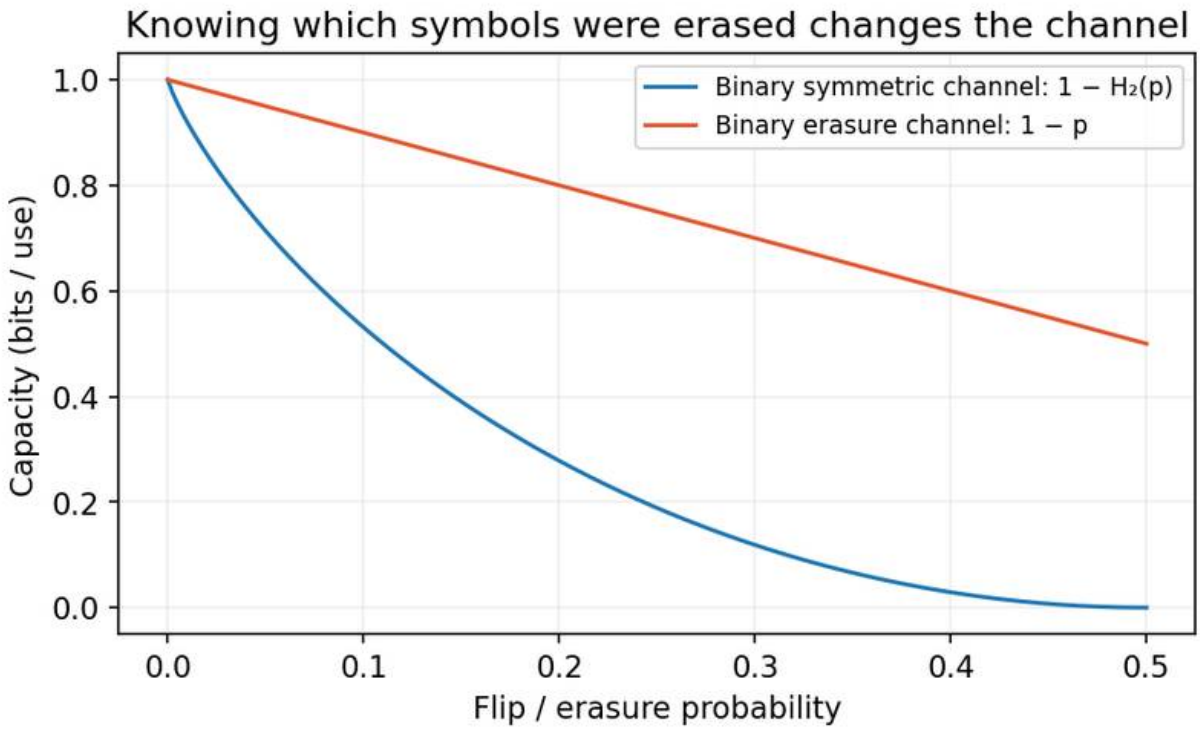


Figure 1 — Analytical capacities under one binary input per channel use. Marked erasures and unmarked flips are different channel models.

Asymmetric channels need asymmetric signaling

When one input is more reliable than another, a uniform input distribution need not be best. Capacity optimizes how often inputs are used.

Consider a Z channel where zero arrives correctly and one becomes zero with probability a:

$$W = \begin{pmatrix} 1 & 0 \\ a & 1-a \end{pmatrix}, \qquad I(q) = h_2(q(1-a)) - q\,h_2(a).$$

Here a is the one-to-zero transition probability, and $q = \Pr(X = 1)$ is the scalar input parameter. Output one has probability $q(1 - a)$; conditional output entropy is $qh_2(a)$.

The first term rewards varied outputs, while the second accounts for noise paid when using input one. Their tradeoff can place the maximum away from $q = 1/2$. Transmitting only the reliable zero is not useful either: a constant input conveys no message information.

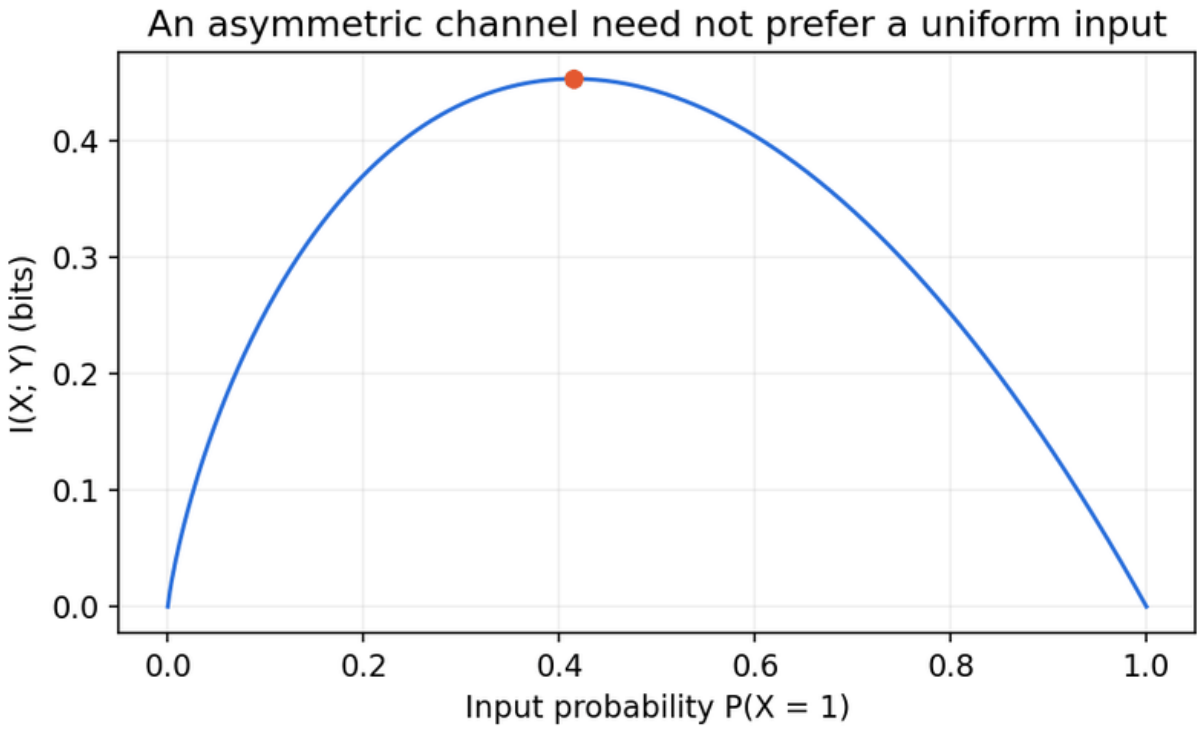


Figure 2 — Analytical input-distribution optimization. Reliability of one symbol does not justify transmitting it exclusively.

Mutual information is concave as a function of the input distribution because output entropy is concave and conditional output entropy is linear. Concavity makes numerical optimization tractable. Flat maxima can still exist when different inputs are redundant, so an optimal input distribution need not be unique.

Why rates below capacity are achievable

Choose many long codewords that look like valid source blocks. A received block usually matches its true codeword's channel relationship, while it rarely matches an unrelated codeword.

Fix an input distribution q and generate approximately 2^{nR} codewords independently from q^n. A jointly typical decoder looks for one codeword consistent with the received output under the joint law $q_x W_{xy}$ [1].

There are two main errors: the true input-output pair fails its typicality test, or another codeword passes. The first probability vanishes by the joint AEP. An independent competing codeword has a joint-typicality probability at an exponential scale near $2^{-nI_q(X;Y)}$.

A union bound over the competing codewords gives the schematic scale

$$P_{\text{competing}} \lesssim 2^{nR} 2^{-n(I_q(X;Y)-\eta)}.$$

Here $P_{\text{competing}}$ is the probability of a competing typical codeword, $\eta > 0$ absorbs the typicality tolerances, and $\lesssim$ suppresses finite prefactors and detailed bounds. This is proof intuition, not an exact finite-length performance formula.

When $R < I_q(X;Y) - \eta$, the competing-error bound vanishes. Choosing a maximizing input and sufficiently small tolerance gives rates below capacity. A small average error over random codebooks implies at least one deterministic codebook has small error.

The argument does not require transmitting the random seed for each message. The codebook is fixed and known to both endpoints before communication. If a practical system chooses or sends new codebooks, that setup protocol must be counted separately.

Random coding proves existence. Searching every codeword in an exponentially large codebook is generally infeasible, and typicality is not a direct recipe for a deployed decoder. A capacity theorem identifies a target; practical structured codes solve the additional complexity problem.

Fano's inequality limits what a receiver can infer

If the receiver frequently guesses wrong, uncertainty remains about the message. Conversely, a small error probability means little conditional message uncertainty remains.

With $N \geq 2$ possible messages and any estimate based on Y^n,

$$H(M \mid Y^n) \leq h_2(P_e) + P_e \log_2(N-1) \leq 1 + P_e \log_2 N.$$

Here M is the true message, Y^n the received block, $P_e = \Pr(\widehat{M} \neq M)$, and h_2 binary entropy. This is Fano's inequality [1,3].

To see the logic, introduce an error indicator. Describing whether an error happened costs at most its binary entropy. If no error happened, the estimate identifies the message. If an error happened, at most $N-1$ alternatives remain. Averaging yields the first bound.

For a uniform message, $H(M) = \log_2 N$. Data processing and channel memorylessness bound

$$I(M;Y^n) \leq I(X^n;Y^n) \leq \sum_{t=1}^{n} I(X_t;Y_t) \leq nC.$$

Here X^n is the transmitted codeword. The middle inequality allows dependent code symbols: output entropy is at most the sum of marginal output entropies, while conditional output entropy separates under the memoryless channel.

Combining the two bounds gives

$$(1 - P_e)\log_2 N \le nC + 1, \qquad P_e \ge 1 - \frac{C}{R_n} - \frac{1}{nR_n}.$$

Here $R_n = \log_2 N/n$ and the lower bound is meaningful only when its right-hand side is positive. A fixed rate strictly above capacity cannot have vanishing error.

This elementary converse establishes a positive error floor above capacity, not by itself convergence of error to one. Strong-converse statements require an additional theorem and assumptions. It is good practice to state exactly what the displayed inequality proves.

Worked rate, error and source-budget calculations

For crossover probability $e = 0.1$, the BSC capacity is $1 - h_2(0.1) = 0.531004$ bits per use. The BEC at the same erasure probability has capacity 0.9. This is a difference between observation models: the BEC reveals which symbols were lost.

A length-100 code at rate 0.8 contains 2^{80} possible messages. Substitute the BSC capacity into the elementary converse:

$$P_e \ge 1 - \frac{0.5310044064}{0.8} - \frac{1}{100 \times 0.8} \approx 0.323744.$$

Here P_e is average whole-message error, 100 is the number of channel uses and 0.8 the message bits per use. The rounded capacity in the substitution retains more digits than the reported answer. This code cannot achieve a 1% message-error target under the stated channel, whatever its decoder. The bound does not determine the exact error of any specific code.

At rate 0.5 the same displayed lower bound is negative, so it provides only the trivial bound zero. That is consistent with asymptotic achievability below capacity, but it does not demonstrate that length 100 already provides the desired reliability. The two questions are answered by different evidence: a theorem for growing block length and a finite-code calculation or experiment.

Now take an independent binary source with probability 0.2 of one. Its entropy is 0.721928 bits per source symbol. Transporting it almost losslessly over this BSC requires more than $0.721928/0.531004 \approx 1.359552$ channel uses per source symbol in the asymptotic separation setting. One use is insufficient; two uses leave a rate margin. A raw-source representation would require one bit per source symbol before channel coding, illustrating why exploiting source bias and protecting against noise are separate design steps.

The bandwidth ratio need not be an integer. Over a long block, a rational or approximate ratio is implemented by allocating the corresponding total number of channel uses. Latency and finite-block overhead remain additional constraints.

Computing capacity with Blahut–Arimoto

Start with an input mixture, examine how distinguishable each channel row is from its output mixture, then move probability toward informative rows. Repeat until an upper and lower bound nearly meet.

For current positive input probabilities q_x and induced r_y, define row divergences in natural units:

$$d_x = \sum_{y:W_{xy}>0} W_{xy} \ln \frac{W_{xy}}{r_y}, \qquad q_x^{\text{new}} = \frac{q_x e^{d_x}}{\sum_j q_j e^{d_j}}.$$

Here d_x is the divergence between channel row x and the current output distribution, ln is the natural logarithm, and j indexes inputs. The denominator uses the same current distribution; the update does not replace it with a uniform prior. Values in nats convert to bits by division by $\ln 2$.

For the current distribution, the lower and upper bounds are

$$L(q) = \frac{\sum_x q_x d_x}{\ln 2}, \qquad U(q) = \frac{\max_x d_x}{\ln 2}, \qquad L(q) \le C \le U(q).$$

Here $L(q)$ is current mutual information and $U(q)$ is a dual-style bound obtained by using the current output mixture as a candidate reference distribution. The gap gives a useful numerical stopping criterion.

This unconstrained algorithm applies to a fixed finite channel. An input-cost constraint requires a modified optimization; one cannot compute unconstrained capacity and claim compliance with a power or symbol-use budget afterward.

Why the numerical upper bound is valid

A stopping gap is useful only if its endpoints actually bracket the optimum. Fix the current output mixture r, then consider **any** candidate input distribution u, whose output mixture is $r_u = uW$. Expanding the logarithmic ratios gives

$$\sum_x u_x D_{\text{nats}}(W_x \| r) = I_{u,\text{nats}}(X;Y) + D_{\text{nats}}(r_u \| r).$$

Here W_x is channel row x, u_x its candidate usage probability, r the fixed reference output distribution, r_u the candidate output marginal and $I_{u,\text{nats}}$ mutual information under u. All quantities in this identity use natural logarithms.

The final divergence is nonnegative, so the candidate information is no larger than the weighted sum on the left. That sum is no larger than its largest row divergence. Because this holds for every candidate u, the current maximum row divergence upper-bounds capacity. The current input's own information is achievable in the single-letter optimization and supplies the lower bound.

At an optimum, inputs assigned positive probability have equal informative row divergences, while unused inputs cannot offer a larger marginal gain. The updates redistribute probability toward rows with greater current divergence until these conditions nearly balance. The gap therefore measures how much improvement could remain for this matrix, rather than merely how small the latest parameter update was.

Code: validate analytic capacities and optimize the Z channel

The complete implementation removes output columns that are impossible for all inputs, masks zero row entries and uses a stable logarithmic normalization. It returns the current input distribution associated with its lower-bound rate.

```
import numpy as np
from scipy.special import logsumexp
from scipy.optimize import minimize_scalar

def binary_entropy(p):
    if p == 0 or p == 1:
        return 0.0
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)

def capacity(W, tolerance=1e-10, maximum=20000):
    W = np.asarray(W, dtype=float)
    assert W.ndim == 2 and np.all(W >= 0)
    assert np.allclose(W.sum(axis=1), 1)
    W = W[:, W.sum(axis=0) > 0]
    q = np.full(W.shape[0], 1 / W.shape[0])
    positive = W > 0
    for iteration in range(maximum):
        r = q @ W
        ratio = np.zeros_like(W)
        np.log(np.divide(W, r, out=np.ones_like(W), where=positive),
               out=ratio, where=positive)
        d = (W * ratio).sum(axis=1)
        lower, upper = q @ d / np.log(2), d.max() / np.log(2)
        if upper - lower < tolerance:
            return lower, q, upper - lower, iteration + 1
        log_q = np.log(q) + d
        q = np.exp(log_q - logsumexp(log_q))
    raise RuntimeError("Capacity iteration did not meet the requested gap")

e = 0.1
bsc = np.array([[1-e, e], [e, 1-e]])
bec = np.array([[1-e, 0, e], [0, 1-e, e]])
for W, analytic in [(bsc, 1-binary_entropy(e)), (bec, 1-e)]:
    value, q, gap, steps = capacity(W)
    print("Capacity / analytic / input / gap:", value, analytic, q, gap)
    assert abs(value - analytic) < 1e-8
a = 0.35
z = np.array([[1, 0], [a, 1-a]])
value, q, gap, steps = capacity(z)
objective = lambda p: binary_entropy(p*(1-a)) - p*binary_entropy(a)
reference = minimize_scalar(lambda p: -objective(p), bounds=(0, 1), method="bounded")
assert abs(value + reference.fun) < 1e-8
print("Z capacity / P(input one) / gap / iterations:", value, q[1], gap, steps)
```

Agreement with analytic binary cases checks units and orientation. Agreement with independent scalar optimization checks the asymmetric case. These are meaningful tests because they would

catch using column-stochastic probabilities, omitting the current q factor or mixing natural and base-two logarithms.

Nearly redundant inputs and extreme probabilities can converge slowly or cause floating-point underflow. A small reported gap is a numerical certificate for this finite matrix within computational precision, not evidence that the matrix accurately describes a real channel.

A repetition experiment shows the finite-block tradeoff

Repeating one bit helps majority decoding, but consumes more uses per message bit. Higher reliability does not automatically mean a capacity-efficient code.

Repeating a bit k times through an independent BSC and majority decoding, with odd k, gives

$$P_e(k) = \sum_{j=(k+1)/2}^{k} \binom{k}{j} e^j (1-e)^{k-j}, \qquad R = \frac{1}{k}.$$

Here j is the number of flipped copies, e the crossover probability, $P_e(k)$ the one-bit message error, and R the code rate. This is an exact finite-code calculation, not a capacity formula.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(35)
e = 0.1
for copies in [1, 3, 5, 9]:
    noise = rng.binomial(1, e, size=(50000, copies))
    observed_error = np.mean(noise.sum(axis=1) > copies // 2)
    exact_error = binom.sf(copies // 2, copies, e)
    print("Copies / rate / simulated error / exact error:",
          copies, 1 / copies, observed_error, exact_error)
    assert abs(observed_error - exact_error) < 0.005
```

We may assume a zero message bit when simulating this symmetric code, because either transmitted bit has the same error probability. More copies reduce error for this one-bit problem while driving rate toward zero. Capacity instead permits codebooks carrying a growing number of bits at a positive asymptotic rate.

A finite evaluation should specify block length, rate, message-error criterion, channel parameters and decoding rule. Reporting only a capacity number does not characterize a short code's latency, complexity or reliability.

Resource constraints and different channel assumptions

A capacity calculation answers one precisely specified question. Changing a transmitter's budget, adding memory or providing a different observation changes that question.

If input x has nonnegative cost $c(x)$ and the average budget per use is B, the constrained single-letter problem is

$$C(B) = \max_{q:\sum_x q_x c(x) \le B} I_q(X;Y).$$

Here $c(x)$ is the input-use cost, B the allowed mean, and q a probability distribution satisfying it. The channel remains the same finite memoryless matrix. This formula concerns an average cost convention; a strict peak constraint on each symbol defines a different feasible set.

For an asymmetric binary channel, a constraint on the frequency of input one can exclude the unconstrained maximizing distribution. The result should be a capacity–cost function rather than one unconstrained number with a power label attached.

Likewise, throughput in bits per second requires a specified number of channel uses per second. Bits per use are not automatically bits per sample, pixel or token. When comparing modulation or representation choices, state how each choice changes the use count and available resources.

Feedback lets an encoder observe earlier channel outputs before choosing the next input. For the ordinary memoryless point-to-point model, noiseless causal feedback does not increase the asymptotic capacity [1,3]. It can still change finite-code design, stopping rules and achievable error behavior. This result should not be transferred without checking to channels with memory or multiple users.

A channel with a persistent hidden noise state may produce bursts rather than independent flips. A single transition matrix estimated by pooling all uses hides that dependence. The capacity proof above uses the conditional product law and cannot simply treat those bursts as independent draws with an average flip probability.

These distinctions also arise when a receiver supplies side information. If the side information reveals an erasure position, a noise state or part of the original message, it changes the receiver's observation. Compare the entire observation model, rather than crediting a decoder for information that a baseline was denied.

Estimating a channel is a separate experiment

Numerical optimization can be exact for a wrong matrix. Before trusting the rate, determine whether the transition probabilities describe the operating conditions.

With labeled input–output observations, a simple row estimate is

$$\widehat{W}_{xy} = \frac{N_{xy}}{\sum_z N_{xz}}.$$

Here N_{xy} counts observed input x paired with output y, z ranges over outputs, and $\widehat{W}$ is an empirical channel matrix. Every input row needs observations. A zero empirical count does not prove an event impossible.

Finite samples create uncertainty in both the estimated capacity and the maximizing distribution. Repeating experiments across independent acquisitions can expose variability; conditioning on

known operating regimes can reveal different channel laws. The optimizer's gap measures computational convergence for one matrix, while experimental uncertainty concerns which matrix should be optimized.

Training and evaluation should also use separate observations when a decoder or input strategy is fitted from data. Selecting a favorable input distribution on the same tiny sample used to estimate its performance can produce an optimistic rate estimate.

A useful report therefore gives the sampling protocol, counts per input, operating conditions, empirical row probabilities, and held-out reconstruction results. Rare output events deserve explicit treatment because assigning them zero probability can distort both a decoder and an information estimate.

How source information meets channel information

A source coder packs useful uncertainty into bits; a channel coder carries those bits reliably. Their rates must be compared using the same source-to-channel use budget.

If a finite memoryless source has entropy $H(S)$ bits per source symbol and b channel uses are available per source symbol, the familiar asymptotic separation condition is

$$H(S) < bC.$$

Here S is a source symbol, b is the use ratio and C the point-to-point capacity. Under the standard long-block assumptions, source and channel coding can separately approach their limits with a margin [1].

The inequality does not specify a practical block size or require independent design to be optimal at finite latency. It also does not assert general source–channel separation for every network. Keeping its assumptions visible prevents the attractive formula from being applied to unrelated constraints.

For lossy reconstruction, the needed source rate becomes a rate–distortion function rather than full entropy. For continuous amplitudes, physical constraints determine channel capacity. These two extensions motivate the next chapters.

Modern view

Nguyen and colleagues' 2025 published survey [4] organizes semantic communication into receiver-intention modeling, generative models, and deep joint source–channel coding. Its taxonomy distinguishes systems by what they aim to preserve and how transmitter and receiver coordinate, rather than treating every learned system as the same codec.

Those objectives can change the required reconstruction. A task receiver might need a class label rather than every original pixel. That can reduce required source information, but it does not increase the Shannon capacity of an unchanged channel under unchanged constraints.

Generative reconstruction similarly needs an explicit criterion. A plausible output can have a different relationship to the original than an exact decoded message. Evaluating task accuracy or perceptual distortion is legitimate when stated, but those quantities cannot replace message-error probability in a bit-reliability claim.

Our inference from the classical model is to keep three layers separate: what the source receiver needs, what the channel physically conveys, and how the code uses finite resources. Joint optimization can improve their practical match without invalidating any of the corresponding limits.

Open questions include robust performance when the channel law shifts, reliable operation at short block lengths, shared-model synchronization, and evaluation across different tasks and receiver knowledge. Survey results for one task or channel do not establish a universal semantic rate.

The next chapter changes alphabets from finite symbols to continuous amplitudes. It introduces differential entropy and power constraints before deriving Gaussian capacity; substituting a continuous density into a discrete entropy formula would miss essential unit dependence.

Key takeaways

- A memoryless channel factors outputs given inputs; code symbols need not be independent.
- Capacity is a reliable-message rate, equal to optimized mutual information under stated assumptions.
- Flips and marked erasures have different channel capacities.
- Asymmetric channels may favor nonuniform input probabilities.
- Random coding establishes existence through typicality and packing.
- Fano and data processing bound rates with vanishing message error.
- Blahut–Arimoto optimizes a finite channel and supplies a numerical gap.
- Finite reliability and semantic objectives require their own explicit evaluation.

Exercises

1. Explain why a BSC with crossover probability 0.9 has the same capacity as one with probability 0.1. Describe the receiver's output relabeling.

Hint

Invert every output.

2. Derive the BEC mutual information for an arbitrary binary input probability rather than assuming a fair input.

Hint

Erasures leave the original input entropy.

3. Change the Z-channel parameter to 0.1 and 0.7. Compare optimized input probabilities with one half and explain the competing entropy terms.

Hint

Optimize output entropy minus weighted row entropy.

4. For a length-100 code of rate 0.8 through a BSC with crossover probability 0.1, calculate the elementary Fano lower bound on message error. State what it does and does not prove.

Hint

Use $1 - C/R - 1/(nR)$ and keep a zero lower bound if negative.

5. Duplicate a row in a channel matrix and rerun the capacity routine. Explain why the value is unchanged while an optimal input mixture may cease to be unique.

Hint

Identical rows produce identical conditional outputs.

6. Compare repetition lengths 3 and 9 at equal total channel-use budgets. Explain why the smaller one-bit error cannot alone decide which system carries more reliably decoded information.

Hint

Count message bits as well as their errors.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 7 and the computational capacity/rate–distortion discussion. Publisher book and contents.
2. T. Weissman, "Communication and Channel Capacity," Stanford EE276 Lecture 7 notes, 2024. Primary lecture notes.
3. T. Weissman, "Channel Coding Theorem: Converse Part," Stanford EE276 Lecture 10 notes, 2024. Primary lecture notes.
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, "A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding," *IEEE Communications Surveys & Tutorials*, 2025, DOI: 10.1109/COMST.2025.3616973. Published-paper metadata and author version.

IT 05 • Differential Entropy and Gaussian Channels

Handle continuous information with explicit units, derive Gaussian capacity under a power budget, and optimize parallel channels by water-filling.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-05-continuous-gaussian-channels/

What you'll learn

- Define differential entropy without treating a density as a probability.
- Track units and coordinate changes while preserving mutual information.
- Derive the Gaussian maximum-entropy property from relative entropy.
- Obtain real Gaussian-channel capacity under an average-power budget.
- Allocate power across parallel channels through water-filling.
- Distinguish analytical information limits from estimation and implementation costs.

The big picture

An amplitude can take a continuum of values, but a receiver with noise cannot distinguish infinitely many messages for free. More precise measurements require more resources, and a transmitter cannot generally use arbitrarily large amplitudes. Continuous information theory keeps those constraints visible.

> **Explain like I'm five**
>
> You can point anywhere along a ruler, but your friend's blurry eyes cannot tell apart every tiny difference. Pointing farther apart helps, yet the ruler has a size limit. The useful question is how many reliably different signals fit within the available space.

This chapter follows Cover and Thomas's Chapters 8–9 [1], supplemented by Stanford's continuous-variable and Gaussian-channel notes [2,3]. The progression matters: we first define density-based information, then prove a distributional bound, then use it to optimize communication under a power constraint.

Continuous formulas also appear in vision features, latent representations and sensor noise models. Their numerical values can change under a change of units even when the scene has not changed. Mutual information is the more stable object for comparing relationships.

A density is not a point probability

A continuous density describes probability per unit of measurement. A density value above one is perfectly possible and does not violate probability rules.

For a real random variable X with density f_X,

$$\Pr(X \in A) = \int_A f_X(x)\,dx, \qquad \int_{\mathbb{R}} f_X(x)\,dx = 1.$$

Here A is a measurable region, x is a coordinate in the chosen units, and $f_X(x)$ is density rather than the probability of the exact point. Under an ordinary continuous density, each exact point has probability zero.

A uniform variable on an interval of width one tenth has density ten on that interval. Its probability over the whole interval is still one. Taking the negative logarithm of a density can therefore yield a negative value.

The differential entropy is

$$h(X) = -\int f_X(x) \log_2 f_X(x)\,dx.$$

Here h is differential entropy in base-two units and the integral is taken where the density is defined. The expression assumes the integral exists in the relevant sense. It can be negative or infinite, and some distributions have no well-defined finite entropy.

Unlike discrete entropy, differential entropy alone is not a minimum count of bits needed for exact representation of a continuously distributed value. Exact infinite-precision reconstruction is a different, generally unbounded requirement.

The notation deliberately uses lowercase h for differential entropy and uppercase H for discrete entropy. They share algebraic relationships under appropriate conditions, but their interpretation and sign constraints differ.

Quantization explains the relationship to actual symbols

A real instrument records bins, not infinitely precise points. As bins shrink, describing the bin index costs more bits.

Quantize a scalar density into bins of width $\Delta > 0$. For a sufficiently regular density and fine quantization,

$$H(X_\Delta) \approx h(X) + \log_2 \frac{1}{\Delta}.$$

Here X_Δ is the discrete bin index, Δ is measured in the same coordinate units as x, and the approximation is a fine-bin asymptotic relationship subject to regularity and tail conditions.

Each bin probability is approximately density times bin width. Its negative log-probability is therefore approximately negative log-density minus log-width. Averaging gives the formula.

The extra term is not a small correction. It diverges as resolution increases. A finite differential entropy does not make arbitrarily precise values cheap to encode.

At coarse resolution, sharp discontinuities or heavy tails, the approximation may be poor. Real quantizers may have saturation, nonuniform bins or a finite range. These details change the discrete distribution and must be included in an actual coding experiment.

For a sensor, both resolution and physical noise matter. Increasing stored numerical precision beyond measurement precision need not create new information about the scene. It can merely create more digits representing uncertain amplitudes.

Code: coordinate units and quantized entropy

This standalone experiment uses a unit uniform variable, computes empirical bin-index entropy at different resolutions, and compares an analytic differential-entropy change under scaling.

```
import numpy as np

rng = np.random.default_rng(36)
x = rng.uniform(0, 1, size=100000)
def discrete_entropy(values):
    _, counts = np.unique(values, return_counts=True)
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))
for bins in [10, 100, 1000]:
    indices = np.floor(x * bins).astype(int)
    estimated = discrete_entropy(indices)
    theoretical = np.log2(bins)
    print("Bins / estimated index entropy / exact:", bins, estimated, theoretical)
    assert abs(estimated - theoretical) < 0.03
scale = 0.1
h_x, h_scaled = 0.0, np.log2(scale)
print("Differential entropy before / after scaling:", h_x, h_scaled)
assert h_scaled < 0
```

The negative scaled entropy is not negative uncertainty in a discrete message. It reflects density per coordinate unit. The finite bin estimate remains nonnegative and has sampling bias that grows when there are many rare bins.

Coordinate changes alter differential entropy

Measuring the same length in millimeters instead of meters stretches the numerical distribution. Differential entropy changes with that stretching.

For a nonzero scalar a and translation b,

$$h(aX + b) = h(X) + \log_2 |a|.$$

Here a scales coordinates and b translates them. A translation does not alter density volume; scaling multiplies it by $|a|$. For an invertible matrix A applied to a d-dimensional vector,

$$h(AX + b) = h(X) + \log_2 |\det A|.$$

Here A is a nonsingular d by d matrix, b a translation vector, and $\det A$ its determinant. Both sides require well-defined entropies.

A smooth one-to-one nonlinear coordinate map T has the corresponding Jacobian correction:

$$h(T(X)) = h(X) + \mathbb{E} \log_2 |\det J_T(X)|.$$

Here J_T is the derivative matrix of T, evaluated at random X, and the expectation averages its log-volume change. This identity requires appropriate differentiability, invertibility and integrability.

It explains why latent density values depend on representation coordinates. An invertible transformation can change differential entropy without discarding information. Comparing two feature spaces by raw differential entropy alone can confuse coordinate expansion with information quality.

Derive the scale correction from the density

The correction can be understood directly rather than memorized. Let $V = aX + b$ with $a \neq 0$. Probability is preserved under the coordinate change, so

$$f_V(v) = \frac{1}{|a|} f_X\left(\frac{v - b}{a}\right).$$

Here V is the transformed variable, v its coordinate, f_V its density, and a, b are scale and translation. Density decreases by $|a|$ when the coordinate interval expands by that factor. Consequently its negative log-density gains the constant $\log_2 |a|$. Averaging over the unchanged event probabilities gives the entropy correction.

Take a uniform source on an interval of width 0.1. Its differential entropy is $\log_2(0.1) = -3.321928$. Quantize it into ten equal bins of width 0.01: each bin has probability one tenth, and the discrete entropy is $\log_2(10) = 3.321928$ bits. Here the fine-bin relationship is exact because the density is constant and the bins align with the support.

Change units by multiplying all coordinates by 100. The interval width becomes ten, differential entropy becomes $\log_2(10) = 3.321928$, and bin width becomes one. The same ten outcomes still cost 3.321928 bits. Scaling changes the density entropy and bin-width correction in opposite directions; it does not create additional distinguishable events. Changing only the number of bins would change the observation, whereas changing coordinate units does not.

Relative entropy and mutual information remain meaningful

Comparing densities by a ratio cancels shared coordinate units. A relationship between two variables should not disappear when either is measured in a different reversible coordinate system.

For densities f and g,

$$D_{\mathrm{KL}}(f\|g) = \int f(x) \log_2 \frac{f(x)}{g(x)}\, dx, \qquad I(X;Y) = D_{\mathrm{KL}}(f_{XY}\|f_X f_Y).$$

Here f_{XY} is a joint density and $f_X f_Y$ is the independent product reference. Divergence is nonnegative, with an infinite value when support or integrability demands it.

Under separately invertible coordinate changes of X and Y, the Jacobian factors cancel in the density ratio and integral, so mutual information is unchanged. This statement does not apply to a noninvertible projection or quantization; those can discard information and are governed by data processing.

When all entropies are finite and compatible,

$$I(X;Y) = h(X) + h(Y) - h(X,Y) = h(Y) - h(Y \mid X).$$

Here $h(X,Y)$ is joint differential entropy and $h(Y \mid X)$ is expected conditional differential entropy. Conditional differential entropy may be negative. The resulting mutual information remains nonnegative.

The density-ratio definition is safer when entropy differences would subtract infinities. A noiseless deterministic identity between genuinely continuous variables can have infinite mutual information and a singular joint law; ordinary full-dimensional joint-density formulas then need care.

For a jointly Gaussian scalar pair with correlation ρ,

$$I(X;Y) = -\frac{1}{2} \log_2 (1 - \rho^2).$$

Here ρ is the correlation coefficient and $|\rho| < 1$ makes covariance nonsingular. Correlation is sufficient for this formula only under joint Gaussianity; a general dependence cannot be summarized by one covariance number.

Gaussian distributions maximize entropy at fixed covariance

Among distributions with a specified covariance, a Gaussian spreads uncertainty most broadly in the differential-entropy sense. This is a constrained statement, not a claim that every source is Gaussian.

A nonsingular d-dimensional Gaussian with covariance Σ has

$$h(G) = \frac{1}{2} \log_2 \left((2\pi e)^d \det \Sigma\right).$$

Here G is the Gaussian vector, d its dimension, Σ its positive-definite covariance, and e the base of natural logarithms. Its mean does not affect entropy.

For another density f with the same mean and covariance, compare it with Gaussian density g. The Gaussian log-density is a constant plus a quadratic form. Its expectation under f equals its expectation under g because the first two moments match. Thus

$$D_{\mathrm{KL}}(f\|g) = h(G) - h(X) \geq 0.$$

Here X has density f, and the equality assumes the needed finite quantities. It proves the maximum-entropy property and identifies equality with the Gaussian distribution [1,2].

In one dimension, fixed variance σ^2 gives maximum entropy $\frac{1}{2}\log_2(2\pi e\sigma^2)$. Specifying only support gives a different extremizer: a uniform density on that bounded support. Constraints determine the maximum-entropy distribution.

This proof is valuable beyond communication. It makes a modeling choice transparent: adopting a Gaussian from a mean and covariance adds no further shape constraint in the maximum-entropy formulation. It does not certify the fit, rule out multimodality or make a Gaussian likelihood robust to outliers.

Code: Gaussian information and reversible scaling

The complete example estimates covariance from a synthetic Gaussian pair, computes its Gaussian-model information, and verifies that an invertible scaling changes entropy while preserving the information formula.

```
import numpy as np

rng = np.random.default_rng(37)
rho = 0.7
covariance = np.array([[1, rho], [rho, 1]])
samples = rng.multivariate_normal([0, 0], covariance, size=100000)
estimated_cov = np.cov(samples, rowvar=False)
def gaussian_entropy(cov):
    sign, logdet = np.linalg.slogdet(np.atleast_2d(cov))
    assert sign > 0
    d = np.atleast_2d(cov).shape[0]
    return 0.5 * (d * np.log(2*np.pi*np.e) + logdet) / np.log(2)
def gaussian_information(cov):
    return (gaussian_entropy([[cov[0, 0]]]) +
            gaussian_entropy([[cov[1, 1]]]) - gaussian_entropy(cov))
estimated = gaussian_information(estimated_cov)
exact = -0.5 * np.log2(1-rho**2)
transform = np.diag([100, 0.2])
scaled_cov = transform @ estimated_cov @ transform.T
assert abs(gaussian_information(scaled_cov) - estimated) < 1e-12
assert abs(gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov)
           - np.log2(abs(np.linalg.det(transform)))) < 1e-12
assert abs(estimated - exact) < 0.02
print("Estimated Gaussian-model MI / exact MI:", estimated, exact)
print("Entropy shift:", gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov))
```

This is a Gaussian parametric calculation, not a general-purpose mutual-information estimator. On a non-Gaussian dataset, plugging in covariance answers a Gaussian-model question and can miss substantial nonlinear dependence.

Derive the real Gaussian-channel capacity

Signal power increases the spread of distinguishable outputs; noise blurs them. The capacity depends on their ratio, under the precise Gaussian model.

Consider one real amplitude per use:

$$Y = X + Z, \qquad Z \sim \mathcal{N}(0, N), \qquad Z \perp X, \qquad \mathbb{E}[X^2] \le P.$$

Here X is the input, Y the output, Z independent zero-mean Gaussian noise, $N > 0$ noise variance, $P \ge 0$ average input power, and $\perp$ denotes independence. Across uses the noise is independent with the same law.

Conditioning on X leaves only translated noise, so $h(Y \mid X) = h(Z)$. Output variance is at most $P + N$. Gaussian maximum entropy therefore yields

$$I(X;Y) \le \frac{1}{2}\log_2(2\pi e(P+N)) - \frac{1}{2}\log_2(2\pi eN) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right).$$

Here P/N is the signal-to-noise ratio. Independent Gaussian input $X \sim \mathcal{N}(0, P)$ makes the output Gaussian and attains this single-letter bound. The corresponding coding theorem establishes

$$C(P) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right)$$

bits per real channel use under the standard average-power model [1,3]. The maximizing input has zero mean because a deterministic mean consumes power without increasing distinguishable variation.

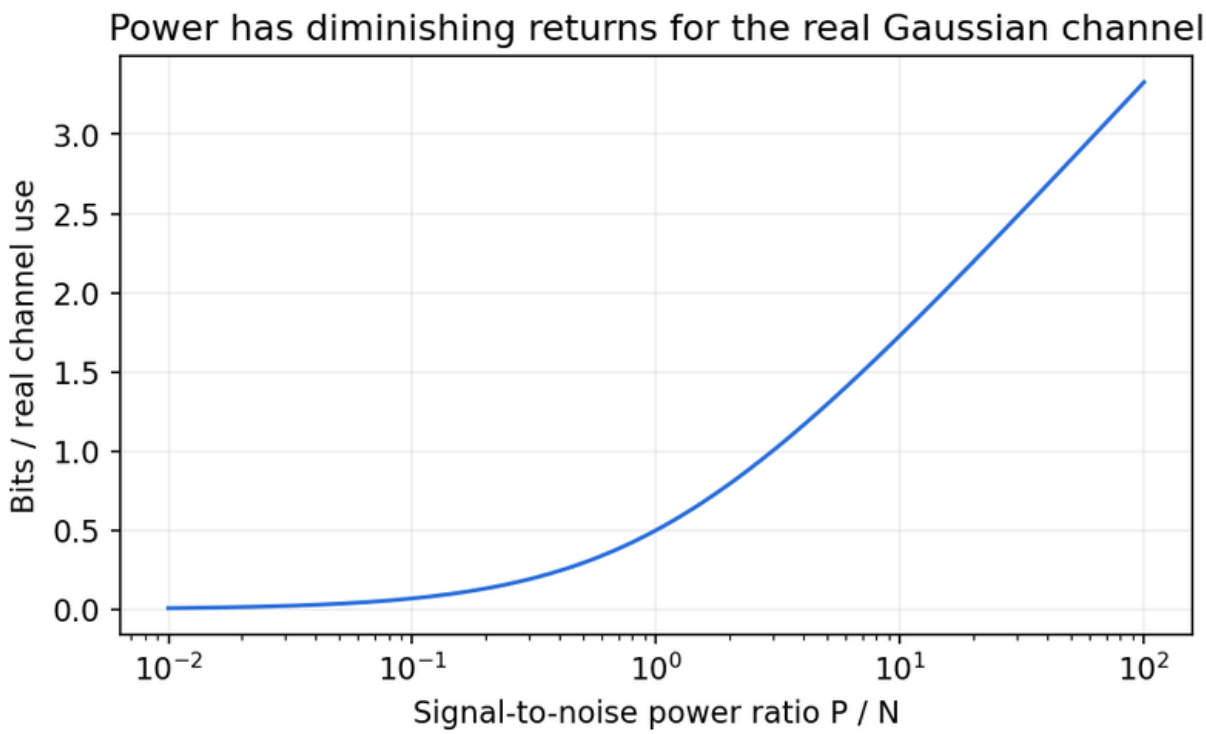


Figure 1 — One real amplitude per use, independent Gaussian noise and average-power constraint. The factor one half belongs to this real-valued convention.

An average-power constraint does not impose a strict maximum on every amplitude. Adding a peak-amplitude constraint changes the optimizing input distribution and generally the value. A finite constellation also defines a more restricted signaling problem.

The analogous complex-valued formula often lacks the one-half factor because a complex use includes two real dimensions, under a corresponding noise and power convention. Comparing the expressions without defining a use can create a false factor-of-two improvement.

A geometric view and its limits

Over many uses, noise places the observation in a cloud around the transmitted codeword. Good codewords are far enough apart that the clouds are usually distinguishable.

For n independent Gaussian noise samples, squared norm concentrates near nN:

$$\frac{1}{n}\sum_{t=1}^{n} Z_t^2 \to N.$$

Here Z_t is noise at use t, and the arrow denotes the appropriate law-of-large-numbers convergence. Power-constrained codewords lie near an input radius on the scale $\sqrt{nP}$, while outputs occupy a larger scale involving $P + N$.

Comparing high-dimensional output volume with noise-cloud volume suggests a count on the scale $(1 + P/N)^{n/2}$, whose normalized binary logarithm is the capacity expression. This is a geometric explanation, not an assertion that Gaussian noise has bounded support.

Noise can leave any finite sphere with positive probability. Reliable communication requires that atypical observations become sufficiently rare, not that noise clouds are literally disjoint for every possible outcome. That distinction separates vanishing-error capacity from zero-error communication.

The geometry also explains why a one-use diagram can be misleading. Long blocks allow many coordinates to cooperate, and their aggregate norms concentrate. A short-block constellation has its own distance and decoding tradeoffs.

Parallel channels and water-filling

When several independent channels have different noise levels, distribute power where it creates the greatest marginal increase in information. Some very noisy channels may receive no power.

For m parallel real Gaussian channels with variances $N_i > 0$, allocate powers $P_i \geq 0$ under total P_{total}:

$$\max_{\sum_i P_i \leq P_{\text{total}}} \sum_{i=1}^{m} \frac{1}{2} \log_2 \left(1 + \frac{P_i}{N_i}\right).$$

Here i indexes one parallel dimension, m is their count, and the objective is bits per joint use of all dimensions. Independent Gaussian inputs attain the component rates.

The derivative with respect to an active power is $1/[2 \ln 2 (N_i + P_i)]$. Equalizing derivatives under a common budget yields

$$P_i = (\mu - N_i)_+, \qquad \sum_i (\mu - N_i)_+ = P_{\text{total}}.$$

Here μ is the water level and $(u)_+ = \max(u, 0)$. The notation reflects filling above noise floors: active channels reach a common total signal-plus-noise level.

This follows from the optimality conditions of a concave optimization problem. A channel whose noise floor exceeds the level receives zero; assigning it positive power would give a smaller marginal return than an active lower-noise channel.

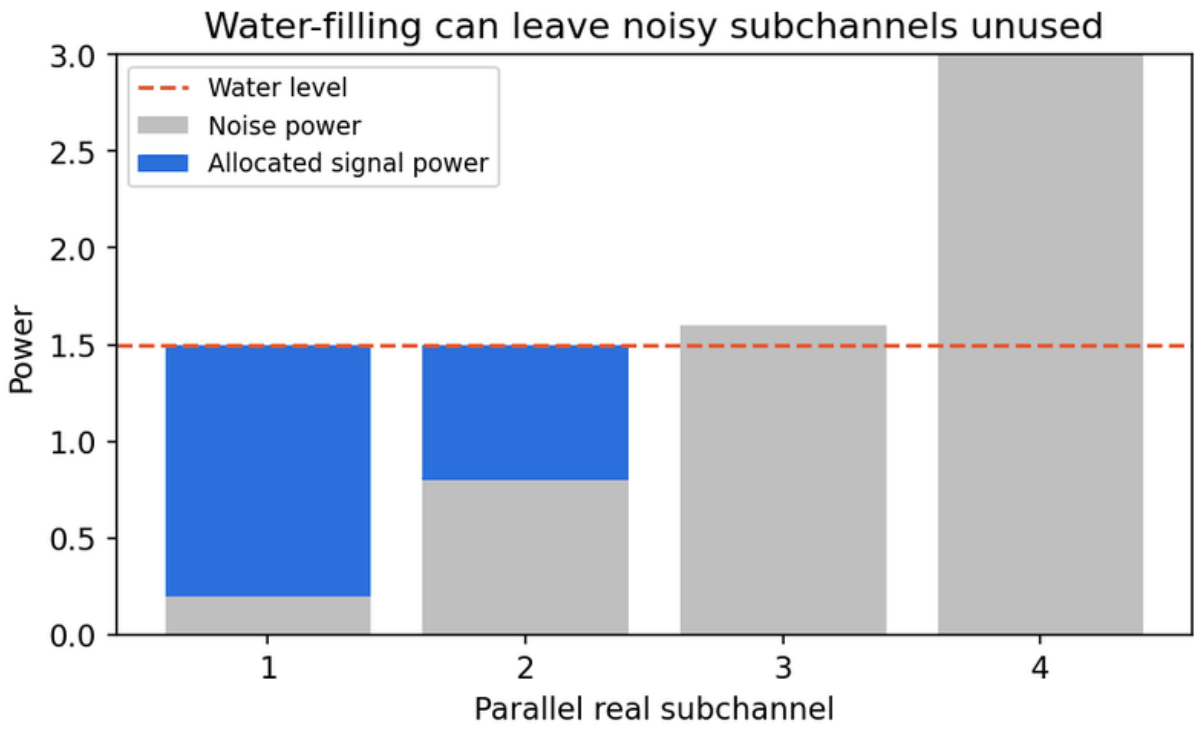


Figure 2 — Total signal power 2.0 across four independent real channels. Water-filling is an analytical resource allocation under the stated model.

Solve the plotted allocation by hand

The noise floors are 0.2, 0.8, 1.6 and 3.0, and the total signal-power budget is two. First suppose only the first channel is active. Its total level would be $0.2 + 2 = 2.2$, which rises above the second floor. Leaving the second channel unused cannot be optimal.

Try the first two active channels. Their powers sum to $(\mu - 0.2) + (\mu - 0.8) = 2$, giving $\mu = 1.5$. This is above both active floors and below the next floor 1.6, so the assumed active set is consistent. Allocate powers 1.3, 0.7, zero and zero.

The resulting joint-use rate is $\frac{1}{2}\log_2(7.5) + \frac{1}{2}\log_2(1.875) = 1.906891$ bits. Equal allocation of 0.5 to each channel gives about 1.561252 bits. Both designs spend total power two; the improvement comes from its placement, not an enlarged budget.

The third channel becomes active when the level exceeds 1.6, which first requires budget $(1.6 - 0.2) + (1.6 - 0.8) = 2.2$. The fourth becomes active above level three, at budget $(3 - 0.2) + (3 - 0.8) + (3 - 1.6) = 6.4$. These thresholds explain the changing active set in a capacity–power curve.

Equal signal-plus-noise levels apply only to the active channels. A channel with a floor above the level keeps zero signal power. This complementary condition is why blindly forcing equal total levels across all four channels would produce impossible negative powers.

Code: solve the level and compare equal allocation

This complete example solves the level by bisection, verifies the budget and active optimality condition, and compares its rate with equal power.

```
import numpy as np

noise = np.array([0.2, 0.8, 1.6, 3.0])
budget = 2.0
low, high = noise.min(), noise.max() + budget
for _ in range(100):
    level = (low + high) / 2
    if np.maximum(level-noise, 0).sum() > budget:
        high = level
    else:
        low = level
level = (low + high) / 2
power = np.maximum(level-noise, 0)
equal = np.full(len(noise), budget / len(noise))
rate = lambda allocation: 0.5*np.log2(1+allocation/noise).sum()
assert abs(power.sum()-budget) < 1e-12
assert np.allclose(noise[power > 0]+power[power > 0], level)
assert rate(power) >= rate(equal)
print("Water level / power allocation:", level, power)
print("Water-filled / equal-allocation joint rate:", rate(power), rate(equal))
```

The objective counts one simultaneous use of every dimension. Dividing by the number of dimensions gives a different per-dimension average. Neither number becomes bits per second until a use rate is supplied.

With colored Gaussian noise, a covariance eigenbasis can create parallel noise modes under compatible linear input and power assumptions. With channel gains, the effective noise floor depends on gain as well as physical noise. A zero gain cannot be inverted into a useful channel.

Vector channels connect geometry with covariance

Several measurements can carry overlapping information about several input coordinates. A matrix describes mixing, while covariance describes the noise cloud and the chosen signal spread.

Consider the real linear channel

$$Y = AX + Z, \qquad Z \sim \mathcal{N}(0, K_Z), \qquad X \sim \mathcal{N}(0, K_X), \qquad Z \perp X.$$

Here X is a d-dimensional input, Y an m-dimensional output, A an m by d channel matrix, K_X an input covariance and K_Z a positive-definite noise covariance. The input covariance can be positive semidefinite, allowing unused directions.

The output covariance is $AK_XA^\top + K_Z$. Subtracting Gaussian output and conditional noise entropies gives

$$I(X;Y) = \frac{1}{2}\log_2 \frac{\det(AK_XA^\top + K_Z)}{\det K_Z}.$$

Here $A^\top$ is the matrix transpose and determinants refer to output-dimensional covariance matrices. The positive noise covariance keeps the denominator and output covariance nonsingular, even if the signal has lower rank.

Under an average total-power convention, $\operatorname{tr} K_X \le P$ constrains the sum of input coordinate variances. Maximizing the displayed information over positive-semidefinite K_X gives the corresponding Gaussian vector-channel problem [1]. The trace notation means the sum of diagonal entries, and P is total power per vector use.

The useful directions depend jointly on A and K_Z. A large raw coordinate variance may lie in a direction strongly attenuated by the channel. Correlated noise may make a direction less informative than its marginal variance suggests. Diagonalizing a compatible transformed problem reveals parallel modes where the same marginal-return principle applies.

This example makes clear why preprocessing and physical channel changes must be distinguished. An invertible receiver coordinate transform changes both the signal matrix and the noise covariance; the determinant ratio stays invariant. A projection discards output directions and can reduce information. A noninvertible transmitter map changes which input signals can be produced.

For numerical work, compute log-determinants with a stable factorization instead of taking a large determinant and then its logarithm. A huge determinant can overflow while its logarithm is moderate. Check positive definiteness and matrix dimensions before interpreting the answer.

Conditional entropy and deterministic representations

A representation can be numerically continuous but carry only a finite set of possible states. Its mathematical distribution depends on how it was created, not the array's floating-point storage type.

The differential-entropy definitions assume densities with respect to continuous coordinate volume. A discrete label stored as a floating-point number remains discrete. A vector produced by a deterministic map of a finite dataset has finite empirical support, even if its coordinates are real-valued.

A lower-dimensional manifold inside a larger feature space has a singular distribution relative to full-dimensional volume. Fitting a full-dimensional density implicitly introduces a noise or smoothing model. The resulting entropy belongs to that model and can depend strongly on its scale.

Likewise, adding tiny independent noise to a deterministic continuous relationship makes a density-based calculation possible, but the noise level becomes part of the information question. Letting that level approach zero can send mutual information to infinity. Reporting a finite value without the smoothing convention hides this dependency.

These observations are useful when comparing neural representations. The information relevant

to a discrete task label can remain finite, while information between an exactly deterministic continuous input and representation can be singular. A compression or predictive objective must specify which variables, noise mechanisms and quantizers define its rate.

The practical remedy is to state the observation model before estimating information: discrete bins, an explicit noisy channel, a parametric density or a fixed-resolution coding protocol. Each is legitimate for its stated purpose, and each answers a different question.

Why vanishing added noise can make information diverge

Consider $X \sim \mathcal{N}(0,1)$ and representation $V = X + \varepsilon$, where independent noise $\varepsilon \sim \mathcal{N}(0,\tau^2)$ has positive variance τ^2. This is a Gaussian channel with signal power one, so $I(X;V) = \frac{1}{2}\log_2(1 + 1/\tau^2)$. With $\tau^2 = 1$, information is one half bit. With $\tau^2 = 0.01$, it is approximately 3.329106 bits. As noise variance tends to zero, the number grows without bound.

At exactly zero noise, $V = X$ is an exact continuous copy. The pair lies on a line in the two-dimensional joint space, so it has no ordinary two-dimensional joint density. This is why directly subtracting marginal and joint differential entropies at that endpoint is unsafe. Finite-resolution bins would instead give a finite discrete information question. The noise or resolution convention determines which question a neural-representation estimate answers.

Modeling and estimation: what the formulas certify

A Gaussian formula certifies a Gaussian problem. It does not convert every covariance estimate into a reliable physical communication limit.

The scalar capacity derivation uses independent additive Gaussian noise, an average second-moment budget and an unbounded real input alphabet. A sensor with clipping, quantization, signal-dependent noise or a changing noise distribution may violate one or more assumptions.

For a bounded output quantizer, information cannot exceed the logarithm of its number of possible labels. A real-valued capacity formula calculated before quantization is an upper reference for a different observation model; it is not automatically attainable by the quantized receiver.

Likewise, using a Gaussian approximation for a non-Gaussian source's entropy produces a moment-based upper bound under the needed conditions. It is not a measured entropy value. Good agreement of means and covariances does not establish agreement of tails or dependence.

Finite sample estimates of covariance can be singular when dimension is too high relative to observations. Then a log-determinant formula may fail. Regularizing the covariance creates a modified statistical model; the regularization value affects the reported information and should be stated.

For evaluation, keep the physical noise model, coordinate convention, estimated parameters and observed decoder performance separate. A synthetic test can verify algebra and implementation, while independently collected data must verify the assumptions.

Modern view

Butakov and colleagues' NeurIPS 2024 paper [4] studies mutual-information estimation through normalizing-flow transformations. It provides a modern example of using invertible density models to help estimation, while retaining the need to assess the estimator rather than interpreting coordinate entropy as intrinsic information.

The review perspective in contemporary semantic-communication literature [5] distinguishes receiver modeling, generative reconstruction and learned joint coding. For continuous channels, our inference is that those designs should report power normalization, real or complex use conventions, receiver quantization and channel mismatch explicitly.

An attractive reconstructed image can coexist with low faithful information about a particular original detail. A task-based objective may intentionally discard that detail; this is a rate–distortion choice rather than a different differential-entropy convention.

Open problems include robust information estimation in high dimension, evaluating likelihood models under shift, handling finite precision consistently at both endpoints, and designing learned systems whose resource accounting matches the physical channel. Invertibility and Gaussian assumptions simplify mathematical analysis, but neither substitutes for those checks.

The next chapter asks how much source information is needed when some reconstruction error is acceptable. Gaussian capacity concerns transporting information; Gaussian rate–distortion concerns how much information the source demands.

Key takeaways

- Density is probability per coordinate unit, not a point probability.
- Differential entropy can be negative and changes under coordinate scaling.
- Fine quantization adds a resolution-dependent discrete description cost.
- Relative entropy and mutual information survive reversible coordinate changes.
- Gaussian maximum entropy is a consequence of fixed moment constraints.
- Real Gaussian capacity includes an explicit average-power budget and one-half factor.
- Water-filling equalizes marginal returns across active parallel channels.
- Parametric information formulas and empirical decoder reliability answer different questions.

Exercises

1. Find the differential entropy of a uniform variable on an interval of width 0.01. Explain why its negative value does not permit a negative-length code.

Hint

Use log interval width and separate bin coding.

2. Quantize a unit uniform source into 16, 64 and 256 equal bins. Compute exact discrete entropy and relate it to differential entropy.

Hint

Each bin is equiprobable.

3. Verify mutual-information invariance under separate nonzero scalings of a jointly Gaussian pair. What changes if one variable is rounded to a single bit?

Hint

Determinants cancel for reversible scales; rounding is not invertible.

4. Compare capacity gains from doubling power at signal-to-noise ratios 0.1 and 10. Explain why absolute rate increments behave differently from relative power increments.

Hint

Subtract capacities before and after doubling.

5. Increase the water-filling budget until the third and fourth channels become active. Calculate the activation thresholds from the noise floors.

Hint

Set the water level to the next noise floor.

6. Explain which assumptions fail for a clipped receiver with two output labels. Give an information upper bound independent of signal power.

Hint

Output entropy is at most one bit.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 8–9. Publisher book and contents.
2. T. Weissman, "Continuous Random Variables," Stanford EE276 Lecture 8 notes, 2024. Primary lecture notes.
3. T. Weissman, "AWGN Channel," Stanford EE276 Lecture 9 notes, 2024. Primary lecture notes.
4. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, "Mutual Information Estimation via Normalizing Flows," *NeurIPS*, 2024. Published paper.

5. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025. Published-paper metadata and author version.

IT 06 • Rate–Distortion and Lossy Representations

Derive binary and Gaussian rate–distortion limits, compute finite-alphabet test channels, and evaluate quantization and learned representations with explicit distortion.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-06-rate-distortion/

What you'll learn

- Specify a distortion function before calling a reconstruction acceptable.
- Distinguish operational rate from an informational test channel.
- Derive Bernoulli/Hamming and Gaussian/squared-error rate–distortion functions.
- Explain covering achievability and the converse's information accounting.
- Compute finite-alphabet rate–distortion points with alternating updates.
- Relate practical quantization and learned objectives to honest rate and quality measurements.

The big picture

Lossless coding asks for the exact original. Lossy coding asks for a useful substitute. The second question cannot be answered until "useful" has a mathematical meaning: which differences matter, how much they cost, and whether the constraint concerns average error or rare large failures.

> **Explain like I'm five**
>
> You describe a drawing to a friend with only a few words. You cannot mention every tiny dot. If the goal is to recognize the animal, its shape matters more than every background speck. If the goal is to reproduce the drawing exactly, those same omissions are errors.

This chapter follows Cover and Thomas's Chapter 10 [1] and Stanford's primary rate–distortion notes [2,3]. The central result minimizes mutual information over hypothetical reconstruction laws. That optimization defines a limit; it does not by itself build a finite file format or guarantee the behavior of a neural decoder.

The distinction connects directly to vision. A pixel reconstruction, a machine-readable feature and a plausible generated image preserve different properties. Their rates and distortions should be compared only after the objectives and resource conventions are made explicit.

Choose the distortion before the codec

An error measure specifies which source differences the receiver tolerates. There is no universal distortion score that captures every task.

Let X be a source symbol and $\widehat{X}$ a reconstruction in a possibly different alphabet. A nonnegative distortion function is $d(x, \hat{x})$. For a length-n block,

$$d_n(x^n, \hat{x}^n) = \frac{1}{n} \sum_{t=1}^{n} d(x_t, \hat{x}_t).$$

Here t indexes symbols, n is block length, and d_n is average per-symbol distortion. A common constraint is $\mathbb{E}[d_n(X^n, \widehat{X}^n)] \leq D$, with D the allowed average and expectation under the specified source and code.

Hamming distortion is zero for a correct symbol and one otherwise. Squared-error distortion is $(x - \hat{x})^2$ for real amplitudes. They answer different questions: the first counts mismatches; the second weights large numerical deviations more heavily.

Squared error also depends on units. Scaling source and reconstruction by a multiplies distortion by a^2. A rate–distortion curve should therefore state source normalization, dimensional averaging and the distortion's units.

The usual additive model assigns a cost to each symbol and averages. A structured task may need a block-level criterion: preserving a relation, a rare feature or a decision. It can still be studied, but it is not automatically the same single-letter problem.

Average distortion does not bound the worst individual reconstruction. A small mean can coexist with rare large errors. If the application requires a tail probability, a maximum error or per-group guarantees, include that separate condition rather than reading it into a mean.

The reproduction alphabet is another part of the definition. Restricting reconstructions to a small codebook, integer pixels or a particular model family reduces the feasible set. An unconstrained theoretical optimum may then be unattainable for that design. Conversely, allowing a receiver to output a task label rather than an image changes both the alphabet and the distortion question. A fair comparison should state these choices before showing any rates, so readers can tell whether two systems preserve the same object.

Operational and informational rate–distortion

A real code sends one of finitely many descriptions. A test channel describes a possible statistical relationship between source and reconstruction. The theorem connects their asymptotic limits.

A block encoder maps X^n to one of M indices, and a decoder maps the index to $\widehat{X}^n$. Its fixed rate is

$$R_n = \frac{1}{n} \log_2 M.$$

Here M is the number of possible transmitted descriptions and R_n is bits per source symbol. The operational function $R(D)$ is the infimum of asymptotically achievable rates at the specified distortion.

For an independent memoryless source and an appropriate additive distortion setting, the rate–distortion theorem gives

$$R(D) = \min_{p(\hat{x}|x):\, \mathbb{E}[d(X,\widehat{X})]\le D} I(X;\widehat{X}).$$

Here the source law $p(x)$ is fixed, while $p(\hat{x} \mid x)$ varies over normalized reconstruction laws. The minimum notation applies to settings where a minimizer exists; broader continuous cases can require an infimum and regularity conditions [1,2].

This reconstruction law is often called a **test channel**. Unlike communication capacity, its input source is not optimized. Capacity maximizes transmitted information under a given physical channel; rate–distortion minimizes required information under a reconstruction constraint.

A stochastic test channel is a mathematical object describing the desired joint distribution. The operational theorem uses block codes to approach its information rate. Independently drawing a noisy reconstruction for every source symbol does not itself explain how few bits a receiver needs to reproduce those draws.

Basic shape and zero-rate reconstruction

Allowing more error cannot require more bits. At some distortion, the receiver can always output one fixed reconstruction and need no message.

Define

$$D_0 = \min_{\hat{x}} \mathbb{E}[d(X,\hat{x})].$$

Here D_0 is the best constant-reconstruction average distortion and $\hat{x}$ is chosen independently of the observed source. At $D \ge D_0$, zero rate is feasible.

The function $R(D)$ is nonnegative, nonincreasing and convex in the standard setting. Nonincrease follows because a larger distortion allowance includes all previously feasible codes.

Convexity has a time-sharing interpretation. For fraction α of a long block, use one code; for the rest, use another. Then

$$R(\alpha D_1 + (1-\alpha)D_2) \le \alpha R(D_1) + (1-\alpha)R(D_2).$$

Here $\alpha \in [0,1]$ is the share, D_1, D_2 the component distortions, and the right side their average rates. Predetermined block boundaries avoid per-symbol signaling of the choice.

At zero distortion, a finite source with an identity-only zero-cost condition returns the ordinary entropy limit. If the distortion considers several different source values equally correct, zero distortion may require less than source entropy. For a genuinely continuous Gaussian source under squared error, the zero-distortion rate diverges.

Those qualifications matter. Saying "zero distortion means entropy" without specifying the distortion's zero set or alphabet can erase the very reason that a task representation compresses.

Binary sources with Hamming error

A fair binary source costs one bit per symbol to preserve exactly. Permitting a controlled mismatch fraction reduces the necessary rate.

For $X \sim \text{Bernoulli}(p)$ with $0 < p \leq 1/2$ and Hamming distortion,

$$R(D) = \begin{cases} h_2(p) - h_2(D), & 0 \leq D \leq p, \\ 0, & D \geq p. \end{cases}$$

Here p is the probability of one, D is mismatch probability, and h_2 is binary entropy. The zero-rate reconstruction is always zero, which makes errors on fraction p of source symbols.

To obtain a lower bound, let $E = X \oplus \widehat{X}$ be the mismatch indicator, where $\oplus$ denotes binary exclusive-or. Then

$$I(X;\widehat{X}) = H(X) - H(E \mid \widehat{X}) \geq h_2(p) - h_2(D).$$

Here E has probability of one at most $D \leq p \leq 1/2$, and binary entropy increases over that range. Conditioning cannot increase discrete entropy.

Equality is reached by a **backward** construction: choose $\widehat{X}$ and independent $E \sim \text{Bernoulli}(D)$, then set $X = \widehat{X} \oplus E$. The reconstruction probability satisfies

$$\Pr(\widehat{X} = 1) = \frac{p - D}{1 - 2D}.$$

Here $D < p \leq 1/2$ avoids the singular fair-source endpoint; endpoints follow by continuity or a direct constant reconstruction. The construction gives the required source law and makes mismatch independent of reconstruction.

For a fair source, the curve reduces to $1 - h_2(D)$ through $D = 1/2$. For a biased source, naively flipping X forward with probability D generally does not attain the same optimum. The direction of the independence condition is part of the derivation.

Build the optimal binary joint table

The backward construction becomes clearer when its probabilities are written out. Take source probability $p = 0.2$ and allowed mismatch $D = 0.1$. If $q = \Pr(\widehat{X} = 1)$, independence of reconstruction and mismatch gives

$$p = q(1 - D) + (1 - q)D = D + q(1 - 2D).$$

Here p is the required source-one probability, q the reconstruction-one probability and D the independent backward flip probability. Solving gives $q = (0.2 - 0.1)/(1 - 0.2) = 0.125$. Multiplying reconstruction probabilities by independent error probabilities yields:

Joint probability	$\widehat{X} = 0$	$\widehat{X} = 1$	Source marginal
$X = 0$	0.7875	0.0125	0.8
$X = 1$	0.0875	0.1125	0.2
Reconstruction marginal	0.875	0.125	1.0

The off-diagonal entries sum to 0.1, exactly the allowed distortion. Dividing either reconstruction column by its total shows a 0.1 probability that the source differs from that reconstruction. Thus $H(X \mid \widehat{X}) = h_2(0.1)$ and required information is $h_2(0.2) - h_2(0.1) = 0.252933$ bits per source symbol.

The equivalent **forward** conditional channel is asymmetric. Given $X = 0$, it outputs one with probability $0.0125/0.8 = 0.015625$. Given $X = 1$, it outputs zero with probability $0.0875/0.2 = 0.4375$. The large error on rare ones and small error on common zeros achieve the specified overall error while meeting the optimal posterior uncertainty.

By contrast, independently flipping the source forward with probability 0.1 preserves the same average distortion, but produces reconstruction-one probability 0.26. Its information is $h_2(0.26) - h_2(0.1) = 0.357751$ bits, greater than the optimum. "Ten percent noise" is therefore insufficient to identify an optimal test channel: the direction and independence relationship matter.

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    positive = p > 0
    return -np.sum(p[positive] * np.log2(p[positive]))

source_one, D = 0.2, 0.1
q = (source_one-D)/(1-2*D)
joint = np.array([[(1-q)*(1-D), q*D],
                  [(1-q)*D, q*(1-D)]])
px, pr = joint.sum(axis=1), joint.sum(axis=0)
information = entropy(px)+entropy(pr)-entropy(joint)
assert np.allclose(px, [0.8, 0.2])
assert np.isclose(joint[0, 1]+joint[1, 0], D)
assert np.allclose([joint[1, 0]/pr[0], joint[0, 1]/pr[1]], D)
exact = entropy(px)-entropy([D, 1-D])
assert np.isclose(information, exact)
print("Backward joint table:", joint)
print("Optimal information:", information)
```

This calculation verifies a statistical optimum. It still needs a long-block code to translate that information rate into a transmitted reconstruction index.

A Gaussian source under squared error

A broad Gaussian source needs more bits for the same absolute precision. Requiring smaller error grows rate logarithmically.

For $X \sim \mathcal{N}(0, \sigma^2)$ and $d(x, \hat{x}) = (x - \hat{x})^2$,

$$R(D) = \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma^2}{D}\right\}, \qquad D > 0.$$

Here σ^2 is source variance and D the mean squared error in the same squared units. At $D \geq \sigma^2$, a zero reconstruction requires no bits. As $D \to 0$, rate grows without bound [1,3].

Let $E = X - \widehat{X}$ be reconstruction error. For compatible finite differential entropies,

$$\begin{aligned} I(X; \widehat{X}) &= h(X) - h(E \mid \widehat{X}) \\ &\geq h(X) - h(E) \geq \frac{1}{2}\log_2 \frac{\sigma^2}{D}. \end{aligned}$$

Here conditioning reduces differential entropy when mutual information is well defined, and Gaussian maximum entropy bounds error entropy by its second moment. The expression is a lower bound in the nonzero-rate range.

Achievability of the informational minimum uses independent $\widehat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ and $E \sim \mathcal{N}(0, D)$, with $X = \widehat{X} + E$. This is again a backward test channel; the error is independent of reconstruction, not of the source.

The equivalent distortion–rate relation is $D(R) = \sigma^2 2^{-2R}$. It refers to asymptotically optimized block coding. A simple scalar quantizer generally lies above the limit at its resulting rate.

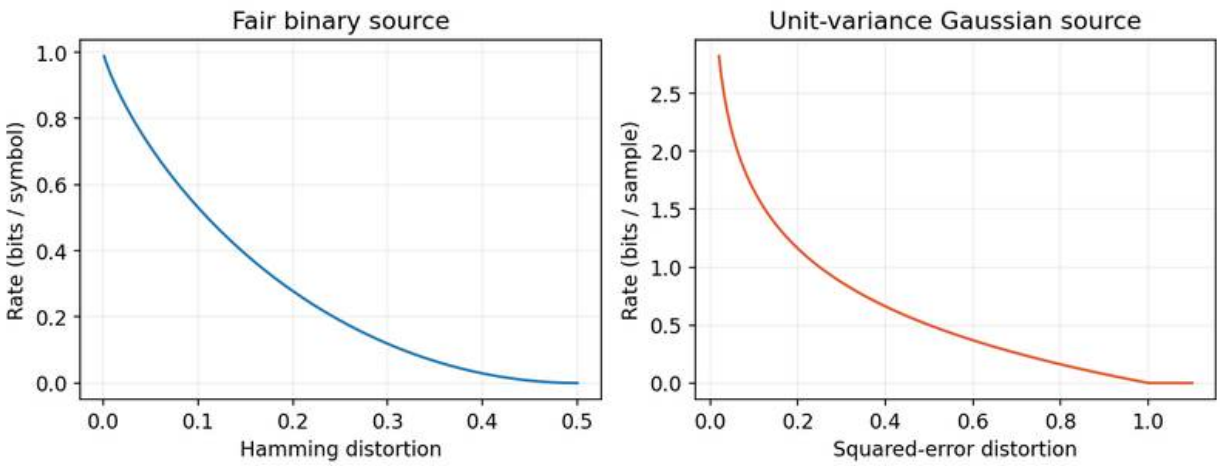


Figure 1 — Analytical examples with different distortion meanings. The horizontal values cannot be compared as a common quality score.

Code: test the Gaussian backward construction

This complete synthetic experiment verifies the source variance, error variance and information formula. It does not serialize a block code; it tests the optimizing joint distribution.

```
import numpy as np

rng = np.random.default_rng(38)
variance, distortion = 1.0, 0.2
reconstruction = rng.normal(0, np.sqrt(variance-distortion), size=100000)
error = rng.normal(0, np.sqrt(distortion), size=100000)
source = reconstruction + error
rho = np.corrcoef(source, reconstruction)[0, 1]
information = -0.5 * np.log2(1-rho**2)
exact = 0.5 * np.log2(variance/distortion)
print("Source variance / error MSE:", source.var(), np.mean(error**2))
print("Error-reconstruction correlation:", np.corrcoef(error, reconstruction)[0, 1])
print("Gaussian-model information / exact rate:", information, exact)
assert abs(source.var()-variance) < 0.02
assert abs(np.mean(error**2)-distortion) < 0.01
assert abs(information-exact) < 0.02
```

Independence is built into the sampling, while the finite correlation is only a check. Small empirical correlation alone would not establish independence for arbitrary non-Gaussian variables.

Why covering differs from channel packing

A lossy source code needs enough reconstruction representatives to cover likely source blocks within acceptable error. A channel code instead separates transmitted representatives to resist noise.

Fix a feasible source–reconstruction joint distribution. Draw many reconstruction codewords from its marginal. For a typical source block, a random reconstruction has a matching joint relationship with a probability on the exponential scale $2^{-nI(X;\widehat{X})}$.

With more than approximately $2^{nI(X;\widehat{X})}$ representatives and a small rate margin, a suitable match becomes likely. The encoder selects its index; the decoder returns that shared reconstruction. Joint typicality connects the chosen pair to the target distortion.

This is a covering argument. The exact proof needs typicality definitions and source/distortion assumptions; the exponential scale explains the rate without serving as a finite-code performance formula.

The converse uses the transmitted index J:

$$nR_n \geq H(J) \geq I(X^n;\widehat{X}^n) \geq \sum_{t=1}^{n} R(D_t) \geq nR\left(\frac{1}{n}\sum_t D_t\right).$$

Here J ranges over M descriptions, $R_n = \log_2 M/n$, $D_t = \mathbb{E}[d(X_t, \widehat{X}_t)]$, and the source is memoryless. The first information bound uses that reconstruction is determined by the index; the per-symbol step uses source independence and the informational definition; the last step is convexity.

The code cannot convey more information about the source than its index contains. This accounting explains why a decoder's sophisticated prior cannot recover arbitrary missing details at no source rate. It can choose useful or probable reconstructions according to the specified distortion.

Compute a finite test channel

For discrete alphabets, alternate between a reconstruction marginal and a source-conditioned reconstruction law. A penalty parameter moves along the rate–distortion frontier.

Minimize the natural-unit objective $I_{\text{nats}}(X; \widehat{X}) + \beta \mathbb{E}[d]$ with $\beta \geq 0$. Given current reconstruction marginal $q_{\hat{x}}$,

$$V(\hat{x} \mid x) = \frac{q_{\hat{x}} \exp[-\beta d(x, \hat{x})]}{\sum_z q_z \exp[-\beta d(x, z)]}, \qquad q_{\hat{x}}^{\text{new}} = \sum_x p_x V(\hat{x} \mid x).$$

Here p_x is the fixed source law, V is the test channel, z indexes reconstructions, and β measures distortion penalty in natural units. Rates reported in bits divide the natural-unit information by $\ln 2$.

The Blahut–Arimoto alternating computation is discussed in the textbook [1]. A positive initial marginal covers all candidate reconstruction symbols. At a solution, compute actual distortion and mutual information; do not label β itself as the distortion.

Where the exponential test-channel update comes from

The alternating rule follows from a constrained minimization, not a heuristic resemblance to a probability distribution. Temporarily fix a positive reconstruction marginal q_j. For source state i, minimize the row contribution

$$\sum_j V_{ij}[\ln(V_{ij}/q_j) + \beta d_{ij}], \qquad \sum_j V_{ij} = 1.$$

Here V_{ij} is the conditional probability of reconstruction j given source i, q_j the fixed reference marginal, d_{ij} distortion and β the natural-unit distortion penalty. The positive source weight p_i multiplies this row and does not change its minimizing conditional distribution.

Add a row-normalization multiplier η_i and differentiate with respect to V_{ij}. The stationary equation is $\ln(V_{ij}/q_j) + 1 + \beta d_{ij} + \eta_i = 0$. Exponentiation makes V_{ij} proportional to $q_j \exp(-\beta d_{ij})$; row normalization supplies the denominator in the displayed update.

Next fix V. Its induced marginal $q_j^{\text{new}} = \sum_i p_i V_{ij}$ minimizes the cross-entropy part over normalized q. Alternating these two steps lowers the penalized objective under the stated finite-alphabet

setting. It also explains the geometry: larger β concentrates each row toward low-distortion reconstructions, while the marginal factor favors reconstructions used by the population.

For a fair binary source with Hamming distortion, symmetry keeps q uniform. Off-diagonal probability is therefore $D = 1/(1+e^{\beta})$, and the rate is $1-h_2(D)$. This analytic relation is what the code checks. It gives a direct interpretation of the penalty without mistaking the penalty value for an achieved distortion.

Code: compare with exact binary curves

The implementation checks convergence of the reconstruction marginal and validates the resulting points against independent analytic formulas. Its tolerance is a numerical update tolerance, not the upper–lower capacity gap from the previous chapter.

```
import numpy as np
from scipy.special import logsumexp

def entropy(p):
    p = np.asarray(p)
    return -np.sum(p[p > 0] * np.log2(p[p > 0]))
def rd_point(p, distortion_matrix, beta):
    p = np.asarray(p, dtype=float)
    d = np.asarray(distortion_matrix, dtype=float)
    q = np.full(d.shape[1], 1/d.shape[1])
    for _ in range(50000):
        scores = np.log(q)[None, :] - beta*d
        V = np.exp(scores-logsumexp(scores, axis=1, keepdims=True))
        updated = p @ V
        if np.max(abs(updated-q)) < 1e-13:
            q = updated
            break
        q = updated
    else:
        raise RuntimeError("Rate-distortion updates did not converge")
    joint = p[:, None]*V
    positive = joint > 0
    ratio = np.divide(V, q[None, :], out=np.ones_like(V), where=positive)
    rate = np.sum(joint[positive]*np.log2(ratio[positive]))
    distortion = np.sum(joint*d)
    return rate, distortion, V

d = np.array([[0, 1], [1, 0]])
for beta in [0.0, 1.0, 2.0, 4.0]:
    rate, D, V = rd_point([0.5, 0.5], d, beta)
    exact = 1-entropy([D, 1-D])
    assert abs(rate-exact) < 1e-10
    print("Penalty / D / rate / exact:", beta, D, rate, exact)
rate, D, V = rd_point([0.8, 0.2], d, 4.0)
exact = entropy([0.8, 0.2])-entropy([D, 1-D])
assert D < 0.2 and abs(rate-exact) < 1e-9
```

```
print("Biased-source D / rate / exact:", D, rate, exact)
```

For arbitrary distortion matrices, a small marginal-update difference is a useful convergence check but not a rigorous global numerical certificate. Inspect objective changes, supports and independent special cases. Very large penalties can underflow probability entries or require a more careful implementation.

Scalar quantization is a concrete achievable design

Round source values to a finite-resolution grid, then losslessly describe their grid indices. The rounding creates distortion; probability coding reduces index rate.

With step Δ, use

$$K = \mathrm{round}(X/\Delta), \qquad \widehat{X} = \Delta K, \qquad D_\Delta = \mathbb{E}[(X - \widehat{X})^2].$$

Here K is the discrete index, $\widehat{X}$ the grid reconstruction, and D_Δ its squared-error distortion. An entropy-coded index has a rate governed by the index distribution, plus practical coding costs.

For sufficiently smooth densities at small step, a common approximation is $D_\Delta \approx \Delta^2/12$. The approximation treats quantization error as approximately uniform inside a cell; it need not hold at coarse steps, near clipping or for structured concentrated values.

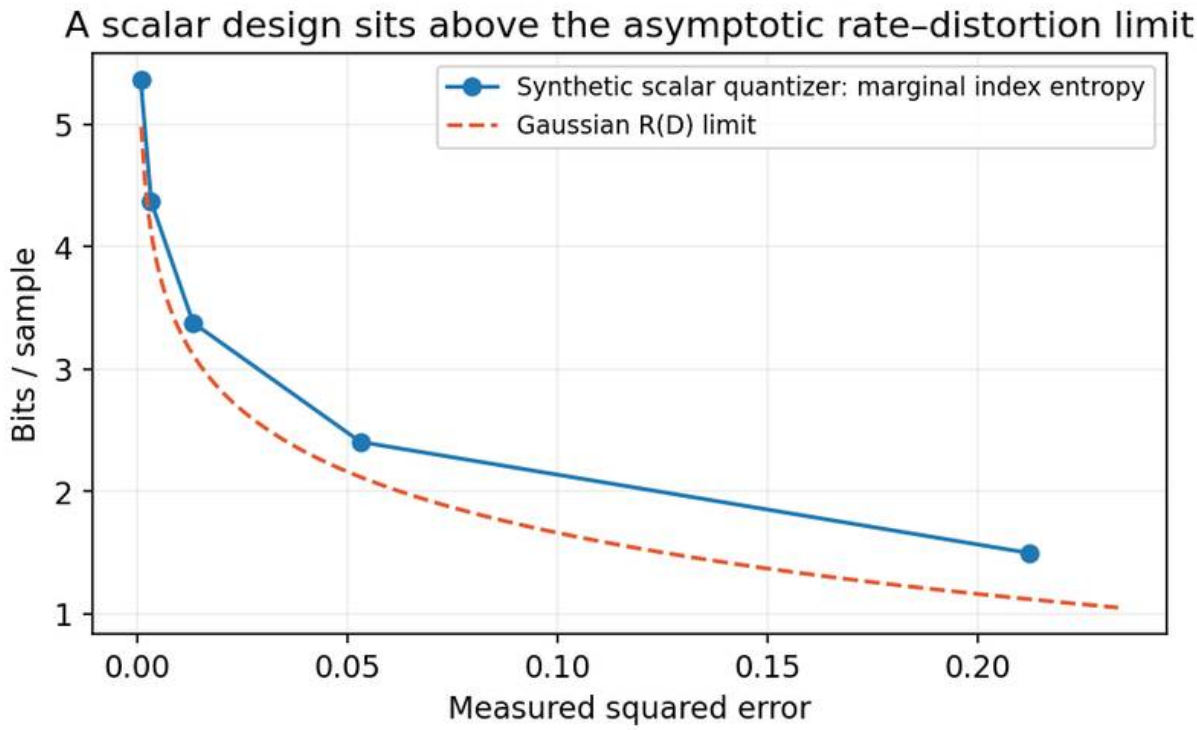


Figure 2 — 20,000 synthetic unit-Gaussian samples. Index entropy is an ideal marginal coding statistic; no actual bitstream length is claimed.

```
import numpy as np

rng = np.random.default_rng(26)
source = rng.normal(size=20000)
for step in [0.1, 0.2, 0.4, 0.8, 1.6]:
    index = np.rint(source/step).astype(int)
    reconstruction = step*index
    _, counts = np.unique(index, return_counts=True)
    p = counts/counts.sum()
    rate_estimate = -np.sum(p*np.log2(p))
    mse = np.mean((source-reconstruction)**2)
    gaussian_limit = max(0, 0.5*np.log2(1/mse))
    print("Step / index entropy / MSE / reference rate:",
          step, rate_estimate, mse, gaussian_limit)
```

The empirical distribution is measured on the same sample being quantized, so it is a descriptive statistic. To compare a learned entropy model, fit it on separate training observations and score a fixed held-out set. Transmit or share its parameters under an explicit convention.

Index entropy is not an actual compressed file size. A codec also specifies index support, out-of-range handling, reconstruction step, message length and entropy-coder termination. Finite empirical estimates can fluctuate around theoretical relationships without invalidating the population theorem.

Parallel Gaussian sources use reverse water-filling

A limited description budget should preserve the most variable source directions more carefully. Low-variance directions can be discarded when their entire variation is within the error allowance.

For independent Gaussian source coordinates with variances σ_i^2, use total squared-error budget D_{total}:

$$R(D_{\text{total}}) = \min_{\sum_i D_i \le D_{\text{total}}} \sum_i \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma_i^2}{D_i}\right\}.$$

Here i indexes coordinates, D_i is their allocated mean squared error, and R is bits per complete source vector. Within the positive-rate range, differentiating the objective makes the optimal active distortions equal:

$$D_i = \min(\theta, \sigma_i^2), \qquad \sum_i \min(\theta, \sigma_i^2) = D_{\text{total}}.$$

Here θ is the reverse water level, chosen for a budget below total source variance. A coordinate with variance below the level is reconstructed as its mean, paying its full variance and no rate.

The similarity to channel water-filling is instructive, but the roles differ. Channel allocation spends signal power above noise floors to maximize transported information. Source allocation permits error up to source variances to minimize required description information.

For a correlated Gaussian vector, an orthogonal covariance eigenbasis produces independent coordinates and preserves total squared error. The allocation then acts on eigenvalues. A nonlinear task distortion or a non-Gaussian distribution need not reduce to this same eigenvalue calculation.

This also explains the benefit and limitation of a low-dimensional reconstruction. Discarding a low-variance coordinate is inexpensive for total squared error, but that coordinate could still be decisive for a particular label. Task relevance and variance are different criteria.

A reverse-water-filling allocation

Take three independent Gaussian coordinates with variances 4, 1 and 0.25, and allow total squared error one per vector. Giving each coordinate error one third would waste the third coordinate's budget: reconstructing it as zero already incurs only 0.25.

The optimal common error level for the remaining two coordinates solves $\theta + \theta + 0.25 = 1$, so $\theta = 0.375$. The distortions are 0.375, 0.375 and 0.25; their sum is one. Required rate is

$$R = \frac{1}{2}\log_2 \frac{4}{0.375} + \frac{1}{2}\log_2 \frac{1}{0.375} + 0 \approx 2.415037.$$

Here R is bits per three-coordinate source vector. The first two coordinates receive positive description rate, while the third is reconstructed as its mean and costs zero rate. The distortion budget is a sum per vector, not a mean per coordinate; dividing the budget by three would express the same operating point using a different convention.

The shared error level follows by differentiating each active coordinate's rate: the magnitude of the rate decrease per extra unit of distortion is $1/(2 \ln 2\, D_i)$. Equalizing that marginal saving makes active D_i equal. Once D_i reaches its source variance, there is no further rate to save, so that coordinate leaves the active set.

A small-variance coordinate may still encode an important label. Reverse water-filling is optimal for this Gaussian squared-error problem, while a label-preservation requirement would need a different distortion. This example makes the link between optimization conditions and the actual receiver objective explicit.

Comparing operating points without changing the problem

A fair rate–distortion comparison keeps source data, quality criteria and resource assumptions fixed. Otherwise a better-looking curve may be solving an easier problem.

An empirical operating point consists of measured rate and measured distortion under one complete protocol. Its description should specify whether the rate is fixed length, average variable length, ideal negative log-probability or actual serialized bits.

The theoretical curve in this chapter uses a memoryless population source and asymptotically long codes. A finite experiment has sampling uncertainty, finite blocks, a restricted model family and implementation costs. Reporting all of those as one gap obscures which component is responsible.

Keep the same held-out observations for different codec settings. Fit quantizers, probability models and any learned transforms using training data. Use validation data to choose penalties or operating points. Reserve the final test set to estimate the selected protocol's performance.

Report distortion across meaningful groups when the population is heterogeneous. An average over common cases can hide a subgroup whose reconstructions are much worse. This does not invalidate a mean-distortion theorem; it shows that the selected mean may not fully express the desired application.

Also state decoding randomness. A deterministic decoder and a randomized generative decoder can have different error distributions at the same stored index. Repeated reconstructions of the same test input reveal that variability; selecting the best random reconstruction after seeing the original introduces a selection step that a real receiver may not have.

Finally, keep shared side information comparable. A decoder with a task-specific database, pretrained prior or transmitted adaptation receives a different protocol from one without it. Shared resources can be useful, but their availability and cost convention need to accompany the claimed frontier.

Representations, tasks and learned rate penalties

A representation can discard irrelevant detail while preserving a task. That helps only when the retained task is specified and evaluated on the intended population.

A learned encoder may produce quantized latent Z with entropy model q, while a decoder returns $\widehat{X}$. A common training objective has the form

$$\mathcal{L} = \mathbb{E}[-\log_2 q(Z)] + \lambda \mathbb{E}[d(X, \widehat{X})].$$

Here the first term is an ideal coding-cost surrogate in bits, $\lambda \geq 0$ balances the specified distortion, and expectations use the training distribution. This objective does not automatically equal the population rate–distortion function.

The latent alphabet, entropy-model mismatch, finite coder, side streams and model costs affect actual rate. The architecture restricts achievable reconstructions. Optimization can find a local or model-constrained tradeoff rather than the information-theoretic minimum.

If the receiver needs a label Y, an appropriate criterion can evaluate the decision inferred from Z instead of reconstructing all of X. That is a different source-distortion problem. A feature preserving one label can discard information needed by a later, unrelated task.

A perceptual criterion may reward plausible texture even when texture differs from the source. A faithful reconstruction criterion may penalize the same change. Neither objective is universally correct; their distinction needs to remain visible in text, figures and evaluation.

Multiple distortions can be reported together without collapsing them into one unexplained score. For example, separate numerical fidelity, a stated task error and a distributional realism criterion.

Modern view

The 2023 monograph by Yang, Mandt and Theis [4] separates lossless entropy coding from lossy transform design, quantization, perceptual objectives and task-oriented coding. Its taxonomy helps locate which component of a learned system changes rate, which changes distortion, and which changes the evaluation goal.

Huang and Wu's 2024 review [5] explicitly contrasts human and machine consumers of learned image compression. The shared lesson is that a compressed representation's usefulness is receiver-dependent; one reconstruction score cannot establish preservation for every downstream analysis.

Hofer and Böhme's 2024 preprint [6] develops a taxonomy of neural-image miscompression. It motivates examining the kinds of changed content, rather than assuming that average numerical error completely characterizes reconstruction behavior. We cite its verified preprint status.

Our inference from rate–distortion theory is to evaluate a **frontier**, not one favorable operating point: measured bitstream size versus explicitly defined distortion on a fixed test population. Keep source normalization, model-sharing assumptions and decoder resource budgets consistent.

Open questions include task transfer after irreversible compression, distortion measures for rare details, performance under distribution shift, and fair resource accounting for generative priors. Longer blocks or more elaborate decoders can improve a particular tradeoff while adding latency or relying on stronger shared assumptions.

The next chapter extends information beyond coding: divergence describes statistical evidence, maximum entropy selects distributions under constraints, and logarithmic growth connects prediction with decision-making.

Key takeaways

- Distortion defines what a lossy reconstruction is allowed to change.
- Rate–distortion fixes the source and optimizes a reconstruction law.
- Its standard frontier is nonincreasing and convex.
- Binary and Gaussian optima use carefully defined backward test channels.
- Covering and information accounting connect test channels to block codes.
- Alternating numerical updates require explicit logarithm and penalty units.
- Quantizer index entropy differs from actual serialized rate.
- Learned or perceptual objectives must be judged against their stated receiver and distortion.

Exercises

1. For a Bernoulli source with probability 0.2, compute the zero-rate distortion and rates at distortions 0, 0.05 and 0.1.

   Hint

   Use $h_2(p) - h_2(D)$ within the nonzero-rate range.

2. Construct the backward binary joint law at $p = 0.2$, $D = 0.1$. Compare it with independently flipping the source forward by 0.1.

   Hint

   Make mismatch independent of reconstruction.

3. For a Gaussian source of variance 4, compute the rates needed for squared errors 1 and 0.25. Explain the result after changing coordinate units.

   Hint

   Rate depends on a variance-to-distortion ratio.

4. Extend the finite test-channel code to a three-symbol source and an asymmetric distortion matrix. Report the penalty, actual rate and actual distortion separately.

   Hint

   Check row normalization and reconstruct the joint law.

5. Add clipping to scalar quantization. Compare errors near the center and tails instead of reporting only the mean.

   Hint

   Track saturation cases separately.

6. Design two different distortion functions for a tiny binary image: exact pixel preservation and preservation of whether any foreground pixel exists. Explain why their zero-distortion rates differ.

   Hint

   The task identifies equivalence classes of images.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 10. Publisher book and contents.
2. T. Weissman, "Lossy Compression & Rate Distortion Theory," Stanford EE276 Lecture 13 notes, 2024. Primary lecture notes.
3. T. Weissman, "Lossy Compression & Rate Distortion Theory Continued," Stanford EE276 Lecture 14 notes, 2024. Primary lecture notes.
4. Y. Yang, S. Mandt and L. Theis, "An Introduction to Neural Data Compression," *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. Published monograph, author version.
5. C.-H. Huang and J.-L. Wu, "Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression," *Entropy*, 26(5), 357, 2024. Published review.
6. N. Hofer and R. Böhme, "A Taxonomy of Miscompressions: Preparing Image Forensics for Neural Compression," arXiv preprint, 2024. Primary preprint.

IT 07 • Statistical Evidence, Maximum Entropy and Growth

Connect likelihood ratios, rare empirical distributions, Fisher information, maximum entropy and logarithmic growth through explicit statistical models.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-07-statistics-decision-growth/

What you'll learn

- Interpret a likelihood ratio as evidence rather than a posterior probability.
- Use empirical types to explain divergence-controlled rare-event rates.
- Distinguish hypothesis-testing error tradeoffs from parameter estimation.
- Derive Fisher information and a regular unbiased-estimation bound.
- Construct maximum-entropy distributions from explicit constraints.
- Relate log-optimal allocation and universal mixtures to coding regret.

The big picture

Information measures are not restricted to storage and communication. They also quantify how observations distinguish competing explanations, how strongly a distribution changes with a parameter, and how costly a poor predictive decision can become over repeated trials.

> **Explain like I'm five**
>
> Two bags have different color mixtures. After seeing many beads, you can ask which bag better explains them. The same probabilities can help you describe the bead string, estimate the mixture, or choose a strategy in a repeated game. Each question uses the observations differently.

The book map is Cover and Thomas's Chapter 11 for statistics, Chapter 12 for maximum entropy, Chapter 6 for gambling and coding, and Chapter 16 for portfolio theory [1]. Stanford's method-of-types notes [2] provide an accessible primary finite-alphabet derivation.

We use mathematical decision games with fully specified probabilities and payoffs. Their purpose is to expose information identities, not to infer practical returns from historical data. In every section, distinguish the source law, the chosen model, the observed sample and the decision criterion.

Likelihood ratios compare explanations

A sample supports one model over another when that model assigns the sample a larger probability. Repeated independent observations add their log evidence.

For two fully specified distributions P and Q with compatible support, define

$$\Lambda(x^n) = \log \frac{P(x^n)}{Q(x^n)} = \sum_{t=1}^{n} \log \frac{P(x_t)}{Q(x_t)}.$$

Here x^n is an observed length-n sample, the final equality assumes independent observations under each model, and log may be natural or base two if used consistently. Positive values favor P in the likelihood comparison.

Under P, expected base-two log ratio is $nD_{\mathrm{KL}}(P\|Q)$. Under Q, it is $-nD_{\mathrm{KL}}(Q\|P)$. The different directions arise because expectation is taken under different laws.

A likelihood ratio is not a posterior probability. To calculate posterior odds, multiply by prior odds. To choose an action, incorporate the losses of wrong decisions. A threshold of zero is appropriate for certain symmetric prior-and-loss choices, not every test.

For simple hypotheses, the Neyman–Pearson likelihood-ratio principle describes the most powerful test at a specified false-alarm constraint [1]. “Simple” means the distributions are fixed, not families with unknown fitted parameters.

If parameters are estimated from the same sample, the test is a different procedure. Model selection, nuisance parameters and threshold calibration can alter its finite error behavior. It is useful to start with fully specified toy laws before interpreting a fitted classifier’s score as evidence.

Testing errors form a tradeoff

Making it harder to declare an alternative reduces false alarms but can miss more real alternatives. Neither error should disappear from the report.

Set null distribution P_0 and alternative P_1. A rejection region A gives

$$\alpha = P_0(X^n \in A), \qquad \beta = P_1(X^n \notin A).$$

Here α is the type-I error or false-alarm probability, β the type-II error or missed-alternative probability, and A the set where the test declares the alternative.

For Bernoulli probabilities $p_0 < p_1$, the log likelihood ratio of alternative to null is an increasing function of the count of ones. A threshold on that count therefore implements a likelihood-ratio test.

Discrete counts may prevent reaching an arbitrary false-alarm target exactly with a deterministic threshold. Randomization at a boundary can achieve an intermediate operating point mathematically; a deployed rule should state whether it uses such randomization.

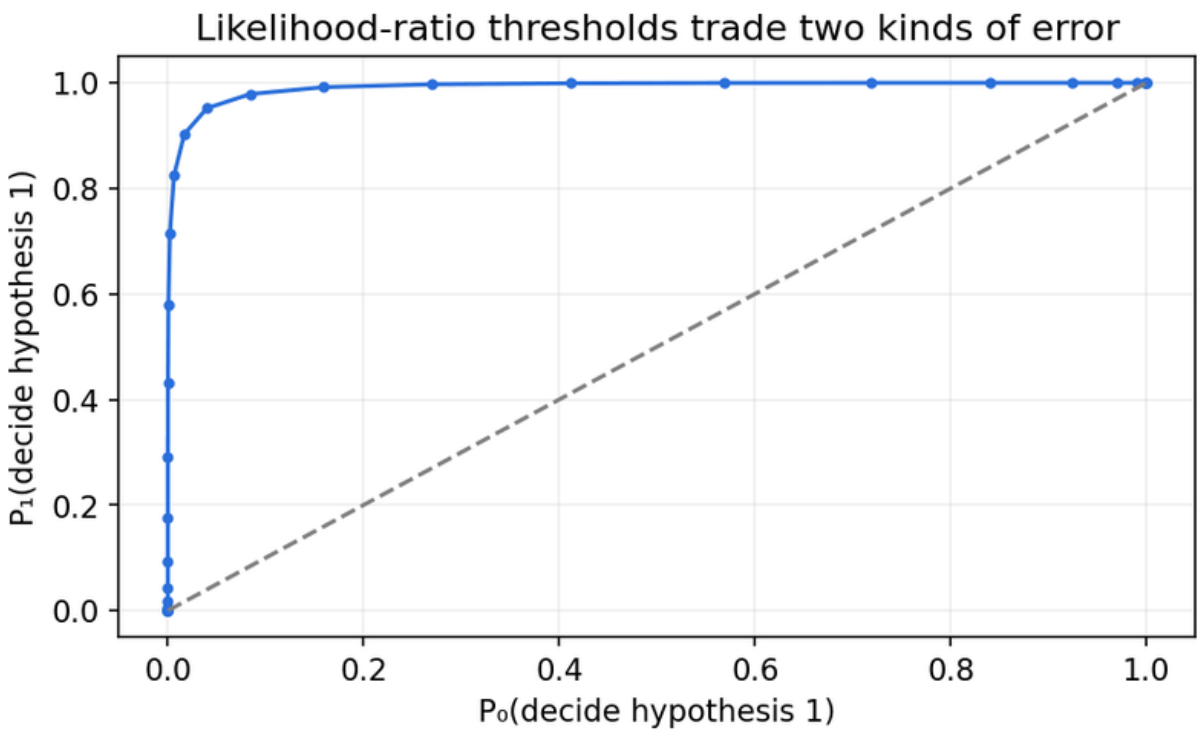


Figure 1 — Exact binomial probabilities. Detection probability is one minus type-II error; each point corresponds to a stated count threshold.

Code: exact errors and a Monte Carlo check

This complete example chooses the first count threshold satisfying a 5% false-alarm limit and calculates its missed-alternative probability. It then verifies the two laws independently.

```
import numpy as np
from scipy.stats import binom

n, p0, p1 = 30, 0.3, 0.6
thresholds = np.arange(n+2)
false_alarm = binom.sf(thresholds-1, n, p0)
detection = binom.sf(thresholds-1, n, p1)
threshold = thresholds[np.flatnonzero(false_alarm <= 0.05)[0]]
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
rng = np.random.default_rng(40)
observed_alpha = np.mean(rng.binomial(n, p0, 100000) >= threshold)
observed_beta = np.mean(rng.binomial(n, p1, 100000) < threshold)
print("Threshold / alpha / beta:", threshold, alpha, beta)
print("Simulation alpha / beta:", observed_alpha, observed_beta)
assert abs(observed_alpha-alpha) < 0.005
```

```
assert abs(observed_beta-beta) < 0.005
```

The selected threshold uses known distributions, not a test-set search. In a learned application, tune the threshold on validation data and estimate its errors on separate test observations. Data groups or time dependence can change the effective independence assumption.

The proportion of alternatives in an evaluation set affects the frequency of correct declarations, but it does not change the two conditional errors for a fixed test and fixed laws. A test can have a small false-alarm probability and still produce many false declarations when alternatives are extremely rare. To interpret those declarations, combine the conditional errors with the intended prior prevalence. This is a direct consequence of conditional probability, not a failure of the likelihood-ratio principle. State whether a reported quantity is conditioned on the null, conditioned on the alternative, or conditioned on the test's output.

Worked evidence is not the same as confidence

For the example's null probability 0.3 and alternative probability 0.6, a length-30 block with k ones has alternative-to-null log likelihood ratio

$$\Lambda_k = k \ln \frac{0.6}{0.3} + (30 - k) \ln \frac{0.4}{0.7}.$$

Here k is the observed count, Λ_k uses natural logarithms, and the two fractions compare the probabilities of one and zero under the competing laws. With 18 ones, $\Lambda_{18} \approx 5.761260$, giving likelihood ratio about 317.748375. This evidence favors the alternative strongly.

If the prior alternative probability is 0.01, prior odds are $0.01/0.99$. Posterior odds become approximately $317.748375 \times 0.01/0.99 = 3.209580$, corresponding to posterior probability about 0.762447. The likelihood ratio is not a 99.7% confidence value: the prior changes the posterior interpretation.

An even more practical distinction concerns all positive declarations from the threshold test. The code chooses threshold 14, with false-alarm probability about 0.040053 and missed-alternative probability about 0.048112. If only 1% of cases come from the alternative, the probability that a positive declaration is truly alternative is

$$\Pr(H_1 \mid \text{positive}) = \frac{\pi(1-\beta)}{\pi(1-\beta) + (1-\pi)\alpha} \approx 0.193588.$$

Here $\pi = 0.01$ is prior alternative prevalence, α the test's false-alarm probability and β its missed-alternative probability. The notation H_1 denotes the event that the alternative generated the case. Most positives in this population are false alarms, even though each conditional error is below 5%.

This conclusion follows from counting expected positive cases from the two populations. It does not change the likelihood-ratio optimality result or the measured conditional errors. A decision report should name its conditioning event so readers can distinguish a particular sample's posterior from the average composition of all positive declarations.

```
import numpy as np
from scipy.stats import binom

n, p0, p1, prior = 30, 0.3, 0.6, 0.01
count = 18
log_ratio = count*np.log(p1/p0)+(n-count)*np.log((1-p1)/(1-p0))
posterior = 1/(1+((1-prior)/prior)*np.exp(-log_ratio))
threshold = 14
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
positive_precision = prior*(1-beta)/(prior*(1-beta)+(1-prior)*alpha)
assert alpha <= 0.05 and binom.sf(threshold-2, n, p0) > 0.05
print("Likelihood ratio / sample posterior:", np.exp(log_ratio), posterior)
print("Probability alternative given positive:", positive_precision)
```

Empirical types reveal why divergence controls rare events

Many strings share the same histogram. Their number grows like an entropy exponent, while their individual source probabilities shrink like a cross-entropy exponent. The difference is divergence.

For a finite alphabet of size k, the type $\hat{p}$ of a length-n sequence is its empirical symbol distribution. Its type class $T(\hat{p})$ contains every sequence with those counts. Under independent source law q,

$$q(x^n) = 2^{-n[H(\hat{p})+D_{\mathrm{KL}}(\hat{p}\|q)]}.$$

Here x^n has type $\hat{p}$, $H(\hat{p})$ is empirical entropy, and divergence is measured in bits. The formula follows by grouping identical log probabilities by count.

The class size is bounded, with a conservative polynomial factor, by

$$(n+1)^{-k}2^{nH(\hat{p})} \le |T(\hat{p})| \le 2^{nH(\hat{p})}.$$

Here $|T|$ is the number of sequences; n and k are block length and alphabet size. Multiplying by one sequence probability gives

$$(n+1)^{-k}2^{-nD_{\mathrm{KL}}(\hat{p}\|q)} \le q(T(\hat{p})) \le 2^{-nD_{\mathrm{KL}}(\hat{p}\|q)}.$$

Here $q(T)$ is total type-class probability. The polynomial factor matters at finite length but disappears from the normalized logarithmic exponent [1,2].

The orientation is worth emphasizing: the first argument is the unusual empirical type; the second is the source actually generating observations. This differs from a coding mismatch calculation averaged under a fixed true source.

For sets of empirical distributions, the dominant exponential behavior is governed by the smallest divergence among allowed types. Sanov's theorem formalizes that large-deviation statement under its finite-alphabet and set-regularity conditions [1].

Code: exact binary type probability versus its exponent

This standalone example compares a binomial probability with the exponential bounds. It uses logarithms to avoid underflow and checks the finite polynomial gap.

```
import numpy as np
from scipy.stats import binom

q, target = 0.3, 0.6
for n in [30, 100, 300, 1000]:
    count = int(round(n*target))
    empirical = count/n
    divergence = (empirical*np.log2(empirical/q) +
                  (1-empirical)*np.log2((1-empirical)/(1-q)))
    negative_log_probability = -binom.logpmf(count, n, q)/np.log(2)
    exponent = n*divergence
    assert exponent <= negative_log_probability + 1e-10
    assert negative_log_probability <= exponent + 2*np.log2(n+1) + 1e-10
    print("Length / observed normalized exponent / divergence:",
          n, negative_log_probability/n, divergence)
```

A rare exact type is not the same event as a one-sided tail or a typical neighborhood. The exponent governing a tail comes from optimizing over all counts in that event, not blindly substituting its most extreme count.

From one type to a set of types

For an event defined by a collection $\mathcal{A}$ of empirical distributions, there are at most $(n+1)^k$ possible types. Summing the individual upper bounds gives

$$q(\hat{p} \in \mathcal{A}) \le (n+1)^k 2^{-n \inf_{v \in \mathcal{A}_n} D_{\mathrm{KL}}(v\|q)}.$$

Here $\mathcal{A}_n$ is the subset of $\mathcal{A}$ realizable at length n, v is a candidate type, and the infimum selects the least surprising type under source q. An empty event has probability zero and is treated separately.

The most probable portion of a rare event often lies at its boundary closest to the source, rather than at its most dramatic extreme. For a binary source with probability 0.3, a tail requiring frequency at least 0.6 is governed near frequency 0.6, not frequency one.

Finite type grids matter: the required boundary may fall between attainable fractions. Exact binomial sums handle that discreteness. The asymptotic divergence picture explains the trend while leaving rounding and polynomial factors visible.

This viewpoint is useful for interpreting apparent distribution shifts. A statistically unusual histogram can be evidence against a source law, but it does not identify why the law failed. Dependence, selection bias, measurement changes and a different source can all alter observed frequencies.

Asymmetric and symmetric hypothesis exponents

Different ways of budgeting the two testing errors lead to different information limits. Divergence is one such limit, not the answer to every classification problem.

Suppose observations are independent under null P or alternative Q, and type-I error is kept below a fixed nontrivial tolerance. The optimal type-II exponent approaches

$$-\frac{1}{n}\ln \beta_n \to D_{\text{nats}}(P\|Q).$$

Here β_n is alternative probability of incorrectly accepting the null, and D_{nats} is divergence with natural logarithms. This is the classical Stein asymptotic regime under suitable source/support conditions [1].

The direction follows the acceptance region: typical null observations must remain accepted, and the alternative probability of looking null has a P-to-Q divergence exponent. Relabeling hypotheses reverses the divergence and can change the exponent.

With fixed nonzero priors and a symmetric Bayesian error objective, the exponent instead involves Chernoff information:

$$C_{\text{Ch}}(P,Q) = -\min_{0\le s\le 1} \ln \sum_x P(x)^s Q(x)^{1-s}.$$

Here s interpolates between the two laws and x ranges over their finite alphabet. This quantity is symmetric despite the asymmetric appearance of the summand.

These exponents describe large independent samples, not exact errors for a short sample. The binomial experiment above is a direct finite calculation. Using an asymptotic exponent as an exact probability can be inaccurate when prefactors and decision boundaries matter.

Composite hypotheses, dependence, changing populations and unequal losses require their own analysis. An information-theoretic limit can explain which source differences help discrimination while leaving practical calibration and sampling design unresolved.

Sufficiency and data processing in inference

A summary is sufficient for a parameter when it retains all the sample's evidence about that parameter within the specified model family.

For a family $p_\theta(x^n)$, a statistic $T(X^n)$ is sufficient when the conditional sample law given T does not depend on θ. A factorization criterion expresses this as

$$p_\theta(x^n) = g_\theta(T(x^n))h(x^n).$$

Here θ is the parameter, T the statistic, g_θ the parameter-dependent factor and h a parameter-independent factor, under the relevant family and domination assumptions [1].

For independent Bernoulli trials, the number of ones is sufficient for the unknown probability. The ordering carries no extra parameter evidence in that family. For a Markov family, the ordering can

matter; a single count of ones is no longer generally sufficient.

If a random parameter Θ has a specified prior, sufficiency gives the conditional relation needed for $I(\Theta; X^n) = I(\Theta; T(X^n))$. Arbitrary summaries satisfy only the data-processing upper bound.

The qualification "within the family" matters. A statistic sufficient for one assumed model may discard evidence that the model is wrong. A mean can estimate a location while hiding multimodality, dependence or an outlier pattern.

This connects to learned representations: retaining information about one target is different from retaining every diagnostic property of the raw data. The task and source family must accompany a sufficiency claim.

Fisher information measures local parameter sensitivity

Fisher information measures how strongly the log probability changes when a parameter moves slightly. It is a local property of a model, distinct from mutual information between random variables.

For a scalar parameter θ and regular density or mass function p_θ, define score and information:

$$s_\theta(X) = \frac{\partial}{\partial \theta} \ln p_\theta(X), \qquad \mathcal{J}(\theta) = \mathbb{E}_\theta[s_\theta(X)^2].$$

Here s_θ is the score, $\mathcal{J}$ Fisher information per observation, and expectation is under the parameterized model. Under regularity, score expectation is zero.

For n independent observations, information adds to $n\mathcal{J}(\theta)$. The Cramér–Rao inequality for a regular unbiased estimator $\hat{\theta}$ gives

$$\mathrm{Var}_\theta(\hat{\theta}) \geq \frac{1}{n\mathcal{J}(\theta)}.$$

Here variance describes repeated samples at the fixed parameter. The bound requires differentiability, exchange of differentiation and integration, and appropriate support and unbiasedness assumptions [1].

For Bernoulli probability $\theta \in (0,1)$, $\mathcal{J}(\theta) = 1/[\theta(1-\theta)]$. The sample mean is unbiased with variance $\theta(1-\theta)/n$, achieving that bound.

The information changes with parameter coordinates. Reparameterizing a probability as a log odds multiplies the information by the squared parameter derivative. This is compatible with an unchanged underlying statistical family; the units of parameter error have changed.

Irregular models with parameter-dependent support or biased estimators need different bounds. A small empirical estimator variance alone does not prove accuracy if its mean is displaced from the parameter.

Derive the Bernoulli information and variance bound

For one Bernoulli observation $x \in \{0, 1\}$, log likelihood is $x \ln \theta + (1 - x) \ln(1 - \theta)$. Differentiating gives

$$s_\theta(x) = \frac{x}{\theta} - \frac{1 - x}{1 - \theta} = \frac{x - \theta}{\theta(1 - \theta)}.$$

Here $\theta \in (0, 1)$ is the probability of one and s_θ the score. Its two possible values are $1/\theta$ and $-1/(1 - \theta)$. Averaging their squares with probabilities θ and $1 - \theta$ yields $\mathcal{J}(\theta) = 1/[\theta(1 - \theta)]$.

Why does its reciprocal bound estimation variance? Let T be a regular unbiased estimator based on n independent observations, and let S_n be the sum of their scores. Differentiate unbiasedness, $\mathbb{E}_\theta T = \theta$, under the regularity assumptions:

$$1 = \frac{d}{d\theta} \mathbb{E}_\theta T = \mathbb{E}_\theta[(T - \theta) S_n].$$

Here T is a fixed function of the observed sample, $S_n = \sum_{i=1}^{n} s_\theta(X_i)$ is its total score, and $\mathbb{E}_\theta S_n = 0$. The right side is the covariance between estimator and score. Squared covariance cannot exceed the product of variances. Since $\mathrm{Var}_\theta(S_n) = n\mathcal{J}(\theta)$, the variance bound follows.

For $\theta = 0.3$ and $n = 100$, information per observation is approximately 4.761905 and the variance lower bound is 0.0021. The sample mean reaches it because its centered error is proportional to the total score. Increasing independent sample size by four divides this variance by four and its standard deviation by two. Independence is what makes score variances add; repeatedly measuring the same correlated trial would not justify that scaling.

Fisher information is local divergence curvature

Under suitable twice-differentiable regularity, neighboring parameter distributions satisfy

$$D_{\mathrm{nats}}(p_\theta \| p_{\theta+\delta}) = \frac{1}{2} \mathcal{J}(\theta) \delta^2 + o(\delta^2).$$

Here δ is a small parameter displacement, $\mathcal{J}(\theta)$ is the natural-log score information, and $o(\delta^2)$ denotes a remainder whose ratio to δ^2 tends to zero. With base-two divergence, divide the leading term by $\ln 2$.

The first derivative vanishes because the score has zero expectation. The second derivative gives the Fisher information under the regular model. This links local estimation sensitivity with the global divergence used in testing and coding.

It is a local approximation. A large parameter move can involve different curvature, changing supports or multiple modes. Treating one Fisher value as an exact distance across a whole model family would discard those differences.

Maximum entropy makes constraints explicit

If only a few expectations are known, choose the distribution with largest entropy among those matching them. The answer depends on the stated constraints and allowed support.

On a finite alphabet with features $\phi_j(x)$, impose $\mathbb{E}[\phi_j(X)] = m_j$. Maximizing entropy with normalization gives an exponential-family form

$$p_\lambda(x) = \frac{\exp(\sum_j \lambda_j \phi_j(x))}{Z(\lambda)}, \qquad Z(\lambda) = \sum_x \exp(\sum_j \lambda_j \phi_j(x)).$$

Here λ_j are multipliers chosen to satisfy the moment constraints, Z normalizes the mass function, and the positive-sign convention is a choice of multiplier sign. Existence and interior-support conditions matter.

Differentiating $\ln Z$ gives the constrained feature expectations. The entropy optimum is not determined by a mean alone unless support is also known. On a bounded interval, fixed support without other constraints gives a uniform density; fixed real mean and variance give a Gaussian; nonnegative support and a fixed mean give an exponential density [1].

Maximum entropy is a transparent modeling principle, not evidence that the real source belongs to the chosen family. Unmeasured dependencies and higher moments remain possible. A useful analysis records constraints, fits multipliers and then tests the resulting predictions on separate data.

The exponential form also explains its relationship to divergence. If a candidate distribution matches the specified feature expectations of the maximum-entropy model, averaging the model's log probability depends only on those shared moments. Relative entropy from the candidate to the model then equals the model entropy minus candidate entropy, using the same logarithm base.

That is another instance of a constraint turning a cross-entropy term into a fixed quantity. The Gaussian maximum-entropy proof from the previous chapter is a continuous moment-constrained example. This common algebra clarifies why maximum entropy, exponential families and information projections repeatedly appear together.

Derive and interpret the maximum-entropy distribution

Use natural logarithms during the optimization; this only rescales entropy and its multipliers. Add the normalization constraint and feature constraints to the entropy objective:

$$\mathcal{F}(p) = -\sum_x p_x \ln p_x + \alpha\left(\sum_x p_x - 1\right) + \sum_j \lambda_j \left(\sum_x p_x \phi_j(x) - m_j\right).$$

Here p_x are candidate probabilities, α is the normalization multiplier, λ_j the feature multipliers, $\phi_j(x)$ feature j and m_j its required expectation. At an interior optimum, differentiating each probability gives $-\ln p_x - 1 + \alpha + \sum_j \lambda_j \phi_j(x) = 0$. Exponentiating and normalizing yields the earlier exponential form.

For states 0, 1, 2 with mean 0.7, there is one feature $\phi(x) = x$. Relative weights are $1, e^{\lambda}, e^{2\lambda}$. A uniform distribution would have mean one, so the desired smaller mean requires a negative multiplier. The code finds $\lambda \approx -0.466121$, with probabilities approximately 0.494780, 0.310440 and 0.194780.

This model spreads probability over all allowed states while matching the mean. A point mass or a two-state mixture could satisfy some moment constraints too, but would have lower entropy here. At a feasible boundary, some probabilities can vanish and finite interior multipliers may no longer suffice. The support, moments and existence conditions are therefore part of the mathematical problem, rather than choices made after selecting a favored density.

Code: solve a finite maximum-entropy model and verify Fisher variance

This complete example finds a three-state distribution with a specified mean and compares Bernoulli estimator variance with its exact unbiased bound.

```
import numpy as np
from scipy.optimize import brentq
from scipy.special import softmax

states = np.array([0.0, 1.0, 2.0])
target_mean = 0.7
probability = lambda lam: softmax(lam*states)
lam = brentq(lambda value: probability(value) @ states-target_mean, -20, 20)
p = probability(lam)
assert abs(p @ states-target_mean) < 1e-12
print("Maximum-entropy multiplier / probabilities:", lam, p)

rng = np.random.default_rng(41)
theta, n = 0.3, 100
estimates = rng.binomial(n, theta, size=100000)/n
fisher = 1/(theta*(1-theta))
bound = 1/(n*fisher)
print("Empirical estimator variance / exact bound:", estimates.var(), bound)
assert abs(estimates.var()-bound) < 0.0001
```

The maximum-entropy calculation uses exactly the enumerated support. Allowing other states would change the optimization. The variance calculation checks a synthetic regular case, not a universal lower bound for every estimator design.

Logarithmic growth turns mismatch into a decision cost

In a repeated multiplicative game, a strategy's errors compound. Maximizing expected logarithmic growth makes the best allocation match the outcome probabilities under the stated fair-payoff model.

Let outcome i occur with probability p_i. Allocate fraction $q_i > 0$ to its contingent payoff, with $\sum_i q_i = 1$. A fair reference law $r_i > 0$ pays multiplier $1/r_i$ when outcome i occurs. Then expected base-two log growth is

$$g(p, q; r) = \sum_i p_i \log_2 \frac{q_i}{r_i} = D_{\mathrm{KL}}(p\|r) - D_{\mathrm{KL}}(p\|q).$$

Here p is the outcome law, q the allocation, r the reciprocal-payoff probability vector, and g is log multiplier per trial. The identity uses normalized laws and the stipulated contingent-payoff game [1].

The optimum is $q = p$ on the source support. A mismatched strategy loses exactly $D(p\|q)$ expected log growth. For two outcomes with equal multiplier two, r is fair and the optimum value is $1 - h_2(p)$.

This is the Kelly connection between prediction and growth. It does not mean every portfolio has contingent fair payoffs or that probabilities are known. Adding cash, transaction costs, leverage restrictions or different payoffs changes the optimization.

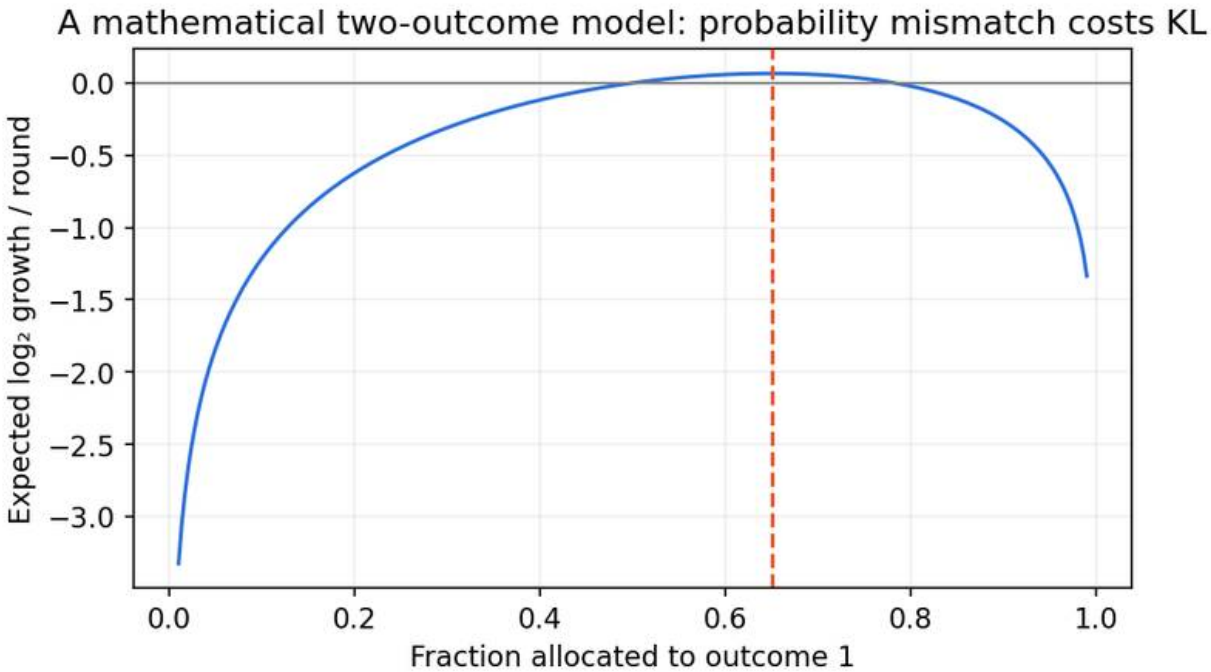


Figure 2 — Analytical two-outcome payoff model. The decrease from the optimum is probability-model divergence.

If a side observation Y is available before each outcome, optimize allocation conditionally. The improvement over a fixed optimal allocation is

$$\mathbb{E}_Y[g(p_{X|Y}, p_{X|Y}; r)] - g(p_X, p_X; r) = I(X; Y).$$

Here X is the outcome, Y the side information, and the reference payoff law r is unchanged. Mu-

tual information has an operational value in this particular log-growth game.

Universal mixtures connect portfolios with coding

Instead of committing to one fixed allocation, mix the wealth of many fixed strategies. Past outcomes then determine how much each strategy contributes.

For a positive return vector R_t and a fixed simplex allocation b, cumulative wealth is

$$W_n(b) = \prod_{t=1}^{n} b^\top R_t, \qquad W_n^{\mathrm{mix}} = \int W_n(b)\, \pi(db).$$

Here t indexes trials, b has nonnegative entries summing to one, R_t contains each component's relative payoff, and π is a prior measure over fixed allocations. The mixture has an equivalent sequential allocation obtained by weighting strategies by their past wealth.

Cover and Thomas's Chapter 16 develops universal-portfolio ideas and their long-run comparisons with a best constant allocation under stated conditions [1]. The benchmark is a fixed allocation in hindsight, not an unrestricted future-knowing trader.

For the two-outcome fair contingent game, a uniform prior on the probability parameter yields the same predictive mechanism as a beta mixture. Its sequential probability of one is $(N_1 + 1)/(t + 2)$ after t observations. This is a whole-count prior, distinct from the half-count predictor in lossless coding.

Code: mixture growth and the best constant allocation

This complete example computes wealth in logarithms and uses the beta integral for the uniform mixture. It compares a known source strategy with the retrospectively best fixed strategy and the causal mixture.

```
import numpy as np
from scipy.special import betaln

rng = np.random.default_rng(42)
p, n = 0.65, 2000
outcomes = rng.binomial(1, p, size=n)
ones, zeros = int(outcomes.sum()), n-int(outcomes.sum())
fitted = ones/n
oracle_log_wealth = n*np.log(2)+ones*np.log(p)+zeros*np.log(1-p)
best_log_wealth = n*np.log(2)+ones*np.log(fitted)+zeros*np.log(1-fitted)
mixture_log_wealth = n*np.log(2)+betaln(ones+1, zeros+1)
sequential_log_wealth, previous_ones = 0.0, 0
for t, outcome in enumerate(outcomes):
    q = (previous_ones+1)/(t+2)
    sequential_log_wealth += np.log(2*(q if outcome else 1-q))
    previous_ones += int(outcome)
assert abs(sequential_log_wealth-mixture_log_wealth) < 1e-9
```

```
assert mixture_log_wealth <= best_log_wealth
print("Source / best fixed / mixture growth bits per trial:",
      oracle_log_wealth/n/np.log(2), best_log_wealth/n/np.log(2),
      mixture_log_wealth/n/np.log(2))
print("Mixture regret to best fixed, bits:",
      (best_log_wealth-mixture_log_wealth)/np.log(2))
```

This is a controlled synthetic payoff experiment. The best fixed parameter uses the entire sequence and is a benchmark, while the mixture updates causally. Log wealth minus a best benchmark has the same algebra as negative-log-probability regret, with opposite orientation.

Modern view

Hellström, Durisi, Guedj and Raginsky's published 2025 monograph [3] unifies information-theoretic and PAC-Bayesian generalization bounds. Its taxonomy separates expectation, probability and conditional-mutual-information formulations, making clear that a statistical guarantee needs a specified random object and evaluation criterion.

El Gamal's 2026 proposed learning chapter [4] connects divergence identities with statistical learning through a textbook-oriented presentation. It is a draft supplement. We use its current perspective alongside the published second edition, without treating a proposed chapter as a completed new edition.

Our inference is that the same quantity can have several operational meanings only after the model is specified. Divergence can be a coding penalty, an unusual-type exponent, an evidence rate or a log-growth loss. The direction, logarithm base and limiting regime can differ among those uses.

Open problems in learned inference include reliable estimation of information in large parameter spaces, guarantees under dependent or shifted data, and bounds that remain informative for actual model selection procedures. A formal inequality can become vacuous when its information term is infinite or difficult to estimate.

For practical work, pair theoretical identities with independent finite experiments: exact toy probabilities to verify implementation, held-out likelihood to assess model fit, and decision-specific errors to evaluate use. No single entropy number replaces all three.

The final chapter combines multiple sources and receivers, develops stronger inequalities, and connects these foundations to modern learning and a small distributed-information capstone.

Key takeaways

- Likelihood evidence, posterior odds and decision thresholds are distinct.
- Testing reports both false-alarm and missed-alternative probabilities.
- Type probabilities have divergence exponents with explicit argument orientation.
- Hypothesis-testing exponents depend on the chosen error regime.
- Sufficiency is relative to a model family and target parameter.

- Fisher information gives local regular-estimation limits.
- Maximum entropy depends on support and stated constraints.
- Log-growth mismatch and universal mixture regret share coding identities.

Exercises

1. Reverse the null and alternative in the binomial example. Find a threshold meeting a 5% false-alarm limit and compare the resulting missed-alternative error.

   Hint

   The rejection region becomes a low-count event.

2. Derive the type-probability identity by grouping symbols with equal empirical counts. Explain why the divergence arguments have their displayed order.

   Hint

   Use counts times log source probability.

3. Compare the Stein divergences in both directions for Bernoulli probabilities 0.3 and 0.6. Explain why the asymmetric results do not contradict a symmetric Chernoff exponent.

   Hint

   Fix the acceptance region before choosing a divergence.

4. Reparameterize a Bernoulli probability by log odds. Derive its Fisher information and explain the changed error units.

   Hint

   Multiply by the squared probability derivative.

5. Solve a maximum-entropy distribution on states 0, 1, 2, 3 with mean 1.2. Then add a second-moment constraint and describe the extra multiplier.

   Hint

   The second feature is squared state.

6. In the fair contingent game, calculate expected growth at source probability 0.65 for allocations 0.5, 0.65 and 0.8. Verify the divergence loss and explain which strategy knows future outcomes.

   Hint

   None knows the future; the hindsight benchmark only fits a constant after observation.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 6, 11–12 and 16. Publisher book and contents.
2. T. Weissman, “Method of Types,” Stanford EE276 Lecture 15 notes, 2024. Primary lecture notes.
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, “Generalization Bounds: Perspectives from Information Theory and PAC-Bayes,” *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. Publisher metadata, author version.
4. A. El Gamal, “Information Theory and Statistical Learning,” proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

IT 08 • Networks, Information Inequalities and Learning

Analyze distributed sources and multiple users, connect advanced inequalities with learning, and run a finite distributed-decoding capstone.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-08-networks-information-learning/

What you'll learn

- Describe rate regions for distributed sources and multiple-access channels.
- Explain why broadcast, relay and general networks need different arguments.
- Apply log-sum, Pinsker, entropy-power and conditional-information inequalities.
- Distinguish representation information from information about a training sample.
- Run a small distributed parity-code decoder with explicit finite errors.
- Connect the book's complete chapter map to current learning research without overstating limits.

The big picture

One sender and one receiver lead to a single capacity. Several sources or receivers lead to a region of compatible rates. Improving one rate may reduce another, and information can be useful jointly even when each component looks useless alone.

> **Explain like I'm five**
>
> Two friends each know one clue. Either clue alone may tell you nothing, but together they solve the puzzle. If both friends speak at once, they also have to share the same noisy room. We must track which clues are shared, who hears what, and how much each friend can say.

This final tutorial follows Cover and Thomas's Chapter 15 on networks and Chapter 17 on inequalities [1]. Earlier chapters covered the other fifteen book chapters through the reading map in IT 01. We end with a finite synthetic experiment that connects source dependence, coding, decoding and task information.

The course uses the published second edition, supplemented by current surveys and a clearly labeled 2026 proposed chapter.

Distributed source coding: correlation can save rate

Two encoders can separately describe correlated observations while a joint decoder uses their relationship. The encoders need not each send every bit independently.

Let independent pairs (X_t, Y_t) follow a fixed finite joint law. Encoder one observes only X^n, encoder two only Y^n, and a common decoder reconstructs both blocks. The Slepian–Wolf asymptotically almost-lossless region is

$$R_X \geq H(X \mid Y), \qquad R_Y \geq H(Y \mid X), \qquad R_X + R_Y \geq H(X, Y).$$

Here R_X, R_Y are bits per paired source sample sent by the separate encoders, and the entropies use the specified joint source. Boundary inequalities describe the closure; codes operate with suitable rate margins and vanishing block error [1].

The individual bounds assume the other source can help at decoding. The sum bound counts total joint uncertainty. Neither encoder is assumed to see the other's current block; allowing that observation would define a different problem.

For fair X and $Y = X \oplus E$ with independent Bernoulli noise probability e, both marginal entropies are one and joint entropy is $1 + h_2(e)$. Separately ignoring dependence would require two bits per pair asymptotically; joint decoding can exploit the smaller joint uncertainty.

One corner sends Y at its full marginal rate and compresses X toward $H(X \mid Y)$. The decoder already has the reconstructed Y, which acts as side information. This corner motivates the parity-syndrome experiment below.

Random binning supplies an existence argument: assign source blocks to bins and use the other decoded source to find the likely matching block in the signaled bin. The bin index tells less than a full source index, while the correlation resolves ambiguity.

This result is not exact reconstruction of every possible finite pair. Rare or ambiguous blocks can fail. Truly lossless finite protocols require an additional escape or other treatment if every allowed realization must be recovered.

Why random bins produce three rate constraints

Each encoder randomly assigns every possible source block to a bin and sends its bin index. The decoder searches for a jointly typical pair in the two indicated bins. A competing pair can be wrong in three ways: only X is wrong, only Y is wrong, or both are wrong.

Competing pair	Typical candidate count scale	Collision probability per candidate
Wrong X, correct Y	$2^{nH(X\|Y)}$	2^{-nR_X}
Correct X, wrong Y	$2^{nH(Y\|X)}$	2^{-nR_Y}
Both wrong	$2^{nH(X,Y)}$	$2^{-n(R_X+R_Y)}$

Here n is paired block length, R_X, R_Y the sent bin-index rates, and the entropy exponents use the fixed joint source. These scales omit the small typicality tolerances and finite prefactors. Independent bin assignments give the collision factors, while joint typicality limits how many candidates need consideration.

Multiply each count by its collision probability and use a union bound. All three terms vanish with suitable margins exactly when the two individual rates exceed their conditional entropies

and their sum exceeds joint entropy. The true pair fails typicality with a probability that also vanishes. Averaging over assignments then proves some fixed shared binning rule has small block error [1].

This proof explains both the correlation gain and its limit. Decoder side information reduces the number of candidate X blocks from the full source catalog to a conditional catalog. It does not eliminate the need to distinguish the remaining candidates. The sum constraint handles simultaneous ambiguity that neither individual bound can remove alone.

For fair bits with flip probability 0.1, conditional entropy is about 0.468996 and joint entropy 1.468996 bits per pair. The corner rates are approximately $(0.468996, 1)$ and $(1, 0.468996)$. A symmetric point has rate 0.734498 for each encoder; assigning 0.5 to each passes the individual bounds but fails the sum bound. These are limiting boundary values, so actual asymptotic constructions use margins rather than promising a short-block code at the boundary.

Multiple access: independent messages share one output

Two senders can both communicate through a common receiver. The receiver's observation limits their combined information as well as each individual rate.

For a finite memoryless channel $W(y \mid x_1, x_2)$ with independent messages, the region for a fixed product input law includes

$$R_1 \le I(X_1; Y \mid X_2), \qquad R_2 \le I(X_2; Y \mid X_1), \qquad R_1 + R_2 \le I(X_1, X_2; Y).$$

Here X_1, X_2 are sender inputs, Y the shared output, and R_1, R_2 bits per common channel use. Union over product input laws and time-sharing gives the ordinary two-user multiple-access capacity region [1].

Conditioning in an individual bound reflects the receiver being able to distinguish that user's message when the other input is known. The sum bound addresses joint decoding. Maximizing each individual expression separately does not remove the shared sum constraint.

Time-sharing introduces a common predetermined variable T and product inputs given T. The corresponding bounds condition each expression on T, and convexify operating points. The senders do not gain arbitrary message-dependent cooperation simply because they share a schedule.

For independent real Gaussian signals with powers P_1, P_2 and additive noise variance N,

$$\begin{aligned} R_1 &\le \tfrac{1}{2} \log_2(1 + P_1/N), \\ R_2 &\le \tfrac{1}{2} \log_2(1 + P_2/N), \\ R_1 + R_2 &\le \tfrac{1}{2} \log_2(1 + (P_1 + P_2)/N). \end{aligned}$$

Here one use contains both simultaneous real inputs; the signals have independent messages and the standard average-power constraints. The one-half factors retain the real-valued convention.

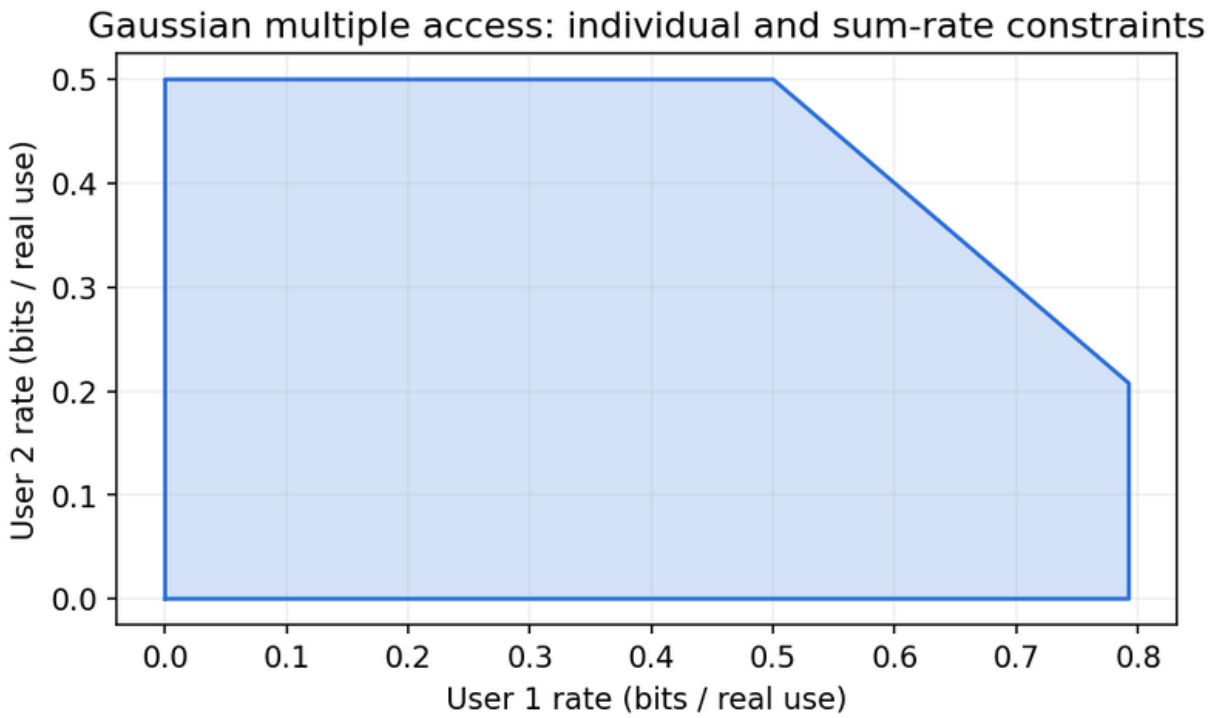


Figure 1 — P_1=2, P_2=1 and N=1. Individual maxima cannot both be reached simultaneously because of the sum-rate constraint.

Successive decoding can achieve corner points by first decoding one message while treating the other as noise, then removing the decoded contribution. The order changes which user receives the more favorable conditional rate. Joint decoding and time-sharing describe the broader region.

Work through the two Gaussian decoding orders

For the figure's powers $P_1 = 2$, $P_2 = 1$ and noise variance $N = 1$, user one's individual bound is about 0.792481 bits per use, user two's is 0.5, and the shared sum bound is one. The pair $(0.75, 0.4)$ satisfies both individual bounds but violates the sum. The pair $(0.6, 0.35)$ satisfies all three.

Decode user one first. Its decoder initially treats user two's independent Gaussian signal as additional noise of variance one, so its rate is limited to $\frac{1}{2}\log_2(1 + 2/(1 + 1)) = 0.5$. After successful decoding and subtraction, user two sees only the original noise and reaches 0.5. This order gives corner $(0.5, 0.5)$.

Decode user two first instead. Its initial rate is $\frac{1}{2}\log_2(1 + 1/(1 + 2)) \approx 0.207519$. After removing that signal, user one reaches $\frac{1}{2}\log_2(1 + 2/1) \approx 0.792481$. This gives corner $(0.792481, 0.207519)$. Both corners have total rate one, but decoding order allocates that total differently.

The subtraction argument assumes the first message was decoded correctly. A wrong first decision can propagate to the second, so a finite receiver must measure joint message error as well

as individual errors. The asymptotic corner calculation guides design without hiding that implementation issue.

Broadcast and relay channels change the information pattern

One sender with two receivers is not the reverse of two senders with one receiver. A relay that hears and retransmits creates another distinct pattern.

In a physically degraded broadcast channel, $X \to Y_1 \to Y_2$ makes receiver one stronger in the specified statistical sense. A superposition construction uses an auxiliary variable U with $U \to X \to Y_1 \to Y_2$:

$$R_2 \le I(U;Y_2), \qquad R_1 \le I(X;Y_1 \mid U).$$

Here R_2 is the weaker receiver's private-message rate, R_1 the stronger receiver's private-message rate, and U the layer both can resolve. Union over appropriate $p(u,x)$ gives the degraded-channel capacity region [1].

The Markov order is an assumption about receiver observations, not simply a ranking by sample accuracy. A general broadcast channel need not be degraded and does not inherit this exact region.

For a memoryless relay, let source input be X_s, relay input X_r, relay observation Y_r and destination observation Y_d. A cut-set upper bound is

$$C_{\text{relay}} \le \max_{p(x_s,x_r)} \min\{I(X_s;Y_r,Y_d \mid X_r), I(X_s,X_r;Y_d)\}.$$

Here the first cut separates the source from relay plus destination; the second separates source plus relay from the destination. The input joint law in the upper bound permits statistical cooperation as an outer relaxation.

A cut-set bound is not automatically achievable. Relays may decode a message, describe an observation, or use other causal strategies, each with its own conditions. General network capacity remains unresolved in many settings.

The lesson is to draw the actual information pattern before applying a formula: independent messages, correlated sources, side observations, causal relays and receiver demands are different assumptions. A scalar point-to-point capacity cannot summarize all of them.

Joint usefulness can be invisible to marginal information

Two individually uninformative features can jointly determine a target. Ranking only one feature at a time may discard both.

Let X_1, X_2 be independent fair bits and $Y = X_1 \oplus X_2$. Then

$$I(X_1;Y) = I(X_2;Y) = 0, \qquad I(X_1,X_2;Y) = 1, \qquad I(X_1;Y \mid X_2) = 1.$$

Here Y is the binary exclusive-or target. Each marginal input leaves the target fair; knowing both gives an exact answer. Conditioning on the other input reveals the first one's full relevance.

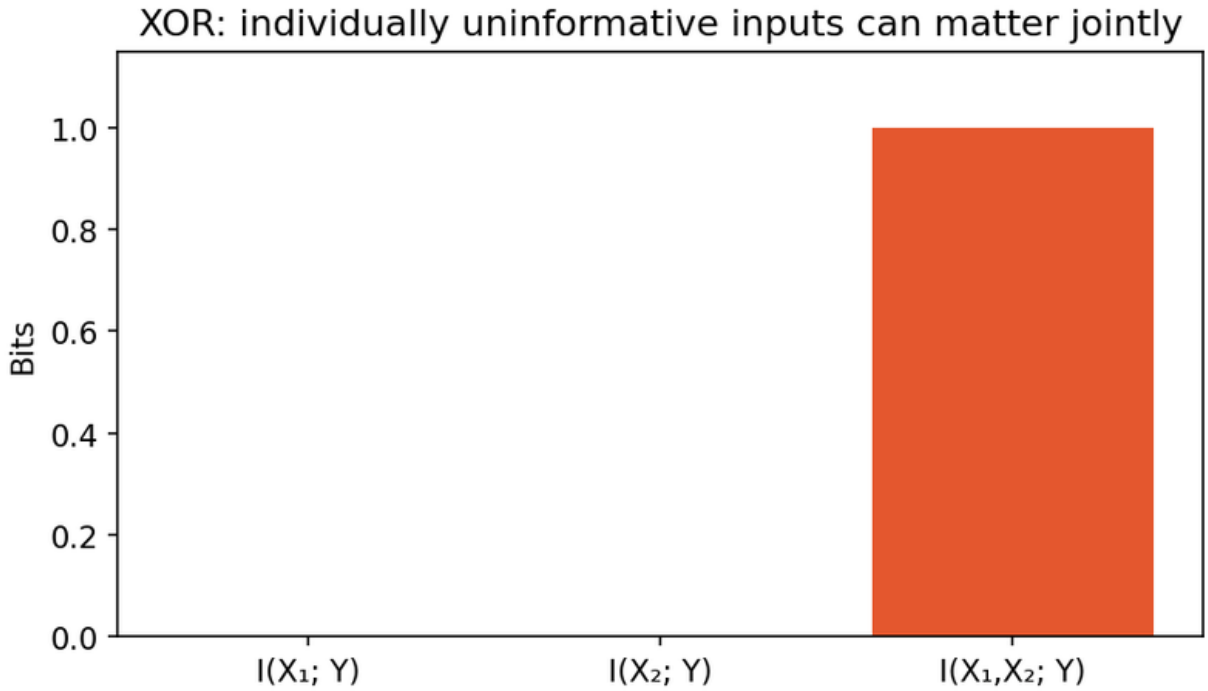


Figure 2 — Exact synthetic joint law. This illustrates joint usefulness without invoking a particular decomposition of redundancy and synergy.

The word synergy is useful informally for this joint-only information, but a general quantitative redundancy–synergy decomposition requires an additional definition. The three mutual-information values alone do not define a universal decomposition for arbitrary variables.

This matters for multimodal learning. A modality that seems weak by itself may be useful in context. Conversely, two strong marginal features may carry largely overlapping information. Analyze the actual joint law and receiver task rather than equating marginal relevance with unique contribution.

Computing a function is another distributed problem

Suppose separate encoders observe independent fair blocks X_1^n and X_2^n, and the decoder needs only their coordinatewise exclusive-or. The output has one bit of entropy per pair. It is tempting to conclude that one total bit per pair must suffice.

That conclusion ignores who sees the inputs. Give the decoder the second raw block as extra information. If it can recover the exclusive-or from the two transmitted indices, it can then recover the first block. The second encoder's index supplies no additional first-block information because the inputs are independent.

Consequently the first encoder must convey approximately one bit per coordinate for vanishing error in this example. By symmetry, so must the second. The distributed sum requirement can therefore be two bits even though the desired function has entropy one.

If one central encoder observed both inputs, it could instead transmit the exclusive-or itself at one bit per pair. The target is unchanged, but the observation pattern differs. This is a concrete reason to keep the network diagram beside a task-oriented rate claim.

The example also separates two meanings of "joint information." The receiver may need a joint computation, while the senders lack joint observation. A learned system cannot assume a centrally available feature merely because that feature would be sufficient after it had been computed.

Real applications can have correlated inputs, interaction, common randomness or additional side information, each changing the problem. Do not transfer the independent noninteractive example's rate conclusion to those settings without deriving the corresponding constraints.

Stronger inequalities extend the toolkit

Information inequalities often compare what remains after merging outcomes, changing a distribution, or adding independent noise. Their units and assumptions decide what they imply.

For nonnegative numbers a_i and positive b_i, the log-sum inequality states

$$\sum_i a_i \ln \frac{a_i}{b_i} \geq A \ln \frac{A}{B}, \qquad A = \sum_i a_i, \quad B = \sum_i b_i.$$

Here i indexes entries, A, B are their sums, and zero terms use the standard continuous convention. The all-zero A case has both sides zero. This inequality underlies divergence convexity and aggregation forms of data processing [1].

Applying it inside each cell of a partition shows that merging outcomes cannot increase relative entropy. Fine measurements can distinguish distributions in ways a coarse summary no longer reveals.

For finite probability vectors p, q, Pinsker's inequality gives

$$\mathrm{TV}(p, q) = \frac{1}{2} \sum_i |p_i - q_i| \leq \sqrt{\frac{1}{2} D_{\mathrm{nats}}(p \| q)}.$$

Here total variation bounds differences in event probabilities, and divergence uses natural logarithms. If divergence is reported in bits, multiply it by $\ln 2$ inside the square root. A small divergence therefore controls event-probability differences, while large divergence can make the bound uninformative.

Discrete entropy is also submodular:

$$H(X_A) + H(X_B) \geq H(X_{A \cup B}) + H(X_{A \cap B}).$$

Here A, B are sets of variable indices, X_A is the corresponding collection, and the empty collection has entropy zero. The difference is a conditional mutual information between the nonoverlapping

parts given the intersection, so it is nonnegative.

This is a relation among entropies of subsets, not a statement that every task-relevance function is submodular. The XOR target is a useful reminder that feature information can increase when another feature is supplied.

Entropy power and added Gaussian noise

Differential entropy can be converted into a variance-like quantity. Adding independent continuous signals cannot make their combined entropy power smaller than the sum.

For a d-dimensional vector with finite natural-unit differential entropy, define entropy power

$$\mathcal{N}(X) = \frac{1}{2\pi e} \exp\left(\frac{2h_{\text{nats}}(X)}{d}\right).$$

Here d is dimension and h_{nats} uses natural logarithms. A scalar Gaussian has entropy power equal to its variance; an isotropic Gaussian vector has the common coordinate variance.

The entropy-power inequality for independent vectors of the same dimension states

$$\mathcal{N}(X + Y) \geq \mathcal{N}(X) + \mathcal{N}(Y).$$

Here X, Y must satisfy the relevant density and finiteness assumptions. Equality holds for independent Gaussian vectors with proportional covariance matrices in the standard nondegenerate setting [1].

The de Bruijn identity connects noise smoothing with Fisher information:

$$\frac{d}{dt} h_{\text{nats}}(X + \sqrt{t}\, G) = \frac{1}{2} \mathcal{J}_{\text{loc}}(X + \sqrt{t}\, G).$$

Here G is an independent standard Gaussian vector, $t > 0$ is noise variance per coordinate, and $\mathcal{J}_{\text{loc}}(V) = \mathbb{E}\|\nabla \ln f_V(V)\|^2$ is location Fisher information. Smoothness and integrability conditions are required.

This Fisher information is a density-gradient quantity related to location estimation, not an arbitrary parameter's Fisher information from the previous chapter. Keeping that definition explicit avoids confusing two similarly named objects.

Adding noise can increase observation entropy while reducing information about the original. There is no contradiction: entropy power concerns output spread; data processing concerns retained relationships. Modern noisy representation models must distinguish those two claims.

Representation information and sample information

A feature can preserve a target while limiting raw input detail. A trained model can also reveal how much it depends on a particular training sample. These are different information questions.

An information-bottleneck objective for $Y \to X \to Z$ is

$$\min_{p(z|x)} I(X;Z) - \beta I(Z;Y).$$

Here X is the observation, Y the chosen relevance target, Z a possibly stochastic representation, and $\beta \geq 0$ balances compression against target information. Goldfeld and Polyanskiy's published tutorial [2] develops this theory and its learning connections.

The objective selects a task-relative statistical representation. It does not establish that every neural network spontaneously follows the same optimization, nor that one representation preserves every future target.

A bottleneck that removes nuisance without losing the label

A finite example makes the representation objective operational. Let label Y and nuisance bit N be independent fair bits, and let observation $X = (Y, N)$ contain both. Compare a full representation $Z = X$, a label-only representation $Z = Y$, and a constant representation.

Representation	$I(X;Z)$	$I(Z;Y)$	Objective $I(X;Z) - \beta I(Z;Y)$
Both bits	2	1	$2 - \beta$
Label only	1	1	$1 - \beta$
Constant	0	0	0

All information values are bits. The relevance weight β is dimensionless. The label-only representation retains everything needed for this label and spends one less bit than the full representation. For $\beta < 1$, a constant wins; for $\beta > 1$, retaining the label wins among these choices. At $\beta = 1$, label-only and constant tie.

This conclusion is not limited to those three examples. The information chain rule gives

$$I(X;Z) - \beta I(Z;Y) = (1-\beta) I(Y;Z) + I(N;Z \mid Y).$$

Here $X = (Y, N)$, N is the independent nuisance bit and Z any representation generated from X. The conditional term is nonnegative. For $\beta \leq 1$, zero is therefore a global lower bound, attained by a constant. For $\beta > 1$, using $I(Y;Z) \leq H(Y) = 1$ gives lower bound $1 - \beta$, attained by $Z = Y$. The task-preserving optimum follows from the chain rule and an explicit source, rather than an informal claim that compression always improves learning.

Now change the future target to $Y' = Y \oplus N$. The old label-only representation has zero information about this new fair target, while the full observation determines it. Irreversible task-specific compression limits future task transfer even when it is optimal for the original relevance criterion.

The notation also separates the two information questions in this section. These values concern one observation and its representation. The learning bound below instead concerns a model W and the whole training sample S. Reducing nuisance information in Z does not directly establish a smaller $I(W;S)$ or a numerical generalization guarantee.

For learning, let $S = (Z_1, \ldots, Z_n)$ be an independent training sample and W the learned model. Define population and empirical losses

$$L(W) = \mathbb{E}_{Z\sim P}[\ell(W,Z)], \qquad \hat{L}_S(W) = \frac{1}{n}\sum_{i=1}^{n} \ell(W,Z_i).$$

Here P is the population law, a fresh Z is independent of training, and ℓ is the chosen loss. Now the relevant information term can be $I(W;S)$, rather than feature information $I(X;Z)$.

If every fixed model's loss is σ-sub-Gaussian under P, a standard expectation bound has the form

$$\left|\mathbb{E}[L(W) - \hat{L}_S(W)]\right| \le \sqrt{\frac{2\sigma^2 I_{\text{nats}}(W;S)}{n}}.$$

Here expectation averages the sample and any learner randomness, σ is the common concentration parameter, and information uses natural logarithms. Bounded loss in $[0,1]$ permits $\sigma = 1/2$ [3].

This is an expectation bound, not automatically a high-probability certificate for one trained network. A deterministic continuous learner can have infinite sample information, making it vacuous. Conditional-information frameworks and other refinements address different randomization and sampling conventions [3].

Code: an exact finite learning bound

This complete example learns only a constant binary classifier by majority label. Because its output is a deterministic bit, $I(W;S) = H(W)$; all expectations can be summed over binomial counts.

```
import numpy as np
from scipy.stats import binom

p = 0.65
for n in [5, 20, 100]:
    counts = np.arange(n+1)
    mass = binom.pmf(counts, n, p)
    prediction = counts >= n/2
    training_loss = np.where(prediction, (n-counts)/n, counts/n)
    population_loss = np.where(prediction, 1-p, p)
    gap = mass @ (population_loss-training_loss)
    probability_one = mass[prediction].sum()
    probabilities = np.array([probability_one, 1-probability_one])
    positive = probabilities > 0
    information_nats = -np.sum(probabilities[positive]*np.log(probabilities[positive]))
    bound = np.sqrt(information_nats/(2*n))
    assert abs(gap) <= bound + 1e-12
    print("Length / expected gap / information bound:", n, gap, bound)
```

The calculation includes the selection dependence between model and sample. Treating the chosen classifier as independent of its training labels would erase the very generalization gap being analyzed.

Capstone: side-information decoding with parity bins

The encoder sends a short parity label for its binary block. The decoder combines that label with a correlated block to select the most likely original.

For binary block $x \in \{0,1\}^n$ and public binary matrix H with r independent rows, transmit syndrome

$$s = Hx \pmod 2.$$

Here H has shape r by n, s contains r parity bits, and multiplication uses binary-field addition and multiplication. Each syndrome identifies a bin of candidate blocks.

The decoder has $y = x \oplus e$, with independent flip probability below one half, and chooses a syndrome-compatible candidate minimizing Hamming distance to y. For this specified law, that is maximum conditional likelihood.

The encoder never sees y. The matrix and block length are shared; the decoder obtains y as side information. The payload rate is r/n, excluding any protocol for delivering or reconstructing that side information.

Thus the experiment corresponds to a source-coding corner, rather than compressing both observations for free. If the second block must also be transmitted, its description length belongs in the total rate. Shared side information is an assumption with a source, not an unexplained decoder advantage.

Code: exhaustive finite decoding and full-block error

The complete CPU example uses ten-bit blocks so exhaustive search is practical. It checks matrix rank over the binary field rather than using real-valued numerical rank.

```
import numpy as np

rng = np.random.default_rng(43)
n, trials, flip = 10, 3000, 0.1
patterns = ((np.arange(2**n)[:, None] >> np.arange(n)) & 1).astype(np.uint8)
source_indices = rng.integers(2**n, size=trials)
source = patterns[source_indices]
side = source ^ rng.binomial(1, flip, size=source.shape).astype(np.uint8)

def binary_rank(matrix):
    work = matrix.copy()
    rank = 0
    for column in range(work.shape[1]):
        pivots = np.flatnonzero(work[rank:, column])
        if len(pivots) == 0:
            continue
        pivot = rank + pivots[0]
        work[[rank, pivot]] = work[[pivot, rank]]
        for row in range(work.shape[0]):
```

```
            if row != rank and work[row, column]:
                work[row] ^= work[rank]
        rank += 1
        if rank == work.shape[0]:
            break
    return rank

for parity_bits in [4, 6, 8]:
    while True:
        H = rng.integers(2, size=(parity_bits, n), dtype=np.uint8)
        if binary_rank(H) == parity_bits:
            break
    weights = 2**np.arange(parity_bits)
    syndromes = ((patterns @ H.T) % 2) @ weights
    bins = [np.flatnonzero(syndromes == s) for s in range(2**parity_bits)]
    assert all(len(group) == 2**(n-parity_bits) for group in bins)
    decoded = np.empty(trials, dtype=int)
    for t, (index, observation) in enumerate(zip(source_indices, side)):
        candidates = bins[syndromes[index]]
        distances = np.count_nonzero(patterns[candidates] != observation, axis=1)
        decoded[t] = candidates[np.argmin(distances)]
    block_error = np.mean(decoded != source_indices)
    print("Parity bits / payload rate / full-block error:",
          parity_bits, parity_bits/n, block_error)
conditional_entropy = -flip*np.log2(flip)-(1-flip)*np.log2(1-flip)
print("Asymptotic side-information reference:", conditional_entropy)
```

This is a finite demonstration, not a capacity-achieving construction. Different random matrices produce different errors; increasing parity count need not improve every unrelated sampled matrix monotonically. The reference entropy is a long-block limit, not an exact ten-bit prediction.

Tie handling is deterministic through candidate order. Noise can move the side block closer to an incorrect syndrome-compatible source. A full round-trip assertion for every noisy trial would therefore be the wrong test: the protocol intentionally reports finite message errors.

For a truly lossless file protocol, provide an escape mechanism or interactive correction with explicit transmitted cost. Here the learning goal is to see how side information narrows bins and how finite ambiguity remains.

Modern view

The published 2025 monograph by Hellström and colleagues [3] maps information-theoretic and PAC-Bayesian generalization through expectation, probability and conditional-information formulations. That taxonomy helps distinguish a bound on average selection dependence from a guarantee for one deployed predictor.

Nguyen and colleagues' 2025 semantic-communication survey [4] distinguishes receiver-intention modeling, generative decoding and learned joint coding. Its network perspective makes receiver knowledge and task goals central; those must be included in the mathematical

information pattern.

El Gamal's proposed 2026 chapter [5] extends the textbook conversation toward statistical learning. Its draft status remains explicit. Modern inference and generative models can use the same divergence machinery while posing additional computational and evaluation problems.

Open questions include general multiuser regions, short-block distributed codes, realistic dependence and channel shift, task criteria for rare content, and useful information bounds for high-dimensional learned systems. A new model architecture does not solve these questions merely by producing plausible samples.

Our course-level inference is to ask five questions before interpreting an information claim: which random variables are involved; who observes each one; what source and channel laws are assumed; which reconstruction or decision is required; and how rate, error and resource cost are measured.

An information audit can turn those questions into a compact experiment record:

Choice	What to record
Source	Population, grouping and dependence assumptions
Observation	Variables available separately at each encoder and receiver
Description	Alphabet, actual bitstream, shared parameters and side streams
Channel	Conditional law, memory, power and use convention
Goal	Exact message, lossy reconstruction or specified task
Evaluation	Fixed test set, error definition, rate and runtime

The record is useful even when no tight theorem is known. It prevents a number from silently changing meaning between training, mathematical analysis and deployment. It also makes negative results informative: a poor rate may come from a probability model, a reconstruction constraint or a finite decoder, rather than from the same underlying limitation.

A reading and project path after the course

Revisit the full book in three passes. First consolidate Chapters 1–5 and 7–10 with the entropy, coding, channel and distortion examples. Then connect Chapters 6 and 11–14 to prediction, evidence, maximum entropy, universal codes and description complexity. Finally study Chapters 15–17 with the network assumptions and inequalities kept beside every theorem.

For a small project, choose one explicit joint law or independently collected dataset and one receiver objective. Implement a baseline encoder–decoder, report actual rate and decision error, then change only one component at a time. A probability model, a quantizer and a channel code answer different experimental questions.

The autoregressive vision chapter connects sequential probability factorization with modern models. The representation chapter develops task features, and the language-model chapter connects conditional prediction with token sequences.

These connections are strongest when the information definitions stay concrete. A tokenization choice changes the alphabet and unit; a learned latent changes the observation; a generated reconstruction changes the distortion question. State those choices before comparing numerical information values.

Key takeaways

- Networks produce compatible rate regions rather than one universal scalar rate.
- Distributed decoding exploits correlation without granting encoders unobserved inputs.
- Multiple-access individual rates remain subject to a shared sum constraint.
- Broadcast and relay formulas depend on their specific information pattern.
- Joint task information can be invisible to individual feature information.
- Advanced inequalities require explicit units, supports and independence assumptions.
- Representation information and sample-dependent generalization are distinct objects.
- Finite decoding, actual rate and honest error reporting complete the theoretical picture.

Exercises

1. For fair X and $Y = X \oplus E$ with flip probability 0.2, calculate the Slepian–Wolf corner rates and joint entropy. Explain what each encoder observes.

   Hint

   Use $h_2(0.2)$ for conditional entropy.

2. Compute the two Gaussian multiple-access corner points for the figure parameters. Verify the sum constraint and describe the successive-decoding order.

   Hint

   One individual rate reaches its maximum and the other fills the sum bound.

3. Explain why an arbitrary broadcast channel cannot inherit the physically degraded region. Identify the missing conditional relation.

   Hint

   Check $X \to Y_1 \to Y_2$.

4. Verify Pinsker's bound for probability vectors $(0.6, 0.4)$ and $(0.5, 0.5)$, using both natural and base-two divergence consistently.

   Hint

   Convert bits to nats with $\ln 2$.

5. Change parity-code block length to 12 and compare at several parity counts. Keep exhaustive-search cost visible and report block errors rather than only bit errors.

Hint

The candidate table has 2^n rows.

6. For the XOR target, compare keeping only X_1 with keeping the pair. Then explain why a deterministic continuous representation's raw-input information can be infinite while its discrete task information stays bounded.

Hint

Separate target entropy from exact continuous relationships.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 15 and 17. Publisher book and contents.
2. Z. Goldfeld and Y. Polyanskiy, "The Information Bottleneck Problem and Its Applications in Machine Learning," *IEEE Journal on Selected Areas in Information Theory*, 1(1), 2020, DOI: 10.1109/JSAIT.2020.2991561. Institutional final-manuscript record.
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, "Generalization Bounds: Perspectives from Information Theory and PAC-Bayes," *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. Publisher metadata, author version.
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, "A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding," *IEEE Communications Surveys & Tutorials*, 2025. Published-paper metadata and author version.
5. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

Part III

Blog Essays

Introducing Pixels → Perception

A bilingual place to study image processing, machine learning, and the ideas behind them through explanations, code, and interactive experiments.

Online edition: https://blog.shyandram.dev/en/blog/introducing-pixels-perception/

An image begins as an array of numbers. Understanding what those numbers represent leads to questions about signals, patterns, learning, and uncertainty. How can a small operation change the appearance of an image? How does a model learn to recognize something? What should we check before trusting its output?

I'm introducing Pixels → Perception, a tutorial website in English and Traditional Chinese that brings these questions into a connected learning path. It covers image processing, pattern recognition, machine learning, deep learning, computer vision, natural language processing, optimization, and information theory. The aim is to help readers move between an intuitive explanation, the mathematics behind it, and an experiment they can inspect for themselves.

There is a particular gap this site tries to address: the distance between recognizing an equation and understanding what happens when you use it. A definition may be clear on the page, yet its assumptions can become much easier to see when a parameter changes, a plot moves, or an example fails. The tutorials are organized to make those connections explicit.

The learning section has eight tracks:

- **Foundations** introduces the site, suggested learning paths, and basic ways to represent and work with images.
- **Image Processing** develops the tools for treating images as signals, from intensity and spatial operations to restoration, compression, and segmentation.
- **Natural Language Processing** explores how to represent language and build systems that work with text, connecting linguistic ideas with learning methods.
- **Pattern Recognition** focuses on features, statistical decisions, classification, and the uncertainty involved in recognizing patterns.
- **Machine Learning** covers mathematical foundations, neural networks, learned representations, generative models, and evaluation.
- **Deep Learning for Computer Vision** follows those ideas into tasks such as image classification, object detection, restoration, enhancement, and tracking.
- **Optimization** examines how to formulate a problem, express its constraints, and choose a way to solve it.

- **Information Theory** studies uncertainty, compression, and communication, providing another perspective on ideas that appear throughout the other tracks.

These subjects share a great deal of common ground. The mathematical tools used to reason about a visual model also help explain a language model. A question about image quality can lead back to the assumptions of an imaging process. A training problem can become clearer when you examine the objective being optimized. Keeping the tracks together makes it easier to follow those connections without losing the structure of an individual subject.

The tutorials follow a consistent rhythm. They begin with learning goals and a plain-language picture of the topic, then develop definitions, equations, and examples. Code and figures give readers something concrete to examine. Later sections connect the fundamentals with research literature, discuss limitations, and end with takeaways and exercises. Prerequisite links help identify where to go when an explanation assumes something unfamiliar.

For example, the interactive playground includes an image-filtering experiment. You can edit a small grid of weights and see the output change. That creates a useful learning loop: make a prediction, adjust the weights, inspect the result, and return to the explanation. Other experiments explore ideas from the broader curriculum. The controls run in the browser, so you can start experimenting without setting up a programming environment.

The code examples serve a different purpose: they let you inspect the operations themselves. Readers who want to go further can change an example, compare outputs, or work through an exercise. Some tutorials also link to a runnable notebook. The small examples are intended to explain mechanisms; they do not reproduce the scale or performance of a full research system.

Both language versions are intended to support the same learning experience. The Traditional Chinese pages use terminology familiar to readers in Taiwan, and the language switcher connects corresponding pages. This can be useful when you understand an idea more comfortably in one language but need to recognize its terminology in the other.

The site also has a blog alongside the structured tutorials. The curriculum provides an ordered route through a subject; the blog provides room for introductions, notes, and discussions that sit outside that sequence. Search is available when you want to return to a specific idea without following a track from the beginning.

I also want to be clear about how the site was made. Much of its draft content and code was created with AI assistance. Human direction shapes the topics and course structure, and human review remains essential. **The content is still under review.** An explanation can be readable and still contain a mistake, and a working example can leave an important assumption unstated. Readers should check the cited sources when a detail matters and treat small demonstrations as a starting point for investigation.

If you are new to the subjects, start with the getting-started guide and Images as Arrays. If you already have a particular question, browse the tracks or open the playground and follow an experiment back to its related lesson. You can read in sequence, revisit a mathematical prerequisite, or spend a few minutes exploring one idea.

The invitation is simple: choose a question, make a prediction, and try an example. I hope Pixels → Perception helps turn something that looks abstract on a page into something you can explain, test, and use.

Visit Pixels → Perception in English or 繁體中文.

Welcome to Pixels → Perception

What this site covers, how the tutorials are organized, and how to run the code.

Online edition: https://blog.shyandram.dev/en/blog/welcome/

This site is a computer vision course you can read in order or dip into. It starts with images as arrays of numbers and builds up to modern deep networks.

How it is organized

The **Learn** section has eight tracks: Foundations, Image Processing, Natural Language Processing, Pattern Recognition, Machine Learning, Deep Learning for Computer Vision, Optimization and Information Theory. Each tutorial lists its prerequisites, so you can always see what to read first.

Running the code

Some tutorials provide a matching Jupyter notebook; others include standalone code blocks.

When a notebook is available, click **Open in Colab** at the top of the page to run it in your browser.

The blog, where this post lives, is for everything outside the curriculum: paper notes, tool reviews and longer essays.